REACT · END-TO-END · SDE2 & SSE

React

> Fiber, reconciliation, hooks, RSC, React 19 – "ఇది ఎందుకు re-render అవుతోంది?" అనే ప్రశ్నకి నిజమైన జవాబు.

Hooks ని గుర్తుపెట్టుకోవడం కాదు – rendering model ని అర్థం చేసుకోవడం. అప్పుడు ప్రతి hook తనంతట తానే అర్థమవుతుంది.

తెలుగు + ENGLISH

YASWANTH · REFERENCE

విషయ సూచిక (Table of Contents)

Part 1 – పునాదులు (Foundations)

1. React అంటే ఏమిటి, ఎందుకు పుట్టింది (declarative vs imperative, library vs framework, SPA, React history)
2. Setup & Project Structure (Vite, CRA ఎందుకు చనిపోయింది, folder structure, npm scripts)
3. JSX Deep (Babel transform, createElement, new JSX runtime, rules, expressions, Fragments)
4. Components & Props (function vs class, children, composition, prop drilling, defaults)
5. Rendering, Virtual DOM & Reconciliation (diffing algorithm, keys, ఎందుకు VDOM)
6. State & useState (batching, functional updates, lazy init, immutability, state ఎప్పుడు stale అవుతుంది)
7. Events (SyntheticEvent, root delegation, e.preventDefault, React 17+ మార్పులు)
8. Conditional Rendering (&&, ternary, early return, గోతులు)
9. Lists & Keys Deep (index key bug, reorder, stable ids)
10. Forms – Controlled vs Uncontrolled (multi-input, validation, React 19 Actions)

Part 2 – Hooks Deep Dive

11. Hooks Rules & ఎందుకు (call order, hooks linked list internals, linter)
12. useEffect Deep (lifecycle mapping, deps array, cleanup, StrictMode double-run, race conditions)
13. "You Might Not Need an Effect" (derived state, event handler vs effect – senior signal)
14. useLayoutEffect vs useEffect (paint timing, flicker fix)
15. useRef (DOM ref, mutable box, forwardRef, useImperativeHandle, callback refs, ref-as-prop in 19)
16. useMemo & useCallback (referential equality, ఎప్పుడు వాడకూడదు, cost model)
17. React.memo & re-render mental model
18. useContext & Context API (provider, re-render problem, context splitting, selector patterns)
19. useReducer (ఎప్పుడు useState కంటే మేలు, reducer + context architecture)
20. useTransition & useDeferredValue (concurrent rendering, priority)
21. useId, useSyncExternalStore, useDebugValue, useInsertionEffect
22. React 19 కొత్త APIs – use(), useActionState, useFormStatus, useOptimistic, ref cleanup
23. Custom Hooks (rules, composition, 10 production-grade custom hooks)

Part 3 – Class Components, Lifecycle & Patterns

24. Class Components & Lifecycle Methods (hooks తో mapping, ఇంకా ఎందుకు నేర్చుకోవాలి)
25. Error Boundaries (class only, react-error-boundary, error handling strategy)
26. HOC (Higher-Order Components) Pattern
27. Render Props Pattern

28. Compound Components, Control Props, State Reducer (library-grade component API design)

Part 4 – Advanced Rendering & React Internals

29. Fiber Architecture Deep (fiber node, work loop, render vs commit phase, double buffering)
30. Concurrent React (lanes, priorities, interruptible rendering, time slicing, tearing)
31. Suspense & React.lazy (code splitting, suspense boundaries, fallback strategy)
32. Portals (modals, tooltips, event bubbling through portals)
33. StrictMode (ఏం double-invoke అవుతుంది, ఎందుకు)
34. Refs & Imperative escape hatches, DOM measurement

Part 5 – State Management (Client & Server)

35. State రకాలు – local, lifted, global, server, URL, form (సరైన decision tree)
36. Context + useReducer architecture (mini-Redux, ఎప్పుడు సరిపోతుంది)
37. Redux Toolkit Deep (store, slices, thunks, selectors, normalization, RTK Query)
38. Zustand, Jotai, Recoil, MobX, Signals – comparison
39. TanStack Query (React Query) Deep – server state, caching, invalidation, optimistic updates
40. Forms at scale – React Hook Form + Zod

Part 6 – Routing

41. React Router v6/v7 Deep (nested routes, params, loaders/actions, protected routes, lazy, navigation blocking)

Part 7 – Performance Engineering

42. Re-render ఎందుకు జరుగుతుంది – పూర్తి mental model + DevTools Profiler
43. Optimization techniques (memo, virtualization, code splitting, debounce, transitions)
44. React Compiler (React 19) – auto-memoization
45. Bundle size, lazy loading, images, Core Web Vitals (LCP, INP, CLS)

Part 8 – Styling

46. CSS Modules, CSS-in-JS (styled-components/emotion), Tailwind, design tokens, RSC compatibility

Part 9 – Data Fetching, SSR & React Server Components

47. Client-side data fetching patterns (waterfalls, race conditions, caching)
48. CSR vs SSR vs SSG vs ISR vs Streaming SSR + Hydration internals
49. React Server Components (RSC) Deep – 'use client', 'use server', Server Actions
50. Next.js App Router (layouts, metadata, caching layers, route handlers, middleware, deployment)

Part 10 – TypeScript + React

51. Props typing, children, events, generic components, hooks typing, discriminated unions

Part 11 – Testing

52. React Testing Library + Vitest/Jest (queries, user-event, async, mocking, MSW, hooks testing, E2E)

Part 12 – Quality, Security & Accessibility

53. Accessibility (a11y) in React – semantic HTML, ARIA, focus management, keyboard nav
54. Security in React – XSS, dangerouslySetInnerHTML, env leaks, auth token storage
55. Error handling, logging, monitoring (Sentry), feature flags

Part 13 – Architecture & Frontend System Design

56. Folder structure & scalable architecture (feature-sliced, barrel files, dependency rules)
57. Component API design principles (props vs composition, controlled/uncontrolled)
58. Frontend System Design round (design Netflix/Twitter feed/autocomplete/chat – SSE interview)
59. Micro-frontends, monorepo, design systems, module federation

Part 14 – Interview Preparation

60. Machine Coding Round – 10 must-practice components (full code)
61. Rapid-fire Interview Q&A (SDE2 + SSE level, 100+ questions)
62. Anti-patterns & Common Mistakes cheat-sheet
63. Memory Tips – ఒకసారి చదివి గుర్తుపెట్టుకునే framework

Part 1 – పునాదులు (Foundations)

1. React అంటే ఏమిటి, ఎందుకు పుట్టింది

వివరణ

React = UI build చేయడానికి ఒక JavaScript library. ఇది Facebook (Meta) 2013 లో open-source చేసింది. దీని ఒకే ఒక్క పని: **"data ఇవ్వు, నేను UI చూపిస్తాను. Data మారితే, UI ని నేనే update చేస్తాను."**

React కి ముందు మనం **imperative** గా code రాసేవాళ్ళం – అంటే "ఎలా చేయాలో" step-by-step చెప్పడం:

```
// Imperative (vanilla JS / jQuery) – "ఎలా" చేయాలో మనమే చెప్పాలి
const btn = document.getElementById("like");
let count = 0;
btn.addEventListener("click", () => {
  count++;
  document.getElementById("count").textContent = count;      // manual DOM update
```

```
  if (count > 10) document.getElementById("badge").style.display = "block";
  else document.getElementById("badge").style.display = "none";
});
```

React లో **declarative** – అంటే *"ఏం కావాలో"* మాత్రమే చెప్తాం:

```
// Declarative (React) — "ఏం" కావాలో చెప్తాం, "ఎలా" React చూసుకుంటుంది
function LikeButton() {
  const [count, setCount] = useState(0);
  return (
    <>
      <button onClick={() => setCount(count + 1)}>Like</button>
      <span>{count}</span>
      {count > 10 && <Badge />}
    </>
  );
}
```

కీలక తేడా: imperative లో మనం *DOM* ని నిర్వహిస్తాం. Declarative లో మనం *state* ని నిర్వహిస్తాం, DOM ని React నిర్వహిస్తుంది.

React యొక్క core equation ఇదే – దీన్ని మర్చిపోకూడదు:

```
UI = f(state)
```

అంటే **UI అనేది state యొక్క function.** అదే state ఇస్తే అదే UI రావాలి (deterministic). State మారితే React `f` ని మళ్ళీ run చేసి, కొత్త UI ని పాత UI తో compare చేసి, తేడా ఉన్న DOM part ని మాత్రమే update చేస్తుంది.

Real-life Scenario

> **Imperative = మీరే వంట చేయడం.** "గిన్నె పెట్టు, నూనె వేయి, ఉల్లిపాయ వేయించు, 5 నిమిషాలు కలుపు…" – ప్రతి step మీరే చెప్పాలి. ఏదైనా మిస్ అయితే వంట పాడవుతుంది.
>
> **Declarative = restaurant లో order చేయడం.** "నాకు బిర్యానీ కావాలి" అని చెప్తారు. ఎలా వండాలో chef (React) చూసుకుంటాడు. మీరు మళ్ళీ "ఇప్పుడు రైతా కూడా" అంటే (state change), chef మొత్తం మళ్ళీ వండడు – రైతా మాత్రమే తెచ్చి పెడతాడు (DOM diffing).

React ఎందుకు పుట్టింది – అసలు సమస్య

Facebook లో ఒక bug పదే పదే వచ్చేది: chat message read అయ్యింది కానీ **unread badge** ఇంకా చూపిస్తోంది. కారణం – ఒకే data (unread count) ని UI లో 5 చోట్ల manual గా update చేయాలి; ఒక చోట మర్చిపోతే UI **inconsistent** అవుతుంది.

React చెప్పింది: *"UI ని manual గా update చేయకు. State ని మార్చు. UI మొత్తం మళ్ళీ describe చెయ్యి. నేను చూసుకుంటాను."* – ఈ ఆలోచనే React యొక్క ఆత్మ.

React లోని 4 core ideas

Idea	అర్థం	ఎందుకు ముఖ్యం
Declarative	UI ని "ఏం" అని describe చేస్తాం	Bugs తగ్గుతాయి, code చదవడం సులభం
Component-based	UI ని చిన్న reusable ముక్కలుగా విడగొట్టడం	Reuse, testing, team parallel work
Unidirectional data flow	Data ఎప్పుడూ parent → child (top-down)	Debug సులభం – data ఎక్కడ నుండి వచ్చిందో తెలుస్తుంది
Learn once, write anywhere	Web (react-dom), Mobile (React Native), 3D, CLI	ఒకే mental model, అనేక platforms

Library vs Framework – React ఏది?

React ఒక library, framework కాదు. ఇది UI rendering మాత్రమే చేస్తుంది. Routing, data fetching, forms, state management, build – ఇవన్నీ మనం ఎంచుకోవాలి.

	Library (React)	Framework (Angular, Next.js)
Control	మనం library ని call చేస్తాం	Framework మన code ని call చేస్తుంది (Inversion of Control)
Decisions	మనం తీసుకోవాలి (routing, state…)	Framework ఇచ్చేస్తుంది
Flexibility	ఎక్కువ	తక్కువ, కానీ consistency ఎక్కువ
Learning curve	తక్కువ start, ఎక్కువ ecosystem confusion	ఎక్కువ start, తక్కువ confusion

> **Interview answer:** "React is a library for building user interfaces – it only owns the view layer. Everything else (routing, data fetching, state) is chosen from the ecosystem. That's why Next.js exists – it's the framework wrapper around React that makes those decisions for you."

React vs Angular vs Vue vs Svelte

	React	Angular	Vue	Svelte
రకం	Library	Full framework	Progressive framework	Compiler
Language	JS/TS + JSX	TypeScript (mandatory)	JS/TS + templates	Svelte syntax
Reactivity	Re-render + VDOM diff	Zone.js / Signals	Proxy-based reactive	Compile-time reactivity, **no VDOM**

	React	Angular	Vue	Svelte
Bundle	~45 KB (react+dom, gzip)	~150 KB+	~35 KB	~2 KB runtime
Learning	Medium	High	Easy	Easy
Jobs (India)	అత్యధికం	Enterprise/banking	Medium	తక్కువ

SPA (Single Page Application) అంటే

MPA (traditional): ప్రతి link click → server కి request → పూర్తి కొత్త HTML page → browser reload (తెల్ల flash).

SPA (React): ఒకే HTML file (`index.html`) load అవుతుంది. తర్వాత అన్ని "page changes" JavaScript లోనే జరుగుతాయి – URL మారుతుంది కానీ page reload అవ్వదు. Data మాత్రమే API నుండి JSON గా వస్తుంది.

```
<!-- ఒక SPA యొక్క మొత్తం HTML – ఇంతే! -->
<body>
  <div id="root"></div>          <!-- React ఇక్కడ మొత్తం app ని inject చేస్తుంది -->
  <script src="/assets/index-a3f2.js"></script>
</body>
```

	SPA (CSR)	MPA (SSR)
First load	నెమ్మది (JS bundle download + parse + render)	వేగం (server ready HTML పంపుతుంది)
Navigation	తక్షణం (instant, no reload)	ప్రతిసారి reload
SEO	బలహీనం (crawler కి JS run అవ్వాలి)	బలం
Server load	తక్కువ (static files)	ఎక్కువ
Offline	సాధ్యం (PWA)	కష్టం

> **ఇక్కడే Next.js / RSC వస్తుంది** – SSR యొక్క first-load వేగం + SPA యొక్క navigation వేగం రెండూ కలిపి ఇవ్వడానికి. Part 9 లో deep గా చూద్దాం.

React చరిత్ర – versions timeline (interview లో అడుగుతారు)

Version	సంవత్సరం	ముఖ్య మార్పు
0.14 / 15	2015-16	`react` + `react-dom` విడిపోయాయి
16 (Fiber)	2017	పూర్తి rewrite – Fiber architecture, Error Boundaries, Portals, Fragments, return arrays

Version	సంవత్సరం	ముఖ్య మార్పు
16.8	2019	**Hooks** 🎉 – function components కి state వచ్చింది
17	2020	"no new features" – event delegation `document` నుండి **root container** కి మారింది (gradual upgrade కోసం)
18	2022	**Concurrent React** – automatic batching, `createRoot`, `useTransition`, `useDeferredValue`, `useId`, `useSyncExternalStore`, Streaming SSR + Suspense
19	2024-25	**Actions**, `use()`, `useActionState`, `useFormStatus`, `useOptimistic`, **ref as prop** (forwardRef అవసరం లేదు), document metadata support, **React Compiler** (auto memo), RSC stable

Gotchas (సాధారణ తప్పులు)

- **"React fast ఎందుకంటే Virtual DOM"** అనడం – ఇది **తప్పు answer.** Direct DOM manipulation ఎప్పుడూ VDOM కంటే వేగం. VDOM ఇచ్చేది **"declarative code రాసినా, పనితీరు ఆమోదయోగ్యంగా ఉంటుంది"** అనే హామీ – maintainability కోసం చేసిన trade-off. (Topic 5 లో deep.)
- **React = MVC లో V** అని ఆగిపోవడం – ఇప్పుడు React కి state, effects, server components అన్నీ ఉన్నాయి; ఇది కేవలం view కంటే ఎక్కువ.
- **React నేర్చుకునే ముందు JavaScript skip చేయడం** – closures, `this`, event loop, promises, immutability, array methods (`map/filter/reduce`), destructuring, spread – ఇవి తెలియకపోతే React లో ప్రతి bug మిస్టరీ అవుతుంది. (`JavaScript_Telugu.md` చూడండి.)
- **SPA అన్నిటికీ సరైనది అనుకోవడం** – blog, e-commerce PLP వంటి SEO-heavy pages కి SSR/SSG మేలు.

Key Points

- React = **UI library**, `UI = f(state)`.
- **Declarative** (ఏం కావాలి) vs **imperative** (ఎలా చేయాలి).
- Core ideas: components, unidirectional data flow, declarative UI, learn-once-write-anywhere.
- React 16 = Fiber, 16.8 = Hooks, 18 = Concurrent, 19 = Actions + Compiler.
- React fast ఎందుకంటే VDOM కాదు – **VDOM వల్ల declarative రాసినా fast గా ఉంటుంది.**

Interview దృష్టి

- *"Why React over vanilla JS?"* → declarative UI + component reuse + ecosystem; manual DOM sync bugs పోతాయి.
- *"Is React MVC?"* → కాదు, React unopinionated view layer; component-based composition model.
- *"React ఎందుకు fast?"* → batching + reconciliation తో minimal DOM writes; కానీ నిజమైన లాభం maintainability, raw speed కాదు.
- *"What changed in React 18?"* → automatic batching, concurrent rendering, `createRoot`, streaming SSR, new hooks.

2. Setup & Project Structure

వివరణ

React project మొదలుపెట్టడానికి 2026 లో ప్రామాణిక మార్గం – **Vite** (SPA కోసం) లేదా **Next.js** (full-stack/SSR కోసం). **Create React App (CRA) deprecated** – React team అధికారికంగా దాన్ని 2025 లో retire చేసింది (నెమ్మది webpack config, పాత dependencies).

```
# Vite తో React + TypeScript project (recommended for SPA)
npm create vite@latest my-app -- --template react-ts
cd my-app && npm install && npm run dev
```

```
# Next.js (SSR / RSC / full-stack కావాలంటే)
npx create-next-app@latest my-app
```

Real-life Scenario

> **CRA vs Vite = పాత Ambassador car vs కొత్త EV.** రెండూ నిన్ను గమ్యానికి చేరుస్తాయి. కానీ CRA start అవ్వడానికి 40 సెకన్లు (webpack మొత్తం bundle చేస్తుంది), Vite 300 milliseconds (native ESM – browser అడిగిన file ని మాత్రమే transform చేస్తుంది). Team లో 10 మంది × రోజుకి 30 సార్లు restart = గంటల కొద్దీ ఆదా.

Vite ఎందుకు వేగం – internals

- **Dev లో bundling లేదు.** Browser native ES modules ని support చేస్తుంది. Vite ఒక dev server మాత్రమే – browser `import './App.jsx'` అడిగినప్పుడు, ఆ ఒక్క file ని **esbuild** (Go లో రాసింది, Babel కంటే 20-30× వేగం) తో transform చేసి పంపుతుంది.
- **Dependencies pre-bundled** (`node_modules` ని ఒకసారి esbuild తో bundle చేసి cache చేస్తుంది).
- **Production లో Rollup** తో bundle (tree-shaking, code-splitting).
- **HMR (Hot Module Replacement)** – file save చేస్తే, ఆ module మాత్రమే మారుతుంది, page reload అవ్వదు, state అలాగే ఉంటుంది.

Code – Entry point (React 18/19)

```
// src/main.jsx — app యొక్క entry point
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";   // ⚠️ 'react-dom' కాదు, 'react-
dom/client'
import App from "./App";
import "./index.css";

// React 18+ — createRoot (concurrent features enable అవుతాయి)
createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
```

```
);

// ❌ React 17 పాత విధానం (ఇప్పుడు వాడితే warning వస్తుంది):
// import ReactDOM from "react-dom";
// ReactDOM.render(<App />, document.getElementById("root"));
```

`createRoot` vs `render` తేడా ఎందుకు ముఖ్యం: `ReactDOM.render` legacy mode – automatic batching లేదు, `useTransition` వంటి concurrent features పని చేయవు. `createRoot` వాడితేనే React 18 యొక్క అన్ని ప్రయోజనాలు వస్తాయి. **Interview లో తరచూ అడుగుతారు.**

Code – package.json ఏం చెప్తుంది

```
{
  "name": "my-app",
  "type": "module",              // ESM (import/export) వాడతాం, CommonJS కాదు
  "scripts": {
    "dev": "vite",               // dev server
    "build": "tsc -b && vite build",  // type-check + production bundle → dist/
    "preview": "vite preview",   // build ని locally serve చేసి test చేయడం
    "lint": "eslint .",
    "test": "vitest"
  },
  "dependencies": {              // runtime లో కావాల్సినవి (bundle లోకి వెళ్తాయి)
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  },
  "devDependencies": {           // build/dev టైంలో మాత్రమే (bundle లోకి వెళ్ళవు)
    "vite": "^6.0.0",
    "@vitejs/plugin-react": "^4.3.0",
    "typescript": "^5.6.0"
  }
}
```

> **`^19.0.0` అంటే** – 19.x.x లో ఏదైనా (minor/patch upgrade OK), కానీ 20 కి వెళ్ళదు. `~19.0.0` = patch మాత్రమే (19.0.x). Exact కావాలంటే `19.0.0`. Production లో **`package-lock.json` commit చేయడం తప్పనిసరి** – అందరికీ ఒకే versions install అవుతాయి.

Folder Structure – beginner vs scalable

```
❌ చిన్న projects కి OK, పెద్దవాటికి నరకం (type-based grouping):
src/
  components/    ← 200 files ఇక్కడ
  hooks/
  utils/
  pages/

✅ Feature-based (scalable – ఒక feature మార్చాలంటే ఒకే folder చూస్తే చాలు):
src/
  app/                    # router, providers, global setup
    router.tsx
    providers.tsx
  features/
```

```
    auth/
      components/LoginForm.tsx
      hooks/useLogin.ts
      api/authApi.ts
      types.ts
      index.ts             # public API — బయటివారు ఇది మాత్రమే import చేయాలి
    cart/
      components/
      hooks/
      api/
  shared/                  # feature-agnostic reusable code
    ui/                    # Button, Input, Modal (design system)
    hooks/                 # useDebounce, useMediaQuery
    lib/                   # axios instance, formatters
    types/
  assets/
  main.tsx
```

నియమం (dependency rule): `features/*` → `shared/*` ని import చేయవచ్చు. `shared/*` ఎప్పుడూ `features/*` ని import చేయకూడదు. Feature-to-feature imports `index.ts` (public API) ద్వారానే. ఇది circular dependency లను ఆపుతుంది. (Part 13 లో deep.)

Gotchas (సాధారణ తప్పులు)

- **CRA తో కొత్త project మొదలుపెట్టడం** — deprecated. Vite లేదా Next.js వాడండి.
- **`react-dom/client` బదులు `react-dom` import చేయడం** — React 18 features రావు.
- **Secrets ని `VITE_` / `NEXT_PUBLIC_` env variables లో పెట్టడం** — ఇవి **bundle లోకి plain text గా వెళ్తాయి**, browser లో ఎవరైనా చూడొచ్చు! API keys ఎప్పుడూ backend లోనే. (Topic 54.)
- **`node_modules` ని git లో commit చేయడం** — `.gitignore` లో ఉండాలి.
- **Barrel files (`index.ts`) అతిగా వాడటం** — పెద్ద projects లో tree-shaking పాడై bundle పెరుగుతుంది, circular imports వస్తాయి.

Key Points

- **Vite** = SPA కి default; **Next.js** = SSR/RSC/full-stack కి; **CRA = dead.**
- Vite dev లో bundle చేయదు (native ESM + esbuild) → instant start + HMR.
- `createRoot` (react-dom/client) వాడాలి — లేకపోతే React 18/19 concurrent features రావు.
- **Feature-based folder structure** scale అవుతుంది; type-based కాదు.
- Client env vars = public. Secrets ఎప్పుడూ frontend లో వద్దు.

Interview దృష్టి

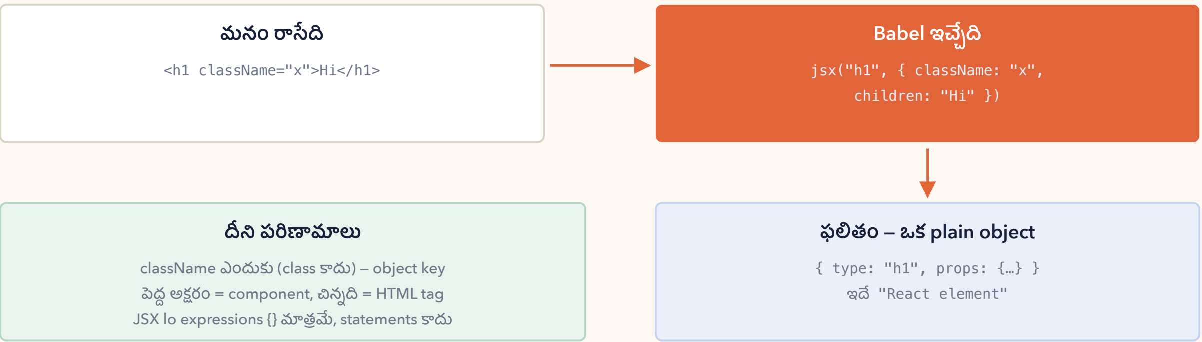
- *"CRA vs Vite?"* → webpack full bundle vs native ESM + esbuild; dev server speed, HMR, maintenance.
- *"createRoot ఎందుకు?"* → concurrent rendering + automatic batching enable చేస్తుంది.
- *"How do you structure a large React app?"* → feature-sliced, public API per feature, shared UI layer, unidirectional dependency rule.

---

3. JSX Deep

JSX · BABEL ఏం చేస్తుంది

JSX ఒక syntax మాత్రమే – Babel దీన్ని function call గా మారుస్తుంది



ఎందుకు ఇది తెలియాలి

JSX ఒక object ని **వర్ణిస్తుంది** – DOM ని సృష్టించదు. అది render సమయంలో జరుగుతుంది.
అందుకే JSX ని variable lo పెట్టొచ్చు, array lo పెట్టొచ్చు, function నుంచి return చేయొచ్చు.

వివరణ

JSX = JavaScript XML. ఇది JavaScript లో HTML లాంటి syntax రాయనిచ్చే **syntax extension**. Browser కి JSX అర్థం కాదు – **Babel** (లేదా Vite లోని esbuild/SWC) దాన్ని సాధారణ JavaScript function calls గా మారుస్తుంది.

```
// మనం రాసేది:
const el = <h1 className="title">Hello, {name}!</h1>;
```

```
// Babel మార్చేది (React 17 కి ముందు — classic runtime):
const el = React.createElement("h1", { className: "title" }, "Hello, ", name, "!");

// React 17+ (automatic JSX runtime — React ని import చేయాల్సిన అవసరం లేదు):
import { jsx as _jsx } from "react/jsx-runtime";
const el = _jsx("h1", { className: "title", children: ["Hello, ", name, "!"] });
```

అంటే **JSX కేవలం syntactic sugar.** JSX లేకుండా కూడా React రాయొచ్చు – కానీ చదవడం నరకం.

`React.createElement` return చేసేది DOM node కాదు – ఒక **plain JavaScript object** (దీన్నే "React element" అంటాం):

```
{
  $$typeof: Symbol(react.element),  // XSS నుండి రక్షణ కోసం (JSON injection ఆపుతుంది)
  type: "h1",                       // string = host element, function = component
  key: null,
  ref: null,
  props: { className: "title", children: [...] }
}
```

> ఈ object యే **Virtual DOM node.** అంటే VDOM అనేది ఒక మాయ కాదు – అది కేవలం "UI ఎలా ఉండాలో" చెప్పే **JS objects tree**, చవకైనది (DOM node create చేయడం ఖరీదు).

Real-life Scenario

> **JSX = ఇంటి blueprint (design plan).** Blueprint ఒక ఇల్లు కాదు – అది కేవలం కాగితం మీద ఇల్లు ఎలా ఉండాలో వివరణ. చవక, త్వరగా మార్చొచ్చు, పోల్చొచ్చు. React ఆ blueprint ని (React element tree) చూసి, real DOM (అసలు ఇల్లు) లో అవసరమైన మార్పులు మాత్రమే చేస్తుంది. కొత్త blueprint వచ్చినప్పుడు ఇల్లు మొత్తం కూల్చి కట్టడు – "ఈ గోడ రంగు మారింది" అని ఆ గోడ మాత్రమే మారుస్తాడు.

JSX నియమాలు (ప్రతి ఒక్కటి interview లో వస్తుంది)

```
function Rules() {
  const user = { name: "Surya", isAdmin: true };
  const items = ["a", "b"];

  return (
    // 1️⃣ ఒకే root element ఉండాలి → Fragment (<>) వాడతాం (extra div వద్దు)
    <>
      {/* 2️⃣ Attributes camelCase — HTML కాదు, JS objects */}
      <label htmlFor="name">Name</label>          {/* for → htmlFor */}
      <input id="name" className="box" tabIndex={0} readOnly />  {/* class →
className */}

      {/* 3️⃣ {} లోపల ఏ JS *expression* అయినా — statements కాదు (if/for రాయలేం) */}
      <h1>{user.name.toUpperCase()}</h1>
      <p>{2 + 2}</p>
      <p>{user.isAdmin ? "Admin" : "User"}</p>     {/* ternary OK, if కాదు */}
      <p>{items.map((i) => <span key={i}>{i}</span>)}</p>

      {/* 4️⃣ Style = object, camelCase, numbers → px */}
      <div style={{ backgroundColor: "red", fontSize: 16, marginTop: "1rem" }} />

      {/* 5️⃣ Self-closing tags తప్పనిసరి */}
      <br />
      <img src="/a.png" alt="a" />

      {/* 6️⃣ Comments ఇలా */}
    </>
  );
}
```

JSX లో ఏం render అవుతుంది, ఏం అవ్వదు

```
<div>
  {"text"}        {/* ✅ string */}
  {42}            {/* ✅ number */}
  {[1, 2]}        {/* ✅ arrays — flatten అవుతాయి */}
  {<Comp />}      {/* ✅ elements */}
```

```
  {null}          {/* 🚫 ఏమీ render కాదు (fine) */}
  {undefined}     {/* 🚫 ఏమీ render కాదు */}
  {false}         {/* 🚫 ఏమీ render కాదు — conditional rendering కి ఇదే ఆధారం */}
  {true}          {/* 🚫 ఏమీ render కాదు */}

  {0}             {/* ⚠️ "0" print అవుతుంది! — {arr.length && <X/>} bug కి కారణం */}
  {NaN}           {/* ⚠️ "NaN" print అవుతుంది */}
  {{ a: 1 }}      {/* ❌ Error: Objects are not valid as a React child */}
</div>
```

Code – JSX transform ని స్వయంగా చూడటం

```
// ఈ మూడూ ఒకటే:
const a = <div id="x">hi</div>;
const b = React.createElement("div", { id: "x" }, "hi");
const c = { type: "div", props: { id: "x", children: "hi" }, key: null, ref: null };

console.log(a.type);          // "div"
console.log(a.props.id);      // "x"
console.log(typeof a);        // "object" — DOM node కాదు!

// Component అయితే type = function reference
const d = <MyComp n={1} />;
// → React.createElement(MyComp, { n: 1 })
// → { type: MyComp (function), props: { n: 1 } }
```

> **ఇందుకే component పేరు capital letter తో ఉండాలి!** `<button />` → `createElement("button")` (string = HTML tag). `<Button />` → `createElement(Button)` (variable = మన component). Lowercase వాడితే React దాన్ని unknown HTML tag అనుకుంటుంది.

`dangerouslySetInnerHTML` – ఎందుకు "dangerous"

React by default అన్ని strings ని **escape** చేస్తుంది → XSS నుండి రక్షణ:

```
const evil = '<img src=x onerror="alert(document.cookie)" />';
<div>{evil}</div>   // ✅ safe — text గా print అవుతుంది, HTML గా కాదు

<div dangerouslySetInnerHTML={{ __html: evil }} />  // ❌ XSS! script run అవుతుంది
```

అవసరమైతే (CMS content, markdown) `DOMPurify` తో sanitize చేయాలి:

```
import DOMPurify from "dompurify";
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(html) }} />
```

Fragments – ఎందుకు

```
// ❌ ప్రతి component కి wrapper div → "div soup", CSS grid/flex పాడవుతుంది
function Row() { return <div><td>A</td><td>B</td></div>; }  // table లో invalid HTML!
```

```
// ✅ Fragment — DOM లో ఏమీ create కాదు
function Row() { return <><td>A</td><td>B</td></>; }

// key అవసరమైనప్పుడు long form వాడాలి (<> </> కి key ఇవ్వలేం)
{items.map((it) => (
  <React.Fragment key={it.id}>
    <dt>{it.term}</dt>
    <dd>{it.desc}</dd>
  </React.Fragment>
))}
```

Gotchas (సాధారణ తప్పులు)

- **`class` వాడటం** → `className`. (`class` JS reserved word.)
- **`{}` లో statement రాయడం** – `{if (x) ...}` ❌. Expression మాత్రమే – ternary లేదా `&&`, లేదా function లోకి తీసుకెళ్ళాలి.
- **`return` తర్వాత newline** – ASI (automatic semicolon insertion) వల్ల `undefined` return అవుతుంది:

  ```
  return          // ❌ ఇక్కడే `return undefined;` అయిపోతుంది!
    <div>hi</div>;
  return (        // ✅ parenthesis తో
    <div>hi</div>
  );
  ```

- **Component పేరు lowercase** → HTML tag అనుకుంటుంది, silent bug.
- **`{0 && <X/>}`** → screen మీద `0` కనిపిస్తుంది (Topic 8).
- **Object ని child గా render చేయడం** → crash. `JSON.stringify()` వాడాలి debug కి.
- **JSX comments HTML style `<!-- -->` రాయడం** – పని చేయదు, `{/* */}` వాడాలి.

Key Points

- JSX = sugar over `React.createElement` → **plain JS object** (React element / VDOM node).
- React 17+ **automatic runtime** – `import React` అవసరం లేదు.
- Element ≠ DOM node; element చవక, immutable description.
- Capital letter = component, lowercase = HTML tag.
- `false/null/undefined/true` render అవ్వవు; `0` అవుతుంది.
- React strings ని escape చేసి XSS ఆపుతుంది; `dangerouslySetInnerHTML` ఆ రక్షణని తీసేస్తుంది.

Interview దృష్టి

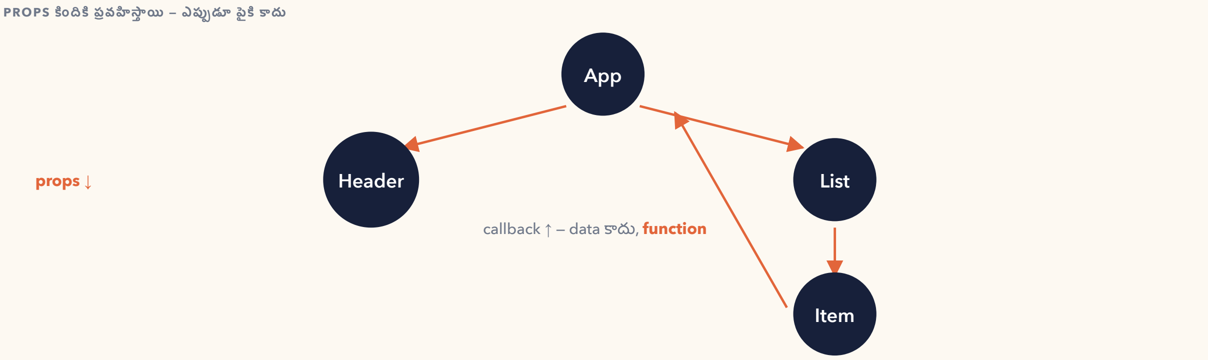
- *"Is JSX mandatory?"* → కాదు, `createElement` రాయొచ్చు; JSX developer experience కోసం.
- *"What does JSX compile to?"* → `jsx()` / `createElement()` calls → element objects tree.
- *"Why key `$$typeof: Symbol(react.element)`?"* → Symbols JSON లో serialize కావు, కాబట్టి server నుండి వచ్చిన JSON ని React element గా నటించనివ్వదు – XSS defense.

- *"Fragment ఎందుకు?"* → extra DOM nodes లేకుండా multiple children return చేయడానికి; table/flex/grid layouts కి కీలకం.

---

4. Components & Props

PROPS · ఒక దిక్కు DATA FLOW



"State ని పైకి ఎత్తడం"
ఇద్దరు siblings ఒకే data వాడాలంటే – వాళ్ళ ఉమ్మడి parent lo state పెట్టడం.

Prop drilling
3+ స్థాయిలు props ని కేవలం దాటిస్తుంటే – Context లేదా composition ఆలోచించాలి.

వివరణ

Component = ఒక JavaScript function, ఇది props తీసుకొని JSX return చేస్తుంది. ఇదే React లో UI యొక్క ప్రాథమిక unit – ఒక LEGO ముక్క.

```
// Function component (2026 లో ప్రామాణికం)
function Greeting({ name, age = 18 }) {   // props destructure + default value
  return <h1>Hello {name}, you are {age}</h1>;
}

// వాడటం
<Greeting name="Surya" age={25} />
```

Props = properties = parent నుండి child కి పంపే data. Props **read-only (immutable)** – child ఎప్పుడూ props ని మార్చకూడదు. ఇదే React యొక్క *unidirectional data flow*.

```
function Bad({ user }) {
  user.name = "changed";   // ❌ props mutate చేయడం – parent కి తెలియకుండా data మారుతుంది
  return <p>{user.name}</p>;
}
```

React components **pure functions** లా ప్రవర్తించాలి: *అదే props ఇస్తే అదే JSX రావాలి*, render సమయంలో బయటి ప్రపంచాన్ని మార్చకూడదు (no DOM writes, no API calls, no mutating outer variables).

Real-life Scenario

> **Component = వంటగదిలో "మిక్సీ".** ఒకసారి కొంటే ఎన్నిసార్లైనా వాడొచ్చు. **Props = మీరు అందులో వేసే పదార్థాలు.** టమాటా వేస్తే టమాటా చట్నీ, కొబ్బరి వేస్తే కొబ్బరి చట్నీ – **మిక్సీ మారదు, ఇచ్చే input మారుతుంది.** మిక్సీ మీ టమాటాని మీ కూరగాయల బుట్టలోకి తిరిగి పెట్టదు (props mutate చేయదు) – కొత్త output ఇస్తుంది.

Code – props యొక్క అన్ని రూపాలు

```
// 1. Basic + destructuring + defaults
function Card({ title, subtitle = "", isActive = false }) {
  return <div className={isActive ? "card active" : "card"}>{title}{subtitle}</div>;
}

// 2. children — component tags మధ్య ఉన్నదంతా `children` prop అవుతుంది
function Panel({ title, children }) {
  return (
    <section className="panel">
      <h2>{title}</h2>
      <div className="body">{children}</div>
    </section>
  );
}
<Panel title="Profile">
  <Avatar />        {/* ఇదంతా children */}
  <p>Bio here</p>
</Panel>

// 3. Function as prop (callback — child నుండి parent కి "పైకి" మాట్లాడటం)
function SearchBar({ onSearch }) {
  return <input onChange={(e) => onSearch(e.target.value)} />;
}
<SearchBar onSearch={(q) => setQuery(q)} />

// 4. Component as prop (slots — layout flexibility)
function Layout({ sidebar, content }) {
  return <div className="grid">{sidebar}{content}</div>;
}
<Layout sidebar={<Nav />} content={<Feed />} />

// 5. Spread props (wrapper components కి ఉపయోగం — కానీ జాగ్రత్త)
function Button({ variant = "primary", ...rest }) {
  return <button className={`btn ${variant}`} {...rest} />;  // onClick, disabled,
type… అన్నీ pass
}
<Button variant="danger" onClick={del} disabled type="submit">Delete</Button>
```

Composition vs Inheritance – React యొక్క ముఖ్య తీర్పు

React లో **inheritance ఎప్పుడూ వాడకూడదు.** Facebook వేల components లో ఒక్కసారి కూడా component inheritance అవసరం పడలేదని React docs చెప్తాయి. బదులు **composition** (children / props గా components పంపడం):

```
// ❌ Inheritance (OOP అలవాటుతో చేసే తప్పు)
class FancyButton extends Button {}   // React లో ఇది anti-pattern

// ✅ Composition — specialization ని props/children తో సాధించడం
function Dialog({ title, body, footer }) { /* ... */ }
function ConfirmDialog(props) {
  return <Dialog {...props} footer={<><Cancel /><Confirm /></>} />;
}
```

Prop Drilling – సమస్య మరియు పరిష్కారాలు

```
// ❌ Prop drilling — user ని 4 levels కిందికి పంపడం; మధ్యలో ఉన్నవాళ్ళకి అది అవసరం లేదు
<App user={user}>
  <Layout user={user}>
    <Sidebar user={user}>
      <Profile user={user} />   // ఇక్కడ మాత్రమే కావాలి!
```

పరిష్కారాలు (ఈ క్రమంలో ఆలోచించాలి):

పరిష్కారం	ఎప్పుడు
Composition (children)	మొదటి ఎంపిక – చాలా సందర్భాల్లో drilling ని పూర్తిగా తొలగిస్తుంది
Context	theme, auth user, locale – అరుదుగా మారే global data
State library (Zustand/Redux)	complex, తరచుగా మారే global state
Server state lib (TanStack Query)	server data – ఇది "global state" కానే కాదు

```
// ✅ Composition తో drilling తొలగింపు — Layout కి user గురించి తెలియనవసరం లేదు
<Layout sidebar={<Sidebar><Profile user={user} /></Sidebar>} />
```

Props validation – PropTypes vs TypeScript

```
// పాత విధానం — PropTypes (runtime check, React 19 లో core నుండి తీసేశారు)
import PropTypes from "prop-types";
Card.propTypes = { title: PropTypes.string.isRequired, count: PropTypes.number };

// ✅ 2026 ప్రామాణికం — TypeScript (compile-time, zero runtime cost)
type CardProps = {
  title: string;
  count?: number;                    // optional
  variant: "primary" | "danger";     // union — typo చేస్తే compile error
  onSelect: (id: string) => void;
  children?: React.ReactNode;
};
function Card({ title, count = 0, variant, onSelect, children }: CardProps) { /* ...
*/ }
```

Gotchas (సాధారణ తప్పులు)

- **Props ని mutate చేయడం** – `props.items.push(x)` ❌. కొత్త array create చేసి callback ద్వారా parent కి పంపాలి.
- **Component లోపల component define చేయడం** – ప్రతి render కి కొత్త function identity → React దాన్ని కొత్త type అనుకొని **DOM మొత్తం unmount/remount** చేస్తుంది → state పోతుంది, inputs focus కోల్పోతాయి:

```
function Parent() {
  function Child() { return <input />; }   // ❌ ప్రతి render కి కొత్త Child
  return <Child />;
}
```

- `{...props}` **గుడ్డిగా spread చేయడం** – తెలియని props DOM కి వెళ్ళి React warning (`Unknown prop`) ఇస్తుంది; typos silent గా పోతాయి.
- **Boolean prop** `={true}` **రాయడం** – `<Btn disabled />` సరిపోతుంది.
- **Render లోపల side-effect** (`document.title = x` , API call) – impure; StrictMode/concurrent mode లో రెండుసార్లు జరిగి bugs వస్తాయి. అవి `useEffect` లో లేదా event handler లో ఉండాలి.

Key Points

- Component = props → JSX, **pure function** లా ఉండాలి.
- Props **read-only**, data flow **top-down**.
- `children` = ఒక special prop → composition కి ఆధారం.
- **Composition > inheritance** (React లో inheritance వాడొద్దు).
- Prop drilling కి మొదటి పరిష్కారం Context కాదు – **composition**.
- Component ని ఇంకో component లోపల define చేయకూడదు.

Interview దృష్టి

- *"Props vs State?"* → props parent నుండి, immutable; state component లోపల, mutable (setter ద్వారా), మారితే re-render.
- *"Prop drilling ఎలా ఆపుతావు?"* → composition మొదట, తర్వాత context, తర్వాత state library – ఏది ఎందుకో చెప్పగలగాలి.
- *"Why must components be pure?"* → concurrent rendering లో React ఒక render ని ఆపి మళ్ళీ మొదలుపెట్టొచ్చు; impure అయితే duplicate side-effects వస్తాయి.

---

5. Rendering, Virtual DOM & Reconciliation

RENDER VS COMMIT · ఎందుకు COMPONENTS PURE గా ఉండాలి

RENDER యొక్క రెండు దశలు – ఇది తెలిస్తే చాలా clear అవుతుంది

1 · RENDER phase
component function నడుస్తుంది · pure గా ఉండాలి

→

2 · COMMIT phase
DOM కి మార్పులు · refs, effects

Render phase lo చేయకూడనివి

setState నేరుగా (అనంత loop)
DOM ని ముట్టుకోవడం
API calls, subscriptions

ఎందుకు pure గా ఉండాలి

React render ని **ఆపి, పారేసి, మళ్ళీ** నడపొచ్చు
StrictMode dev lo రెండుసార్లు నడిపి ఇది test చేస్తుంది
Side effect ఉంటే – అది రెండుసార్లు జరుగుతుంది

StrictMode double-render ఒక bug కాదు

అది ఒక **పరీక్ష** – "నీ component నిజంగా pure నా?" అని. Production lo ఒకసారే నడుస్తుంది.
Double-render వల్ల bug కనిపిస్తే – ఆ bug నిజంగా ఉంది, StrictMode దాన్ని బయటపెట్టింది.

వివరణ

React లో "render" అంటే **screen మీద గీయడం కాదు** – అది కేవలం *"నీ component function ని React call చేయడం, తిరిగి వచ్చిన element tree ని తీసుకోవడం."* అసలు DOM మార్పు తర్వాతి దశ.

React యొక్క ప్రతి update **మూడు దశలుగా** జరుగుతుంది – ఇది గుర్తుపెట్టుకోవడం interview లో పెద్ద score:

దశ	ఏం జరుగుతుంది	ఆపగలమా?
1. Trigger	Initial render లేదా `setState` call	–
2. Render (reconcile)	Component functions run → కొత్త element tree → పాత tree తో diff	✅ ఆపొచ్చు/మళ్ళీ మొదలుపెట్టొచ్చు (concurrent)
3. Commit	తేడాలను real DOM కి apply (`appendChild`, `setAttribute`…), refs set, layout effects run	❌ ఆపలేం (synchronous, atomic)

తర్వాత browser **paint** చేస్తుంది (screen మీద pixels). `useEffect` paint తర్వాత run అవుతుంది; `useLayoutEffect` paint కి ముందు (commit లో).

Virtual DOM అంటే

VDOM = real DOM యొక్క తేలికపాటి JS object copy. Real DOM node లో వందల properties, layout data, event system ఉంటాయి – దాన్ని create/read చేయడం ఖరీదు. VDOM node కేవలం `{ type, props, key }`.

React ప్రతి render కి కొత్త VDOM tree తయారు చేసి, పాత tree తో పోల్చి (**diffing**), **కనీస DOM operations** మాత్రమే చేస్తుంది. ఈ మొత్తం ప్రక్రియనే **reconciliation** అంటారు.

Real-life Scenario

> **VDOM = రాత ప్రతి (draft) మీద edits.** మీరు 50 పేజీల పుస్తకం printed copy లో ఒక అక్షరం మార్చాలంటే – మొత్తం 50 పేజీలు మళ్ళీ print చేయడం (= innerHTML మొత్తం replace) బాగా ఖరీదు. బదులు, draft (VDOM) లో మార్చి, పాత draft తో పోల్చి, "పేజీ 12 లో 3వ లైన్ మాత్రమే మారింది" అని కనిపెట్టి **ఆ ఒక్క పేజీ మాత్రమే** reprint చేస్తారు (= targeted DOM update).

Diffing Algorithm – React యొక్క 2 heuristics

సాధారణ tree-diff algorithm complexity **O(n³)** – 1000 nodes కి బిలియన్ operations. React దాన్ని **O(n)** కి తగ్గించడానికి రెండు అంచనాలు (heuristics) పెట్టుకుంది:

1️⃣ **వేరే `type` = వేరే tree.** Element type మారితే React diff చేయదు – పాత subtree మొత్తం **destroy** చేసి కొత్తది create చేస్తుంది (state మొత్తం పోతుంది).

```
// <div><Counter /></div>  →  <span><Counter /></span>
// div ≠ span → Counter unmount + remount → దాని state RESET!
```

2️⃣ **`key` prop ద్వారా children ని identify చేస్తుంది** (list reorder కోసం).

```
// keys లేకపోతే React position ద్వారా match చేస్తుంది
// [A, B] → [Z, A, B]  : మూడు nodes నీ మార్చేస్తుంది (A→Z, B→A, +B) — వృథా
// keys ఉంటే          : Z ని insert మాత్రమే చేస్తుంది — optimal
```

Code – reconciliation ని ప్రత్యక్షంగా చూడటం

```
function Demo() {
  const [show, setShow] = useState(true);
  return (
    <div>
      <button onClick={() => setShow((s) => !s)}>toggle</button>

      {/* ❌ ఇక్కడ type మారుతోంది (div ↔ section) → Input unmount అవుతుంది → typed text పోతుంది */}
      {show ? <div><Input /></div> : <section><Input /></section>}

      {/* ✅ type ఒకటే → అదే DOM node reuse → state ఉంటుంది */}
      <div className={show ? "a" : "b"}><Input /></div>
    </div>
  );
}
function Input() { return <input placeholder="type here" />; }
```

నియమం: *"State DOM position లో ఉంటుంది, component లో కాదు."* అదే component, అదే position → state ఉంటుంది. Position/type మారితే → state పోతుంది. ఇది React లో అత్యంత తప్పుగా అర్థం చేసుకునే విషయం.

`key` తో ఉద్దేశపూర్వకంగా state reset చేయడం (senior trick)

```
// User మారినప్పుడు form లోని అన్ని state ని reset చేయాలంటే —
// useEffect లో manual reset రాయకుండా, key మార్చితే React తనే unmount+remount చేస్తుంది
<ProfileForm key={userId} userId={userId} />
```

Render ≠ DOM update

```
function Parent() {
  const [n, setN] = useState(0);
  console.log("Parent rendered");          // ప్రతి click కి print అవుతుంది
  return (
    <>
      <button onClick={() => setN(n + 1)}>{n}</button>
      <Child />     {/* Child కూడా re-render అవుతుంది (props మారకపోయినా!) */}
    </>
  );
}
function Child() {
  console.log("Child rendered");           // ఇదీ print అవుతుంది
  return <p>static text</p>;               // కానీ DOM లో ఏమీ మారదు (diff = empty)
}
```

కీలకం: parent re-render అయితే **అన్ని children default గా re-render అవుతాయి** – props మారకపోయినా. కానీ VDOM diff ఖాళీ అయితే **DOM touch అవ్వదు.** అందుకే re-render ≠ నెమ్మది. నెమ్మది అయ్యేది render function ఖరీదు ఎక్కువైనప్పుడు మాత్రమే – అప్పుడే `React.memo` (Topic 17).

VDOM ఉన్న vs లేని frameworks

	VDOM (React, Vue 2)	No VDOM (Svelte, Solid, Angular Signals)
విధానం	ప్రతి update కి tree diff	Compile/signal time లోనే ఏ DOM node మారాలో తెలుసు
Runtime cost	diff overhead ఉంది	దాదాపు లేదు
Mental model	"మొత్తం మళ్ళీ describe చేయి" – సులభం	fine-grained subscriptions
Bundle	పెద్దది	చిన్నది

> **Honest interview answer:** "VDOM makes React *fast enough*, not fastest. It buys us a simple mental model – re-describe the whole UI – while keeping DOM writes minimal. Fine-grained reactive systems like Solid/Svelte are faster at runtime; that's why React added the compiler."

Gotchas (సాధారణ తప్పులు)

- **"VDOM ఎప్పుడూ DOM కంటే వేగం"** – తప్పు. Direct targeted DOM update ఎప్పుడూ వేగం. VDOM = developer experience కోసం చేసిన trade-off.

- **Re-render = performance problem అనుకోవడం** – కాదు. DOM commit ఖరీదు, render function కాదు (అది ఖరీదైనది అయితే తప్పు). ముందు Profiler తో measure చేయాలి.
- **Conditional rendering లో type మార్చడం** వల్ల అనుకోకుండా state reset.
- **Array index ని key గా వాడటం** (Topic 9 లో deep) – reorder/delete లో తప్పు DOM reuse.
- `key` **ని props లా వాడొచ్చు అనుకోవడం** – `props.key` access చేయలేం, అది React కి మాత్రమే.

Key Points

- Update = **trigger** → **render** → **commit** (+ browser paint).
- Render = component function call; DOM change కాదు.
- VDOM = చవకైన JS object tree; diff → minimal DOM ops.
- Heuristics: **వేరే type = tree పారేయ్**, **key ద్వారా children match**.
- **State position లో ఉంటుంది** – type/position మారితే state పోతుంది; `key` మార్చి ఉద్దేశపూర్వకంగా reset చేయొచ్చు.
- Parent re-render → children re-render (default), కానీ DOM మారకపోవచ్చు.

Interview దృష్టి

- *"Explain reconciliation."* → కొత్త element tree vs current fiber tree diff, heuristics, effects list, commit.
- *"Why keys?"* → children ని identity ద్వారా match చేయడానికి; లేకపోతే index-based match → wrong reuse, state bugs.
- *"Does re-render mean DOM update?"* → కాదు; diff ఖాళీ అయితే DOM touch అవ్వదు.
- *"How would you reset a child's state?"* → `key` మార్చడం.

---

6. State & useState

USESTATE · BATCHING మరియు FUNCTIONAL UPDATES

useState – ఎందుకు batching ఉంది

Batching లేకపోతే	React 18 batching
setA(1) → re-render setB(2) → re-render setC(3) → re-render ఒక handler lo 3 renders!	setA, setB, setC – అన్నీ queue lo handler ముగిశాక **ఒకే** re-render Promise, timeout lo కూడా (automatic)

FUNCTIONAL UPDATE ఎప్పుడు తప్పనిసరి

count + 1 (stale)	c => c + 1
setCount(count + 1); setCount(count + 1); → ఫలితం +1 మాత్రమే. count ఆ render lo స్థిరం.	setCount(c => c+1); setCount(c => c+1); → +2. React తాజా విలువని ఇస్తుంది.

మానసిక నమూనా

ప్రతి render ఒక **ఫోటో** – ఆ render lo state ఒక స్థిరమైన విలువ, అది మారదు.
కొత్త విలువ కావాలంటే కొత్త render కావాలి. అందుకే closure lo పాత విలువ కనిపిస్తుంది.

వివరణ

State = component యొక్క జ్ఞాపకశక్తి (memory). Render లు మధ్య నిలిచి ఉండే, మారితే re-render trigger చేసే data.

```
const [count, setCount] = useState(0);
//      ↑ current value  ↑ setter    ↑ initial value (మొదటి render లో మాత్రమే
వాడబడుతుంది)
```

సాధారణ variable ఎందుకు పని చేయదు?

```
function Broken() {
  let count = 0;                          // ప్రతి render కి మళ్ళీ 0 అవుతుంది
  return <button onClick={() => { count++; console.log(count); }}>{count}</button>;
  // console లో 1,2,3 కనిపిస్తుంది కానీ UI ఎప్పటికీ 0 — ఎందుకంటే (1) re-render trigger కాలేదు
  // (2) re-render అయినా count మళ్ళీ 0 అవుతుంది
}
```

State రెండు పనులు చేస్తుంది: **(1) విలువను renders మధ్య నిలుపుతుంది, (2) మారితే React కి "మళ్ళీ render చేయి" అని చెప్తుంది.**

Real-life Scenario

> **State = హోటల్ లో మీ table యొక్క order slip.** Waiter (React) ప్రతిసారి మీ table దగ్గరికి వచ్చినప్పుడు, ఆ slip చూసి ఏం serve చేయాలో తెలుసుకుంటాడు. Slip లో మార్పు రాస్తే (setState), వంటగదికి కబురు వెళ్ళి కొత్త plate వస్తుంది (re-render). Waiter మీ slip ని గుర్తుపెట్టుకోడు – table number (component position) తో దాన్ని map చేసి బయట ఉంచుతాడు (React internal fiber లో state నిల్వ). మీరు table మారితే (position/key మారితే) కొత్త slip – పాత order పోతుంది.

State ఎక్కడ నిల్వ ఉంటుంది – internals

State component function లో ఉండదు (function ప్రతిసారి కొత్తగా run అవుతుంది). React ప్రతి component instance కి ఒక **fiber node** ఉంచుతుంది; ఆ fiber లో hooks యొక్క **linked list** ఉంటుంది:

```
fiber.memoizedState → hook1 { state: 0, queue, next } → hook2 { ... } → hook3 { ... }
→ null
                        (useState)                          (useEffect)     (useRef)
```

ప్రతి render లో React ఈ list ని **క్రమంలో** చదువుతుంది. అందుకే **hooks ని condition/loop లో call చేయకూడదు** – క్రమం మారితే state తప్పు hook కి వెళ్తుంది (Topic 11).

Code – setState యొక్క రెండు రూపాలు (అత్యంత ముఖ్యం)

```
function Counter() {
  const [count, setCount] = useState(0);

  // ❌ Direct value — ఒకే render లో మూడుసార్లు call చేసినా +1 మాత్రమే!
```

```
  const wrong = () => {
    setCount(count + 1);   // count = 0 → 1
    setCount(count + 1);   // count ఇంకా 0 (closure లో పాత value) → 1
    setCount(count + 1);   // → 1
  }; // ఫలితం: 1

  // ✅ Functional updater — queue లో వరుసగా apply అవుతాయి
  const right = () => {
    setCount((c) => c + 1);   // 0 → 1
    setCount((c) => c + 1);   // 1 → 2
    setCount((c) => c + 1);   // 2 → 3
  }; // ఫలితం: 3

  return <button onClick={right}>{count}</button>;
}
```

నియమం: *కొత్త value పాత value మీద ఆధారపడితే ఎప్పుడూ functional updater `setX(prev => ...)` వాడాలి.*

State ఒక "snapshot" – stale closure

```
function Alert() {
  const [n, setN] = useState(0);
  const handle = () => {
    setN(n + 1);
    console.log(n);              // ⚠️ ఇంకా పాత value (0) — state variable ఆ render కి
"frozen"
    setTimeout(() => alert(n), 3000);  // ⚠️ 3 సెకన్ల తర్వాత కూడా పాత value చూపిస్తుంది!
  };
  return <button onClick={handle}>{n}</button>;
}
```

ఎందుకు: ప్రతి render ఒక **snapshot** – ఆ render లోని `n` ఒక const, ఎప్పటికీ మారదు. `setN` కొత్త render ని schedule చేస్తుంది, ప్రస్తుత render లోని variable ని మార్చదు. ఇదే **stale closure** – React లో అత్యంత సాధారణ bug source. (Deep: `JavaScript_Telugu.md` – closures.)

Batching – React 18 లో ఏం మారింది

```
const onClick = () => {
  setA(1);
  setB(2);
  setC(3);
};
// React 17: event handlers లో batch అవుతుంది → 1 re-render ✅
//           కానీ setTimeout/promise/native event లో batch కాదు → 3 re-renders ❌

// React 18+ (createRoot తో): **Automatic batching** — ఎక్కడైనా batch అవుతుంది
setTimeout(() => { setA(1); setB(2); }, 0);      // → 1 re-render ✅
fetch(...).then(() => { setA(1); setB(2); });    // → 1 re-render ✅

// Batching వద్దు అనుకుంటే (అరుదు):
import { flushSync } from "react-dom";
```

```
flushSync(() => setA(1));   // వెంటనే re-render + DOM update
setB(2);
```

Objects & Arrays — immutability తప్పనిసరి

React state ని `Object.is` తో compare చేస్తుంది. Object ని mutate చేస్తే reference మారదు → **React కి మార్పు తెలియదు → re-render కాదు.**

```
const [user, setUser] = useState({ name: "Surya", address: { city: "HYD" } });

user.name = "Ravi"; setUser(user);          // ❌ ఏమీ జరగదు (same reference)
setUser({ ...user, name: "Ravi" });         // ✅ కొత్త object

// Nested — ప్రతి level కాపీ చేయాలి
setUser({ ...user, address: { ...user.address, city: "BLR" } });   // ✅

// Arrays — mutating methods వద్దు
const [todos, setTodos] = useState([]);
todos.push(x); setTodos(todos);                        // ❌
setTodos([...todos, x]);                               // ✅ add
setTodos(todos.filter((t) => t.id !== id));            // ✅ remove
setTodos(todos.map((t) => t.id === id ? { ...t, done: true } : t));  // ✅ update
setTodos([...todos].sort((a, b) => a.n - b.n));        // ✅ sort (copy మొదట!)
setTodos(todos.toSorted((a, b) => a.n - b.n));         // ✅ ES2023 non-mutating
```

Deep nesting నరకంగా ఉంటే — `immer` వాడండి (Redux Toolkit లో ఇది built-in):

```
import { useImmer } from "use-immer";
const [user, updateUser] = useImmer({ address: { city: "HYD" } });
updateUser((draft) => { draft.address.city = "BLR"; });   // mutate రాస్తాం, immer
immutable చేస్తుంది
```

Lazy initial state

```
// ❌ ప్రతి render కి expensive function run అవుతుంది (result మొదటిసారి మాత్రమే వాడినా!)
const [data, setData] = useState(parseHugeJSON(raw));

// ✅ Lazy initializer — function pass చేస్తే React దాన్ని మొదటి render లో మాత్రమే call
చేస్తుంది
const [data, setData] = useState(() => parseHugeJSON(raw));
const [id] = useState(() => crypto.randomUUID());
```

State design principles (senior-level)

1. **Group related state** — `x, y` బదులు `{ x, y }` (కలిసి మారితే).
2. **Contradictions నివారించండి** — `isLoading + isError + isSuccess` బదులు `status: 'idle'|'loading'|'success'|'error'` (impossible states impossible).
3. **Redundant state వద్దు** — `fullName` ని state లో పెట్టొద్దు, render లోనే `first + last` compute చేయండి (Topic 13).
4. **Duplication వద్దు** — `items` + `selectedItem` (పూర్తి object) బదులు `items` + `selectedId`.

5. **Deep nesting తగ్గించండి** – flat/normalized structure (`{ byId, allIds }`).

```
// ❌ impossible states సాధ్యం (loading + error ఒకేసారి?)
const [isLoading, setLoading] = useState(false);
const [error, setError] = useState(null);
const [data, setData] = useState(null);

// ✅ state machine style
const [state, setState] = useState({ status: "idle" });
// { status: 'loading' } | { status: 'success', data } | { status: 'error', error }
```

Gotchas (సాధారణ తప్పులు)

- **setState synchronous అనుకోవడం** – కాదు; తర్వాతి లైన్లో పాత value ఉంటుంది.
- **State mutate చేయడం** → re-render కాదు (silent bug).
- **Loop లో `setCount(count+1)`** → ఒక్క increment మాత్రమే. Functional updater వాడాలి.
- **props ని state లోకి copy చేయడం** – `useState(props.value)` → props మారినా state update కాదు (initial value మొదటిసారి మాత్రమే). నిజంగా అవసరమైతే `key` pattern వాడాలి.
- **Derived data ని state లో పెట్టడం** – sync bugs. Render లో compute చేయాలి.
- **Render లోపల setState call చేయడం** → infinite loop (`Too many re-renders`).

Key Points

- `useState` → `[value, setter]`; initial value మొదటి render కి మాత్రమే.
- **State snapshot** – ఆ render కి frozen; setter కొత్త render ని schedule చేస్తుంది.
- పాత value మీద ఆధారపడితే **functional updater**.
- React 18 = **automatic batching** ప్రతిచోటా.
- **Immutability తప్పనిసరి** (`Object.is` comparison).
- Lazy init `useState(() => expensive())`.
- State design: group, no contradictions, no redundancy, no duplication, flat.

Interview దృష్టి

- *"Is setState async?"* → asynchronous scheduling; batched; updater form ద్వారా latest value.
- *"Why does my state show old value?"* → snapshot/closure explanation.
- *"React 17 vs 18 batching?"* → automatic batching everywhere with `createRoot`.
- *"Why immutability?"* → `Object.is` reference check, time-travel debugging, memo comparisons, concurrent safety.

---

7. Events & SyntheticEvent

వివరణ

React లో events HTML లాగే కనిపిస్తాయి కానీ లోపల వేరు:

```
<button onClick={handleClick}>Click</button>       {/* camelCase, function reference
*/}
<!-- HTML: <button onclick="handleClick()">         lowercase, string -->
```

React ఇచ్చే event object **native event కాదు** – అది **SyntheticEvent**, browser events చుట్టూ React వేసిన wrapper. ఎందుకు?

- **Cross-browser consistency** – పాత IE/Safari తేడాలను normalize చేస్తుంది.
- **Performance** – React అన్ని events ని ప్రతి DOM node కి attach చేయదు; **root container కి ఒక్కసారి** attach చేసి **event delegation** వాడుతుంది.
- **React scheduling తో integration** – batching, priority (click = urgent, scroll = less urgent).

Real-life Scenario

> **Event delegation = apartment లో ఒకే security guard.** 200 flats కి 200 guards పెట్టడం (ప్రతి button కి listener) ఖరీదు. బదులు main gate దగ్గర ఒక్క guard (root listener) – ఎవరు వచ్చినా అతనే receive చేసి "ఇది 12వ flat కి" అని పంపుతాడు (`event.target` చూసి సరైన handler ని కనుగొంటాడు).

Root delegation – React 17 లో మారింది (interview favourite)

```
React 16: అన్ని events → document కి attach
React 17+: అన్ని events → root container (createRoot లో ఇచ్చిన DOM node) కి attach
```

ఎందుకు మార్చారు: ఒకే page లో రెండు React versions (gradual migration), లేదా React app ని jQuery/ఇతర app లోపల embed చేసినప్పుడు `document` మీద conflicts వచ్చేవి. Root కి attach చేస్తే ఒక React app ఇంకో దానితో ఢీకొట్టదు.

Code – events యొక్క అన్ని అంశాలు

```
function EventDemo() {
  const [text, setText] = useState("");

  // 1. Event object
  const onClick = (e) => {
    e.preventDefault();      // default behaviour ఆపు (form submit, link navigation)
    e.stopPropagation();     // parent కి bubble కాకుండా ఆపు
    console.log(e.type, e.target, e.currentTarget);
    console.log(e.nativeEvent);   // అసలు browser event కావాలంటే
  };

  // 2. Argument పంపడం – arrow function తో wrap చేయాలి
  const del = (id) => console.log("delete", id);
  // <button onClick={del(id)}>   ❌ render సమయంలోనే call అవుతుంది!
  // <button onClick={() => del(id)}>  ✅

  // 3. Input handling
  const onChange = (e) => setText(e.target.value);
```

```
  // 4. Keyboard
  const onKeyDown = (e) => {
    if (e.key === "Enter" && !e.shiftKey) { e.preventDefault(); submit(); }
    if (e.key === "Escape") close();
    if ((e.metaKey || e.ctrlKey) && e.key === "k") openSearch();   // Cmd/Ctrl+K
  };

  return (
    <form onSubmit={(e) => { e.preventDefault(); submit(); }}>
      <input value={text} onChange={onChange} onKeyDown={onKeyDown} />
      <button type="submit" onClick={onClick}>Save</button>
    </form>
  );
}
```

`target` vs `currentTarget` (తరచూ అడుగుతారు)

	అర్థం
`e.target`	**ఎక్కడ** event పుట్టిందో ఆ element (deep child కావచ్చు)
`e.currentTarget`	**ఏ element మీద** handler attach అయిందో అది

```
<div onClick={(e) => {
  e.target        // నొక్కిన <span> కావచ్చు
  e.currentTarget // ఎప్పుడూ ఈ <div>
}}>
  <span>click me</span>
</div>
```

Event bubbling, capturing & delegation in React

```
// Bubbling (default): child → parent
<div onClick={() => console.log("parent")}>
  <button onClick={() => console.log("child")}>x</button>
</div>
// output: "child", ఆపై "parent"

// Capturing: parent → child (Capture suffix)
<div onClickCapture={() => console.log("parent capture")}>

// React లో delegation pattern — 1000 rows కి 1000 handlers వద్దు
<ul onClick={(e) => {
  const li = e.target.closest("li[data-id]");
  if (li) select(li.dataset.id);
}}>
  {items.map((it) => <li key={it.id} data-id={it.id}>{it.name}</li>)}
</ul>
```

Event pooling – React 16 లో ఉండేది, 17 లో తీసేశారు

```
// React 16 లో ఇది fail అయ్యేది (event recycle అయ్యేది):
const onClick = (e) => {
  setTimeout(() => console.log(e.target), 0);  // ❌ 16: null! (pooled)
  // పరిష్కారం: e.persist();
};
// React 17+ లో pooling తీసేశారు → ఇది సరిగా పని చేస్తుంది ✅
```

> **Interview:** "React removed event pooling in v17 – `e.persist()` is no longer needed." ఇది చెప్తే మీరు versions ని follow చేస్తున్నారని తెలుస్తుంది.

Portals లో events (surprise!)

Portal DOM లో వేరే చోట render అయినా, **React tree లో ఉన్న parent కే event bubble అవుతుంది** – ఎందుకంటే delegation React tree ఆధారంగా పని చేస్తుంది, DOM tree కాదు. (Topic 32.)

Gotchas (సాధారణ తప్పులు)

- `onClick={handler()}` – render లోనే call అవుతుంది (infinite loop కూడా కావచ్చు). `onClick={() => handler()}` వాడాలి.
- **`onclick` (lowercase)** – React లో పని చేయదు (JSX camelCase).
- **`return false` తో default ఆపడం** – HTML/jQuery లో పని చేసేది; React లో `e.preventDefault()` తప్పనిసరి.
- **Form లో `<button>` కి `type` ఇవ్వకపోవడం** – default `type="submit"` → అనుకోని page reload/submit. Non-submit buttons కి `type="button"` తప్పనిసరి.
- **`stopPropagation()` గుడ్డిగా వాడటం** – outside-click handlers, analytics tracking పాడవుతాయి.
- **Native listener (`addEventListener`) + React handler కలిపి వాడినప్పుడు order confusion** – native listeners React delegation కంటే ముందు fire కావచ్చు.
- **`onChange` semantics** – HTML లో `change` blur తర్వాత fire అవుతుంది; React లో ఇది `input` event లా **ప్రతి keystroke కి** fire అవుతుంది.

Key Points

- React events = **SyntheticEvent** wrapper (cross-browser, batched, scheduled).
- **Root container delegation** (React 17+; అంతకుముందు `document`).
- `e.preventDefault()` / `e.stopPropagation()` ; `return false` పని చేయదు.
- `target` = origin, `currentTarget` = handler owner.
- Event **pooling తీసేశారు** React 17 లో.
- `onChange` = ప్రతి keystroke (native `input` లా).

Interview దృష్టి

- *"Why SyntheticEvent?"* → normalization + delegation + integration with React's scheduler.
- *"React 17 event change?"* → document → root container attachment; multi-version coexistence.

- *"How to handle 10k row clicks?"* → single parent handler + `data-*` + `closest()`.

---

8. Conditional Rendering

వివరణ

JSX లో `if` statement రాయలేం (అది expression స్థానం). కాబట్టి conditions ని expressions గా రాయాలి.

```
function Status({ user, items, error, loading }) {
  // 1️⃣ Early return — అత్యంత చదవగలిగేది (ఇదే మొదటి ఎంపిక)
  if (loading) return <Spinner />;
  if (error) return <ErrorBox error={error} />;
  if (!user) return <Login />;

  return (
    <div>
      {/* 2️⃣ Ternary — రెండు వైపులా UI ఉంటే */}
      {user.isAdmin ? <AdminPanel /> : <UserPanel />}

      {/* 3️⃣ && — ఒక వైపు మాత్రమే ఉంటే */}
      {items.length > 0 && <List items={items} />}

      {/* 4️⃣ ?? / optional chaining */}
      <p>{user.bio ?? "No bio yet"}</p>
      <p>{user.address?.city}</p>

      {/* 5️⃣ null return — ఏమీ render చేయకూడదంటే */}
      {items.length === 0 ? null : <Footer />}
    </div>
  );
}
```

Real-life Scenario

> **Conditional rendering = restaurant menu board.** "Chicken అయిపోయింది" అంటే ఆ item board మీద కనిపించదు (`&&`). "Veg / Non-veg" toggle అయితే రెండు వేరే boards (`ternary`). Restaurant మూసి ఉంటే మొత్తం board బదులు "Closed" board (`early return`).

⚠️ `&&` యొక్క నెంబర్ ఉచ్చు (React లో టాప్ bug)

```
const items = [];
{items.length && <List />}      // ❌ screen మీద "0" కనిపిస్తుంది!
// ఎందుకు: 0 && X → 0 (falsy కానీ number). React 0 ని render చేస్తుంది!

// ✅ పరిష్కారాలు
{items.length > 0 && <List />}
```

```
{!!items.length && <List />}
{items.length ? <List /> : null}
```

అదే విధంగా `{count && ...}` (count=0), `{str && ...}` (str="" → ఏమీ కనిపించదు, ఇది OK), `{NaN && ...}` (NaN print అవుతుంది).

Patterns – పెద్ద conditions ని ఎలా చక్కబెట్టాలి

```
// ❌ nested ternary నరకం
{a ? <A /> : b ? <B /> : c ? <C /> : <D />}

// ✅ 1. Object map (lookup table)
const VIEWS = { idle: Idle, loading: Spinner, error: ErrorBox, success: Result };
const View = VIEWS[status] ?? Idle;
return <View {...props} />;

// ✅ 2. Switch in a function
function renderBody(status) {
  switch (status) {
    case "loading": return <Spinner />;
    case "error":   return <ErrorBox />;
    case "empty":   return <Empty />;
    default:        return <List />;
  }
}

// ✅ 3. IIFE inside JSX (అరుదుగా, కానీ చెల్లుతుంది)
{(() => { if (a) return <A />; return <B />; })()}
```

Toggle vs mount/unmount – CSS hide చేయాలా, unmount చేయాలా?

```
{open && <HeavyModal />}                    // unmount — state పోతుంది, DOM తగ్గుతుంది,
animation కష్టం
<HeavyModal style={{ display: open ? "block" : "none" }} />  // mounted — state
ఉంటుంది, DOM ఖరీదు
```

ఎంపిక	ఎప్పుడు
Unmount (`&&`)	ఖరీదైన component, state reset కావాలి, DOM తక్కువ ఉండాలి
CSS hide	తరచూ toggle అవుతుంది, state/scroll position నిలవాలి, exit animation కావాలి

Gotchas (సాధారణ తప్పులు)

- `{0 && <X/>}` → "0" కనిపిస్తుంది (పైన).
- **Nested ternaries** – 2 కంటే ఎక్కువ అయితే refactor చేయండి.
- **Conditional lo hooks** – `if (x) { useState() }` ❌ Rules of Hooks ఉల్లంఘన (Topic 11).
- **Loading/empty/error states మర్చిపోవడం** – production లో `data.map is not a function` crashes. ప్రతి async UI కి **4 states**: loading, error, empty, success.
- **Conditional లో element `type` మార్చడం** → అనుకోని state reset (Topic 5).

Key Points

- `if` రాయలేం → early return, ternary, `&&`, lookup map.
- `&&` తో **numbers జాగ్రత్త** – `> 0` లేదా `!!` వాడాలి.
- ప్రతి async UI కి **loading / error / empty / success** నాలుగూ handle చేయాలి.
- Unmount vs CSS-hide – state persistence & cost ఆధారంగా ఎంచుకోవాలి.

Interview దృష్టి

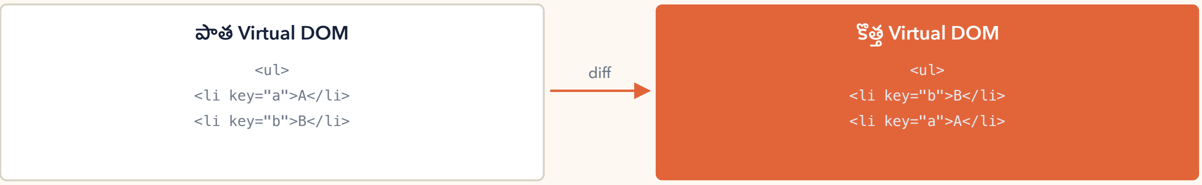
- "`{list.length && <X/>}` *లో bug ఏమిటి?"* → 0 renders; classic screening question.
- *"How do you handle loading/error UI at scale?"* → status state machine + shared `<AsyncBoundary>` (Suspense + ErrorBoundary).

---

9. Lists & Keys (Deep)

RECONCILIATION · KEYS ఎందుకు అంత ముఖ్యం

RECONCILIATION – React ఏం మారిందో ఎలా కనుక్కుంటుంది



కీలకమైన రెండు అంచనాలు (heuristics)

1 · వేరే **type** ఉన్న elements వేరే trees – పాతదాన్ని పూర్తిగా పడేసి కొత్తది కట్టడం.
2 · **key** ఒకటే అయితే అదే element – దాన్ని కదిలించడం, మళ్ళీ కట్టడం కాదు.
ఈ రెండింటి వల్ల $O(n^3)$ diff సమస్య $O(n)$ అవుతుంది.

index ని key గా వాడితే ఏమవుతుంది

List మధ్యలో ఒకటి తీసేస్తే – అన్ని indices జరుగుతాయి.
React "key 2 ఇప్పుడు వేరే data" అనుకుని **DOM ని తప్పుగా reuse** చేస్తుంది.
ఫలితం: input lo టైప్ చేసినది తప్పు row కి జారడం. ఇది అత్యంత సాధారణమైన React bug.

వివరణ

Arrays ని UI గా మార్చడానికి `map()` వాడతాం, ప్రతి item కి **stable unique** `key` ఇవ్వాలి.

```
{users.map((u) => <UserCard key={u.id} user={u} />)}
```

`key` **ఎందుకు:** React కి "ఈ element అదే element" అని identify చేయడానికి. Key లేకపోతే React **position (index)** ద్వారా match చేస్తుంది → list మారినప్పుడు తప్పు DOM node ని reuse చేస్తుంది.

Real-life Scenario

> **Key = student roll number.** Class లో పిల్లలు కూర్చున్న వరుస (index) ప్రతిరోజూ మారొచ్చు. Teacher "3వ bench లో కూర్చున్నవాడు" అని గుర్తుపెడితే (index key), ఒక్కరు absent అయితే మొత్తం records తప్పు అవుతాయి – 3వ bench లో ఇప్పుడు వేరే వ్యక్తి! Roll number (stable id) వాడితే, ఎవరు ఎక్కడ కూర్చున్నా records సరిగ్గా ఉంటాయి.

Index-as-key bug – ప్రత్యక్ష demo

```
function BuggyList() {
  const [items, setItems] = useState([
    { id: "a", label: "Apple" },
    { id: "b", label: "Banana" },
    { id: "c", label: "Cherry" },
  ]);

  return (
    <>
      <button onClick={() => setItems(items.slice(1))}>Remove first</button>
      {items.map((item, i) => (
        // ❌ index key: మొదటిది తీసేస్తే – Banana కి index 0 వస్తుంది
        // React అనుకుంటుంది "index 0 అలాగే ఉంది, label మాత్రమే మారింది"
        // → Apple row యొక్క <input> లో టైప్ చేసిన text ఇప్పుడు Banana row లో కనిపిస్తుంది!
        <li key={i}>
          {item.label} <input placeholder="note" />
        </li>
      ))}
    </>
  );
}
// ✅ పరిష్కారం: key={item.id}
```

Index key వల్ల వచ్చే మూడు రకాల bugs: (1) input values తప్పు rows లోకి జారడం, (2) component state (checkbox, expanded) తప్పు item కి అతుక్కోవడం, (3) unnecessary DOM updates → performance.

Index key ఎప్పుడు OK?

మూడూ నిజమైతే మాత్రమే: **(1)** list ఎప్పుడూ reorder/filter/sort కాదు, **(2)** items add/remove చివర తప్ప జరగదు, **(3)** items లో local state లేదు. Static footer links లాంటివి – OK.

Key యొక్క నియమాలు

```
{items.map((it) => <Row key={it.id} />)}          // ✅ stable, unique (siblings మధ్య)
{items.map((it) => <Row key={Math.random()} />)}  // ❌ ప్రతి render కి కొత్త key → పూర్తి
remount!
{items.map((it) => <Row key={it.name} />)}        // ⚠️ names duplicate కావచ్చు
{items.map((it, i) => <Row key={`${it.type}-${i}`} />)}  // ⚠️ index ఉంటే అదే సమస్య

// Keys siblings మధ్య మాత్రమే unique అయితే చాలు – globally కాదు
```

```
// key React కే — component లో props.key చదవలేం
function Row({ key, id }) { console.log(key); }   // ❌ undefined
```

Backend id లేకపోతే?

```
// Data create అయ్యే క్షణంలోనే id generate చేయాలి — render లో కాదు
const addTodo = (text) =>
  setTodos((prev) => [...prev, { id: crypto.randomUUID(), text }]);
```

Nested lists & Fragments తో keys

```
{groups.map((g) => (
  <React.Fragment key={g.id}>          {/* <> </> కి key ఇవ్వలేం */}
    <h3>{g.title}</h3>
    {g.items.map((it) => <Item key={it.id} item={it} />)}   {/* లోపలి list కి దాని own
keys */}
  </React.Fragment>
))}
```

పెద్ద lists — performance

```
// 10,000 rows → DOM లో 10,000 nodes → browser ఇబ్బంది
// ✅ Virtualization — screen మీద కనిపించే ~20 rows మాత్రమే render
import { useVirtualizer } from "@tanstack/react-virtual";

function BigList({ rows }) {
  const parentRef = useRef(null);
  const v = useVirtualizer({
    count: rows.length,
    getScrollElement: () => parentRef.current,
    estimateSize: () => 40,       // ప్రతి row యొక్క అంచనా ఎత్తు (px)
    overscan: 5,                  // కనిపించే వాటికి అటూఇటూ కొన్ని extra
  });

  return (
    <div ref={parentRef} style={{ height: 500, overflow: "auto" }}>
      <div style={{ height: v.getTotalSize(), position: "relative" }}>
        {v.getVirtualItems().map((vi) => (
          <div
            key={rows[vi.index].id}
            style={{ position: "absolute", top: 0, left: 0, width: "100%",
                     height: vi.size, transform: `translateY(${vi.start}px)` }}
          >
            {rows[vi.index].name}
          </div>
        ))}
      </div>
    </div>
  );
}
```

Gotchas (సాధారణ తప్పులు)

- **Index as key** (reorder/delete జరిగే lists లో) – top interview + production bug.
- **`Math.random()` / `Date.now()` key** – ప్రతి render కి remount, state + focus పోతాయి, performance నాశనం.
- **Key ని `<li>` లోపల element కి ఇవ్వడం** – key **`map` return చేసే outermost element** మీద ఉండాలి.
- **`forEach` వాడి return మర్చిపోవడం** – `map` వాడాలి, లేదా `return` రాయాలి.
- **Missing key warning ని ignore చేయడం** – silent state bugs.
- **Array ని render లో sort/mutate చేయడం** – `arr.sort()` props ని mutate చేస్తుంది; `[...arr].sort()` వాడాలి.

Key Points

- `key` = **identity**, index కాదు; siblings మధ్య unique + renders మధ్య stable.
- Index key → reorder/delete లో state jumbling bugs.
- `key` ని props గా చదవలేం.
- `key` మార్చడం = **ఉద్దేశపూర్వక remount** (state reset trick).
- 1000+ rows → **virtualization**.

Interview దృష్టి

- *"Why not index as key?"* → identity vs position; concrete input-state bug demo చెప్పగలగాలి.
- *"When is index acceptable?"* → static, append-only, stateless lists.
- *"How to render 100k rows?"* → virtualization + pagination + memoized rows.

---

10. Forms – Controlled vs Uncontrolled

వివరణ

React లో form inputs కి రెండు విధానాలు:

	Controlled	Uncontrolled
Value ఎక్కడ	React state లో	DOM లోనే (browser handle చేస్తుంది)
Read ఎలా	`value` state	`ref.current.value`
Re-render	ప్రతి keystroke కి	లేదు
Validation	instant, per-keystroke	submit లో
ఎప్పుడు	dynamic UI, live validation, conditional fields	simple forms, file inputs, పెద్ద forms perf

```
// Controlled — React "single source of truth"
function Controlled() {
  const [email, setEmail] = useState("");
  return <input value={email} onChange={(e) => setEmail(e.target.value)} />;
}

// Uncontrolled — DOM source of truth
function Uncontrolled() {
  const ref = useRef(null);
  const submit = (e) => { e.preventDefault(); console.log(ref.current.value); };
  return <form onSubmit={submit}><input ref={ref} defaultValue="" /></form>;
}
```

Real-life Scenario

> **Controlled = బ్యాంకులో officer ముందు కూర్చుని form నింపడం.** మీరు రాసే ప్రతి అక్షరాన్ని అతను చూసి వెంటనే "ఆ PAN number తప్పు" అని చెప్తాడు (live validation). **Uncontrolled = form ని ఇంట్లో నింపి, చివర్లో counter లో ఇవ్వడం.** వేగం (ప్రతి అక్షరానికీ ఎవరూ జోక్యం చేసుకోరు), కానీ తప్పులు చివర్లోనే తెలుస్తాయి.

Code — production-grade multi-field form (controlled)

```
function SignupForm() {
  const [values, setValues] = useState({ name: "", email: "", password: "" });
  const [touched, setTouched] = useState({});
  const [submitting, setSubmitting] = useState(false);
  const [serverError, setServerError] = useState(null);

  // ఒకే handler అన్ని fields కి — name attribute ఆధారంగా
  const onChange = (e) => {
    const { name, value, type, checked } = e.target;
    setValues((v) => ({ ...v, [name]: type === "checkbox" ? checked : value }));
  };
  const onBlur = (e) => setTouched((t) => ({ ...t, [e.target.name]: true }));

  // ✅ errors ని state లో పెట్టొద్దు — render లో derive చేయాలి (Topic 13)
  const errors = {
    name: values.name.trim().length < 2 ? "Name too short" : null,
    email: !/^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(values.email) ? "Invalid email" : null,
    password: values.password.length < 8 ? "Min 8 characters" : null,
  };
  const isValid = Object.values(errors).every((e) => e === null);

  const onSubmit = async (e) => {
    e.preventDefault();
    setTouched({ name: true, email: true, password: true });
    if (!isValid) return;
    setSubmitting(true);
    setServerError(null);
    try {
      await api.signup(values);
    } catch (err) {
      setServerError(err.message);
```

```
    } finally {
      setSubmitting(false);      // ⚠️ finally లో — success/fail రెండింటిలోనూ reset
కావాలి
    }
  };

  return (
    <form onSubmit={onSubmit} noValidate>
      <input name="name" value={values.name} onChange={onChange} onBlur={onBlur}
             aria-invalid={!!(touched.name && errors.name)} />
      {touched.name && errors.name && <span role="alert">{errors.name}</span>}

      <input name="email" type="email" value={values.email} onChange={onChange}
onBlur={onBlur} />
      {touched.email && errors.email && <span role="alert">{errors.email}</span>}

      <input name="password" type="password" value={values.password} onChange=
{onChange} onBlur={onBlur} />
      {touched.password && errors.password && <span role="alert">{errors.password}
</span>}

      {serverError && <p role="alert">{serverError}</p>}
      <button type="submit" disabled={submitting || !isValid}>
        {submitting ? "Creating…" : "Sign up"}
      </button>
    </form>
  );
}
```

> `touched` **ఎందుకు:** user ఇంకా టైప్ చేయకముందే "Invalid email" చూపిస్తే చిరాకు. Field ని touch చేసిన (blur అయిన) తర్వాతే error చూపించాలి — ఇది UX detail, interview లో మెచ్చుకుంటారు.

అన్ని input రకాలు

```
<input type="text" value={v} onChange={e => set(e.target.value)} />
<input type="checkbox" checked={v} onChange={e => set(e.target.checked)} />     {/*
checked! */}
<input type="radio" name="g" value="a" checked={v === "a"} onChange={e =>
set(e.target.value)} />
<select value={v} onChange={e => set(e.target.value)}>                          {/*
selected కాదు */}
  <option value="a">A</option>
</select>
<select multiple value={arr} onChange={e => set([...e.target.selectedOptions].map(o
=> o.value))} />
<textarea value={v} onChange={e => set(e.target.value)} />                      {/*
children కాదు */}
<input type="file" ref={fileRef} />        {/* ⚠️ ఎప్పుడూ uncontrolled — value set
చేయలేం (security) */}
<input type="number" value={v} onChange={e => set(e.target.valueAsNumber)} />   {/*
string కాదు */}
```

Controlled input యొక్క ప్రసిద్ధ warning

```
Warning: A component is changing an uncontrolled input to be controlled.
```

కారణం: initial value `undefined` (ఉదా. `value={user?.name}` – user ఇంకా load కాలేదు), తర్వాత string వచ్చింది. **పరిష్కారం:** ఎప్పుడూ `value={user?.name ?? ""}` – `undefined` ఎప్పుడూ ఇవ్వొద్దు.

React 19 – Form Actions (కొత్త, పెద్ద మార్పు)

React 19 లో `<form action={fn}>` native గా support చేస్తుంది; loading/error state ని React నిర్వహిస్తుంది:

```
import { useActionState } from "react";
import { useFormStatus } from "react-dom";

function SubmitButton() {
  const { pending } = useFormStatus();     // ⚠️ form లోపలి child లోనే పని చేస్తుంది
  return <button disabled={pending}>{pending ? "Saving…" : "Save"}</button>;
}

function ProfileForm() {
  // (prevState, formData) => newState ; React pending/error ని నిర్వహిస్తుంది
  const [state, formAction, isPending] = useActionState(
    async (prev, formData) => {
      const name = formData.get("name");
      try {
        await api.updateName(name);
        return { ok: true, message: "Saved" };
      } catch (e) {
        return { ok: false, message: e.message };
      }
    },
    { ok: true, message: "" }
  );

  return (
    <form action={formAction}>
      <input name="name" defaultValue="Surya" />
      <SubmitButton />
      {state.message && <p>{state.message}</p>}
    </form>
  );
}
```

లాభాలు: manual `isSubmitting` state అవసరం లేదు, JS load అవ్వకముందే form పని చేస్తుంది (progressive enhancement), Server Actions తో నేరుగా కలుస్తుంది.

పెద్ద forms – React Hook Form (production standard)

```
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";
```

```
const schema = z.object({
  email: z.string().email("Invalid email"),
  age: z.coerce.number().min(18, "Must be 18+"),
});

function Fast() {
  const { register, handleSubmit, formState: { errors, isSubmitting } } =
    useForm({ resolver: zodResolver(schema) });

  return (
    <form onSubmit={handleSubmit(async (data) => await api.save(data))}>
      <input {...register("email")} />
      {errors.email && <span>{errors.email.message}</span>}
      <input {...register("age")} />
      <button disabled={isSubmitting}>Save</button>
    </form>
  );
}
```

RHF ఎందుకు వేగం: inputs ని uncontrolled గా (refs తో) ఉంచి, subscription model వాడుతుంది → **ప్రతి keystroke కి form మొత్తం re-render కాదు.** 50 fields form లో ఇది భారీ తేడా.

Gotchas (సాధారణ తప్పులు)

- `value` **ఇచ్చి** `onChange` **ఇవ్వకపోవడం** → input read-only అవుతుంది + warning. Read-only కావాలంటే `readOnly` prop.
- `undefined` **initial value** → controlled/uncontrolled warning.
- `checked` **బదులు** `value` **(checkbox)**, `selected` **(option)** వాడటం.
- `e.preventDefault()` **మర్చిపోవడం** → page reload, state పోతుంది.
- **ప్రతి keystroke కి API call** → debounce అవసరం (Topic 43).
- **Password/token ని localStorage లో పెట్టడం** – XSS లో దొంగిలించబడుతుంది (Topic 54).
- **Client validation మాత్రమే నమ్మడం** – server లో మళ్ళీ validate చేయాలి, ఎప్పుడూ.
- **Submit button** `type` **ఇవ్వకపోవడం** → అనుకోని submits.

Key Points

- **Controlled** = state source of truth (default ఎంపిక); **uncontrolled** = ref/DOM.
- File input ఎప్పుడూ uncontrolled.
- Validation errors ని **derive** చేయాలి, state లో duplicate చేయకూడదు.
- `touched` తో error timing UX.
- React 19: `<form action>` + `useActionState` + `useFormStatus` + `useOptimistic`.
- పెద్ద forms → **React Hook Form + Zod** (re-render తగ్గించడానికి).

Interview దృష్టి

- *"Controlled vs uncontrolled – ఏది ఎప్పుడు?"* → live validation/dynamic UI vs perf/simple submit; file inputs always uncontrolled.
- *"50-field form నెమ్మదిగా ఉంది, ఏం చేస్తావు?"* → RHF/uncontrolled, field-level subscription, split into steps, memoize field components.

- *"React 19 Actions ఏం మార్చాయి?"* → built-in pending/error state, progressive enhancement, server action integration.

Part 2 – Hooks Deep Dive

11. Hooks Rules & ఎందుకు (internals)

వివరణ

Hook = "use" తో మొదలయ్యే special function, ఇది React యొక్క internal features (state, lifecycle, context) ని function components కి అందిస్తుంది.

Hooks ఎందుకు వచ్చాయి (2019, React 16.8) – class components లోని 3 సమస్యలు:

1. **Stateful logic reuse కష్టం** – HOC/render props వల్ల "wrapper hell" (`<A><B><C><D>…`).
2. **సంబంధిత logic విడిపోవడం** – ఒకే feature యొక్క code `componentDidMount` + `componentDidUpdate` + `componentWillUnmount` లో ముక్కలుగా చెల్లాచెదురు.
3. `this` **గందరగోళం** – binding, arrow functions, `this` context bugs.

Rules of Hooks (రెండే నియమాలు)

1️⃣ **Hooks ని top level లో మాత్రమే call చేయాలి** – conditions, loops, nested functions, `return` తర్వాత కాదు.

```
// ❌ అన్నీ తప్పు
if (isLoggedIn) { const [x, setX] = useState(0); }
for (let i = 0; i < n; i++) { useEffect(...); }
if (!user) return null;  const [y] = useState(0);   // early return తర్వాత hook

// ✅ సరైనది — condition ని hook లోపలికి తీసుకెళ్ళాలి
const [x, setX] = useState(0);
useEffect(() => { if (isLoggedIn) doThing(); }, [isLoggedIn]);
```

2️⃣ **Hooks ని React functions నుండి మాత్రమే call చేయాలి** – component లేదా custom hook (`useXxx`) నుండి; సాధారణ JS function/class నుండి కాదు.

Real-life Scenario

> **Hooks = టిఫిన్ box లో అరలు (compartments).** React మీ component కి ఒక box ఇస్తుంది. మొదటి render లో మీరు "1వ అరలో అన్నం, 2వ అరలో కూర, 3వ అరలో పెరుగు" అని పెట్టారు. React అరలకి **పేర్లు పెట్టుకోదు – వరుస సంఖ్య మాత్రమే గుర్తుపెట్టుకుంటుంది.** తర్వాతి render లో మీరు "ఈ రోజు కూర వద్దు" అని 2వ అరని skip చేస్తే – పెరుగు 2వ అరలోకి వచ్చేస్తుంది, React "ఇది కూర" అనుకుంటుంది. అందుకే **ప్రతి render లోనూ hooks అదే వరుసలో, అదే సంఖ్యలో ఉండాలి.**

లోపల ఏం జరుగుతుంది – hooks linked list

React hooks ని **పేరు ద్వారా కాదు, call order ద్వారా** గుర్తిస్తుంది. ప్రతి fiber లో ఒక linked list:

```
// React యొక్క (బాగా సరళీకరించిన) implementation — ఇది అర్థమైతే hooks మొత్తం అర్థమైనట్లే
let hooks = [];          // నిజంగా ఇది fiber.memoizedState linked list
let currentIndex = 0;

function useState(initial) {
  const i = currentIndex++;                         // ⚠️ ఇక్కడే order మీద ఆధారపడుతోంది
  hooks[i] = hooks[i] ?? initial;                   // మొదటి render అయితే initial
  const setState = (next) => {
    hooks[i] = typeof next === "function" ? next(hooks[i]) : next;
    rerender();                                     // schedule re-render
  };
  return [hooks[i], setState];
}

function render(Component) {
  currentIndex = 0;                                 // ప్రతి render కి index reset
  return Component();
}
```

ఇప్పుడు condition లో hook పెడితే ఏమవుతుందో స్పష్టం:

```
// Render 1 (loggedIn = true):   useState(name) → index 0,  useState(cart) → index 1
// Render 2 (loggedIn = false):  useState(cart) → index 0  ❌ cart కి name యొక్క value
వచ్చింది!
if (loggedIn) { const [name] = useState("x"); }
const [cart] = useState([]);
```

React ఈ mismatch ని పట్టుకుంటే error ఇస్తుంది: *"Rendered fewer hooks than expected"*.

ESLint plugin – తప్పనిసరి

```
// eslint.config.js
import reactHooks from "eslint-plugin-react-hooks";
export default [{
  plugins: { "react-hooks": reactHooks },
  rules: {
    "react-hooks/rules-of-hooks": "error",         // నియమాలు enforce
    "react-hooks/exhaustive-deps": "warn",         // dependency array తప్పులు
పట్టుకుంటుంది
  },
}];
```

> `exhaustive-deps` **warning ని ఎప్పుడూ** `// eslint-disable-line` **తో ఆపొద్దు.** అది "నీ code లో bug ఉంది" అని చెప్తోంది. సరైన పరిష్కారం: dependency ని add చేయడం, function ని effect లోపలికి తరలించడం, `useCallback`, లేదా functional updater వాడటం. (Topic 12.)

అన్ని built-in hooks – ఒక్క చూపులో

Hook	పని	Version
`useState`	local state	16.8
`useReducer`	complex state transitions	16.8
`useEffect`	side effects (paint తర్వాత)	16.8
`useLayoutEffect`	DOM measurement (paint కి ముందు)	16.8
`useInsertionEffect`	CSS-in-JS style injection	18
`useRef`	mutable box / DOM reference	16.8
`useImperativeHandle`	parent కి imperative API ఇవ్వడం	16.8
`useContext`	context value చదవడం	16.8
`useMemo`	value memoization	16.8
`useCallback`	function memoization	16.8
`useDebugValue`	DevTools label (custom hooks)	16.8
`useId`	SSR-safe unique id	18
`useTransition`	non-urgent update గా mark చేయడం	18
`useDeferredValue`	value యొక్క deferred copy	18
`useSyncExternalStore`	external store subscription (tearing-safe)	18
`use`	promise/context ని read చేయడం (conditional OK!)	19
`useActionState`	form action state	19
`useFormStatus`	parent form యొక్క pending state (`react-dom`)	19
`useOptimistic`	optimistic UI updates	19

Gotchas (సాధారణ తప్పులు)

- **Condition/loop/early-return తర్వాత hook** → "Rendered fewer hooks than expected" crash.
- **Custom hook పేరు `use` తో మొదలుపెట్టకపోవడం** → linter check చేయలేదు, bugs silent.
- **Event handler లోపల hook call చేయడం** – ❌ (handlers render సమయంలో run అవ్వవు).
- **Class component లో hooks** – పని చేయవు.
- **రెండు React copies** (npm link, duplicate `node_modules`) → *"Invalid hook call"* – dedupe చేయాలి.
- **`use()` కూడా conditional OK అని అన్ని hooks కి వర్తిస్తుంది అనుకోవడం** – `use` మాత్రమే exception (React 19).

Key Points

- Hooks = top level లో మాత్రమే, React functions నుండి మాత్రమే.
- React hooks ని **order ద్వారా** track చేస్తుంది (fiber లో linked list).
- Hooks class సమస్యలను (reuse, split logic, `this`) పరిష్కరించాయి.
- `eslint-plugin-react-hooks` తప్పనిసరి; `exhaustive-deps` ని ఆపొద్దు.
- React 19 `use()` – conditional గా call చేయగల ఏకైక hook.

Interview దృష్టి

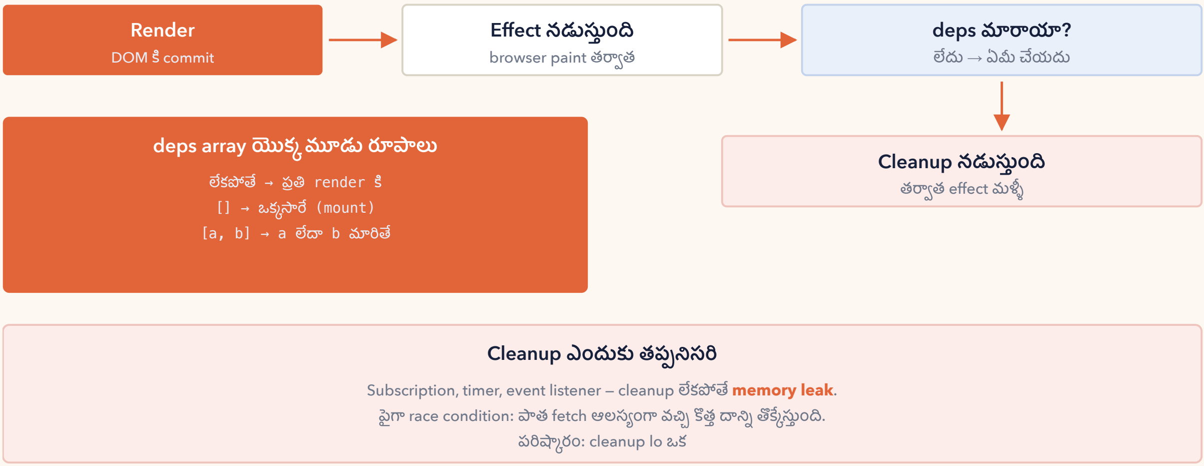
- *"Why can't hooks be conditional?"* → linked list + index-based lookup; order mismatch = wrong state.
- *"Why were hooks introduced?"* → logic reuse without wrapper hell, colocated lifecycle logic, no `this`.
- *"Invalid hook call ఎందుకు వస్తుంది?"* → mismatched React versions, two React copies, hook outside component.

---

12. useEffect Deep

USEEFFECT · ఎప్పుడు నడుస్తుంది, CLEANUP ఎందుకు

useEffect యొక్క జీవితచక్రం



`cancelled` flag లేదా AbortController.

వివరణ

`useEffect` = React బయటి ప్రపంచంతో (external system) మీ component ని sync చేయడానికి. ఇది "lifecycle method" కాదు – official docs దీన్ని *"synchronization"* అని పిలుస్తాయి. ఇది React 18/19 లో అత్యంత తప్పుగా వాడే hook.

```
useEffect(() => {
  // side effect — subscription, timer, DOM API, analytics, non-React library
  return () => { /* cleanup — unmount లో లేదా తర్వాతి effect కి ముందు */ };
}, [deps]);
```

External systems అంటే: browser APIs (`setInterval` , `addEventListener` , `IntersectionObserver`), WebSocket/network subscription, third-party libraries (charts, maps), `document.title` , analytics.

Real-life Scenario

> **useEffect = ఇంటికి కేబుల్ connection.** మీరు ఇంట్లోకి దిగినప్పుడు (mount) connection తీసుకుంటారు. **ఇల్లు మారితే (deps మారితే), పాత ఇంట్లో connection cut చేసి (cleanup) కొత్త ఇంట్లో కొత్తది తీసుకుంటారు.** ఇల్లు ఖాళీ చేసేటప్పుడు (unmount) కూడా cut చేస్తారు. Cleanup మర్చిపోతే – మీరు ఇక్కడ లేకపోయినా bill వస్తూనే ఉంటుంది (**memory leak**).

Dependency array – మూడు రూపాలు

```
useEffect(() => { ... });              // ప్రతి render తర్వాత (అరుదుగా కావాలి – సాధారణంగా bug)
useEffect(() => { ... }, []);          // mount లో ఒకసారి మాత్రమే (+ unmount లో cleanup)
useEffect(() => { ... }, [a, b]);      // a లేదా b `Object.is` ప్రకారం మారితే
```

Code – cleanup ఎందుకు తప్పనిసరి

```
// ❌ Memory leak – component unmount అయినా timer నడుస్తూనే ఉంటుంది
useEffect(() => {
  setInterval(() => setTick((t) => t + 1), 1000);
}, []);

// ✅ Cleanup తో
useEffect(() => {
  const id = setInterval(() => setTick((t) => t + 1), 1000);
  return () => clearInterval(id);
}, []);

// ✅ Event listener
useEffect(() => {
  const onResize = () => setWidth(window.innerWidth);
  window.addEventListener("resize", onResize);
  return () => window.removeEventListener("resize", onResize);
}, []);

// ✅ Subscription
useEffect(() => {
  const sub = socket.subscribe(roomId, onMessage);
  return () => sub.unsubscribe();
}, [roomId]);          // roomId మారితే: పాత room unsubscribe → కొత్త room subscribe
```

Cleanup ఎప్పుడు run అవుతుంది: (1) unmount లో, **(2) ప్రతి తర్వాతి effect run కి ముందు** – ఇది చాలామందికి తెలియదు. `[roomId]` మారితే క్రమం: పాత cleanup → కొత్త effect.

Race condition – async data fetching లో అతిపెద్ద bug

```
// ❌ Race condition — userId త్వరగా 1 → 2 మారితే, 1 యొక్క నెమ్మది response
//    తర్వాత వచ్చి user 2 data ని overwrite చేస్తుంది!
useEffect(() => {
  fetch(`/api/users/${userId}`).then((r) => r.json()).then(setUser);
}, [userId]);

// ✅ పరిష్కారం 1 — ignore flag
useEffect(() => {
  let ignore = false;
  (async () => {
    const res = await fetch(`/api/users/${userId}`);
    const data = await res.json();
    if (!ignore) setUser(data);          // stale response ని పారేయ్
  })();
  return () => { ignore = true; };
}, [userId]);

// ✅ పరిష్కారం 2 — AbortController (network request నే cancel చేస్తుంది — మెరుగు)
useEffect(() => {
  const ctrl = new AbortController();
  (async () => {
    try {
      const res = await fetch(`/api/users/${userId}`, { signal: ctrl.signal });
      if (!res.ok) throw new Error(res.statusText);
      setUser(await res.json());
    } catch (e) {
      if (e.name !== "AbortError") setError(e);    // abort ని error గా చూపొద్దు
    }
  })();
  return () => ctrl.abort();
}, [userId]);
```

> **నిజ ప్రపంచంలో:** ఈ code అంతా స్వయంగా రాయకుండా **TanStack Query** వాడతారు – caching, dedup, retry, stale-while-revalidate అన్నీ ఉచితం (Topic 39). Interview లో "manual గా ఎలా చేస్తావు" అని అడిగితే పైన code, "production లో ఏం వాడతావు" అంటే React Query.

StrictMode double-invoke (React 18+) – గందరగోళానికి కారణం

Development లో `<StrictMode>` ప్రతి effect ని **mount** → **cleanup** → **mount మళ్ళీ** run చేస్తుంది.

```
useEffect(() => {
  console.log("mount");
  return () => console.log("cleanup");
}, []);
// Dev + StrictMode: "mount", "cleanup", "mount"   ← 2 సార్లు!
// Production: "mount" ఒక్కసారి
```

ఎందుకు: cleanup సరిగా రాశారా అని పరీక్షించడానికి. Effect ని రెండుసార్లు run చేసినా app సరిగా పని చేయాలి (idempotent). **ఇది bug కాదు, feature.** `StrictMode` తీసేయడం తప్పు పరిష్కారం – cleanup

రాయడం సరైనది. (Future లో React state ని preserve చేసి components ని remount చేసే feature కి ఇది సన్నద్ధత.)

Effect execution timing – పూర్తి క్రమం

```
1. State update trigger
2. Render phase (component functions run)
3. Cleanup of previous useLayoutEffect
4. Commit phase — DOM mutations
5. useLayoutEffect (synchronous — browser paint కి ముందు)   ⚠️ ఇక్కడ ఆలస్యం = jank
6. Browser paint (screen మీద కనిపిస్తుంది)
7. Cleanup of previous useEffect
8. useEffect (asynchronous — paint తర్వాత)
```

`useEffect` లో object/array deps – infinite loop trap

```
// ❌ Infinite loop — ప్రతి render కి కొత్త object identity
const options = { limit: 10 };
useEffect(() => { fetchData(options); }, [options]);   // options ప్రతిసారి "కొత్తది"

// ✅ 1. Primitive deps వాడు
useEffect(() => { fetchData({ limit }); }, [limit]);

// ✅ 2. Object ని effect లోపలికి తరలించు
useEffect(() => { const options = { limit: 10 }; fetchData(options); }, []);

// ✅ 3. useMemo (అవసరమైతే)
const options = useMemo(() => ({ limit }), [limit]);

// ❌ ఇదే infinite loop — effect లో setState + dep గా అదే state
const [data, setData] = useState([]);
useEffect(() => { setData([...data, x]); }, [data]);   // set → render → effect → set
→ ∞
```

Custom hook – production `useFetch`

```
function useFetch(url, options) {
  const [state, setState] = useState({ status: "idle", data: null, error: null });

  useEffect(() => {
    if (!url) return;
    const ctrl = new AbortController();
    setState({ status: "loading", data: null, error: null });

    fetch(url, { ...options, signal: ctrl.signal })
      .then((r) => {
        if (!r.ok) throw new Error(`HTTP ${r.status}`);
        return r.json();
      })
      .then((data) => setState({ status: "success", data, error: null }))
      .catch((error) => {
        if (error.name !== "AbortError")
          setState({ status: "error", data: null, error });
      });
```

```
    return () => ctrl.abort();
    // ⚠️ options object ప్రతి render కి కొత్తది అయితే loop — caller memoize చేయాలి,
    //    లేదా url మాత్రమే dep గా ఉంచి options ని ref లో ఉంచాలి
  }, [url]);   // eslint-disable-line react-hooks/exhaustive-deps

  return state;
}
```

Gotchas (సాధారణ తప్పులు)

- **Cleanup మర్చిపోవడం** → memory leaks, "setState on unmounted component" warnings, duplicate listeners.
- **Deps array మర్చిపోవడం** → ప్రతి render కి effect → API storm.
- `[]` **పెట్టి stale closure** – effect లోపల పాత state/props శాశ్వతంగా.

```
useEffect(() => {
  const id = setInterval(() => console.log(count), 1000);  // ❌ ఎప్పుడూ 0
  return () => clearInterval(id);
}, []);
// ✅ functional updater లేదా ref లేదా count ని dep గా
```

- `async` **నేరుగా effect callback కి** – `useEffect(async () => {})` ❌ (Promise return అవుతుంది, cleanup అనుకుంటుంది). లోపల IIFE వాడాలి.
- **Effect లో derived state calculate చేసి setState** → extra render (Topic 13).
- **StrictMode double-run ని bug అనుకొని StrictMode తీసేయడం.**
- **Effect లో object dep** → infinite loop.

Key Points

- `useEffect` = **external systems తో synchronization**, lifecycle కాదు.
- Cleanup = unmount + **ప్రతి re-run కి ముందు**.
- Async fetching లో **race condition** ni `ignore` flag లేదా `AbortController` తో handle చేయాలి.
- StrictMode dev లో effects ని double-run చేస్తుంది – cleanup correctness test.
- Deps = `Object.is` comparison → objects/functions ప్రతిసారి కొత్తవి.
- `useEffect` **paint తర్వాత** (async), `useLayoutEffect` paint కి ముందు (sync).

Interview దృష్టి

- *"useEffect ని componentDidMount అనొచ్చా?"* → `[]` దగ్గరగా ఉంటుంది కానీ mental model వేరు – synchronization, lifecycle కాదు; StrictMode లో రెండుసార్లు run అవుతుంది.
- *"Effect లో fetch చేసేటప్పుడు race condition ఎలా ఆపుతావు?"* → cleanup + ignore/abort.
- *"Why does my interval log stale state?"* → closure snapshot; updater/ref పరిష్కారం.

13. "You Might Not Need an Effect" (Senior Signal)

వివరణ

Effect ల **అతివాడకం** React codebases లో అతిపెద్ద సమస్య. React docs కి ఒక ప్రత్యేక page ఉంది: *"You Might Not Need an Effect"*. Interview లో ఇది తెలిస్తే **వెంటనే senior signal.**

బంగారు నియమం:

- **Render సమయంలో లెక్కించగలిగేది** → effect + state వద్దు, నేరుగా compute చేయి.
- **User action వల్ల జరిగేది** → **event handler** లో, effect లో కాదు.
- **External system తో sync** → *అప్పుడే* effect.

Real-life Scenario

> **Effect అతివాడకం = ప్రతి పనికీ courier వాడటం.** పక్క గదిలో ఉన్న వ్యక్తికి పుస్తకం ఇవ్వాలంటే – చేతికి ఇవ్వొచ్చు (render లో compute). కానీ కొందరు courier book చేసి (setState in effect), అది reach అయ్యేదాకా వేచి (extra render), ఆపై ఇస్తారు. ఖర్చు ఎక్కువ, ఆలస్యం ఎక్కువ, tracking bugs ఎక్కువ.

Anti-pattern 1 – derived state

```
// ❌ రెండు renders, sync bugs, extra state
const [fullName, setFullName] = useState("");
useEffect(() => { setFullName(first + " " + last); }, [first, last]);

// ✅ render లోనే derive
const fullName = first + " " + last;

// ❌ filtered list ని state లో
const [visible, setVisible] = useState([]);
useEffect(() => { setVisible(todos.filter((t) => !t.done)); }, [todos]);

// ✅
const visible = todos.filter((t) => !t.done);
// నిజంగా ఖరీదైతే (10k items, heavy compute) అప్పుడు మాత్రమే:
const visible = useMemo(() => todos.filter((t) => !t.done), [todos]);
```

Anti-pattern 2 – event logic ని effect లో పెట్టడం

```
// ❌ POST ఎప్పుడు జరుగుతుందో స్పష్టత లేదు; StrictMode లో రెండుసార్లు!
useEffect(() => {
  if (submitted) { api.post("/orders", cart); }
}, [submitted]);

// ✅ event handler లో – "ఇది user click వల్ల జరిగింది" అని స్పష్టం
const onSubmit = async () => { await api.post("/orders", cart); };
```

నియమం: *"ఇది ఎందుకు జరిగింది?"* అని అడగండి. **User చేసిన దాని వల్ల** → handler. **Component screen మీద కనిపించడం వల్ల** → effect.

Anti-pattern 3 – props మారితే state reset చేయడానికి effect

```
// ❌ extra render + stale UI flash
useEffect(() => { setComment(""); }, [userId]);

// ✅ key తో component ని remount చేయి (React తనే reset చేస్తుంది)
<CommentBox key={userId} userId={userId} />

// ✅ లేదా render సమయంలో adjust (అరుదైన pattern, docs approved)
const [prevId, setPrevId] = useState(userId);
if (prevId !== userId) { setPrevId(userId); setComment(""); }   // render లోనే — effect
కంటే వేగం
```

Anti-pattern 4 – parent కి తెలియజేయడానికి effect

```
// ❌ child state మారిన తర్వాత effect లో parent కి చెప్పడం → double render
useEffect(() => { onChange(isOn); }, [isOn]);

// ✅ ఒకే event లో రెండూ
const toggle = () => { const next = !isOn; setIsOn(next); onChange(next); };
```

Anti-pattern 5 – chained effects (dominoes)

```
// ❌ ప్రతి step ఒక render — 4 renders, debug నరకం
useEffect(() => { if (card) setGoldCount(c => c + 1); }, [card]);
useEffect(() => { if (goldCount > 3) setRound(r => r + 1); }, [goldCount]);
useEffect(() => { if (round > 5) setGameOver(true); }, [round]);

// ✅ ఒకే event handler లో మొత్తం logic (లేదా useReducer)
const onPlayCard = (card) => {
  const nextGold = goldCount + 1;
  const nextRound = nextGold > 3 ? round + 1 : round;
  setGoldCount(nextGold); setRound(nextRound); setGameOver(nextRound > 5);
};
```

Effect నిజంగా అవసరమైన సందర్భాలు

```
useEffect(() => { document.title = title; }, [title]);                   // ✅
browser API
useEffect(() => { const s = chat.connect(room); return () => s.close(); }, [room]);
// ✅ subscription
useEffect(() => { const o = new IntersectionObserver(cb); ... }, []);    // ✅
browser observer
useEffect(() => { analytics.pageView(path); }, [path]);                  // ✅
external logging
useEffect(() => { chartLib.render(ref.current, data); }, [data]);        // ✅ third-
party lib
```

```
useEffect(() => { localStorage.setItem("k", JSON.stringify(v)); }, [v]); // ✅
external store
```

Decision Tree

```
నాకు కావలసిన value…
├ props/state నుండి లెక్కించగలనా?          → render లో compute (అవసరమైతే useMemo)
├ user action వల్ల జరుగుతుందా?              → event handler
├ props మారితే state reset కావాలా?          → key prop లేదా render-time adjust
├ server data కావాలా?                        → TanStack Query / RSC / router loader
└ browser API / subscription / 3rd-party?    → ✅ useEffect (cleanup తో)
```

Gotchas (సాధారణ తప్పులు)

- `useEffect` **= "component load అయినప్పుడు run అయ్యేది"** అనే mental model → అతివాడకం.
- **Derived state ని state లో పెట్టడం** → రెండు source of truth → sync bugs.
- **Data fetching కి effect ని default గా వాడటం** – waterfalls, races, no cache. Query library/loader/RSC మేలు.
- **అన్నిటికీ** `useMemo` – derive చేయడం చవక; profiling లేకుండా memoize చేయకూడదు.

Key Points

- **Render లో compute > effect + state.**
- **Event handler > effect** (user action logic కి).
- Props-change reset → `key`.
- Effect = external systems కోసం మాత్రమే.
- Chained effects = smell → reducer/handler లోకి కలపండి.

Interview దృష్టి

- *"When do you NOT need useEffect?"* → derived state, event logic, prop-based reset, parent notification, chained state – 5 examples చెప్తే బలమైన impression.
- *"Effect లో data fetch చేయడంలో సమస్యలు?"* → races, waterfalls, no dedupe/cache, double-run in StrictMode, no SSR support.

---

14. useLayoutEffect vs useEffect

వివరణ

	`useEffect`	`useLayoutEffect`
ఎప్పుడు	Browser paint **తర్వాత**	DOM mutation తర్వాత, paint **కి ముందు**
Blocking?	కాదు (async)	అవును (synchronous – browser వేచి ఉంటుంది)

	`useEffect`	`useLayoutEffect`
వాడకం	95% cases	DOM measure చేసి వెంటనే adjust చేయాలంటే
SSR	server లో run అవ్వదు	server లో run అవ్వదు + **warning** ఇస్తుంది

Real-life Scenario

> **useEffect = ఫోటో తీసిన తర్వాత edit చేయడం.** అందరూ ఫోటో చూశాక "అరె, జుట్టు సరిచేయాలి" అని edit చేస్తే – వాళ్ళు పాత version ఒక క్షణం చూశారు (**flicker**).
>
> **useLayoutEffect = ఫోటో తీసేముందు జుట్టు సరిచేయడం.** ఎవరూ తప్పు version చూడరు. కానీ అందరూ వేచి ఉండాలి (**blocking**) – ఎక్కువ సేపు తీసుకుంటే అందరికీ చిరాకు (jank).

Code – flicker సమస్య మరియు పరిష్కారం

```
// ❌ useEffect — tooltip మొదట తప్పు స్థానంలో paint అవుతుంది, ఆపై జరుగుతుంది → flicker
కనిపిస్తుంది
function Tooltip({ targetRect, children }) {
  const ref = useRef(null);
  const [top, setTop] = useState(0);

  useEffect(() => {
    const h = ref.current.getBoundingClientRect().height;
    setTop(targetRect.top - h < 0 ? targetRect.bottom : targetRect.top - h);
  }, [targetRect]);

  return <div ref={ref} style={{ position: "absolute", top }}>{children}</div>;
}

// ✅ useLayoutEffect — measure + reposition అంతా paint కి ముందే → flicker లేదు
useLayoutEffect(() => {
  const h = ref.current.getBoundingClientRect().height;
  setTop(targetRect.top - h < 0 ? targetRect.bottom : targetRect.top - h);
}, [targetRect]);
```

ఎప్పుడు `useLayoutEffect` వాడాలి (చాలా అరుదు)

- Element size/position measure చేసి **అదే frame లో** style మార్చాలంటే (tooltip, popover, dropdown positioning).
- Scroll position ని restore చేయాలంటే (chat window ని కిందికి scroll చేయడం).
- Third-party DOM library ని paint కి ముందు initialize చేయాలంటే.
- Animation యొక్క starting state ని set చేయాలంటే (FLIP animations).

```
// Chat — కొత్త message వచ్చినప్పుడు bottom కి scroll (flicker లేకుండా)
useLayoutEffect(() => {
  const el = listRef.current;
  el.scrollTop = el.scrollHeight;
}, [messages]);
```

SSR warning & పరిష్కారం

`useLayoutEffect` server లో run అవ్వదు (DOM లేదు) → Next.js లో warning వస్తుంది. Isomorphic hook:

```
import { useEffect, useLayoutEffect } from "react";
// Browser లో layout effect, server లో plain effect (no-op)
export const useIsomorphicLayoutEffect =
  typeof window !== "undefined" ? useLayoutEffect : useEffect;
```

Gotchas (సాధారణ తప్పులు)

- **Default గా `useLayoutEffect` వాడటం** – main thread block అవుతుంది; ఎక్కువ పని పెడితే visible jank.
- **`useLayoutEffect` లో data fetching** – అర్థరహితం, paint ని అనవసరంగా ఆపుతుంది.
- **SSR warning ని ignore చేయడం** – hydration mismatch గందరగోళం.
- **`useEffect` ఉన్న flicker ని CSS తో పరిష్కరించవచ్చని మర్చిపోవడం** – తరచుగా CSS (`transform`, `visibility`) మెరుగైన పరిష్కారం.

Key Points

- `useEffect` = **paint తర్వాత, non-blocking** – default ఎంపిక.
- `useLayoutEffect` = **paint కి ముందు, blocking** – DOM measurement + immediate adjustment కి మాత్రమే.
- Flicker కనిపిస్తే → layout effect గురించి ఆలోచించండి.
- SSR లో layout effect run అవ్వదు → isomorphic wrapper.

Interview దృష్టి

- *"రెండింటి తేడా?"* → timing (paint ముందు/తర్వాత) + blocking nature + SSR behaviour.
- *"ఎప్పుడు layout effect అవసరం?"* → tooltip positioning, scroll restore, FLIP animation.

---

15. useRef – DOM & Mutable Box

వివరణ

`useRef` రెండు పూర్తిగా వేరే పనులకి వాడతాం:

1. **DOM element ని పట్టుకోవడం** (focus, scroll, measure, media control).
2. **Renders మధ్య నిలిచే mutable value** – మారినా **re-render trigger చేయదు**.

```
const ref = useRef(initialValue);
// ref = { current: initialValue }  ← ఒకే object, renders అన్నిటిలో అదే identity
```

State vs Ref – ముఖ్యమైన comparison:

	`useState`	`useRef`
మారితే re-render	✅ అవును	❌ కాదు
Value	immutable snapshot (per render)	mutable `.current`
Render లో చదవొచ్చా	✅	⚠️ చదవకూడదు (render impure అవుతుంది)
వాడకం	UI లో కనిపించే data	timers, previous values, DOM, instance-like data

Real-life Scenario

> **State = whiteboard** – ఏం రాసినా అందరూ చూస్తారు, రాయగానే meeting మళ్ళీ మొదలవుతుంది (re-render). **Ref = మీ జేబులో నోట్‌బుక్** – ఏం రాసినా మీకు మాత్రమే తెలుసు, meeting కి అంతరాయం లేదు. కానీ దాన్ని presentation లో చూపిస్తే (render లో చదివితే), అది update అయిందని ఎవరికీ తెలియదు – పాత info చూపిస్తారు.

Code – DOM refs

```
function SearchBox() {
  const inputRef = useRef(null);

  // mount లో focus
  useEffect(() => { inputRef.current?.focus(); }, []);

  const scrollToBottom = () => listRef.current?.scrollIntoView({ behavior: "smooth"
});
  const measure = () => {
    const { width, height, top } = boxRef.current.getBoundingClientRect();
  };
  const play = () => videoRef.current.play();     // imperative media API

  return <input ref={inputRef} />;
}
```

Mutable box – re-render లేకుండా value నిలుపుకోవడం

```
function Timer() {
  const [time, setTime] = useState(0);
  const intervalRef = useRef(null);       // ✅ ref – id మారితే re-render అవసరం లేదు

  const start = () => {
    if (intervalRef.current) return;       // double-start ఆపు
    intervalRef.current = setInterval(() => setTime((t) => t + 1), 1000);
  };
  const stop = () => { clearInterval(intervalRef.current); intervalRef.current =
null; };

  useEffect(() => () => clearInterval(intervalRef.current), []);   // unmount cleanup
```

```
  return <><span>{time}</span><button onClick={start}>▶</button><button onClick=
{stop}>⏸</button></>;
}
```

ఇతర సాధారణ ref patterns

```
// 1. Previous value ని గుర్తుపెట్టుకోవడం
function usePrevious(value) {
  const ref = useRef(undefined);
  useEffect(() => { ref.current = value; });   // render తర్వాత update
  return ref.current;                          // ఈ render లో పాత value
}

// 2. Mount అయ్యిందా అని తెలుసుకోవడం (మొదటి render లో effect skip చేయడానికి)
function useUpdateEffect(fn, deps) {
  const mounted = useRef(false);
  useEffect(() => {
    if (!mounted.current) { mounted.current = true; return; }
    return fn();
  }, deps);
}

// 3. Latest callback ని stale closure లేకుండా పట్టుకోవడం (event-handler pattern)
function useEventCallback(fn) {
  const ref = useRef(fn);
  useLayoutEffect(() => { ref.current = fn; });
  return useCallback((...args) => ref.current(...args), []);   // identity ఎప్పుడూ
మారదు
}

// 4. Render count (debugging)
const renders = useRef(0);
renders.current++;    // ⚠️ render లో ref mutate — debugging కి మాత్రమే OK
```

forwardRef (React 18 వరకు) vs ref-as-prop (React 19)

```
// React ≤18 — refs props గా pass కావు, forwardRef అవసరం
const Input = forwardRef(function Input(props, ref) {
  return <input ref={ref} {...props} />;
});
<Input ref={myRef} />

// ✅ React 19 — ref ఒక సాధారణ prop! forwardRef అవసరం లేదు
function Input({ ref, ...props }) {
  return <input ref={ref} {...props} />;
}
<Input ref={myRef} />
```

useImperativeHandle – parent కి పరిమిత API ఇవ్వడం

```
// Parent కి మొత్తం DOM node ఇవ్వకుండా, కొన్ని methods మాత్రమే బయటపెట్టడం
const VideoPlayer = forwardRef(function VideoPlayer(props, ref) {
  const videoRef = useRef(null);
```

```
  useImperativeHandle(ref, () => ({
    play: () => videoRef.current.play(),
    pause: () => videoRef.current.pause(),
    seek: (t) => { videoRef.current.currentTime = t; },
    // getBoundingClientRect, innerHTML వంటివి బయటపెట్టలేదు — encapsulation
  }), []);

  return <video ref={videoRef} src={props.src} />;
});

// వాడకం
const playerRef = useRef(null);
<VideoPlayer ref={playerRef} src="/a.mp4" />
<button onClick={() => playerRef.current.play()}>Play</button>
```

> **జాగ్రత్త:** ఇది escape hatch. మొదట props/state తో declarative గా చేయలేమా అని ఆలోచించాలి. Media players, focus management, animations – ఇక్కడ న్యాయమైనది.

Callback refs – ref గా function

```
// Ref attach/detach అయినప్పుడు code run చేయాలంటే
const measuredRef = useCallback((node) => {
  if (node !== null) setHeight(node.getBoundingClientRect().height);
}, []);
<div ref={measuredRef}>content</div>

// React 19 — callback ref నుండి cleanup function return చేయొచ్చు
<div ref={(node) => {
  const ro = new ResizeObserver(() => {/* ... */});
  ro.observe(node);
  return () => ro.disconnect();     // ✅ 19 లో కొత్తది
}} />

// Dynamic list of refs — Map వాడాలి
const itemRefs = useRef(new Map());
{items.map((it) => (
  <li key={it.id} ref={(node) => {
    node ? itemRefs.current.set(it.id, node) : itemRefs.current.delete(it.id);
  }}>{it.name}</li>
))}
```

Gotchas (సాధారణ తప్పులు)

- **Render లో `ref.current` చదవడం/రాయడం** – impure; concurrent rendering లో inconsistent. Effects/handlers లో మాత్రమే.
- **UI లో కనిపించాల్సిన data ని ref లో పెట్టడం** – update అయినా screen మారదు.
- **Ref initial value `null` కాబట్టి `ref.current.focus()`** → crash. `ref.current?.focus()` వాడాలి.
- **Effect లో ref ని `[ref.current]` dep గా ఇవ్వడం** – ref mutation re-render trigger చేయదు కాబట్టి పనికిరాదు; callback ref వాడాలి.

- `forwardRef` **లేకుండా custom component కి ref ఇవ్వడం** (React ≤18) → `undefined` + warning.
- **DOM ని ref తో నేరుగా మార్చడం** (`ref.current.style.display = 'none'`) – React తర్వాతి render లో overwrite చేస్తుంది. State వాడాలి.

Key Points

- Ref = `{ current }` – **stable object, re-render trigger చేయదు**.
- రెండు వాడకాలు: **DOM access** + **mutable instance data**.
- Render లో ref చదవొద్దు/రాయొద్దు.
- React 19: **ref ఒక సాధారణ prop**, `forwardRef` అవసరం లేదు; callback ref cleanup support.
- `useImperativeHandle` = పరిమిత imperative API (escape hatch).

Interview దృష్టి

- *"useRef vs useState?"* → re-render trigger, mutability, render purity.
- *"Ref ఎప్పుడు state కంటే మేలు?"* → timer ids, previous values, DOM handles, latest-callback pattern.
- *"forwardRef ఎందుకు ఉండేది, ఇప్పుడు?"* → React 19 లో ref-as-prop వచ్చింది.

---

16. useMemo & useCallback

వివరణ

రెండూ **memoization** hooks – renders మధ్య ఏదో ఒకటి cache చేస్తాయి.

```
const value = useMemo(() => computeExpensive(a, b), [a, b]);   // *value* ని cache
చేస్తుంది
const fn    = useCallback((x) => doThing(x, a), [a]);          // *function identity*
ని cache చేస్తుంది

// useCallback(fn, deps) === useMemo(() => fn, deps)   ← ఇవి సమానం
```

ఎందుకు అవసరం: JavaScript లో ప్రతి render కి objects/arrays/functions **కొత్త identity** పొందుతాయి:

```
{} === {}                  // false
[] === []                  // false
(() => {}) === (() => {}) // false
```

అంటే `React.memo` child కి prop గా function పంపితే, ప్రతి render కి కొత్త function → memo విఫలం → child మళ్ళీ render.

Real-life Scenario

> **useMemo = టిఫిన్ ముందురోజే వండి fridge లో పెట్టడం.** పదార్థాలు (deps) మారకపోతే మళ్ళీ వండాల్సిన పని లేదు. కానీ **fridge కి కూడా ఖర్చు ఉంది** (memory + comparison) – ప్రతి చిన్న దానికీ (ఒక కప్పు నీళ్ళు) fridge వాడటం అర్థరహితం. అదే `useMemo` ని అన్నిటికీ వాడటం.

మూడు చెల్లుబాటు అయ్యే వాడకాలు మాత్రమే

1️⃣ నిజంగా ఖరీదైన computation

```
// 50,000 items sort + filter — ప్రతి render కి చేయడం waste
const sorted = useMemo(
  () => hugeList.filter((x) => x.active).sort((a, b) => b.score - a.score),
  [hugeList]
);
```

2️⃣ `React.memo` child కి stable props ఇవ్వడం

```
const Child = React.memo(function Child({ onSelect, config }) { /* ... */ });

function Parent() {
  const [n, setN] = useState(0);
  const onSelect = useCallback((id) => console.log(id), []);   // ✅ stable identity
  const config = useMemo(() => ({ theme: "dark" }), []);       // ✅ stable object
  return <><button onClick={() => setN(n+1)}>{n}</button><Child onSelect={onSelect} config={config} /></>;
}
// ఈ మూడూ (memo + useCallback + useMemo) కలిస్తేనే పని చేస్తుంది. ఒకటి లేకపోయినా వ్యర్థం.
```

3️⃣ Hook dependency గా వాడే value

```
const params = useMemo(() => ({ q, page }), [q, page]);
useEffect(() => { search(params); }, [params]);   // లేకపోతే infinite loop
```

ఎప్పుడు వాడకూడదు (సాధారణ అతివాడకం)

```
const total = useMemo(() => a + b, [a, b]);            // ❌ addition memoize చేయడం hook కంటే చవక!
const name = useMemo(() => `${f} ${l}`, [f, l]);       // ❌
const cls = useMemo(() => active ? "on" : "off", [active]);  // ❌
const onClick = useCallback(() => setOpen(true), []);  // ❌ memo చేయని child కి — వ్యర్థం
```

Memoization ఖరీదు: hook slot allocation + deps array create + ప్రతి render కి comparison + memory retention. చిన్న computations కి **memoize చేయడం చేయకపోవడం కంటే నెమ్మది.**

Code – memoization ఎప్పుడు నిజంగా పని చేసిందో measure చేయడం

```
function useRenderCount(label) {
  const n = useRef(0);
  n.current++;
  console.log(`${label} rendered ${n.current}×`);
}
// React DevTools Profiler → "Why did this render?" (Record why each component
rendered ON)
```

> **Profiler లేకుండా optimize చేయడం = చీకట్లో బాణం వేయడం.** ముందు measure, తర్వాత memoize.

React Compiler (React 19) – ఇదంతా అనవసరం చేస్తుంది

React 19 తో వచ్చిన **React Compiler** (గతంలో "React Forget") మీ code ని build time లో analyze చేసి **automatic గా memoization inject** చేస్తుంది:

```
// మీరు రాసేది — memo hooks ఏవీ లేవు
function Product({ items, onPick }) {
  const sorted = items.filter(i => i.ok).sort(byPrice);
  const handle = (id) => onPick(id);
  return <List items={sorted} onPick={handle} />;
}
// Compiler దీన్ని memoized version గా మారుస్తుంది — మీరు useMemo/useCallback రాయనవసరం లేదు
```

```
npm install -D babel-plugin-react-compiler
```

> **Interview గోల్డ్:** "With the React Compiler, manual `useMemo` / `useCallback` become largely unnecessary — but the compiler only works on code that follows the Rules of React (pure components, no mutation during render). That's why purity matters more than ever."

Gotchas (సాధారణ తప్పులు)

- **అన్నిటినీ memoize చేయడం** – code చదవడం కష్టం, memory ఎక్కువ, benefit సున్నా.
- **`useCallback` ఇచ్చి child ని `React.memo` చేయకపోవడం** → పూర్తిగా వ్యర్థం.
- **Deps తప్పుగా ఇవ్వడం** → stale closures (`useCallback` లో పాత state).
- **`useMemo` లో side effect** – `useMemo(() => { fetchData(); }, [])` ❌; ఇది pure computation కోసం మాత్రమే.
- **`useMemo` cache హామీ అనుకోవడం** – React memory ఒత్తిడిలో cache ని discard చేయొచ్చు (semantics ప్రకారం). Correctness కి memo మీద ఆధారపడొద్దు.
- **Children ని memoize చేయకుండా parent memo చేయడం** – `{children}` prop ప్రతిసారి కొత్తది కావచ్చు.

Key Points

- `useMemo` = value cache; `useCallback` = function identity cache.
- **మూడు చెల్లుబాటు కారణాలు:** ఖరీదైన compute, `React.memo` props, hook deps.
- Memoization కి **ఖరీదు ఉంది** – profile చేయకుండా వాడొద్దు.
- `useCallback` + `React.memo` **జంటగా** పని చేస్తాయి.
- **React Compiler** ఈ manual పని ని తీసేస్తుంది (purity తప్పనిసరి).

Interview దృష్టి

- *"useMemo vs useCallback?"* → value vs function; `useCallback(f, d) === useMemo(() => f, d)`.
- *"Why isn't React.memo working?"* → inline object/function/children props → identity మారుతోంది.
- *"Should you memoize everything?"* → కాదు; cost model + profiler-first + compiler.

---

17. React.memo & Re-render Mental Model

RE-RENDER MENTAL MODEL · ఎప్పుడు, ఎందుకు

ఒక component ఎప్పుడు RE-RENDER అవుతుంది

State మారితే	useState / useReducer setter
Props మారితే	parent కొత్త props ఇస్తే
Parent re-render అయితే	props మారకపోయినా! ← ఇదే ఆశ్చర్యం
Context విలువ మారితే	ఆ context వాడే అన్ని consumers

React.memo ఎప్పుడు

Props నిజంగా మారనప్పుడు మాత్రమే ఆపుతుంది
⚠ object/function props ప్రతిసారీ కొత్తవి –
అప్పుడు memo పనికిరాదు. useMemo/useCallback తోడు కావాలి.

ఎప్పుడు వద్దు

Re-render ఎప్పుడూ చెడ్డది కాదు – React వేగం.
Memo కూడా ఒక ఖర్చు (comparison + memory).
ముందు కొలవండి, తర్వాత optimise చేయండి.

సరైన మానసిక నమూనా

Re-render ≠ DOM update. React component function ని మళ్ళీ నడిపి, kotta virtual DOM ని diff చేస్తుంది.
నిజంగా మారినది మాత్రమే DOM కి వెళ్తుంది. అందుకే చాలా re-renders నిజానికి చౌక.

వివరణ

`React.memo(Component)` – ఒక HOC. Props **shallow equal** అయితే re-render ని skip చేసి, పాత render ఫలితాన్ని reuse చేస్తుంది.

```
const Row = React.memo(function Row({ item, onSelect }) {
  return <li onClick={() => onSelect(item.id)}>{item.name}</li>;
});
```

```
// Custom comparison (అరుదు — సాధారణంగా అవసరం లేదు)
const Row2 = React.memo(Comp, (prevProps, nextProps) => {
  return prevProps.item.id === nextProps.item.id;   // true = skip render (⚠️
shouldComponentUpdate కి తలకిందులు!)
});
```

Component ఎందుకు re-render అవుతుంది – పూర్తి జాబితా

1. **State మారింది** (`setState`) – ఆ component + దాని కింద ఉన్నవన్నీ.
2. **Parent re-render అయింది** – props మారకపోయినా (memo లేకపోతే).
3. **Context value మారింది** – ఆ context ని consume చేసే అన్ని components.
4. **Hook (`useSyncExternalStore` మొ.) ద్వారా external store మారింది.**
5. `key` **మారింది** – re-render కాదు, **remount**.

❌ **Props మారడం *స్వయంగా* re-render కి కారణం కాదు!** Parent re-render అవ్వాలి. అందుకే memo parent chain ని ఆపుతుంది.

Real-life Scenario

> **React.memo = office లో "Do Not Disturb" board.** Manager (parent) ఏదో మారిందని అందరి desk కి వెళ్తాడు (re-render). Memo board పెట్టిన వ్యక్తి "నా input files మారాయా?" అని అడుగుతాడు. మారకపోతే "నేను పని ఆపను, పాత output అదే" అంటాడు. కానీ **manager ప్రతిసారి కొత్త pen తెచ్చి ఇస్తే (inline function prop), input మారినట్టే – memo పనికిరాదు.**

Code – memo ఎందుకు విఫలమవుతుంది

```
const Child = React.memo(function Child({ data, onClick, children }) {
  console.log("Child rendered");
  return <div onClick={onClick}>{data.name}{children}</div>;
});

function Parent() {
  const [n, setN] = useState(0);
  return (
    <>
      <button onClick={() => setN(n + 1)}>{n}</button>

      {/* ❌ మూడు కారణాలవల్ల memo విఫలం */}
      <Child data={{ name: "x" }}             // కొత్త object ప్రతి render
             onClick={() => console.log("hi")} // కొత్త function ప్రతి render
      >
        <span>hello</span>                    {/* children కూడా కొత్త element */}
      </Child>
    </>
  );
}

// ✅ సరైనది
function ParentFixed() {
  const [n, setN] = useState(0);
  const data = useMemo(() => ({ name: "x" }), []);
```

```
  const onClick = useCallback(() => console.log("hi"), []);
  const kids = useMemo(() => <span>hello</span>, []);
  return <>
    <button onClick={() => setN(n + 1)}>{n}</button>
    <Child data={data} onClick={onClick}>{kids}</Child>
  </>;
}
```

Memo కంటే మెరుగైన pattern – composition ("children" trick)

Memoization రాయకుండానే re-renders ఆపొచ్చు – state ని **కిందికి తరలించడం** లేదా **children గా పంపడం** ద్వారా:

```
// ❌ theme మారిన ప్రతిసారి ExpensiveTree re-render
function App() {
  const [theme, setTheme] = useState("dark");
  return <div className={theme}><Toggle onClick={setTheme} /><ExpensiveTree /></div>;
}

// ✅ Pattern A: state ని అవసరమైన చోటికి కిందికి తరలించడం
function App() {
  return <><ThemeToggle /><ExpensiveTree /></>;   // state ThemeToggle లోపల
}

// ✅ Pattern B: children గా పంపడం — children elements parent render లో మళ్ళీ create
కావు
function ThemeWrapper({ children }) {
  const [theme, setTheme] = useState("dark");
  return <div className={theme}><Toggle onClick={setTheme} />{children}</div>;
}
function App() {
  return <ThemeWrapper><ExpensiveTree /></ThemeWrapper>;
  // ThemeWrapper re-render అయినా, <ExpensiveTree /> element App లో create అయింది —
  // అదే element object → React "మారలేదు" అని skip చేస్తుంది. memo అవసరం లేదు!
}
```

> **ఇది interview లో చెప్తే బలమైన signal:** "Before reaching for memo, I try lifting state down or passing children – structural fixes beat memoization."

Memo ఎప్పుడు వాడాలి

వాడు	వాడొద్దు
List rows (100+ items)	చిన్న leaf components (`<Label>`)
Heavy render (charts, tables, editors)	ప్రతి render కి props మారే components
Pure presentational, తరచూ మారని props	Parent కంటే తరచూ మారే state ఉన్న components

Gotchas (సాధారణ తప్పులు)

- **Memo = "ఎప్పుడూ re-render కాదు" అనుకోవడం** – state/context మారితే memo ఉన్నా re-render అవుతుంది.
- **Inline props** (object/array/function/JSX) → memo dead.
- **అన్నిటినీ memo చేయడం** – comparison ఖరీదు + memory; benefit ముందు measure చేయాలి.
- `memo` **custom comparator return value తిరగబడటం** – `true` = **skip** (props equal). `shouldComponentUpdate` కి తలకిందులు.
- **Memo ని deep-compare చేయడం** (`JSON.stringify` comparator) – తరచుగా re-render కంటే ఖరీదు.

Key Points

- `React.memo` = props shallow-equal అయితే re-render skip.
- Re-render కారణాలు: own state, parent render, context, external store, key change (remount).
- Memo పని చేయాలంటే **props identity stable** ఉండాలి → `useCallback` / `useMemo` జంటగా.
- **Composition (children / lift state down) > memoization.**
- React Compiler ఈ manual పని ని ఎక్కువగా తీసేస్తుంది.

Interview దృష్టి

- *"Parent re-render అయితే child ఎప్పుడూ re-render అవుతుందా?"* → అవును, memo/same-element-reference లేకపోతే.
- *"Memo పని చేయట్లేదు, ఎందుకు?"* → identity-changing props; DevTools Profiler తో నిరూపించడం.
- *"Alternative to memo?"* → state colocation, children composition, list virtualization.

---

18. useContext & Context API

CONTEXT · ఎప్పుడు, ఎందుకు జాగ్రత్త

CONTEXT – prop drilling కి పరిష్కారం, కాని ఒక ఖరీదుతో

Provider value మారితే
ఆ context వాడే **అన్ని** consumers
re-render అవుతాయి – memo అయినా సరే

→

సమస్య
value ఒక object అయితే –
ప్రతి render కి కొత్త reference

పరిష్కారాలు

1 · useMemo
value ని memo చేయడం
deps సరిగ్గా ఇవ్వడం

2 · Context ని విడగొట్టడం
State context + Dispatch context
Dispatch ఎప్పుడూ మారదు

3 · Selector library
use-context-selector
లేదా Zustand/Jotai

Context ఏం కాదు
Context ఒక **state management library కాదు** – అది ఒక dependency injection యంత్రాంగం.
తరచుగా మారే data కి (form input) Context తప్పు ఎంపిక – అది అందరినీ re-render చేస్తుంది.

వివరణ

Context = prop drilling లేకుండా component tree లో data ని "బ్రాడ్‌కాస్ట్" చేయడం.

```
// 1. Context create
const ThemeContext = createContext("light");      // default value (Provider లేకపోతే
వాడతారు)

// 2. Provide
<ThemeContext.Provider value={theme}>
  <App />
</ThemeContext.Provider>

// React 19: Provider అవసరం లేదు — Context నే component గా వాడొచ్చు
<ThemeContext value={theme}><App /></ThemeContext>

// 3. Consume
const theme = useContext(ThemeContext);
// React 19 లో: const theme = use(ThemeContext);   ← conditional గా కూడా call చేయొచ్చు
```

Real-life Scenario

> **Context = ఇంట్లో WiFi.** Router (Provider) ఒకసారి signal ఇస్తే, ఇంట్లో ఏ గదిలో ఉన్న device అయినా (ఏ depth లో ఉన్న component అయినా) నేరుగా connect అవుతుంది – ప్రతి గది గుండా cable లాగాల్సిన పని లేదు (prop drilling). కానీ **WiFi password మార్చితే (context value మారితే), ఇంట్లో అన్ని devices disconnect అయ్యి మళ్ళీ connect అవుతాయి** – వాటికి ఆ మార్పు అవసరం లేకపోయినా (unnecessary re-renders).

Code – production-grade context (typed + custom hook + guard)

```
// contexts/AuthContext.tsx
import { createContext, useContext, useState, useCallback, useMemo } from "react";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  const login = useCallback(async (creds) => {
    const u = await api.login(creds);
    setUser(u);
  }, []);
  const logout = useCallback(async () => { await api.logout(); setUser(null); }, []);

  // ⚠️ ఇది కీలకం — useMemo లేకపోతే ప్రతి render కి కొత్త object → అన్ని consumers re-render
  const value = useMemo(() => ({ user, loading, login, logout }), [user, loading,
login, logout]);

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

// ✅ Custom hook — Provider లేకపోతే స్పష్టమైన error (silent undefined కాకుండా)
```

```
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (ctx === null) throw new Error("useAuth must be used within <AuthProvider>");
  return ctx;
}
```

Context యొక్క పెద్ద సమస్య – re-render broadcast

Context value మారితే, ఆ context ని consume చేసే ప్రతి component re-render అవుతుంది – value లోని ఒక్క field మాత్రమే మారినా, మీరు ఆ field వాడకపోయినా. `React.memo` కూడా దీన్ని ఆపదు (context subscription memo ని bypass చేస్తుంది).

```
// ❌ ఒకే context లో అన్నీ – theme మారితే cart consumers కూడా re-render
const AppContext = createContext();
<AppContext.Provider value={{ user, theme, cart, notifications }}>
```

పరిష్కారం 1 – Context splitting (అత్యంత ముఖ్యం)

```
// ✅ ప్రతి concern కి వేరే context – cart మారితే theme consumers ప్రశాంతం
<ThemeContext value={theme}>
  <UserContext value={user}>
    <CartContext value={cart}>{children}</CartContext>
  </UserContext>
</ThemeContext>
```

పరిష్కారం 2 – State/Dispatch విడగొట్టడం

```
// dispatch ఎప్పుడూ మారదు → dispatch మాత్రమే వాడే components ఎప్పుడూ re-render కావు
const StateContext = createContext(null);
const DispatchContext = createContext(null);

function Provider({ children }) {
  const [state, dispatch] = useReducer(reducer, initial);
  return (
    <StateContext.Provider value={state}>
      <DispatchContext.Provider value={dispatch}>{children}
</DispatchContext.Provider>
    </StateContext.Provider>
  );
}
// Button లాంటివి useDispatch() మాత్రమే వాడితే state మారినా re-render కావు ✅
```

పరిష్కారం 3 – External store + `useSyncExternalStore` (selector-based subscriptions) – ఇదే Zustand/Redux చేసేది. తరచూ మారే global state కి context కంటే ఇవి మేలు.

Context ఎప్పుడు వాడాలి / వాడకూడదు

✅ మంచిది	❌ చెడ్డది
Theme, locale, auth user	ప్రతి keystroke కి మారే form data

✅ మంచిది	❌ చెడ్డది
Feature flags, config	Server data (→ TanStack Query)
Design-system context (Tabs, Accordion internals)	High-frequency updates (mouse position, scroll)
Dependency injection (testing లో mock provider)	"అన్ని state కి ఒకే global context"

> **Context ఒక state management library కాదు** – ఇది *dependency injection* mechanism. State ని ఎలా నిర్వహించాలో అది చెప్పదు (అది `useState` / `useReducer` పని). Interview లో ఈ వ్యత్యాసం చెప్తే మంచి ముద్ర.

Gotchas (సాధారణ తప్పులు)

- **Provider `value` ని memoize చేయకపోవడం** → ప్రతి parent render కి అందరూ re-render.

```
<Ctx.Provider value={{ user, setUser }}>   // ❌ కొత్త object ప్రతిసారి
```

- **అన్నిటికీ ఒకే giant context** → performance నాశనం.
- **Provider లోపల లేకపోతే default value silent గా వాడబడటం** → `null` default + custom hook throw.
- **Context ని Redux స్థానంలో వాడటం** (తరచూ మారే complex state కి) → re-render storms.
- **Provider ని component లోపల define చేయడం** → ప్రతి render కి కొత్త context → tree remount.
- **`useContext` ని memo తో ఆపగలం అనుకోవడం** – ఆపలేం.

Key Points

- Context = **prop drilling పరిష్కారం**, state manager కాదు.
- Value మారితే **అన్ని consumers re-render** – memo ఆపదు.
- **Split contexts**, **state/dispatch విభజన**, **value memoization** – మూడు తప్పనిసరి patterns.
- Custom hook + null-guard = ప్రామాణిక అభ్యాసం.
- React 19: `<Context>` నే provider గా వాడొచ్చు; `use(Context)` conditional OK.

Interview దృష్టి

- *"Context vs Redux?"* → DI vs state management; re-render granularity; middleware/devtools/selectors.
- *"Context ని ఎలా optimize చేస్తావు?"* → split, memoize value, dispatch separation, external store + selectors.
- *"Context ఎందుకు తరచూ మారే state కి చెడ్డది?"* → subscription granularity లేదు, whole-subtree broadcast.

---

19. useReducer

వివరణ

`useReducer` = complex state transitions కి `useState` కి ప్రత్యామ్నాయం. State update logic ని component నుండి బయటికి, ఒక **pure reducer function** లోకి తీసుకెళ్తుంది.

```
const [state, dispatch] = useReducer(reducer, initialState, initFn?);

function reducer(state, action) {          // pure: (state, action) => newState
  switch (action.type) {
    case "increment": return { ...state, count: state.count + 1 };
    default: throw new Error(`Unknown action: ${action.type}`);
  }
}
dispatch({ type: "increment" });
```

Real-life Scenario

> **useState = ప్రతి పనికీ నేరుగా bank account లో డబ్బు మార్చడం.** చిన్న లావాదేవీలకి సరిపోతుంది.
>
> **useReducer = బ్యాంకుకి "voucher" ఇవ్వడం.** మీరు "ఇలా మార్చు" అని రాయరు – "withdraw ₹500" అనే **intent** (action) ఇస్తారు. బ్యాంకు నియమాలు (reducer) ప్రకారం ఏం చేయాలో నిర్ణయిస్తుంది. లాభం: ప్రతి లావాదేవీ **audit trail** లో ఉంటుంది (action log), నియమాలు ఒకే చోట, testing సులభం.

`useState` vs `useReducer` – ఎప్పుడు ఏది

useState	useReducer
Independent, simple values	Multiple sub-values కలిసి మారేవి
Update logic చిన్నది	సంక్లిష్ట transitions, validation
ఒక్కో update ఒక్కో చోట	Update logic ఒకే చోట (reducer)
Testing కష్టం (component లోపల)	**Reducer ని component లేకుండా test చేయొచ్చు** ✅
Deep children కి setters పంపడం కష్టం	`dispatch` stable – context తో పంపడం సులభం ✅

Code – form/wizard state (వాస్తవిక ఉదాహరణ)

```
const initial = { step: 0, data: {}, errors: {}, status: "editing" };

function wizardReducer(state, action) {
  switch (action.type) {
    case "field_change":
      return {
        ...state,
```

```
        data: { ...state.data, [action.field]: action.value },
        errors: { ...state.errors, [action.field]: null },   // typing చేస్తే error clear
      };
    case "next":
      if (Object.keys(state.errors).some((k) => state.errors[k])) return state;   //
guard
      return { ...state, step: state.step + 1 };
    case "back":
      return { ...state, step: Math.max(0, state.step - 1) };
    case "submit_start":
      return { ...state, status: "submitting" };
    case "submit_success":
      return { ...initial, status: "done" };
    case "submit_error":
      return { ...state, status: "editing", errors: action.errors };
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}

function Wizard() {
  const [state, dispatch] = useReducer(wizardReducer, initial);

  const onChange = (e) =>
    dispatch({ type: "field_change", field: e.target.name, value: e.target.value });

  const onSubmit = async () => {
    dispatch({ type: "submit_start" });
    try {
      await api.save(state.data);
      dispatch({ type: "submit_success" });
    } catch (err) {
      dispatch({ type: "submit_error", errors: err.fieldErrors });
    }
  };

  return (/* ... UI ... */);
}
```

Reducer testing – component అవసరం లేదు:

```
test("field_change clears that field's error", () => {
  const s = { data: {}, errors: { email: "bad" }, step: 0, status: "editing" };
  const next = wizardReducer(s, { type: "field_change", field: "email", value:
"a@b.c" });
  expect(next.errors.email).toBeNull();
  expect(next.data.email).toBe("a@b.c");
});
```

Reducer + Context = mini Redux (library లేకుండా)

```
const TodoStateContext = createContext(null);
const TodoDispatchContext = createContext(null);

export function TodoProvider({ children }) {
```

```
  const [todos, dispatch] = useReducer(todoReducer, [], () =>
    JSON.parse(localStorage.getItem("todos") ?? "[]")     // lazy init
  );
  useEffect(() => { localStorage.setItem("todos", JSON.stringify(todos)); },
[todos]);

  return (
    <TodoStateContext.Provider value={todos}>
      <TodoDispatchContext.Provider value={dispatch}>{children}
</TodoDispatchContext.Provider>
    </TodoStateContext.Provider>
  );
}
export const useTodos = () => useContext(TodoStateContext);
export const useTodoDispatch = () => useContext(TodoDispatchContext);
```

> `dispatch` **identity ఎప్పుడూ మారదు** – దాన్ని deps array లో, memoized children కి నిర్భయంగా పంపొచ్చు. ఇది `useReducer` యొక్క పెద్ద దాచిన లాభం.

State machine style (impossible states ని అసాధ్యం చేయడం)

```
function fetchReducer(state, action) {
  switch (state.status) {
    case "idle":     return action.type === "FETCH" ? { status: "loading" } : state;
    case "loading":
      if (action.type === "SUCCESS") return { status: "success", data: action.data };
      if (action.type === "FAILURE") return { status: "error", error: action.error };
      return state;
    case "success":  return action.type === "FETCH" ? { status: "loading" } : state;
    case "error":    return action.type === "RETRY" ? { status: "loading" } : state;
    default:         return state;
  }
}
// loading + error ఒకేసారి ఎప్పటికీ సాధ్యం కాదు ✅ (XState library ఇదే idea ని ముందుకు
తీసుకెళ్తుంది)
```

Gotchas (సాధారణ తప్పులు)

- **Reducer ని impure చేయడం** – API call, `Math.random()`, `Date.now()`, mutation ❌. Reducer తప్పనిసరిగా pure. Side effects handler/effect లో.
- **State ని mutate చేయడం** – `state.items.push()` ❌ (immer వాడొచ్చు).
- `default: return state` వదిలేయడం → typo చేసిన action silent గా పోతుంది. Throw చేయడం మేలు (dev లో).
- **అతి-సరళ state కి reducer వాడటం** – `useState` సరిపోతే అదే మేలు (boilerplate తగ్గుతుంది).
- **Action types ని string typo తో రాయడం** – TypeScript discriminated union వాడాలి.

Key Points

- `useReducer` = state transition logic ని pure function లోకి తీసుకెళ్తుంది.
- Multiple related values + complex logic + testability కావాలంటే ఇదే.

- `dispatch` **stable identity** → memo/context కి ideal.
- Reducer + Context = library లేకుండా global state.
- State machine style తో impossible states నివారణ.

Interview దృష్టి

- *"useState vs useReducer?"* → complexity, colocation of logic, testability, stable dispatch.
- *"Reducer + Context Redux ని replace చేస్తుందా?"* → చిన్న/మధ్యస్థ apps కి అవును; devtools, middleware, selector-based subscriptions, async patterns కావాలంటే Redux Toolkit మేలు.

---

20. useTransition & useDeferredValue (Concurrent React)

వివరణ

React 18 తో వచ్చిన **concurrent rendering** యొక్క ప్రధాన developer-facing APIs. ఆలోచన: **అన్ని updates సమానం కాదు.** కొన్ని అత్యవసరం (typing, click), కొన్ని వాయిదా వేయొచ్చు (పెద్ద list filter).

```
const [isPending, startTransition] = useTransition();
startTransition(() => { setSearchResults(query); });   // "ఇది urgent కాదు" అని React కి
చెప్పడం

const deferredQuery = useDeferredValue(query);          // value యొక్క "వెనుకబడిన" copy
```

Real-life Scenario

> **Transition = ambulance vs సరుకు లారీ.** రెండూ రోడ్డు మీద ఉన్నాయి. Ambulance (user typing) కి దారి ఇవ్వాలి – లారీ (heavy list re-render) పక్కకి తప్పుకోవాలి, ఆలస్యమైనా పర్వాలేదు. React 17 లో అన్నీ ఒకే lane లో వరుసగా – ముందు లారీ వస్తే ambulance ఆగాలి (UI freeze). React 18 concurrent rendering లారీ ని **మధ్యలో ఆపి**, ambulance ని పంపి, తర్వాత లారీ ని మళ్ళీ మొదలుపెడుతుంది.

సమస్య – ఇది లేకపోతే ఏమవుతుంది

```
// ❌ 20,000 items filter — ప్రతి keystroke కి UI freeze (input లో typing ఆలస్యం)
function Search({ items }) {
  const [query, setQuery] = useState("");
  const results = items.filter((i) => i.name.includes(query));   // ఖరీదైనది
  return (
    <>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      <List results={results} />
    </>
  );
}
```

పరిష్కారం A – `useTransition`

```
function Search({ items }) {
  const [query, setQuery] = useState("");
  const [results, setResults] = useState(items);
  const [isPending, startTransition] = useTransition();

  const onChange = (e) => {
    setQuery(e.target.value);                         // ✅ urgent — input వెంటనే update
    startTransition(() => {                           // ⏳ non-urgent — interrupt కావొచ్చు
      setResults(items.filter((i) => i.name.includes(e.target.value)));
    });
  };

  return (
    <>
      <input value={query} onChange={onChange} />
      <div style={{ opacity: isPending ? 0.5 : 1 }}>   {/* stale UI ని మసకబారవడం */}
        <List results={results} />
      </div>
    </>
  );
}
```

పరిష్కారం B – `useDeferredValue` (సరళమైనది)

```
function Search({ items }) {
  const [query, setQuery] = useState("");
  const deferredQuery = useDeferredValue(query);        // వెనుకబడిన copy
  const isStale = query !== deferredQuery;

  // ⚠️ deferredQuery మీద memoize చేయడం కీలకం — లేకపోతే ప్రయోజనం లేదు
  const results = useMemo(
    () => items.filter((i) => i.name.includes(deferredQuery)),
    [items, deferredQuery]
  );

  return (
    <>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      <div style={{ opacity: isStale ? 0.5 : 1 }}><List results={results} /></div>
    </>
  );
}
```

	`useTransition`	`useDeferredValue`
నియంత్రణ	*update* ని mark చేస్తాం (setState మన చేతిలో ఉండాలి)	*value* ని defer చేస్తాం (props అయినా OK)
ఎప్పుడు	Tab switch, navigation, filter submit	Search-as-you-type, props నుండి వచ్చే value

	`useTransition`	`useDeferredValue`
Pending flag	`isPending` ఇస్తుంది	`value !== deferred` తో మనం లెక్కించాలి

Debounce vs Transition – ముఖ్యమైన తేడా

Debounce/Throttle	useTransition/useDeferredValue
సమయం ఆధారంగా వేచి ఉంటుంది (300ms)	వేచి ఉండదు – వెంటనే మొదలై, అవసరమైతే **interrupt** అవుతుంది
నెమ్మది device లో కూడా అదే delay	React device speed కి తగ్గట్టు adapt అవుతుంది
Network calls తగ్గించడానికి ఉత్తమం ✅	Rendering work కి ఉత్తమం ✅

> **రెండూ కలిపి వాడొచ్చు:** API calls కి debounce + heavy rendering కి transition.

Concurrent rendering – లోపల ఏమి మారింది

React 18 కి ముందు rendering **synchronous, uninterruptible** – ఒకసారి మొదలైతే మొత్తం tree పూర్తయ్యే వరకు main thread block. Concurrent mode లో React ప్రతి కొన్ని fiber nodes తర్వాత *"browser కి ఏదైనా urgent పని ఉందా?"* అని చూసి, ఉంటే **render ని ఆపి**, urgent పని చేసి, తర్వాత మళ్ళీ మొదలుపెడుతుంది (**time slicing**).

అందుకే **components pure గా ఉండాలి** – render ని discard చేసి మళ్ళీ run చేయొచ్చు కాబట్టి, render లో side effects ఉంటే అవి రెండుసార్లు జరుగుతాయి.

Gotchas (సాధారణ తప్పులు)

- `startTransition` **లో async/await** – callback **synchronous** గా ఉండాలి; `await` తర్వాత setState transition లో ఉండదు. (React 19 లో async transitions Actions ద్వారా support.)
- **Input value ని transition లో పెట్టడం** – typing lag వస్తుంది; controlled input ఎప్పుడూ urgent.
- `useDeferredValue` **వాడి** `useMemo` **మర్చిపోవడం** – heavy work మళ్ళీ ప్రతిసారి జరుగుతుంది, ప్రయోజనం సున్నా.
- `createRoot` **బదులు legacy** `render` → concurrent features పని చేయవు.
- **ఇవి పని ని *వేగం* చేస్తాయి అనుకోవడం** – కాదు; అవి **priority** ని మాత్రమే మారుస్తాయి. అసలు work ఖరీదైతే virtualization/pagination అవసరం.

Key Points

- Concurrent React = **interruptible rendering** + priorities.
- `useTransition` = update ని non-urgent గా mark; `isPending` ఇస్తుంది.
- `useDeferredValue` = value యొక్క lagging copy (props కి కూడా పని చేస్తుంది).
- Debounce = time-based (network కి), transition = priority-based (rendering కి).
- Purity తప్పనిసరి – renders discard/replay కావొచ్చు.

Interview దృష్టి

- *"React 18 concurrent అంటే ఏమిటి?"* → interruptible rendering, lanes, priorities – mode కాదు, opt-in features.
- *"Transition vs debounce?"* → పైన table; ఎప్పుడు ఏది.
- *"UI typing లో freeze అవుతోంది – ఏం చేస్తావు?"* → profile → transition/deferred + memo → virtualization → web worker (చివరి ఎంపిక).

---

21. useId, useSyncExternalStore, useDebugValue, useInsertionEffect

`useId` – SSR-safe unique IDs

```
function Field({ label }) {
  const id = useId();                    // ":r1:" లాంటిది – server & client లో ఒకటే
  return (
    <>
      <label htmlFor={id}>{label}</label>
      <input id={id} aria-describedby={`${id}-hint`} />
      <span id={`${id}-hint`}>Hint text</span>
    </>
  );
}
```

ఎందుకు అవసరం: `Math.random()` లేదా counter వాడితే server లో ఒక id, client లో వేరే id → **hydration mismatch error**. `useId` server/client రెండింటిలోనూ ఒకే value ఇస్తుంది (component tree లో position ఆధారంగా).

⚠️ **list keys కి వాడకూడదు** – ఇది accessibility attributes (`htmlFor` , `aria-*`) కి మాత్రమే.

`useSyncExternalStore` – external stores కి safe subscription

React బయట ఉన్న store (Redux, Zustand, browser API) ని subscribe చేయడానికి. **Concurrent rendering లో "tearing" ని ఆపుతుంది** – tearing అంటే ఒకే render లో వేర్వేరు components వేర్వేరు store values చూడటం.

```
const isOnline = useSyncExternalStore(
  // subscribe: (callback) => unsubscribe
  (cb) => {
    window.addEventListener("online", cb);
    window.addEventListener("offline", cb);
    return () => {
      window.removeEventListener("online", cb);
      window.removeEventListener("offline", cb);
    };
  },
  () => navigator.onLine,          // getSnapshot (client)
```

```
  () => true                     // getServerSnapshot (SSR) — లేకపోతే SSR లో crash
);
```

```
// వాస్తవిక ఉదాహరణ — media query hook
function useMediaQuery(query) {
  const subscribe = useCallback((cb) => {
    const mql = window.matchMedia(query);
    mql.addEventListener("change", cb);
    return () => mql.removeEventListener("change", cb);
  }, [query]);
  return useSyncExternalStore(
    subscribe,
    () => window.matchMedia(query).matches,
    () => false                  // server లో false
  );
}
const isMobile = useMediaQuery("(max-width: 768px)");
```

> ⚠️ `getSnapshot` **cached value return చేయాలి** – ప్రతిసారి కొత్త object return చేస్తే infinite loop (`The result of getSnapshot should be cached`).

`useDebugValue` – DevTools label (custom hooks కి)

```
function useOnlineStatus() {
  const isOnline = useSyncExternalStore(subscribe, getSnapshot);
  useDebugValue(isOnline ? "🟢 Online" : "🔴 Offline");   // DevTools లో కనిపిస్తుంది
  return isOnline;
}
// ఖరీదైన format అయితే lazy: useDebugValue(date, (d) => d.toISOString());
```

`useInsertionEffect` – CSS-in-JS library authors కోసం

Layout effects కంటే ముందు run అవుతుంది; `<style>` tags inject చేయడానికి (layout thrash ఆపడానికి). **App code లో దాదాపు ఎప్పుడూ అవసరం లేదు** – styled-components/emotion వంటి libraries మాత్రమే వాడతాయి.

```
execution order: useInsertionEffect → useLayoutEffect → (paint) → useEffect
```

Key Points

- `useId` = SSR-safe ids (a11y attributes కి; **keys కి కాదు**).
- `useSyncExternalStore` = external store subscription, tearing-safe, SSR snapshot తప్పనిసరి.
- `useDebugValue` = custom hook DevTools label.
- `useInsertionEffect` = CSS-in-JS libraries మాత్రమే.

Interview దృష్టి

- *"Tearing అంటే ఏమిటి?"* → concurrent render మధ్యలో external store మారితే, ఒకే frame లో వేర్వేరు components వేర్వేరు values చూడటం; `useSyncExternalStore` దీన్ని ఆపుతుంది.
- *"Hydration mismatch ఎలా ఆపుతావు?"* → `useId`, server/client లో ఒకే output, `Date/random` ని render లో వాడకపోవడం.

---

22. React 19 కొత్త APIs – use(), Actions, useOptimistic

`use()` – విప్లవాత్మక hook

`use()` ఒక **promise** లేదా **context** ని చదువుతుంది. ఇతర hooks లా కాకుండా – **conditions, loops లోపల call చేయొచ్చు!**

```
import { use, Suspense } from "react";

function Comments({ commentsPromise }) {
  const comments = use(commentsPromise);      // promise resolve అయ్యేవరకు suspend
అవుతుంది
  return comments.map((c) => <p key={c.id}>{c.text}</p>);
}

function Page({ commentsPromise }) {
  return (
    <Suspense fallback={<Spinner />}>        {/* fallback ఇక్కడ */}
      <Comments commentsPromise={commentsPromise} />
    </Suspense>
  );
}

// Context కి కూడా – conditional గా!
function Heading({ children }) {
  if (children == null) return null;          // early return తర్వాత కూడా OK ✅
  const theme = use(ThemeContext);
  return <h1 style={{ color: theme.color }}>{children}</h1>;
}
```

> ⚠️ **Client component లో render సమయంలో promise create చేయొద్దు** (`use(fetch(...))`) – ప్రతి render కి కొత్త promise → infinite loop. Promise ని server component లో లేదా cached source (React Query, framework loader) నుండి రావాలి.

Actions – async transitions

React 19 లో `startTransition` **async functions** ని support చేస్తుంది. ఒక async function ని transition లో పంపితే దాన్ని **"Action"** అంటారు – React pending state, errors, optimistic updates ని నిర్వహిస్తుంది.

```
function UpdateName() {
  const [name, setName] = useState("");
  const [error, setError] = useState(null);
  const [isPending, startTransition] = useTransition();

  const submit = () => {
    startTransition(async () => {              // ✅ 19 లో async OK
      const err = await updateName(name);
      if (err) { setError(err); return; }
      redirect("/profile");
    });
  };
  return <><input value={name} onChange={e => setName(e.target.value)} />
    <button onClick={submit} disabled={isPending}>Save</button>{error && <p>{error}
</p>}</>;
}
```

`useActionState` – action + state ఒకే చోట

```
const [state, formAction, isPending] = useActionState(actionFn, initialState);
```

(Topic 10 లో పూర్తి form ఉదాహరణ చూశాం.)

`useFormStatus` – parent form యొక్క status ని child లో చదవడం

```
import { useFormStatus } from "react-dom";     // ⚠️ 'react' కాదు, 'react-dom'

function SubmitButton() {
  const { pending, data, method, action } = useFormStatus();
  return <button disabled={pending}>{pending ? "Submitting…" : "Submit"}</button>;
}
// <form action={fn}><SubmitButton /></form> — props drilling అవసరం లేదు
```

`useOptimistic` – తక్షణ UI feedback

Server confirm చేయకముందే UI ని update చేసి, fail అయితే automatic గా revert చేయడం.

```
function MessageThread({ messages, sendMessage }) {
  const [optimisticMessages, addOptimistic] = useOptimistic(
    messages,
    (currentMessages, newText) => [
      ...currentMessages,
      { id: "temp", text: newText, sending: true },     // temporary entry
    ]
  );

  const formAction = async (formData) => {
    const text = formData.get("message");
    addOptimistic(text);                        // ⚡ వెంటనే UI లో కనిపిస్తుంది
    await sendMessage(text);                    // ఇది fail అయినా/పూర్తయినా React
optimistic entry ని తీసేసి
                                                // అసలు messages ని చూపిస్తుంది
```

```
  };

  return (
    <>
      {optimisticMessages.map((m) => (
        <div key={m.id} style={{ opacity: m.sending ? 0.5 : 1 }}>{m.text}</div>
      ))}
      <form action={formAction}><input name="message" /><button>Send</button></form>
    </>
  );
}
```

React 19 లోని ఇతర ముఖ్య మార్పులు

మార్పు	వివరణ
ref as prop	`forwardRef` అవసరం లేదు – `function Input({ ref })`
ref cleanup	callback ref నుండి cleanup function return చేయొచ్చు
`<Context>` **as provider**	`<Ctx value={v}>` – `.Provider` అవసరం లేదు
Document metadata	`<title>`, `<meta>`, `<link>` ని ఏ component లోనైనా render చేస్తే React `<head>` కి తరలిస్తుంది
Stylesheets & async scripts	`precedence` తో stylesheet ordering; `<script async>` dedup
Resource preloading	`preload`, `preinit`, `prefetchDNS`, `preconnect` APIs
Better errors	hydration mismatch diffs, error dedup, `onCaughtError` / `onUncaughtError` root options
`propTypes`, `defaultProps` **(function components), string refs, legacy context తొలగింపు**	TypeScript + default parameters వాడాలి
React Compiler	auto memoization (opt-in babel plugin)

```
// Document metadata — ఏ component లోనైనా (react-helmet అవసరం లేదు)
function BlogPost({ post }) {
  return (
    <article>
      <title>{post.title}</title>
      <meta name="description" content={post.excerpt} />
      <link rel="canonical" href={post.url} />
      <h1>{post.title}</h1>
    </article>
  );
}
```

Gotchas (సాధారణ తప్పులు)

- **Client component render లో promise create చేసి `use()` కి పంపడం** → infinite loop.
- **`useFormStatus` ని form component లోనే వాడటం** – child లోనే పని చేస్తుంది, అదే component లో కాదు.
- **`useOptimistic` ని transition/action బయట వాడటం** – action context లోనే revert logic పని చేస్తుంది.
- **React 19 కి upgrade చేసి `propTypes` / `defaultProps` వాడటం** – silent గా పని చేయవు.

Key Points

- `use()` = promise/context read; **conditional call చేయగలిగే ఏకైక hook**.
- **Actions** = async transitions; pending/error handling built-in.
- `useActionState`, `useFormStatus`, `useOptimistic` – forms కి పూర్తి kit.
- ref-as-prop, `<Context>` provider, document metadata – boilerplate తగ్గింపు.
- React Compiler = manual memoization ముగింపు.

Interview దృష్టి

- *"React 19 లో ఏం కొత్తది?"* → Actions + 4 hooks + ref-as-prop + metadata + compiler – ఒక్కో దానికి ఒక్క వాక్యం.
- *"Optimistic update ఎలా implement చేస్తావు?"* → `useOptimistic`, లేదా React Query `onMutate` + rollback (Topic 39).

---

23. Custom Hooks (Production Library)

CUSTOM HOOKS · LOGIC REUSE, STATE REUSE కాదు

CUSTOM HOOK – logic ని పంచుకోవడం, state ని కాదు

కీలకమైన అపోహ

Custom hook state ని **పంచుకోదు** – ప్రతి component కి తన సొంత కాపీ వస్తుంది.
A lo count పెంచితే B lo మారదు. అది logic reuse మాత్రమే.
నిజంగా పంచుకోవాలంటే – Context లేదా బయటి store కావాలి.

మంచి custom hook లక్షణాలు

"use" తో మొదలు · ఒక్క పని చేస్తుంది · state కాకుండా **ఒక API** ని return చేస్తుంది
Component ఎలా render చేయాలో నిర్ణయించదు – దాన్ని component కి వదిలేస్తుంది.

వివరణ

Custom hook = `use` తో మొదలయ్యే function, ఇది ఇతర hooks ని వాడుతుంది. ఇది React లో **stateful logic reuse** చేసే ప్రామాణిక మార్గం (HOC/render props స్థానంలో).

కీలక అవగాహన: Custom hook **state ని share చేయదు – logic ని share చేస్తుంది.** రెండు components ఒకే hook వాడితే, ఒక్కొక్కరికి **వేరే state instance** ఉంటుంది.

Real-life Scenario

> **Custom hook = వంట recipe.** Recipe ని ఇద్దరు వాడితే ఇద్దరికీ **వేరే వేరే వంటకం** వస్తుంది (వేరే state), కానీ **పద్ధతి ఒకటే** (logic). Recipe ని share చేసినంత మాత్రాన ఒకే గిన్నెలో వండరు. State share కావాలంటే Context/store అవసరం.

1. `useToggle` / `useBoolean`

```
function useToggle(initial = false) {
  const [value, setValue] = useState(initial);
  const toggle = useCallback(() => setValue((v) => !v), []);
  const setTrue = useCallback(() => setValue(true), []);
  const setFalse = useCallback(() => setValue(false), []);
  return { value, toggle, setTrue, setFalse, setValue };
}
const { value: isOpen, toggle, setFalse: close } = useToggle();
```

2. `useDebounce` + `useDebouncedCallback`

```
// Value ని debounce చేయడం
function useDebounce(value, delay = 500) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);           // value మారితే పాత timer cancel
  }, [value, delay]);
  return debounced;
}

// Callback ని debounce చేయడం (latest args తో)
function useDebouncedCallback(fn, delay = 500) {
  const fnRef = useRef(fn);
  useEffect(() => { fnRef.current = fn; });         // ఎప్పుడూ latest fn
  const timer = useRef(null);
  const debounced = useCallback((...args) => {
    clearTimeout(timer.current);
    timer.current = setTimeout(() => fnRef.current(...args), delay);
  }, [delay]);
  useEffect(() => () => clearTimeout(timer.current), []);   // unmount cleanup
  return debounced;
}

// వాడకం — search
const debouncedQuery = useDebounce(query, 300);
useEffect(() => { if (debouncedQuery) search(debouncedQuery); }, [debouncedQuery]);
```

3. `useLocalStorage` (SSR-safe, cross-tab sync)

```
function useLocalStorage(key, initialValue) {
  const [stored, setStored] = useState(() => {
    if (typeof window === "undefined") return initialValue;    // SSR guard
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch { return initialValue; }                            // corrupt JSON /
private mode
  });

  const setValue = useCallback((value) => {
    setStored((prev) => {
      const next = value instanceof Function ? value(prev) : value;
      try { window.localStorage.setItem(key, JSON.stringify(next)); } catch { /*
quota */ }
      return next;
    });
  }, [key]);

  // ఇతర tabs లో మారితే sync
  useEffect(() => {
    const onStorage = (e) => {
      if (e.key === key && e.newValue) setStored(JSON.parse(e.newValue));
    };
    window.addEventListener("storage", onStorage);
    return () => window.removeEventListener("storage", onStorage);
  }, [key]);

  return [stored, setValue];
}
```

4. `useOnClickOutside` (modals, dropdowns)

```
function useOnClickOutside(ref, handler) {
  const handlerRef = useRef(handler);
  useEffect(() => { handlerRef.current = handler; });

  useEffect(() => {
    const listener = (e) => {
      if (!ref.current || ref.current.contains(e.target)) return;   // లోపల click —
ignore
      handlerRef.current(e);
    };
    document.addEventListener("mousedown", listener);
    document.addEventListener("touchstart", listener);
    return () => {
      document.removeEventListener("mousedown", listener);
      document.removeEventListener("touchstart", listener);
    };
  }, [ref]);
}
// ⚠️ 'click' కాకుండా 'mousedown' వాడటం ముఖ్యం — button click తో conflict రాకుండా
```

5. `useFetch` – Topic 12 లో చూశాం (AbortController తో)

6. `useIntersectionObserver` (infinite scroll, lazy images)

```
function useIntersectionObserver(options = {}) {
  const [entry, setEntry] = useState(null);
  const [node, setNode] = useState(null);        // ✅ callback ref → node మారితే re-
run

  useEffect(() => {
    if (!node) return;
    const observer = new IntersectionObserver(([e]) => setEntry(e), options);
    observer.observe(node);
    return () => observer.disconnect();
  }, [node, options.root, options.rootMargin, options.threshold]);

  return [setNode, entry?.isIntersecting ?? false, entry];
}

// Infinite scroll
function Feed() {
  const [ref, isVisible] = useIntersectionObserver({ rootMargin: "200px" });
  useEffect(() => { if (isVisible) loadMore(); }, [isVisible]);
  return <><List /><div ref={ref} /></>;      // sentinel
}
```

7. `useMediaQuery` – Topic 21 లో (useSyncExternalStore తో)

8. `useInterval` (Dan Abramov's declarative interval)

```
function useInterval(callback, delay) {
  const savedCallback = useRef(callback);
  useEffect(() => { savedCallback.current = callback; }, [callback]);

  useEffect(() => {
    if (delay === null) return;                        // null = pause
    const id = setInterval(() => savedCallback.current(), delay);
    return () => clearInterval(id);
  }, [delay]);
}
// useInterval(() => setCount(c => c + 1), isRunning ? 1000 : null);
```

> **ఈ ref pattern ఎందుకు:** callback ని నేరుగా `setInterval` కి ఇస్తే stale closure; callback ని dep గా ఇస్తే ప్రతి render కి interval reset. Ref తో రెండూ పరిష్కారం.

9. `useAsync` (loading/error/data + manual trigger)

```
function useAsync(asyncFn, { immediate = true } = {}) {
  const [state, setState] = useState({ status: "idle", data: null, error: null });
  const mounted = useRef(true);
  useEffect(() => () => { mounted.current = false; }, []);
```

```
  const run = useCallback(async (...args) => {
    setState({ status: "pending", data: null, error: null });
    try {
      const data = await asyncFn(...args);
      if (mounted.current) setState({ status: "success", data, error: null });
      return data;
    } catch (error) {
      if (mounted.current) setState({ status: "error", data: null, error });
      throw error;
    }
  }, [asyncFn]);

  useEffect(() => { if (immediate) run(); }, [immediate, run]);
  return { ...state, run, isLoading: state.status === "pending" };
}
```

10. `useCopyToClipboard`, `useWindowSize`, `useHover`

```
function useCopyToClipboard(resetMs = 2000) {
  const [copied, setCopied] = useState(false);
  const copy = useCallback(async (text) => {
    try {
      await navigator.clipboard.writeText(text);
      setCopied(true);
      setTimeout(() => setCopied(false), resetMs);
      return true;
    } catch { setCopied(false); return false; }
  }, [resetMs]);
  return [copied, copy];
}

function useWindowSize() {
  const [size, setSize] = useState({ width: 0, height: 0 });
  useEffect(() => {
    const onResize = () => setSize({ width: innerWidth, height: innerHeight });
    onResize();
    window.addEventListener("resize", onResize);
    return () => window.removeEventListener("resize", onResize);
  }, []);
  return size;
}
```

Custom hooks – design నియమాలు

1. **పేరు `use` తో మొదలవ్వాలి** (linter దీని ఆధారంగానే rules enforce చేస్తుంది).
2. **ఒకే బాధ్యత** – `useUserDashboardDataAndTheme` ❌.
3. **Return shape:** 2 values → array `[value, setter]`; 3+ → object `{ data, error, isLoading }`.
4. **Cleanup తప్పనిసరి** – listeners, timers, subscriptions.
5. **Callbacks ని ref లో ఉంచి stale closures నివారించడం.**
6. **SSR guard** – `typeof window === "undefined"`.
7. **Hook లో business logic OK, కానీ UI (JSX) return చేయకూడదు** – అది component పని.

Gotchas (సాధారణ తప్పులు)

- **State share అవుతుందని అనుకోవడం** – ప్రతి caller కి వేరే instance.
- **Hook లోపల condition తో hooks call చేయడం** – నియమాలు hooks లోపలా వర్తిస్తాయి.
- **Cleanup మర్చిపోవడం** – reusable hook కావడంతో leak అనేక చోట్ల.
- **Deps array లో objects** → infinite loops.
- **అతిగా abstraction** – ఒకే చోట వాడే logic ని hook చేయడం అనవసరం.

Key Points

- Custom hook = **logic reuse**, state reuse కాదు.
- `use` prefix తప్పనిసరి.
- Latest-callback ref pattern = stale closure కి universal పరిష్కారం.
- Cleanup + SSR guard = production quality.
- Return shape convention: tuple (2) vs object (3+).

Interview దృష్టి

- *"Custom hook రాయి – useDebounce/useFetch/useLocalStorage"* → machine coding round లో అత్యంత సాధారణం.
- *"రెండు components ఒకే hook వాడితే state share అవుతుందా?"* → కాదు; share కావాలంటే Context/store.
- *"HOC vs custom hook?"* → wrapper hell లేదు, composition సులభం, TypeScript inference మెరుగు.

Part 3 – Class Components, Lifecycle & Patterns

24. Class Components & Lifecycle

వివరణ

2019 కి ముందు React లో state ఉన్న component అంటే **class component** మాత్రమే. ఇప్పుడు కొత్త code లో వాడరు, కానీ **మూడు కారణాలవల్ల తెలియాలి:**

1. **Legacy codebases** – భారతదేశంలో చాలా enterprise projects ఇంకా class components తో ఉన్నాయి.
2. **Error Boundaries** ఇప్పటికీ **classes లోనే** సాధ్యం (hook equivalent లేదు).
3. **Interview** – "lifecycle methods ని hooks తో ఎలా map చేస్తావు?" అనేది classic SDE2 question.

```
class Counter extends React.Component {
  constructor(props) {
    super(props);                              // ⚠️ తప్పనిసరి — లేకపోతే this.props
undefined
    this.state = { count: 0 };
    this.handleClick = this.handleClick.bind(this);   // this binding
  }

  // ✅ Class fields తో binding అవసరం లేదు (arrow = lexical this)
  handleClick = () => this.setState((s) => ({ count: s.count + 1 }));

  render() {
    return <button onClick={this.handleClick}>{this.state.count}</button>;
  }
}
```

Real-life Scenario

> **Class components = పాత landline phone.** పని చేస్తుంది, నమ్మకమైనది, కానీ ప్రతి పనికీ manual (`this` binding, lifecycle లో logic చెల్లాచెదురు). **Hooks = smartphone** – అదే పని, తక్కువ boilerplate, logic ని app లాగా (custom hooks) package చేసి share చేయొచ్చు.

Lifecycle – మూడు దశలు

```
MOUNTING:    constructor → getDerivedStateFromProps → render → componentDidMount
UPDATING:    getDerivedStateFromProps → shouldComponentUpdate → render
             → getSnapshotBeforeUpdate → componentDidUpdate
UNMOUNTING: componentWillUnmount
ERROR:       getDerivedStateFromError → componentDidCatch
```

Code – పూర్తి lifecycle + hooks equivalent

```
class DataView extends React.Component {
  state = { data: null, loading: true };

  // 1. Mount తర్వాత ఒకసారి – API calls, subscriptions
  componentDidMount() {
    this.controller = new AbortController();
    fetch(this.props.url, { signal: this.controller.signal })
      .then((r) => r.json())
      .then((data) => this.setState({ data, loading: false }));
  }
  // ↔ useEffect(() => { ... }, [])

  // 2. Update తర్వాత – ⚠️ prevProps compare చేయకపోతే infinite loop!
  componentDidUpdate(prevProps, prevState, snapshot) {
    if (prevProps.url !== this.props.url) {
      this.setState({ loading: true });
      this.fetchData();
    }
  }
  // ↔ useEffect(() => { ... }, [url])

  // 3. Unmount ముందు – cleanup
  componentWillUnmount() {
    this.controller?.abort();
  }
  // ↔ useEffect(() => { return () => cleanup; }, [])

  // 4. Re-render అవసరమా? (performance)
  shouldComponentUpdate(nextProps, nextState) {
    return nextProps.url !== this.props.url || nextState.data !== this.state.data;
  }
  // ↔ React.memo (⚠️ return value తలకిందులు: true = render చేయి)

  // 5. Props నుండి state derive (అరుదు – static, this లేదు)
  static getDerivedStateFromProps(props, state) {
    return props.id !== state.prevId ? { data: null, prevId: props.id } : null;
  }
  // ↔ render సమయంలో adjust, లేదా key prop

  // 6. DOM update కి ముందు snapshot (scroll position వంటివి)
  getSnapshotBeforeUpdate(prevProps, prevState) {
    return this.listRef.current.scrollHeight;    // ఈ value componentDidUpdate కి
వెళ్తుంది
  }
  // ↔ useLayoutEffect

  render() {
    const { data, loading } = this.state;
    return loading ? <Spinner /> : <List data={data} />;
  }
}
```

Lifecycle → Hooks mapping table (interview cheat sheet)

Class	Hook equivalent
`constructor` (state init)	`useState(initial)` / lazy initializer
`componentDidMount`	`useEffect(fn, [])`
`componentDidUpdate`	`useEffect(fn, [deps])`
`componentWillUnmount`	`useEffect(() => () => cleanup, [])`
`shouldComponentUpdate`	`React.memo` (+ comparator)
`getDerivedStateFromProps`	render-time adjust / `key` prop
`getSnapshotBeforeUpdate`	`useLayoutEffect`
`getDerivedStateFromError`	❌ classes only (Error Boundary)
`componentDidCatch`	❌ classes only
`this.forceUpdate()`	`useReducer(x => x+1, 0)` (hack)

Deprecated lifecycle methods (వాడొద్దు)

`componentWillMount`, `componentWillReceiveProps`, `componentWillUpdate` — async/concurrent rendering లో ఇవి **అనేకసార్లు** call కావచ్చు. React 17 లో `UNSAFE_` prefix తో మాత్రమే మిగిలాయి.

`setState` in classes — hooks కంటే ఒక ముఖ్య తేడా

```
// Class setState = shallow MERGE (partial state OK)
this.setState({ name: "x" });                    // ఇతర state fields అలాగే ఉంటాయి ✅
this.setState((s, p) => ({ count: s.count + 1 }));  // updater form
this.setState({ a: 1 }, () => console.log("done"));  // ✅ callback — hooks లో ఇది
లేదు!

// Hooks setState = REPLACE (merge కాదు)
setUser({ name: "x" });                          // ❌ ఇతర fields పోతాయి!
setUser((u) => ({ ...u, name: "x" }));           // ✅ manual merge
```

> **Hooks లో setState callback కి ప్రత్యామ్నాయం:** `useEffect(() => { ... }, [state])`.

Class vs Function — comparison

	Class	Function + Hooks
Boilerplate	ఎక్కువ (`constructor`, `this`, `bind`)	తక్కువ
Logic reuse	HOC/render props (wrapper hell)	Custom hooks ✅

	Class	Function + Hooks
Related logic	lifecycle లో చెల్లాచెదురు	ఒకే effect లో కలిసి ✅
`this`	గందరగోళం	లేదు ✅
Bundle size	కొంచెం పెద్దది	చిన్నది (minify better)
Error boundary	✅	❌
React Compiler	support లేదు	✅

Gotchas (సాధారణ తప్పులు)

- **`super(props)` మర్చిపోవడం** → `this.props` undefined in constructor.
- **`this.state.x = 1`** → direct mutation, re-render కాదు. `setState` వాడాలి.
- **Handler ని bind చేయకపోవడం** → `this` undefined (class field arrow వాడాలి).
- **`componentDidUpdate` లో guard లేకుండా `setState`** → infinite loop.
- **`shouldComponentUpdate` return value ని memo తో గందరగోళం** – `sCU: true = render`; `memo comparator: true = skip`.
- **`this.state` ని `setState` తర్వాత వెంటనే చదవడం** → పాత value (async).

Key Points

- Lifecycle: mount → update → unmount (+ error phase).
- `componentDidMount/Update/WillUnmount` = ఒకే `useEffect` యొక్క మూడు ముఖాలు.
- Class `setState` **merges**; hooks setState **replaces**.
- Deprecated `componentWill*` – concurrent-unsafe.
- Error boundaries **classes only** ఇప్పటికీ.

Interview దృష్టి

- *"Map lifecycle methods to hooks"* → పై table (memorize).
- *"Why hooks over classes?"* → logic reuse, colocation, no `this`, smaller bundles, compiler support.
- *"Class లో ఇంకా ఏం చేయగలం, hooks లో కాదు?"* → Error boundaries (`getDerivedStateFromError`, `componentDidCatch`).

---

25. Error Boundaries

ERROR BOUNDARIES · పరిధి

ERROR BOUNDARY ఏం పట్టుకుంటుంది, ఏం పట్టుకోదు

✓ పట్టుకుంటుంది	✗ పట్టుకోదు
Render సమయంలో వచ్చే errors	Event handlers (onClick lo throw)
Lifecycle methods lo errors	async code (setTimeout, fetch .then)
Children యొక్క constructors lo	Server-side rendering · boundary దానిలోనే

ఎందుకు ఈ తేడా

Error boundary React యొక్క **render tree** ని మాత్రమే కాపాడుతుంది.
Event handler React tree బయట నడుస్తుంది – అక్కడ సాధారణ try/catch వాడాలి.
Async కి – .catch() లేదా error state. React 19 lo కొన్ని మెరుగుదలలు వచ్చాయి.

ఇంకా class component మాత్రమే

Hooks తో error boundary రాయలేం –

`getDerivedStateFromError` కి class కావాలి.ఆచరణలో: `react-error-boundary` package వాడటం.

వివరణ

Error Boundary = child component tree లో వచ్చే JavaScript errors ని పట్టుకొని, మొత్తం app crash కాకుండా fallback UI చూపించే class component.

React 16 నుండి, error boundary లేకపోతే **render లో error వస్తే మొత్తం React tree unmount అవుతుంది** – user కి తెల్ల screen. ఇది ఉద్దేశపూర్వక design: *"పాడైన UI చూపించడం కంటే ఏమీ చూపించకపోవడం మేలు"* (బ్యాంకింగ్ app లో తప్పు balance చూపించడం ప్రమాదం).

Real-life Scenario

> **Error boundary = ఇంట్లో MCB (circuit breaker).** ఒక గదిలో short circuit అయితే మొత్తం ఇంటి కరెంటు పోకూడదు – ఆ గది MCB మాత్రమే trip అవ్వాలి. Error boundary లేకపోతే – bedroom లో fan పాడైతే మొత్తం ఇల్లు చీకటి (whole app crash).

Code – production error boundary

```
class ErrorBoundary extends React.Component {
  state = { hasError: false, error: null };

  // 1. Render phase – fallback UI చూపించడానికి state update (pure ఉండాలి)
  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  // 2. Commit phase – side effects (logging) ఇక్కడ
  componentDidCatch(error, errorInfo) {
    console.error("Caught:", error, errorInfo.componentStack);
    Sentry.captureException(error, { extra: errorInfo });   // monitoring కి పంపడం
  }
```

```
  reset = () => this.setState({ hasError: false, error: null });

  render() {
    if (this.state.hasError) {
      if (this.props.fallback) return this.props.fallback(this.state.error,
this.reset);
      return (
        <div role="alert">
          <h2>ఏదో తప్పు జరిగింది</h2>
          <pre>{this.state.error?.message}</pre>
          <button onClick={this.reset}>మళ్ళీ ప్రయత్నించండి</button>
        </div>
      );
    }
    return this.props.children;
  }
}
```

❌ Error Boundary ఏమి పట్టుకోదు (అత్యంత ముఖ్యం)

పట్టుకోదు	ఎందుకు / పరిష్కారం
Event handlers	React rendering బయట → `try/catch` వాడాలి
Async code (`setTimeout` , promises)	call stack వేరు → `.catch()` / `try/catch` in async fn
Server-side rendering	server లో boundaries పని చేయవు (Next.js `error.tsx` వేరుగా handle)
Error boundary లోనే వచ్చిన error	పైన ఉన్న boundary పట్టుకుంటుంది

```
// ❌ boundary దీన్ని పట్టుకోదు
<button onClick={() => { throw new Error("boom"); }}>x</button>

// ✅ handler లో try/catch
<button onClick={() => {
  try { risky(); } catch (e) { setError(e); }
}}>x</button>

// ✅ లేదా error ని state లో పెట్టి render లో throw చేయడం (boundary పట్టుకుంటుంది)
const [err, setErr] = useState(null);
if (err) throw err;
```

`react-error-boundary` library (production standard)

```
import { ErrorBoundary } from "react-error-boundary";

function Fallback({ error, resetErrorBoundary }) {
  return (
    <div role="alert">
      <p>Error: {error.message}</p>
      <button onClick={resetErrorBoundary}>Retry</button>
```

```
    </div>
  );
}

<ErrorBoundary
  FallbackComponent={Fallback}
  onError={(error, info) => logToService(error, info)}
  onReset={() => queryClient.resetQueries()}       // reset లో state ని కూడా clear
  resetKeys={[userId]}                              // ఈ value మారితే automatic reset
>
  <Dashboard />
</ErrorBoundary>

// Async errors ని boundary లోకి పంపడానికి
import { useErrorBoundary } from "react-error-boundary";
const { showBoundary } = useErrorBoundary();
useEffect(() => { fetchData().catch(showBoundary); }, []);
```

Boundaries ని ఎక్కడ పెట్టాలి – granularity strategy

```
<ErrorBoundary fallback={<FullPageError />}>          {/* 1. App level — last resort
*/}
  <Router>
    <ErrorBoundary fallback={<PageError />}>          {/* 2. Route level */}
      <Layout>
        <Sidebar />
        <ErrorBoundary fallback={<WidgetError />}>    {/* 3. Widget level — ideal */}
          <RevenueChart />                            {/* chart crash అయినా page పని
చేస్తుంది */}
        </ErrorBoundary>
        <ErrorBoundary fallback={<WidgetError />}>
          <RecentOrders />
        </ErrorBoundary>
      </Layout>
    </ErrorBoundary>
  </Router>
</ErrorBoundary>
```

నియమం: *"ఈ భాగం విఫలమైతే, మిగిలిన UI ఇంకా ఉపయోగకరంగా ఉంటుందా?"* – అవును అయితే అక్కడ boundary పెట్టండి.

Suspense + ErrorBoundary కలిపి – `AsyncBoundary`

```
function AsyncBoundary({ children, errorFallback, loadingFallback }) {
  return (
    <ErrorBoundary fallback={errorFallback}>
      <Suspense fallback={loadingFallback}>{children}</Suspense>
    </ErrorBoundary>
  );
}
// ⚠️ క్రమం ముఖ్యం — ErrorBoundary బయట, Suspense లోపల
```

React 19 – root-level error handlers

```
createRoot(document.getElementById("root"), {
  onUncaughtError: (error, errorInfo) => log("uncaught", error),
  onCaughtError: (error, errorInfo) => log("caught by boundary", error),
  onRecoverableError: (error) => log("recovered (hydration mismatch)", error),
}).render(<App />);
```

Gotchas (సాధారణ తప్పులు)

- **Event handler errors ని boundary పట్టుకుంటుంది అనుకోవడం** – పట్టుకోదు.
- **App లో ఒకే boundary** – ఏ చిన్న error అయినా మొత్తం page పోతుంది.
- **`getDerivedStateFromError` లో side effects** – అది pure ఉండాలి; logging `componentDidCatch` లో.
- **Fallback UI లో reset ఇవ్వకపోవడం** – user reload తప్ప మార్గం లేదు.
- **Reset తర్వాత అదే error మళ్ళీ** – root cause (bad state/props) ని కూడా reset చేయాలి (`resetKeys`).
- **Development లో error overlay చూసి "boundary పని చేయట్లేదు" అనుకోవడం** – dev లో React overlay చూపిస్తుంది, production లో boundary పని చేస్తుంది.

Key Points

- Error boundary = **class only** (`getDerivedStateFromError` + `componentDidCatch`).
- Render/lifecycle errors పట్టుకుంటుంది; **events, async, SSR పట్టుకోదు**.
- Multi-level granularity: app → route → widget.
- `react-error-boundary` = reset, resetKeys, `useErrorBoundary` (async).
- ErrorBoundary **బయట**, Suspense **లోపల**.

Interview దృష్టి

- *"Error boundary ఏం పట్టుకోదు?"* → events, async, SSR, boundary దానిలోని errors – 4 చెప్పగలగాలి.
- *"Hooks తో error boundary రాయగలవా?"* → లేదు; library వాడతాం లేదా చిన్న class రాస్తాం.
- *"Production లో error strategy?"* → boundaries + Sentry + user-friendly fallback + retry + source maps.

---

26. HOC (Higher-Order Component) Pattern

వివరణ

HOC = component తీసుకొని కొత్త (మెరుగైన) component return చేసే function.

```
const EnhancedComponent = withSomething(WrappedComponent);
```

ఇది hooks కి ముందు **logic reuse** యొక్క ప్రధాన pattern. ఇప్పుడు custom hooks మేలు, కానీ HOC ఇంకా కొన్ని చోట్ల ఉపయోగం (cross-cutting concerns: auth guards, analytics, error handling, theming) మరియు **legacy code + interviews** లో వస్తుంది.

Real-life Scenario

> **HOC = ఫోన్ కి cover.** ఫోన్ (component) అలాగే ఉంటుంది; cover (HOC) దానికి రక్షణ/అదనపు feature (auth check, loading spinner) జోడిస్తుంది. కానీ **cover మీద cover మీద cover** వేస్తే (`withAuth(withTheme(withRouter(withLogger(App))))`) – ఫోన్ ఏదో తెలియని పరిస్థితి. అదే **wrapper hell**, hooks దీన్ని పరిష్కరించాయి.

Code – HOC ఉదాహరణలు

```
// 1. withAuth — route protection
function withAuth(Component, { redirectTo = "/login" } = {}) {
  return function AuthenticatedComponent(props) {
    const { user, loading } = useAuth();
    const navigate = useNavigate();

    useEffect(() => { if (!loading && !user) navigate(redirectTo); }, [user, loading,
navigate]);

    if (loading) return <Spinner />;
    if (!user) return null;
    return <Component {...props} user={user} />;
  };
}
const ProtectedDashboard = withAuth(Dashboard);

// 2. withErrorBoundary
function withErrorBoundary(Component, Fallback) {
  return function Wrapped(props) {
    return (
      <ErrorBoundary fallback={<Fallback />}>
        <Component {...props} />
      </ErrorBoundary>
    );
  };
}

// 3. withLogger (analytics)
function withLogger(Component, eventName) {
  return function Logged(props) {
    useEffect(() => { analytics.track(eventName, { props }); }, []);
    return <Component {...props} />;
  };
}
```

HOC నియమాలు (correctness కి తప్పనిసరి)

```
function withThing(Component) {
  function Wrapper(props) {
```

```
    return <Component {...props} extra="value" />;      // 1️⃣ props అన్నీ pass చేయాలి
  }

  // 2️⃣ DevTools లో మంచి పేరు
  Wrapper.displayName = `withThing(${Component.displayName || Component.name ||
"Component"})`;

  // 3️⃣ Static methods copy చేయాలి (hoist-non-react-statics library)
  hoistNonReactStatics(Wrapper, Component);

  return Wrapper;
}

// 4️⃣ ❌ Render లోపల HOC apply చేయకూడదు — ప్రతి render కి కొత్త component type → remount!
function Bad() {
  const Enhanced = withAuth(Dashboard);     // ❌ ప్రతిసారి కొత్త type
  return <Enhanced />;
}
// ✅ module level లో ఒకసారి
const Enhanced = withAuth(Dashboard);
```

HOC vs Custom Hook vs Render Props

	HOC	Render Props	Custom Hook
Composition	wrapper nesting	JSX nesting	flat function calls ✅
Props origin	అస్పష్టం (ఎక్కడి నుండి వచ్చాయో తెలియదు)	స్పష్టం	స్పష్టం ✅
TypeScript	కష్టం	మధ్యస్థం	సులభం ✅
Ref forwarding	manual (forwardRef)	సమస్య లేదు	సమస్య లేదు
DevTools	wrapper noise	nesting noise	శుభ్రం ✅
JSX structure మార్చాలంటే	✅ (wrapping UI)	✅	❌ (hook JSX return చేయదు)

```
// అదే logic — మూడు విధాలుగా
const WithMouse = withMouse(Comp);                          // HOC
<Mouse render={({x, y}) => <Comp x={x} y={y} />} />         // Render prop
const { x, y } = useMouse();                                // Hook ✅ సరళం
```

> **2026 నియమం:** logic reuse → **custom hook**. Component ని *wrap* చేయాలి (auth gate, error boundary, layout) → **HOC లేదా composition**.

Gotchas (సాధారణ తప్పులు)

- **Props pass చేయకపోవడం** → wrapped component కి props రావు.

- **Render లో HOC apply చేయడం** → unmount/remount, state loss.
- **`displayName` పెట్టకపోవడం** → DevTools లో `Unknown`.
- **Static methods పోగొట్టడం** → `Component.someStatic` undefined.
- **Ref pass కాకపోవడం** → HOC లో `forwardRef` (React ≤18) అవసరం.
- **Prop name collisions** – రెండు HOCs ఒకే prop name ఇస్తే silent overwrite.

Key Points

- HOC = `Component => Component`; hooks కి ముందు reuse pattern.
- నియమాలు: props spread, displayName, statics hoist, module-level apply, ref forward.
- **Wrapper hell** = HOC యొక్క ప్రధాన లోపం → hooks పరిష్కారం.
- Wrapping/JSX structure అవసరమైతే HOC ఇంకా చెల్లుతుంది.

Interview దృష్టి

- *"HOC ఏమిటి, ఉదాహరణ?"* → `withAuth` రాయమని అడుగుతారు.
- *"HOC vs hook?"* → పై table; "hooks logic reuse కి, HOC wrapping కి".
- *"React లో మీకు తెలిసిన HOCs?"* → `React.memo`, `connect` (react-redux), `withRouter` (RRv5).

---

27. Render Props Pattern

వివరణ

Render prop = component కి ఒక *function* ని prop గా ఇచ్చి, ఆ function ద్వారా UI ని render చేయడం. Component logic ని కలిగి ఉంటుంది, UI ని caller నిర్ణయిస్తుంది.

```
<DataProvider render={(data) => <h1>{data.title}</h1>} />
// లేదా children as function (ఎక్కువ ప్రాచుర్యం)
<DataProvider>{(data) => <h1>{data.title}</h1>}</DataProvider>
```

Real-life Scenario

> **Render prop = tiffin box + "నీ ఇష్టం వచ్చినట్లు అలంకరించుకో".** వంటవాడు (component) ఆహారం (data/logic) సిద్ధం చేస్తాడు, కానీ plate లో ఎలా పెట్టాలో (UI) మీరే నిర్ణయిస్తారు. అదే వంటవాడు, వేరే వేరే presentations.

Code

```
// Mouse tracker — logic ఒకటే, UI అనేకం
function MouseTracker({ children }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  useEffect(() => {
    const onMove = (e) => setPos({ x: e.clientX, y: e.clientY });
    window.addEventListener("mousemove", onMove);
```

```
    return () => window.removeEventListener("mousemove", onMove);
  }, []);
  return children(pos);          // ✅ children ఒక function
}

// వాడకం — ఒకే logic, వేరే UI
<MouseTracker>{({ x, y }) => <p>Mouse: {x}, {y}</p>}</MouseTracker>
<MouseTracker>{({ x, y }) => <Cat top={y} left={x} />}</MouseTracker>
```

నిజంగా ఇంకా ఉపయోగపడే చోట్లు

Hooks వచ్చాక render props తగ్గాయి, కానీ **component కి ఎక్కడ render అవ్వాలో తెలియాలంటే** ఇది ఇంకా ఉత్తమం:

```
// 1. Virtualized lists — library కి row ఎలా render చేయాలో తెలియదు
<VirtualList items={items} height={400}>
  {(item, style) => <div style={style}>{item.name}</div>}
</VirtualList>

// 2. Headless UI components (Downshift, react-table)
<Downshift onChange={select}>
  {({ getInputProps, getItemProps, isOpen, highlightedIndex }) => (
    <div>
      <input {...getInputProps()} />
      {isOpen && items.map((it, i) => (
        <li {...getItemProps({ item: it, index: i })}
            style={{ background: highlightedIndex === i ? "#eee" : "" }}>{it.label}
</li>
      ))}
    </div>
  )}
</Downshift>

// 3. Data fetching wrapper with UI states
<Query url="/api/users">
  {({ loading, error, data }) =>
    loading ? <Spinner /> : error ? <Err e={error} /> : <List data={data} />}
</Query>
```

Callback hell సమస్య

```
// ❌ nesting నరకం — hooks తో 3 lines
<Mouse>{(m) => (
  <Theme>{(t) => (
    <Auth>{(a) => (
      <Data>{(d) => <UI {...m} {...t} {...a} {...d} />}
    </Auth>
  )}</Theme>
)}</Mouse>

// ✅ hooks
const m = useMouse(); const t = useTheme(); const a = useAuth(); const d = useData();
```

Gotchas (సాధారణ తప్పులు)

- **Render prop ని inline arrow గా ఇవ్వడం + parent memo** → ప్రతి render కి కొత్త function → memo విఫలం.
- **Deep nesting** → callback hell.
- `children` **ని function గా ఇవ్వడం TypeScript లో typing గందరగోళం.**

Key Points

- Render prop = **function ని prop గా ఇచ్చి UI నియంత్రణ caller కి ఇవ్వడం**.
- `children` as function = ప్రాచుర్యం పొందిన రూపం.
- Logic reuse కి hooks మేలు; **rendering control** కావాలంటే render props ఇంకా ఉత్తమం.
- Headless UI libraries (Downshift, react-table, TanStack Virtual) ఈ pattern మీద నిలబడ్డాయి.

Interview దృష్టి

- *"Render props ఇంకా అవసరమా?"* → logic reuse కి కాదు; rendering control/headless components కి అవును.
- *"Render props vs HOC vs hooks"* → composition, transparency, TS, DevTools ఆధారంగా.

---

28. Compound Components, Control Props & State Reducer

వివరణ

ఇవి **library-grade component API design** patterns – design system build చేసేటప్పుడు (SSE level responsibility) అవసరం.

Compound Components = ఒకదానితో ఒకటి కలిసి పని చేసే components సమూహం, state ని implicit గా (context ద్వారా) share చేస్తాయి. HTML లోని `<select>` + `<option>` లాంటిది.

Real-life Scenario

> **Compound components = TV remote.** Remote (parent) కి channel state ఉంది. మీద ఉన్న buttons (children) – వాటిలో ప్రతి దానికి "ఏ TV, ఏ channel" అని విడిగా చెప్పరు; అవి remote లోపలి wiring (context) ద్వారా తెలుసుకుంటాయి. మీరు buttons ని ఎలా arrange చేసినా (composition freedom) పని చేస్తాయి.

Code – Compound Components (Tabs)

```
const TabsContext = createContext(null);

function Tabs({ children, defaultValue, value: controlledValue, onChange }) {
  const [uncontrolled, setUncontrolled] = useState(defaultValue);
  const isControlled = controlledValue !== undefined;      // 🔑 control props
pattern
  const value = isControlled ? controlledValue : uncontrolled;
```

```
  const select = useCallback((v) => {
    if (!isControlled) setUncontrolled(v);
    onChange?.(v);
  }, [isControlled, onChange]);

  const ctx = useMemo(() => ({ value, select }), [value, select]);
  return <TabsContext.Provider value={ctx}><div className="tabs">{children}</div>
</TabsContext.Provider>;
}

function useTabs() {
  const ctx = useContext(TabsContext);
  if (!ctx) throw new Error("Tabs.* must be used inside <Tabs>");
  return ctx;
}

Tabs.List   = function List({ children }) { return <div role="tablist">{children}
</div>; };
Tabs.Tab    = function Tab({ value, children }) {
  const { value: active, select } = useTabs();
  return (
    <button role="tab" aria-selected={active === value}
            onClick={() => select(value)}
            className={active === value ? "tab active" : "tab"}>{children}</button>
  );
};
Tabs.Panel  = function Panel({ value, children }) {
  const { value: active } = useTabs();
  return active === value ? <div role="tabpanel">{children}</div> : null;
};

// ✅ వాడకం — flexible, self-documenting, HTML లా చదవగలిగేది
<Tabs defaultValue="profile">
  <Tabs.List>
    <Tabs.Tab value="profile">Profile</Tabs.Tab>
    <Tabs.Tab value="settings">Settings</Tabs.Tab>
  </Tabs.List>
  <Tabs.Panel value="profile"><Profile /></Tabs.Panel>
  <Tabs.Panel value="settings"><Settings /></Tabs.Panel>
</Tabs>
```

Config-props API తో పోలిక – compound ఎందుకు మేలో:

```
// ❌ config-based — flexibility సున్నా, props పేలిపోతాయి
<Tabs
  tabs={[{ label: "Profile", content: <Profile />, icon: "user", badge: 3, disabled:
false }]}
  tabClassName="..." panelClassName="..." renderTabLabel={...}
/>
// మధ్యలో divider పెట్టాలంటే? Tab లో custom UI కావాలంటే? → కొత్త prop అవసరం ప్రతిసారి
```

Control Props Pattern (controlled + uncontrolled రెండూ support చేయడం)

HTML `<input>` లాగే – `value` ఇస్తే controlled, `defaultValue` ఇస్తే uncontrolled. అన్ని మంచి libraries ఇలానే చేస్తాయి.

```
function useControllableState({ value, defaultValue, onChange }) {
  const [internal, setInternal] = useState(defaultValue);
  const isControlled = value !== undefined;
  const state = isControlled ? value : internal;

  const setState = useCallback((next) => {
    const resolved = typeof next === "function" ? next(state) : next;
    if (!isControlled) setInternal(resolved);
    onChange?.(resolved);
  }, [isControlled, onChange, state]);

  return [state, setState];
}
```

State Reducer Pattern (Kent C. Dodds) – గరిష్ఠ flexibility

Component యొక్క internal state transitions ని **user override చేయనివ్వడం**.

```
function useToggle({ reducer = (state, action) => action.changes } = {}) {
  const defaultReducer = (state, action) => {
    switch (action.type) {
      case "toggle": return { on: !state.on };
      case "reset":  return { on: false };
      default: throw new Error(action.type);
    }
  };
  const [state, dispatch] = useReducer((s, a) => {
    const changes = defaultReducer(s, a);
    return reducer(s, { ...a, changes });          // 🔑 user కి final say
  }, { on: false });

  return { on: state.on, toggle: () => dispatch({ type: "toggle" }) };
}

// User: "4 సార్ల తర్వాత toggle ఆగిపోవాలి" — library code మార్చకుండా
const [count, setCount] = useState(0);
const { on, toggle } = useToggle({
  reducer(state, action) {
    if (action.type === "toggle" && count >= 4) return { on: state.on };   // block
    return action.changes;
  },
});
```

Headless components (2026 లో ప్రబలమైన approach)

Logic (hook) ని UI నుండి పూర్తిగా విడదీయడం – Radix UI, Headless UI, TanStack Table, React Aria ఇలానే పని చేస్తాయి.

```
// Library logic + a11y మాత్రమే ఇస్తుంది; styling మనది
const { getInputProps, getMenuProps, getItemProps, isOpen } = useCombobox({ items });
```

లాభాలు: ఏ design system అయినా వాడొచ్చు, accessibility built-in, bundle లో అనవసర CSS లేదు.

Gotchas (సాధారణ తప్పులు)

- **Compound components కి `React.Children.map` తో implicit props ఇవ్వడం** – deep nesting లో విఫలం (children ని wrapper లో పెడితే). **Context వాడాలి.**
- **Context provider లేకుండా sub-component వాడితే గందరగోళ error** → custom hook లో throw చేయాలి.
- **Controlled/uncontrolled మధ్య మారడం** – React warning; `isControlled` ని mount లో lock చేయాలి.
- **Provider value memoize చేయకపోవడం** → అన్ని tabs re-render.
- **Accessibility మర్చిపోవడం** – `role="tab"`, `aria-selected`, keyboard arrow navigation తప్పనిసరి.

Key Points

- **Compound components** = context ద్వారా implicit state share + composition freedom.
- **Control props** = controlled + uncontrolled రెండూ support (`value` vs `defaultValue`).
- **State reducer** = internal transitions ని user override చేయడం (max flexibility).
- **Headless** = logic/a11y hook + UI freedom – ఆధునిక design system approach.
- Config props (`items={[...]}`) కంటే composition API scale అవుతుంది.

Interview దృష్టి

- *"Design a Tabs/Accordion/Modal component API"* – SSE frontend rounds లో సాధారణం. Compound + controlled/uncontrolled + a11y చెప్పండి.
- *"Radix/Headless UI ఎందుకు popular?"* → styling freedom + accessibility + logic reuse.

Part 4 – Advanced Rendering & React Internals

29. Fiber Architecture Deep

FIBER · ఆపగలిగే RENDERING

FIBER – ఎందుకు React ని తిరిగి రాశారు

పాత Stack reconciler

ఒకసారి మొదలైతే **ఆపలేం** – recursive.
పెద్ద tree = main thread స్తంభిస్తుంది.
User type చేస్తున్నా UI స్పందించదు.

Fiber

Tree ని ఒక **linked list** గా మార్చడం.
ప్రతి unit తర్వాత "ఇంకా సమయం ఉందా?" అని చూడటం.
లేకపోతే ఆగి, browser కి దారి ఇచ్చి, తర్వాత కొనసాగడం.

రెండు దశలు

RENDER phase → **COMMIT phase**

RENDER phase: ఆపొచ్చు · మళ్ళీ మొదలుపెట్టొచ్చు · పారేయొచ్చు

COMMIT phase: ఆపలేం · ఒకేసారి DOM కి వర్తిస్తుంది

దీని వల్ల వచ్చినవి

useTransition – "ఇది తక్కువ ప్రాధాన్యత, ఆగినా ఫర్వాలేదు" అని చెప్పగలగడం
Suspense – data వచ్చేదాకా ఆ భాగాన్ని ఆపి, మిగతాది చూపడం
Concurrent rendering – అత్యవసరమైన update (typing) ముందు వెళ్ళడం

వివరణ

Fiber = React 16 లో వచ్చిన reconciliation engine యొక్క పూర్తి rewrite. పాత architecture ని "Stack reconciler" అంటారు – అది recursive గా tree ని traverse చేసేది, **ఆపలేని** (uninterruptible) synchronous ప్రక్రియ. Component tree పెద్దదైతే main thread నిమిషాల పాటు block → typing, animations, clicks అన్నీ freeze.

Fiber పరిష్కారం: recursion ని **linked-list ఆధారిత loop** గా మార్చడం – దీనివల్ల React పని ని **మధ్యలో ఆపి, తర్వాత అక్కడి నుండే కొనసాగించగలదు**.

Real-life Scenario

> **Stack reconciler = ఒకే శ్వాసలో 500 పేజీల పుస్తకం చదవడం.** మధ్యలో ఎవరైనా పిలిచినా ఆగలేరు (browser freeze).
>
> **Fiber = bookmark పెట్టుకుంటూ చదవడం.** ప్రతి పేజీ తర్వాత "ఎవరైనా పిలిచారా?" అని చూస్తారు. Doorbell (user click) మోగితే – bookmark పెట్టి, తలుపు తీసి, తిరిగి వచ్చి **అక్కడి నుండే** కొనసాగిస్తారు. అవసరమైతే ఆ పుస్తకాన్ని పక్కన పెట్టి ఇంకో ముఖ్యమైన పుస్తకం (urgent update) ముందు చదువుతారు.

Fiber node – ఇందులో ఏమి ఉంటుంది

ప్రతి React element కి ఒక fiber node ఉంటుంది – ఇది ఒక JS object, "unit of work":

```
const fiber = {
  // Identity
  tag: FunctionComponent,      // component రకం (Function/Class/Host/Fragment…)
  type: MyComponent,           // function reference లేదా "div"
  key: null,

  // Tree structure — linked list (children array కాదు!)
  return: parentFiber,         // parent (return ఎందుకంటే stack frame లా return
అవుతుంది)
  child: firstChildFiber,      // మొదటి child మాత్రమే
  sibling: nextSiblingFiber,   // తర్వాతి sibling

  // State
  memoizedState: hooksLinkedList,   // hooks ఇక్కడ నిల్వ
  memoizedProps: {...},             // చివరి render లో వాడిన props
  pendingProps: {...},              // ఈ render కి కొత్త props

  // Effects & scheduling
  flags: Placement | Update | Deletion,   // ఏం చేయాలో bitmask
  lanes: 0b0000001,                       // priority (lane model)
  alternate: workInProgressFiber,         // double buffering జంట
  stateNode: domNodeOrClassInstance,
};
```

`child` + `sibling` + `return` — ఇదే recursion ని loop గా మార్చిన ట్రిక్. React ఒక pointer ని ముందుకు జరుపుతూ tree మొత్తం traverse చేయగలదు, call stack అవసరం లేకుండా. అందుకే ఆపి మళ్ళీ మొదలుపెట్టగలదు.

```
        App
         │ child
       ┌─▼──┐ sibling  ┌───┐
       │Header│────────▶│ Main│
       └─┬──┘          └───┘
         │ child
       ┌─▼──┐
       │ Logo│
       └────┘

Traversal: App → Header → Logo → (Logo కి child లేదు, sibling లేదు → return) → Header →
Main → …
```

రెండు దశలు — Render (async) & Commit (sync)

1️⃣ Render / Reconciliation phase — ఆపగలిగేది (interruptible)

- `beginWork(fiber)` — component ని render చేసి children fibers create/update చేస్తుంది.
- `completeWork(fiber)` — DOM node create చేస్తుంది (ఇంకా attach చేయదు), effect flags set చేస్తుంది.
- ప్రతి unit తర్వాత `shouldYield()` — browser కి urgent పని ఉందా అని check.
- **Side effects ఇక్కడ ఉండకూడదు** — ఈ phase discard/replay కావొచ్చు. (అందుకే components pure ఉండాలి!)

2️⃣ Commit phase — ఆపలేనిది (synchronous, atomic)

- **Before mutation:** `getSnapshotBeforeUpdate`
- **Mutation:** DOM insert/update/delete, refs detach
- **Layout:** refs attach, `componentDidMount/Update`, `useLayoutEffect` (sync)
- తర్వాత browser paint → ఆ తర్వాత `useEffect` (async, scheduler ద్వారా)

Double buffering – current & workInProgress

React ఎప్పుడూ **రెండు fiber trees** ఉంచుతుంది:

```
current tree  ←→  workInProgress tree
  (screen మీద ఉన్నది)    (తయారవుతున్నది)
```

Render phase లో React `workInProgress` tree ని build చేస్తుంది (ప్రతి fiber యొక్క `alternate` ని reuse చేస్తూ – memory allocation తగ్గించడానికి). పూర్తయ్యాక commit phase లో **pointer ని swap** చేస్తుంది – `workInProgress` ఇప్పుడు `current`.

> **ఇది game engines లోని double buffering లాంటిదే** – ఒక frame ని off-screen లో గీసి, పూర్తయ్యాక ఒక్కసారిగా screen కి swap చేయడం. Half-rendered UI ఎప్పుడూ కనిపించదు.

Work loop (సరళీకృత pseudo-code)

```
function workLoopConcurrent() {
  while (workInProgress !== null && !shouldYield()) {   // 🔑 ఇక్కడే interruption
    workInProgress = performUnitOfWork(workInProgress);
  }
}

function performUnitOfWork(fiber) {
  const next = beginWork(fiber);              // render component, create children
  if (next !== null) return next;             // child కి దిగు

  let node = fiber;
  while (node !== null) {
    completeWork(node);                       // DOM create, effects collect
    if (node.sibling !== null) return node.sibling;   // sibling కి వెళ్ళు
    node = node.return;                       // parent కి ఎక్కు
  }
  return null;                                // tree పూర్తి
}

function shouldYield() {
  return performance.now() >= deadline;       // 5ms time slice అయిపోయిందా?
}
```

Reconciliation – fiber స్థాయిలో ఏం జరుగుతుంది

```
1. setState → fiber ని "dirty" గా mark + lane assign
2. Scheduler ఆ lane priority ప్రకారం work schedule చేస్తుంది
3. Render phase: root నుండి కిందికి
   – props/state మారని subtree ఉంటే → bailout (subtree మొత్తం skip! ⚡)
```

```
   - మారితే → component run → children reconcile
     - key + type match → fiber reuse (alternate)
     - type మారింది → పాత fiber delete + కొత్తది create
4. Effect flags ఉన్న fibers linked list గా collect
5. Commit: ఆ list మీద DOM operations
```

Bailout = React యొక్క ముఖ్యమైన optimization: props reference equal + state మారలేదు + context మారలేదు అయితే **subtree మొత్తం skip**. `React.memo` దీన్నే explicit గా enable చేస్తుంది.

Gotchas (సాధారణ తప్పులు)

- **Fiber = performance boost అనుకోవడం** – raw speed కాదు; **scheduling & interruptibility**. Total work అదే.
- **Render phase లో side effects రాయడం** (mutation, subscriptions) – discard/replay వల్ల duplicates, corrupted state.
- `useEffect` **commit phase లో run అవుతుంది అనుకోవడం** – layout effects commit లో; passive effects **paint తర్వాత** async.
- **Fiber ని internal API గా వాడటం** (`_reactInternals`) – ఎప్పుడూ వద్దు.

Key Points

- **Fiber = unit of work** + linked-list tree (`child` / `sibling` / `return`) → interruptible loop.
- **Render phase** = pure, interruptible, discardable; **Commit phase** = sync, atomic.
- **Double buffering** (current ↔ workInProgress) → torn UI ఎప్పుడూ కనిపించదు.
- **Bailout** = subtree skip; memo దాన్ని explicit చేస్తుంది.
- Fiber ఇచ్చినది speed కాదు – **priority + interruptibility** (concurrent features కి పునాది).

Interview దృష్టి

- *"Explain Fiber."* → stack reconciler సమస్య → linked list → two phases → double buffering → concurrent ఆధారం.
- *"Render phase ఎందుకు pure ఉండాలి?"* → interrupt/discard/replay కావొచ్చు.
- *"Bailout అంటే?"* → unchanged subtree skip.

---

30. Concurrent React – Lanes, Priorities, Tearing

వివరణ

React 18 "concurrent mode" ఒక **mode కాదు** – అది opt-in **features సమూహం** (transitions, Suspense streaming, `useDeferredValue`). వాటి కింద ఉన్న engine: **lane-based priority scheduling**.

Lanes model

React 17 లో priority ఒక number (`expirationTime`). React 18 లో **lanes** – 31-bit bitmask, ప్రతి bit ఒక priority level:

```
SyncLane              0b0000000000000000000000000000001   ← discrete input
(click, keydown)
InputContinuousLane   0b0000000000000000000000000000100   ← drag, scroll, hover
DefaultLane           0b0000000000000000000000000010000   ← normal updates, fetch
responses
TransitionLanes       0b0000000001111111111111110000000   ← startTransition (16
lanes!)
RetryLanes            0b0000111110000000000000000000000   ← Suspense retries
IdleLane              0b0100000000000000000000000000000   ← offscreen work
```

Bitmask ఎందుకు: ఒకేసారి బహుళ priorities ని ఒక్క integer లో represent చేసి, bitwise operations (`&` , `|`) తో అతివేగంగా check చేయొచ్చు – "ఈ fiber లో pending work ఉందా?" అనేది ఒక CPU instruction.

Real-life Scenario

> **Lanes = highway lanes.** Ambulance lane (SyncLane – click, typing), సాధారణ traffic lane (DefaultLane), సరుకు లారీ lane (TransitionLane), రాత్రి delivery lane (IdleLane). ఒకే రోడ్డు (main thread), కానీ lane priority ప్రకారం ఎవరు ముందు వెళ్ళాలో traffic police (Scheduler) నిర్ణయిస్తుంది. Ambulance వస్తే లారీ పక్కకి తప్పుకుంటుంది (interrupt), తర్వాత మళ్ళీ కదులుతుంది.

Priority levels – ఆచరణలో

Update రకం	Priority	ప్రవర్తన
`onClick` , `onKeyDown` లో setState	**Sync/Discrete**	వెంటనే, ఆపలేం
`onScroll` , `onMouseMove`	Continuous	అధిక priority కానీ throttled
`fetch().then(setState)`	Default	సాధారణ
`startTransition(() => setState())`	Transition	**ఆపగలిగేది** ✅
Offscreen/prerender	Idle	ఖాళీ సమయంలో

Starvation నివారణ (expiration)

తక్కువ priority update ఎప్పటికీ run అవ్వకుండా (starvation) ఉండటానికి, React ప్రతి lane కి **expiration time** ఇస్తుంది. ఆ సమయం దాటితే lane **synchronous** గా మారి తప్పనిసరిగా పూర్తవుతుంది.

Tearing – concurrent rendering యొక్క కొత్త ప్రమాదం

Tearing = ఒకే render pass లో వేర్వేరు components ఒకే data యొక్క **వేర్వేరు versions** చూడటం → UI అస్థిరం.

```
1. React ComponentA ని render చేసింది → external store లో count = 5 చదివింది
2. React yield చేసింది (browser కి control ఇచ్చింది)
3. ఆ సమయంలో store update అయింది → count = 6
4. React తిరిగి వచ్చి ComponentB ని render చేసింది → count = 6 చదివింది
→ ఒకే screen లో "5" మరియు "6" — TORN UI ❌
```

React state కి ఈ సమస్య లేదు (React state ని ఒక snapshot గా ఉంచుతుంది). **External stores** (Redux, Zustand, browser APIs) కి ఉంది — అందుకే `useSyncExternalStore` వచ్చింది. అది store ని subscribe చేసి, render సమయంలో మార్పు గుర్తిస్తే **synchronous గా re-render** చేసి tearing ఆపుతుంది.

`startTransition` — లోపల ఏం జరుగుతుంది

```
startTransition(() => setResults(bigFilter(query)));
```

1. React ఈ update కి **TransitionLane** assign చేస్తుంది.
2. Render మొదలవుతుంది, కానీ ప్రతి 5ms తర్వాత `shouldYield()` check.
3. User టైప్ చేస్తే — ఆ update **SyncLane** → React transition render ని **వదిలేసి** (throw away), input update ని ముందు చేసి, ఆ తర్వాత transition ని **మొదటి నుండి** మళ్ళీ మొదలుపెడుతుంది.
4. అందుకే **render pure ఉండాలి** — పని పారేయబడొచ్చు.

`<Offscreen>` / Activity (భవిష్యత్తు)

Component ని unmount చేయకుండా "దాచడం" — state, DOM నిలిచి ఉంటాయి కానీ render తక్కువ priority లో. Tab switching, back-navigation restore కి. (React 19 లో experimental `<Activity>`.)

Gotchas (సాధారణ తప్పులు)

- **"Concurrent mode ON చేస్తే app వేగం అవుతుంది"** — కాదు; total work అదే, **responsiveness** మెరుగవుతుంది.
- `createRoot` **వాడకపోవడం** → concurrent features silent గా పని చేయవు.
- **External store ని** `useEffect` **+** `useState` **తో subscribe చేయడం** → tearing + stale reads. `useSyncExternalStore` వాడాలి.
- **Impure components** → concurrent లో duplicate side-effects, inconsistent UI.
- **అన్ని updates ని transition లో పెట్టడం** → input lag (controlled inputs ఎప్పుడూ urgent).

Key Points

- **Lanes** = bitmask priority model; 31 lanes; bitwise ops తో వేగం.
- Transitions **interruptible**; sync input ఎప్పుడూ ముందు.
- **Starvation** ని expiration తో ఆపుతుంది.
- **Tearing** = external store + concurrent render → `useSyncExternalStore`.
- Concurrent = responsiveness, throughput కాదు.

Interview దృష్టి

- *"Concurrent rendering అంటే ఏమిటి?"* → interruptible rendering with priorities; mode కాదు, features.
- *"Tearing అంటే? ఎలా అవుతారు?"* → external store consistency; `useSyncExternalStore`.
- *"Lane model ఎందుకు?"* → multiple priorities ని ఒకే bitmask లో, వేగవంతమైన checks, batching per lane.

---

31. Suspense & React.lazy (Code Splitting)

REACT.LAZY & SUSPENSE · BUNDLE ని విడగొట్టడం

వివరణ

`Suspense` = "ఈ subtree ఇంకా సిద్ధంగా లేదు – అది ready అయ్యేవరకు ఈ fallback చూపించు" అని చెప్పే declarative boundary.

```
<Suspense fallback={<Skeleton />}>
  <SlowComponent />
</Suspense>
```

Suspense ఏం కోసం: (1) **code splitting** (`React.lazy`), (2) **data fetching** (RSC, `use()`, Relay, TanStack Query with `useSuspenseQuery`), (3) **streaming SSR**.

Real-life Scenario

> **Suspense = restaurant లో starter.** మీ main course (heavy component) వండటానికి సమయం పడుతుంది. Waiter మిమ్మల్ని ఖాళీగా కూర్చోబెట్టడు – starter (fallback skeleton) ఇస్తాడు. Main course సిద్ధమైన క్షణం swap చేస్తాడు. **Streaming SSR = ఒక్కో dish సిద్ధమైన కొద్దీ table కి పంపడం**, అన్నీ సిద్ధమయ్యేదాకా ఆగకుండా.

Code splitting తో `React.lazy`

```
import { lazy, Suspense } from "react";

// ⚠️ default export ఉండాలి; dynamic import() bundler కి "ఇక్కడ split చేయి" అని చెప్తుంది
const Dashboard = lazy(() => import("./pages/Dashboard"));
const Settings = lazy(() => import("./pages/Settings"));

// Named export అయితే:
const Chart = lazy(() => import("./Chart").then((m) => ({ default: m.Chart })));

function App() {
  return (
    <Routes>
      <Route path="/dashboard" element={
        <Suspense fallback={<PageSkeleton />}><Dashboard /></Suspense>
      } />
    </Routes>
  );
}
```

ఏం జరుగుతుంది: bundler (Vite/webpack) `Dashboard` code ని వేరే chunk file గా విడగొడుతుంది. User `/dashboard` కి వెళ్ళినప్పుడే ఆ chunk network నుండి download అవుతుంది → **initial bundle చిన్నది** → **first load వేగం.**

ఎక్కడ split చేయాలి

```
// ✅ 1. Route level — అత్యధిక ప్రయోజనం, తక్కువ శ్రమ
const Admin = lazy(() => import("./Admin"));

// ✅ 2. Heavy libraries — chart, editor, map, PDF viewer
const Editor = lazy(() => import("./RichTextEditor"));   // 500KB library

// ✅ 3. Conditional/rare UI — modals, dialogs, onboarding
{showModal && <Suspense fallback={null}><HeavyModal /></Suspense>}

// ✅ 4. Below-the-fold content
const Comments = lazy(() => import("./Comments"));

// ❌ చిన్న components ని split చేయడం — network round-trip overhead > benefit
```

Preloading – lazy యొక్క latency ని దాచడం

```
const Settings = lazy(() => import("./Settings"));

// Hover/focus లో preload — user click చేసేసరికి chunk సిద్ధం
<Link
  to="/settings"
  onMouseEnter={() => import("./Settings")}     // import() cache అవుతుంది — రెండోసారి
network hit లేదు
  onFocus={() => import("./Settings")}
>Settings</Link>
```

Suspense boundary placement – UX కీలకం

```
// ❌ ఒకే పెద్ద boundary — ఏ ఒక్కటి నెమ్మదైనా మొత్తం page skeleton
<Suspense fallback={<FullPageSpinner />}>
  <Header /><Sidebar /><Feed /><Trending />
</Suspense>

// ✅ granular boundaries — ప్రతి భాగం సిద్ధమైనప్పుడు కనిపిస్తుంది
<Header />                                                    {/* వెంటనే */}
<Suspense fallback={<SidebarSkeleton />}><Sidebar /></Suspense>
<Suspense fallback={<FeedSkeleton />}><Feed /></Suspense>
<Suspense fallback={<TrendingSkeleton />}><Trending /></Suspense>
```

Suspense + Transitions – "unwanted fallback" సమస్య

```
// ❌ Tab మారినప్పుడు ఇప్పటికే కనిపిస్తున్న content పోయి spinner వస్తుంది (jarring)
const onTabChange = (tab) => setTab(tab);

// ✅ transition లో పెడితే — పాత content కనిపిస్తూనే ఉంటుంది, కొత్తది ready అయ్యాక swap
const [isPending, startTransition] = useTransition();
const onTabChange = (tab) => startTransition(() => setTab(tab));
// isPending తో subtle loading indicator (opacity/progress bar) చూపించవచ్చు
```

> **నియమం:** *Initial load* కి fallback సరైనది. *Update* కి transition – పాత content ని ఉంచడం మెరుగైన UX.

Data fetching తో Suspense

```
// 1. React 19 — use()
function Profile({ userPromise }) {
  const user = use(userPromise);           // suspend అవుతుంది
  return <h1>{user.name}</h1>;
}

// 2. TanStack Query
import { useSuspenseQuery } from "@tanstack/react-query";
function Profile({ id }) {
  const { data } = useSuspenseQuery({ queryKey: ["user", id], queryFn: () =>
fetchUser(id) });
  return <h1>{data.name}</h1>;             // loading state అవసరం లేదు — Suspense
handle చేస్తుంది
}

// 3. Next.js App Router — async server component
async function Profile({ id }) {
  const user = await getUser(id);          // server లో await
  return <h1>{user.name}</h1>;
}
```

లోపల ఏం జరుగుతుంది

Component "suspend" అవ్వడం అంటే – render సమయంలో ఒక **promise ని throw** చేయడం (`use()` లోపల ఇదే జరుగుతుంది). React ఆ promise ని పట్టుకొని, దగ్గరలోని Suspense boundary ని కనుగొని fallback render చేస్తుంది. Promise resolve అయ్యాక ఆ subtree ని **retry lane** లో మళ్ళీ render చేస్తుంది.

Gotchas (సాధారణ తప్పులు)

- `lazy` **component కి Suspense boundary లేకపోవడం** → crash ("*A component suspended while responding to synchronous input*").
- **Named export ని** `lazy` **కి నేరుగా ఇవ్వడం** → `default` లేదని error.
- **Render లోపల** `lazy()` **call చేయడం** → ప్రతి render కి కొత్త lazy component → infinite remount. Module scope లో ఉండాలి.
- **అతి-granular splitting** → network waterfall (chunk లోపల chunk).
- **Update లో fallback flash** → `startTransition` వాడాలి.
- **Suspense error handling లేకపోవడం** → chunk load fail (deploy తర్వాత పాత chunk missing) → white screen. ErrorBoundary + retry తప్పనిసరి:

```
<ErrorBoundary fallback={<button onClick={() =>
location.reload()}>Reload</button>}>
  <Suspense fallback={<S />}><LazyPage /></Suspense>
</ErrorBoundary>
```

- `lazy` **ని SSR లో వాడటం** – Next.js లో `next/dynamic` (with `ssr: false` option) మేలు.

Key Points

- `Suspense` = declarative loading boundary; `lazy` = route/component code splitting.
- Boundaries **granular** గా – page-wide spinner కాదు.
- **Transitions** తో fallback flash నివారణ.
- Preload on hover/focus.
- Chunk load failures కి **ErrorBoundary + reload**.

Interview దృష్టి

- *"Code splitting ఎలా చేస్తావు, ఎక్కడ?"* → route-first, heavy libs, modals; measure with bundle analyzer.
- *"Suspense లోపల ఎలా పని చేస్తుంది?"* → promise throw + boundary + retry.
- *"Tab switch లో spinner flash ఎలా ఆపుతావు?"* → `useTransition` + `isPending`.

---

32. Portals

వివరణ

Portal = child ని DOM లో వేరే చోట render చేయడం, కానీ React tree లో అక్కడే ఉంచడం.

```
import { createPortal } from "react-dom";

function Modal({ children, onClose }) {
  return createPortal(
    <div className="overlay" onClick={onClose}>
      <div className="modal" onClick={(e) => e.stopPropagation()}>{children}</div>
    </div>,
    document.getElementById("modal-root")     // DOM లో ఎక్కడ render చేయాలో
  );
}
```

ఎందుకు అవసరం: parent కి `overflow: hidden`, `z-index`, `transform`, లేదా `position: relative` ఉంటే – modal/tooltip/dropdown ఆ container లోపల **కత్తిరించబడతాయి** (clipped). Portal దాన్ని `<body>` కి తీసుకెళ్ళి ఈ CSS జైలు నుండి తప్పిస్తుంది.

Real-life Scenario

> **Portal = apartment నుండి terrace మీద party.** Party మీ ఇంటిది (React tree లో మీ component యొక్క child), కానీ మీ flat లో స్థలం సరిపోదు (`overflow: hidden`), కాబట్టి terrace లో (document.body) పెడతారు. **అతిథులు మాట్లాడేది ఇంకా మీకే వినిపిస్తుంది** (events ఇంకా React parent కి bubble అవుతాయి) – ఇదే portal యొక్క మ్యాజిక్.

Code – production modal (a11y తో)

```
function Modal({ isOpen, onClose, title, children }) {
  const ref = useRef(null);

  // ESC key
  useEffect(() => {
    if (!isOpen) return;
    const onKey = (e) => e.key === "Escape" && onClose();
    document.addEventListener("keydown", onKey);
    return () => document.removeEventListener("keydown", onKey);
  }, [isOpen, onClose]);

  // Body scroll lock
  useEffect(() => {
    if (!isOpen) return;
    const prev = document.body.style.overflow;
    document.body.style.overflow = "hidden";
    return () => { document.body.style.overflow = prev; };
  }, [isOpen]);

  // Focus management – modal తెరిచినప్పుడు focus లోపలికి, మూసినప్పుడు తిరిగి trigger కి
  useEffect(() => {
```

```
    if (!isOpen) return;
    const prevFocused = document.activeElement;
    ref.current?.focus();
    return () => prevFocused?.focus?.();
  }, [isOpen]);

  if (!isOpen) return null;

  return createPortal(
    <div className="overlay" onClick={onClose}>
      <div ref={ref} className="modal" role="dialog" aria-modal="true"
           aria-labelledby="modal-title" tabIndex={-1}
           onClick={(e) => e.stopPropagation()}>
        <h2 id="modal-title">{title}</h2>
        {children}
        <button onClick={onClose} aria-label="Close">×</button>
      </div>
    </div>,
    document.body
  );
}
```

```
<!-- index.html -->
<div id="root"></div>
<div id="modal-root"></div>     <!-- portal target (body కి కూడా render చేయొచ్చు) -->
```

Events portal గుండా bubble అవుతాయి (అత్యంత ముఖ్యం)

```
function Parent() {
  return (
    <div onClick={() => console.log("parent clicked!")}>    {/* ✅ ఇది fire అవుతుంది!
*/}
      <Modal>
        <button>Click me</button>      {/* DOM లో body కింద ఉన్నా */}
      </Modal>
    </div>
  );
}
```

ఎందుకు: React యొక్క synthetic event system **React tree** ఆధారంగా పని చేస్తుంది, DOM tree కాదు. Portal DOM position ని మారుస్తుంది, React tree position ని కాదు. కాబట్టి context, events, error boundaries — అన్నీ normal గా పని చేస్తాయి.

⚠️ **ఇది ఆశ్చర్యకరమైన bug కి కారణం కావొచ్చు:** dropdown ని portal లో పెట్టి, parent లో "outside click closes menu" logic ఉంటే — portal లోపల click కూడా parent కి bubble అయ్యి menu మూసేస్తుంది. `e.stopPropagation()` అవసరం.

Portal వాడకాలు

Modals/dialogs · Tooltips/popovers · Dropdowns & select menus · Toasts/notifications · Context menus · Full-screen overlays, drawers

Gotchas (సాధారణ తప్పులు)

- **Portal target DOM లో లేకపోవడం** → `createPortal(children, null)` crash. Guard: `document.getElementById("x") ?? document.body`.
- **SSR లో `document` access** → Next.js లో crash. `useEffect` లో mount check లేదా `next/dynamic` with `ssr: false`.

```
const [mounted, setMounted] = useState(false);
useEffect(() => setMounted(true), []);
if (!mounted) return null;
return createPortal(...);
```

- **Event bubbling ని మర్చిపోవడం** → అనుకోని outside-click closes.
- **Focus trap లేకపోవడం** → keyboard users modal బయటికి tab చేసేస్తారు (a11y fail). `focus-trap-react` లేదా manual.
- **`aria-modal`, `role="dialog"` లేకపోవడం** → screen readers కి modal అని తెలియదు.
- **Body scroll lock మర్చిపోవడం** → modal వెనుక page scroll అవుతుంది (mobile లో ఘోరం).

Key Points

- `createPortal(children, domNode)` – DOM position మారుతుంది, **React tree position మారదు**.
- Events, context, error boundaries **React tree ప్రకారమే** పని చేస్తాయి.
- `overflow/z-index/transform` clipping సమస్యలకు పరిష్కారం.
- Production modal కి: ESC, scroll lock, focus management, focus trap, ARIA.
- SSR లో mount guard తప్పనిసరి.

Interview దృష్టి

- *"Portal లో event bubble అవుతుందా?"* → అవును, React tree ప్రకారం – favourite trick question.
- *"Modal ఎందుకు portal లో?"* → CSS stacking/clipping contexts.
- *"Accessible modal రాయి"* → machine coding round లో సాధారణం.

---

33. StrictMode

వివరణ

`<StrictMode>` = **development-only** tool. Production build లో **ఏమీ చేయదు** (zero cost). ఇది React యొక్క నియమాలను ఉల్లంఘించే code ని ముందే పట్టుకుంటుంది.

```
createRoot(root).render(<StrictMode><App /></StrictMode>);
// లేదా ఒక subtree కి మాత్రమే
<StrictMode><NewFeature /></StrictMode>
```

StrictMode ఏం చేస్తుంది

తనిఖీ	ఎందుకు
Components ని రెండుసార్లు render చేస్తుంది	Impure render logic (mutation, random, side effects) ని బయటపెట్టడానికి
Effects ని mount → cleanup → mount చేస్తుంది	Cleanup సరిగా రాశారా అని పరీక్షించడానికి
State updater/initializer functions ని రెండుసార్లు call చేస్తుంది	అవి pure గా ఉన్నాయా అని
Deprecated APIs కి warnings	legacy string refs, `findDOMNode`, పాత context
Ref cleanup functions ని double-invoke (React 19)	ref cleanup correctness

Real-life Scenario

> **StrictMode = flight కి ముందు pre-flight check.** Pilot ప్రతి switch ని రెండుసార్లు test చేస్తాడు – ప్రయాణీకులకి ఇది అనవసరమైన ఆలస్యంగా అనిపించొచ్చు (dev లో "నా code రెండుసార్లు run అవుతోంది!"), కానీ గాలిలో problem రాకుండా ఇది కాపాడుతుంది (production bugs).

Code – StrictMode ఏ bugs ని బయటపెడుతుంది

```
// ❌ Impure render – double render లో వెంటనే బయటపడుతుంది
let idCounter = 0;
function Bad() {
  idCounter++;                     // ❌ render లో బయటి variable mutation
  const id = idCounter;            // StrictMode: 1, 2 → ఏదో తప్పు అని స్పష్టం
  return <div id={id} />;
}

// ❌ Props mutation
function BadList({ items }) {
  items.sort((a, b) => a.n - b.n);   // ❌ props mutate – double render లో order
confusion
  return items.map(...);
}
// ✅ [...items].sort(...)

// ❌ Effect cleanup లేకపోవడం – double mount లో duplicate connections కనిపిస్తాయి
useEffect(() => {
  const conn = createConnection(roomId);
  conn.connect();
  // return () => conn.disconnect();  ← ఇది లేకపోతే StrictMode లో రెండు connections!
}, [roomId]);
```

"నా API రెండుసార్లు call అవుతోంది!" – అత్యంత సాధారణ ఫిర్యాదు

```
useEffect(() => { fetch("/api/data").then(setData); }, []);
// Dev + StrictMode: రెండు network calls
```

సరైన స్పందన: ఇది bug కాదు – *"నీ effect idempotent కాదు"* అనే హెచ్చరిక. పరిష్కారాలు:

1. `AbortController` తో cleanup (రెండో call మొదటిదాన్ని cancel చేస్తుంది).
2. TanStack Query వాడటం (deduplication built-in).
3. దీన్ని ignore చేయడం – production లో ఒక్కసారే జరుగుతుంది.

❌ **తప్పు పరిష్కారం:** StrictMode తీసేయడం, లేదా `useRef` "hasFetched" flag పెట్టడం (ఇది నిజమైన cleanup bug ని దాచేస్తుంది).

Gotchas (సాధారణ తప్పులు)

- **StrictMode production లో నెమ్మది చేస్తుందని అనుకోవడం** – dev-only, production లో no-op.
- **Double-invoke ని bug అనుకొని StrictMode తీసేయడం** – నిజమైన bugs దాగిపోతాయి.
- **Third-party library StrictMode-incompatible అయితే** – library ని update చేయాలి, StrictMode ని కాదు (కానీ కొన్నిసార్లు తాత్కాలికంగా ఆ subtree ని బయట పెట్టాల్సి వస్తుంది).
- `console.log` **duplicates** – React వాటిని dev లో గ్రే చేస్తుంది; browser console settings తో filter చేయొచ్చు.

Key Points

- Dev-only; production లో **zero effect**.
- Double render + double effect (mount/cleanup/mount) + double updaters.
- లక్ష్యం: **purity + cleanup correctness** – concurrent/future features కి సన్నద్ధత.
- Effects idempotent గా ఉండాలి.

Interview దృష్టి

- *"StrictMode లో effects ఎందుకు రెండుసార్లు run అవుతాయి?"* → cleanup correctness test; future state-preserving remount కి సన్నద్ధత.
- *"దీన్ని ఎలా fix చేస్తావు?"* → cleanup/abort/query library – flag hack కాదు.

---

34. Refs, Imperative Escape Hatches & DOM Measurement

వివరణ

React declarative – కానీ కొన్ని పనులు DOM తో **imperative** గా మాత్రమే చేయగలం: focus, scroll, text selection, measurement, media playback, canvas, third-party DOM libraries. వీటిని React "escape hatches" అంటుంది.

Focus management (a11y కి కీలకం)

```
// 1. Modal తెరిచినప్పుడు focus లోపలికి, మూసినప్పుడు trigger కి తిరిగి
const triggerRef = useRef(null);
const openModal = () => { triggerRef.current = document.activeElement; setOpen(true);
};
const closeModal = () => { setOpen(false); triggerRef.current?.focus(); };

// 2. Focus trap — Tab modal బయటికి వెళ్ళకూడదు
function useFocusTrap(ref, active) {
  useEffect(() => {
    if (!active || !ref.current) return;
    const sel =
'a[href],button:not([disabled]),input:not([disabled]),select,textarea,
[tabindex]:not([tabindex="-1"])';
    const onKey = (e) => {
      if (e.key !== "Tab") return;
      const nodes = [...ref.current.querySelectorAll(sel)];
      if (!nodes.length) return;
      const first = nodes[0], last = nodes[nodes.length - 1];
      if (e.shiftKey && document.activeElement === first) { e.preventDefault();
last.focus(); }
      else if (!e.shiftKey && document.activeElement === last) { e.preventDefault();
first.focus(); }
    };
    document.addEventListener("keydown", onKey);
    return () => document.removeEventListener("keydown", onKey);
  }, [ref, active]);
}

// 3. Route మారినప్పుడు heading కి focus (screen reader announcement)
useEffect(() => { headingRef.current?.focus(); }, [pathname]);
```

DOM measurement

```
// getBoundingClientRect — layout మీద ఆధారపడుతుంది → useLayoutEffect లో
useLayoutEffect(() => {
  const { width, height, top, left } = ref.current.getBoundingClientRect();
  setSize({ width, height });
}, [deps]);

// ✅ ResizeObserver — resize అయినప్పుడల్లా (window resize event కంటే ఖచ్చితం)
useEffect(() => {
  const el = ref.current;
  if (!el) return;
  const ro = new ResizeObserver(([entry]) => {
    const { width, height } = entry.contentRect;
    setSize({ width, height });
  });
  ro.observe(el);
  return () => ro.disconnect();
}, []);
```

Layout thrashing నివారణ: DOM read (`offsetWidth`, `getBoundingClientRect`) తర్వాత write (style మార్పు), మళ్ళీ read — ఇది forced reflow. **అన్ని reads ముందు, అన్ని writes తర్వాత** చేయాలి.

`flushSync` — batching ని ఉద్దేశపూర్వకంగా bypass చేయడం

```
import { flushSync } from "react-dom";

// కొత్త item add చేసి వెంటనే దానికి scroll చేయాలి
const add = () => {
  flushSync(() => { setItems([...items, newItem]); });   // DOM వెంటనే update
  listRef.current.lastChild.scrollIntoView();            // ఇప్పుడు కొత్త node ఉంది ✅
};
// ⚠️ performance hit — అరుదుగా మాత్రమే; batching ప్రయోజనం పోతుంది
```

Third-party DOM libraries ని integrate చేయడం

```
function Chart({ data }) {
  const ref = useRef(null);
  const chartRef = useRef(null);

  useEffect(() => {                         // 1. mount లో create
    chartRef.current = new ChartLib(ref.current, { data });
    return () => chartRef.current.destroy();   // 2. unmount లో destroy — తప్పనిసరి
  }, []);

  useEffect(() => { chartRef.current?.update(data); }, [data]);   // 3. data మారితే
update

  return <div ref={ref} />;               // React ఈ div లోపల ఏమీ render చేయదు
}
```

> **నియమం:** React నిర్వహించే DOM ని ఎప్పుడూ manually మార్చొద్దు. Third-party library కి **ఖాళీ div** ఇచ్చి, ఆ లోపలిది పూర్తిగా దానికే వదిలేయాలి.

Gotchas

- `ref.current.style.x = ...` తో React-managed element ని మార్చడం → తర్వాతి render లో overwrite.
- **Measurement ని `useEffect` లో చేసి flicker** → `useLayoutEffect`.
- **`flushSync` ని అలవాటుగా వాడటం** → batching ప్రయోజనం నష్టం.
- **Third-party instance ని cleanup చేయకపోవడం** → memory leaks.

Key Points

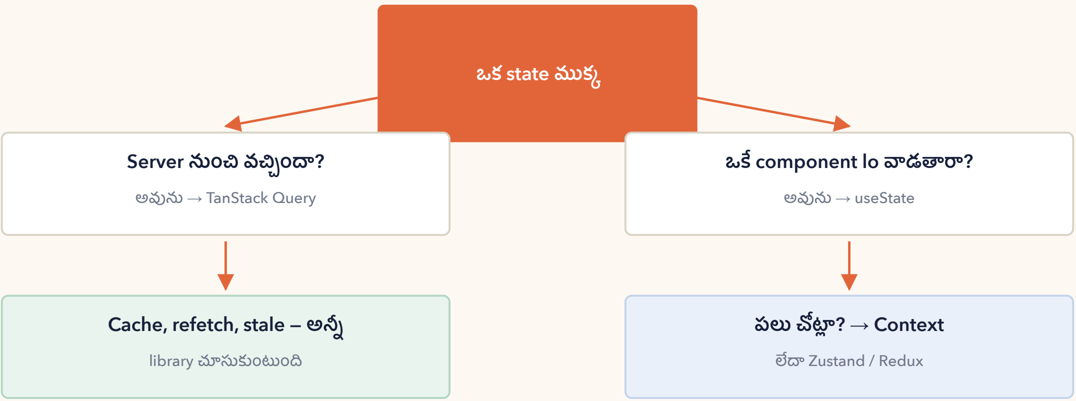
- Escape hatches: focus, scroll, measure, media, canvas, 3rd-party libs.
- Measurement → `useLayoutEffect` / `ResizeObserver`.
- `flushSync` = batching bypass (అరుదుగా).
- 3rd-party DOM libs కి ఖాళీ container + create/update/destroy lifecycle.

Part 5 – State Management (Client & Server)

35. State రకాలు – సరైన Decision Tree

STATE రకాలు · ఏది ఎక్కడ ఉంచాలి

STATE ఎక్కడ ఉండాలి – ఒక నిర్ణయ వృక్షం



అత్యంత సాధారణమైన తప్పు

Server data ని Redux lo పెట్టడం – అప్పుడు caching, refetch, stale, loading, error అన్నీ మీరే రాయాలి. అది ఒక పూర్తి library ని మళ్ళీ కట్టడం. Server state మరియు client state **వేర్వేరు జంతువులు** – వేర్వేరు సాధనాలు.

వివరణ

"State management" లో అతిపెద్ద తప్పు: **అన్ని state ని ఒకేలా చూడటం.** నిజానికి React app లో **6 రకాల state** ఉంటాయి, ప్రతిదానికీ వేరే సరైన సాధనం ఉంది.

రకం	ఉదాహరణ	సరైన సాధనం
1. Local UI state	modal open, input value, hover	`useState` / `useReducer`
2. Lifted/shared state	selected row (2-3 components కి)	parent లో `useState` + props
3. Global client state	theme, auth user, cart, feature flags	Context (అరుదుగా మారితే) / Zustand / Redux
4. Server state	users list, orders, products	**TanStack Query / RTK Query / SWR / RSC**
5. URL state	filters, page number, search query, tab	React Router `useSearchParams`
6. Form state	field values, errors, touched, dirty	React Hook Form / local state

80% "state management" సమస్యలు అసలు #4 (server state) ని #3 (global client state) లా treat చేయడం వల్ల వస్తాయి. Server data ని Redux లో పెట్టి caching/invalidation/loading/error/retry అంతా manual గా రాయడం – వేల lines వృథా.

Real-life Scenario

> **State రకాలు = ఇంట్లో వస్తువులు.** జేబులో డబ్బు (local state – వెంటనే అవసరం), ఇంట్లో లాకర్ (global state – అందరికీ కావాల్సినది), **బ్యాంకులో డబ్బు (server state – అది మీది కాదు, బ్యాంకుది; మీ దగ్గర ఉన్నది కేవలం ఒక *copy*, అది పాతదైపోవచ్చు)**, ఇంటి address (URL state – link share చేస్తే అవతలివారికీ కనిపిస్తుంది).
>
> Server state ని client state లా treat చేయడం = **passbook లోని balance నే నిజమైన balance అనుకోవడం.** ఇంకెవరో డబ్బు తీసేసి ఉండొచ్చు (stale data) – అందుకే refetch, invalidation, cache TTL అవసరం.

Decision Tree

```
ఈ state ఎవరికి అవసరం?
│
├─ ఒకే component కి          → useState / useReducer ✅
│
├─ 2–3 దగ్గరి components కి  → lift state up (సమీప common parent) ✅
│
├─ URL లో ఉండాలా? (share/bookmark/back button)
│   → useSearchParams ✅ (filters, page, sort, tab, search query)
│
├─ Server నుండి వచ్చిందా?
│   → TanStack Query / RTK Query / RSC ✅ (Redux/Context కాదు!)
│
└─ నిజంగా global client state?
    ├─ అరుదుగా మారుతుందా (theme, locale, auth)  → Context ✅
    └─ తరచూ మారుతుందా / complex               → Zustand / Redux Toolkit / Jotai ✅
```

Server state ఎందుకు ప్రత్యేకం

Client state	Server state
మనకి పూర్తి యజమాన్యం	**Remote – ఇంకొకరు మార్చొచ్చు**
ఎప్పుడూ up-to-date	**Stale అవుతుంది**
Synchronous	Asynchronous
Loading/error లేవు	Loading, error, retry, refetch అవసరం
Cache అవసరం లేదు	**Caching, dedup, invalidation తప్పనిసరి**

URL state – తక్కువ వాడే శక్తివంతమైన సాధనం

```
import { useSearchParams } from "react-router-dom";

function ProductList() {
  const [params, setParams] = useSearchParams();
  const page = Number(params.get("page") ?? 1);
  const category = params.get("category") ?? "all";
  const sort = params.get("sort") ?? "newest";

  const setPage = (p) => setParams((prev) => { prev.set("page", String(p)); return
prev; });
  // ✅ లాభాలు: shareable link, bookmark, browser back/forward, refresh లో state
నిలుస్తుంది,
  //    SSR లో server కి కూడా తెలుస్తుంది, analytics లో కనిపిస్తుంది
}
```

> **నియమం:** *"User ఈ view ని ఒక link గా పంపగలగాలా?"* – అవును అయితే **URL state**.

State colocation principle

```
// ❌ అన్నీ App లో (అనవసర re-renders + prop drilling)
function App() {
  const [modalOpen, setModalOpen] = useState(false);      // ఒకే చోట వాడతారు!
  const [tooltipText, setTooltipText] = useState("");
  return <Layout modalOpen={modalOpen} ... />;
}

// ✅ State ని వాడే చోటికి దగ్గరగా తరలించండి
function DeleteButton() {
  const [modalOpen, setModalOpen] = useState(false);      // ఇక్కడే ఉండాలి
}
```

"Lift state up" ని అతిగా అన్వయించడం – ఇదే React apps నెమ్మది అవ్వడానికి #1 కారణం. నియమం: **అవసరమైనంత పైకి మాత్రమే, ఆ తర్వాత వెంటనే కిందికి.**

Key Points

- **6 రకాల state** – ఒక్కో దానికి ఒక్కో సాధనం.
- **Server state ≠ client state** – query library వాడాలి.
- **URL state** = shareable/bookmarkable UI state.
- **Colocation** – వీలైనంత కిందికి; lift అవసరమైనప్పుడే.
- Global store అనేది **చివరి ఎంపిక**, మొదటిది కాదు.

Interview దృష్టి

- *"State management ఎలా ఎంచుకుంటావు?"* → పై decision tree చెప్తే వెంటనే senior signal.
- *"Redux ఎప్పుడు అవసరం లేదు?"* → server state మాత్రమే అయితే; Context+reducer సరిపోతే; app చిన్నదైతే.

- *"Filters ని ఎక్కడ ఉంచుతావు?"* → URL – sharing, back button, SSR కారణాలతో.

---

36. Context + useReducer Architecture (library లేకుండా global state)

వివరణ

చిన్న/మధ్యస్థ apps కి Redux అవసరం లేదు – `useReducer` + Context కలిస్తే **అదే mental model** (actions, reducers, dispatch) zero dependencies తో వస్తుంది.

Code – పూర్తి cart store

```
// store/CartContext.jsx
const CartStateContext = createContext(null);
const CartDispatchContext = createContext(null);

const initialState = { items: [], coupon: null };

function cartReducer(state, action) {
  switch (action.type) {
    case "add": {
      const existing = state.items.find((i) => i.id === action.item.id);
      return existing
        ? { ...state, items: state.items.map((i) =>
              i.id === action.item.id ? { ...i, qty: i.qty + 1 } : i) }
        : { ...state, items: [...state.items, { ...action.item, qty: 1 }] };
    }
    case "remove":
      return { ...state, items: state.items.filter((i) => i.id !== action.id) };
    case "setQty":
      return { ...state, items: action.qty <= 0
        ? state.items.filter((i) => i.id !== action.id)
        : state.items.map((i) => i.id === action.id ? { ...i, qty: action.qty } : i)
};
    case "applyCoupon": return { ...state, coupon: action.coupon };
    case "clear":       return initialState;
    default: throw new Error(`Unknown action: ${action.type}`);
  }
}

export function CartProvider({ children }) {
  const [state, dispatch] = useReducer(cartReducer, initialState, (init) => {
    try { return JSON.parse(localStorage.getItem("cart")) ?? init; } catch { return
init; }
  });

  useEffect(() => { localStorage.setItem("cart", JSON.stringify(state)); }, [state]);

  return (
    <CartStateContext.Provider value={state}>
      <CartDispatchContext.Provider value={dispatch}>{children}
</CartDispatchContext.Provider>
    </CartStateContext.Provider>
```

```
  );
}

// Selector-ish hooks — consumers కి కావాల్సినది మాత్రమే
export function useCart() {
  const ctx = useContext(CartStateContext);
  if (!ctx) throw new Error("useCart must be inside CartProvider");
  return ctx;
}
export function useCartDispatch() {
  const ctx = useContext(CartDispatchContext);
  if (!ctx) throw new Error("useCartDispatch must be inside CartProvider");
  return ctx;
}
// Derived values — state లో duplicate చేయకూడదు
export function useCartTotal() {
  const { items, coupon } = useCart();
  return useMemo(() => {
    const sub = items.reduce((s, i) => s + i.price * i.qty, 0);
    return coupon ? sub * (1 - coupon.percent / 100) : sub;
  }, [items, coupon]);
}
```

ఈ approach యొక్క పరిమితులు (Redux/Zustand ఎప్పుడు అవసరం)

పరిమితి	వివరణ
Selector-based subscription లేదు	State లో ఏ field మారినా అన్ని consumers re-render (Redux `useSelector` లా granular కాదు)
Middleware లేదు	Logging, persistence, analytics ని manual గా రాయాలి
DevTools లేవు	Time-travel debugging, action inspection లేవు
Async patterns	Thunks/sagas లేవు – effects ని components లో రాయాలి
Provider nesting	5-6 stores అయితే provider pyramid

Provider pyramid ని చక్కబెట్టడం

```
// ❌ nesting hell
<ThemeProvider><AuthProvider><CartProvider><ToastProvider><App /></ToastProvider>
</CartProvider></AuthProvider></ThemeProvider>

// ✅ compose helper
const compose = (...providers) => ({ children }) =>
  providers.reduceRight((acc, P) => <P>{acc}</P>, children);

const AppProviders = compose(ThemeProvider, AuthProvider, CartProvider,
ToastProvider);
<AppProviders><App /></AppProviders>
```

Key Points

- `useReducer` + Context = zero-dependency global state, Redux mental model.
- **State/dispatch విభజన** తప్పనిసరి (re-render తగ్గించడానికి).
- Derived values ని hooks లో `useMemo` తో – state లో duplicate వద్దు.
- పరిమితులు: selectors, middleware, devtools, async patterns లేవు.

Interview దృష్టి

- *"Redux ని Context+useReducer తో replace చేయొచ్చా?"* → చిన్న apps కి అవును; granular subscriptions/middleware/devtools కావాలంటే కాదు.

---

37. Redux Toolkit (RTK) Deep

వివరణ

Redux = predictable state container. మూడు సూత్రాలు: (1) **single source of truth** (ఒకే store), (2) **state read-only** (actions ద్వారానే మారుతుంది), (3) **pure reducers** తో మార్పులు.

Redux Toolkit (RTK) = 2026 లో Redux రాయడానికి **అధికారిక, ఏకైక సిఫార్సు** పద్ధతి. పాత "vanilla Redux" (switch reducers, action type constants, `combineReducers`, manual thunk setup) ఇప్పుడు రాయకూడదు.

Real-life Scenario

> **Redux = బ్యాంకు ledger.** ఎవరూ నేరుగా balance ని మార్చలేరు. ప్రతి మార్పుకి ఒక **voucher (action)** ఇవ్వాలి – "deposit ₹500". Ledger keeper (reducer) నియమాల ప్రకారం కొత్త balance రాస్తాడు. **ప్రతి voucher record అవుతుంది** → ఏ మార్పు ఎప్పుడు ఎందుకు జరిగిందో ట్రేస్ చేయొచ్చు (Redux DevTools time-travel). ఇదే Redux యొక్క అసలు విలువ – **auditability**, boilerplate కాదు.

Code – పూర్తి RTK setup

```
// store/cartSlice.js
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";

export const checkout = createAsyncThunk(
  "cart/checkout",
  async (payload, { rejectWithValue, getState, dispatch }) => {
    try {
      const { cart } = getState();
      return await api.checkout(cart.items);
    } catch (err) {
      return rejectWithValue(err.response?.data ?? err.message);
    }
  }
);
```

```
const cartSlice = createSlice({
  name: "cart",
  initialState: { items: [], status: "idle", error: null },
  reducers: {
    // ⚡ Immer వల్ల "mutation" రాయొచ్చు — RTK లోపల immutable copy చేస్తుంది
    itemAdded(state, action) {
      const existing = state.items.find((i) => i.id === action.payload.id);
      if (existing) existing.qty++;                    // ✅ నిజంగా mutate కాదు (immer
draft)
      else state.items.push({ ...action.payload, qty: 1 });
    },
    itemRemoved(state, action) {
      state.items = state.items.filter((i) => i.id !== action.payload);
    },
    // Prepare callback — action payload ని customize చేయడానికి
    itemAddedWithId: {
      reducer(state, action) { state.items.push(action.payload); },
      prepare(item) { return { payload: { ...item, id: nanoid(), qty: 1 } }; },
    },
  },
  extraReducers: (builder) => {
    builder
      .addCase(checkout.pending,   (s) => { s.status = "loading"; s.error = null; })
      .addCase(checkout.fulfilled, (s) => { s.status = "idle"; s.items = []; })
      .addCase(checkout.rejected,  (s, a) => { s.status = "failed"; s.error =
a.payload; });
  },
});

export const { itemAdded, itemRemoved } = cartSlice.actions;   // action creators
auto-generated
export default cartSlice.reducer;

// store/index.js
import { configureStore } from "@reduxjs/toolkit";
export const store = configureStore({
  reducer: { cart: cartReducer, auth: authReducer },
  // DevTools, thunk, immutability & serializability checks — అన్నీ default ✅
  middleware: (getDefault) => getDefault().concat(loggerMiddleware),
});
```

```
// Component లో వాడకం
import { useSelector, useDispatch } from "react-redux";

function Cart() {
  const items = useSelector((s) => s.cart.items);          // ⚠️ narrow selector
  const status = useSelector((s) => s.cart.status);
  const dispatch = useDispatch();

  return (
    <>
      {items.map((i) => (
        <Row key={i.id} item={i} onRemove={() => dispatch(itemRemoved(i.id))} />
      ))}
      <button disabled={status === "loading"} onClick={() =>
dispatch(checkout())}>Checkout</button>
```

```
    </>
  );
}
```

Selectors & memoization

```
// ❌ ప్రతి render కి కొత్త array/object → ప్రతిసారి re-render
const expensive = useSelector((s) => s.cart.items.filter((i) => i.price > 1000));

// ✅ createSelector (reselect) — inputs మారితేనే recompute
import { createSelector } from "@reduxjs/toolkit";
export const selectExpensiveItems = createSelector(
  [(s) => s.cart.items],
  (items) => items.filter((i) => i.price > 1000)
);
const expensive = useSelector(selectExpensiveItems);   // ✅ memoized

// ✅ లేదా shallowEqual
import { shallowEqual } from "react-redux";
const { a, b } = useSelector((s) => ({ a: s.x.a, b: s.x.b }), shallowEqual);
```

Normalization – nested data ని flat చేయడం

```
// ❌ nested — update చేయాలంటే deep clone నరకం, duplicates
{ posts: [{ id: 1, author: { id: 9, name: "A" }, comments: [{...}] }] }

// ✅ normalized (database style)
{
  posts:    { byId: { 1: { id: 1, authorId: 9, commentIds: [5, 6] } }, allIds: [1] },
  users:    { byId: { 9: { id: 9, name: "A" } } },
  comments: { byId: { 5: {...}, 6: {...} } },
}

// RTK createEntityAdapter ఇది automatic గా చేస్తుంది
import { createEntityAdapter } from "@reduxjs/toolkit";
const postsAdapter = createEntityAdapter({ sortComparer: (a, b) =>
b.date.localeCompare(a.date) });
const postsSlice = createSlice({
  name: "posts",
  initialState: postsAdapter.getInitialState({ status: "idle" }),
  reducers: {
    postAdded: postsAdapter.addOne,
    postsReceived: postsAdapter.setAll,
    postUpdated: postsAdapter.updateOne,
  },
});
export const { selectAll: selectAllPosts, selectById: selectPostById } =
  postsAdapter.getSelectors((s) => s.posts);
```

RTK Query – server state కి Redux పరిష్కారం

```
import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";

export const api = createApi({
```

```
  reducerPath: "api",
  baseQuery: fetchBaseQuery({
    baseUrl: "/api",
    prepareHeaders: (headers, { getState }) => {
      const token = getState().auth.token;
      if (token) headers.set("authorization", `Bearer ${token}`);
      return headers;
    },
  }),
  tagTypes: ["Post"],
  endpoints: (builder) => ({
    getPosts: builder.query({
      query: (page = 1) => `/posts?page=${page}`,
      providesTags: (result) =>
        result ? [...result.map(({ id }) => ({ type: "Post", id })), "Post"] :
["Post"],
    }),
    addPost: builder.mutation({
      query: (body) => ({ url: "/posts", method: "POST", body }),
      invalidatesTags: ["Post"],           // ⚡ automatic refetch
    }),
  }),
});
export const { useGetPostsQuery, useAddPostMutation } = api;   // hooks auto-
generated!

// Component
const { data, isLoading, isError, refetch } = useGetPostsQuery(page);
const [addPost, { isLoading: adding }] = useAddPostMutation();
```

Redux ఎప్పుడు అవసరం / అవసరం లేదు

✅ అవసరం	❌ అవసరం లేదు
పెద్ద team, complex shared state	Server data మాత్రమే (→ TanStack Query)
Time-travel debugging కావాలి	చిన్న app (→ useState/Context)
Complex async flows (sagas)	Simple global state (→ Zustand)
State transitions ని audit చేయాలి	Prop drilling సమస్య మాత్రమే (→ composition)
Offline sync, optimistic queues	

Gotchas (సాధారణ తప్పులు)

- **Vanilla Redux boilerplate రాయడం** – RTK వాడాలి (official).
- **Non-serializable values ని store లో** (Date, Map, functions, class instances) → time-travel/persistence పాడవుతుంది (RTK warning ఇస్తుంది).
- **`useSelector` లో మొత్తం state select చేయడం** – `useSelector(s => s)` ❌ ప్రతి action కి re-render.
- **Selector లో కొత్త object/array return** → memoization లేకపోతే infinite re-renders.
- **Server data ని manually Redux లో cache చేయడం** – RTK Query లేదా TanStack Query వాడాలి.

- **Immer draft ని return + mutate రెండూ చేయడం** – ఒకటే చేయాలి.
- **`createAsyncThunk` లో error ని swallow చేయడం** – `rejectWithValue` వాడాలి.

Key Points

- **RTK = official Redux** (createSlice, configureStore, thunks, immer, devtools built-in).
- Immer వల్ల mutation-style syntax, లోపల immutable.
- **Selectors** ని `createSelector` తో memoize.
- **Normalization** (`createEntityAdapter`) పెద్ద data కి.
- **RTK Query** = Redux ecosystem లో server state పరిష్కారం.
- Redux యొక్క నిజమైన విలువ = **predictability + devtools + audit**, boilerplate కాదు.

Interview దృష్టి

- *"Redux ఎలా పని చేస్తుంది?"* → action → dispatch → middleware → reducer → new state → subscribed components re-render.
- *"RTK ఏం పరిష్కరించింది?"* → boilerplate, immutability errors, thunk/devtools setup, non-serializable detection.
- *"Redux vs Context?"* → subscription granularity, middleware, devtools, async patterns.

---

38. Zustand, Jotai, Recoil, MobX, Signals

వివరణ – ఆధునిక ప్రత్యామ్నాయాలు

Zustand (అత్యంత ప్రాచుర్యం 2026) – చిన్నది (~1KB), boilerplate లేదు, provider అవసరం లేదు, selector-based subscriptions.

```
import { create } from "zustand";
import { persist, devtools } from "zustand/middleware";

const useCartStore = create(
  devtools(persist(
    (set, get) => ({
      items: [],
      addItem: (item) => set((s) => {
        const existing = s.items.find((i) => i.id === item.id);
        return existing
          ? { items: s.items.map((i) => i.id === item.id ? { ...i, qty: i.qty + 1 } :
i) }
          : { items: [...s.items, { ...item, qty: 1 }] };
      }),
      removeItem: (id) => set((s) => ({ items: s.items.filter((i) => i.id !== id)
})),
      clear: () => set({ items: [] }),
      // Async actions నేరుగా – thunk అవసరం లేదు
      checkout: async () => {
        set({ status: "loading" });
        try { await api.checkout(get().items); set({ items: [], status: "idle" }); }
```

```
        catch (e) { set({ status: "error", error: e.message }); }
      },
      // Derived — getter
      get total() { return get().items.reduce((s, i) => s + i.price * i.qty, 0); },
    }),
    { name: "cart-storage" }        // localStorage key
  ))
);

// Component — Provider అవసరం లేదు! ⚡ selector ఇచ్చిన field మారితేనే re-render
function Cart() {
  const items = useCartStore((s) => s.items);
  const addItem = useCartStore((s) => s.addItem);      // actions stable
  return <>{items.length}</>;
}
// ⚠️ useCartStore((s) => ({ a: s.a, b: s.b })) → కొత్త object ప్రతిసారి → useShallow వాడాలి
import { useShallow } from "zustand/react/shallow";
const { a, b } = useCartStore(useShallow((s) => ({ a: s.a, b: s.b })));
```

Jotai — atomic state (Recoil స్ఫూర్తి), bottom-up. చిన్న చిన్న atoms ని కలిపి derived atoms.

```
import { atom, useAtom } from "jotai";

const countAtom = atom(0);
const doubledAtom = atom((get) => get(countAtom) * 2);           // derived (read-
only)
const asyncAtom = atom(async (get) => (await fetch(`/api/${get(idAtom)}`)).json());
// async!

function Counter() {
  const [count, setCount] = useAtom(countAtom);
  const [doubled] = useAtom(doubledAtom);               // countAtom మారితేనే re-render
  return <button onClick={() => setCount((c) => c + 1)}>{count} / {doubled}</button>;
}
```

MobX — observable/reactive, OOP style, automatic dependency tracking.

```
import { makeAutoObservable } from "mobx";
import { observer } from "mobx-react-lite";

class CartStore {
  items = [];
  constructor() { makeAutoObservable(this); }
  add(item) { this.items.push(item); }                 // ✅ direct mutation OK (MobX
తో)
  get total() { return this.items.reduce((s, i) => s + i.price, 0); }
}
const cart = new CartStore();
const Cart = observer(() => <div>{cart.total}</div>);  // వాడిన observables మారితేనే re-
render
```

Signals (Preact Signals, Solid, Angular) — fine-grained reactivity; value మారితే **ఆ ఒక్క DOM node** మాత్రమే update (component re-render లేదు). React లో ఇంకా native కాదు (React Compiler వేరే

మార్గం ఎంచుకుంది).

పోలిక (interview cheat sheet)

	Redux Toolkit	Zustand	Jotai	MobX	Context
Bundle	~12KB	~1KB	~3KB	~16KB	0
Boilerplate	మధ్యస్థం	చాలా తక్కువ	తక్కువ	తక్కువ	మధ్యస్థం
Provider అవసరమా	✅	❌	optional	❌	✅
Selector subscriptions	✅	✅	✅ (atom-level)	✅ (auto)	❌
DevTools	✅ అత్యుత్తమం	✅ (redux devtools)	✅	✅	❌
Learning curve	ఎక్కువ	తక్కువ	తక్కువ	మధ్యస్థం	తక్కువ
Immutability	తప్పనిసరి (immer)	తప్పనిసరి	తప్పనిసరి	అవసరం లేదు	తప్పనిసరి
ఎప్పుడు	పెద్ద team/app, audit	చాలా apps కి default ✅	atomic/derived-heavy	OOP teams	theme/auth

ఎంపిక సూత్రం (2026)

```
Server data          → TanStack Query (లేదా RTK Query / RSC)
కొద్దిపాటి global    → Zustand ✅
అరుదుగా మారేది       → Context
పెద్ద enterprise     → Redux Toolkit
Atomic/derived heavy → Jotai
```

Gotchas

- **Zustand selector లో object return** → `useShallow` లేకపోతే ప్రతిసారి re-render.
- **Zustand store ని module scope లో create చేయడం** – SSR (Next.js) లో requests మధ్య state share అవుతుంది! Per-request store + provider వాడాలి.
- **Jotai atom ని component లోపల create చేయడం** → ప్రతి render కి కొత్త atom.
- **MobX లో `observer` మర్చిపోవడం** → component update కాదు.
- **అనేక state libraries ఒకే app లో** – team confusion; ఒకటి ఎంచుకొని నిలబడాలి.

Key Points

- **Zustand** = 2026 లో చాలా apps కి sweet spot (చిన్నది, provider-free, selectors).
- **Jotai** = atomic, derived state కి సొగసైనది.
- **MobX** = mutation-friendly, auto-tracking, OOP.

- **RTK** = enterprise scale, devtools, middleware.
- **Signals** = fine-grained reactivity (React లో native కాదు).
- Server state కి **ఏ client state library వాడకూడదు**.

Interview దృష్టి

- *"Redux బదులు Zustand ఎందుకు?"* → boilerplate, bundle, provider-free, selector simplicity; audit/middleware అవసరమైతే Redux.
- *"Zustand ఎలా re-renders ఆపుతుంది?"* → `useSyncExternalStore` + selector comparison.

---

39. TanStack Query (React Query) Deep

SERVER STATE VS CLIENT STATE · ఎందుకు వేరే సాధనాలు

SERVER STATE vs CLIENT STATE

Server state

మీది కాదు – server దగ్గర ఉంది
పాతబడిపోతుంది (stale)
Async · ఇతరులు మార్చొచ్చు
→ TanStack Query, SWR

Client state

పూర్తిగా మీది
ఎప్పుడూ తాజాదే
Sync · మీరే మార్చేది
→ useState, Zustand

TanStack Query ఏం ఇస్తుంది

Cache + stale-while-revalidate · deduplication (ఒకే key కి ఒకే request)
Background refetch · retry · pagination · optimistic update
ఇవన్నీ చేతితో రాస్తే – అదే ఒక library.

వివరణ

TanStack Query = server state management library. ఇది "data fetching library" కాదు – fetch ని మీరే ఇస్తారు (`fetch` , `axios` , GraphQL). ఇది **cache, deduplication, background refetch, stale-while-revalidate, retry, pagination, optimistic updates** అన్నీ నిర్వహిస్తుంది.

```
const { data, isPending, isError, error, refetch, isFetching } = useQuery({
  queryKey: ["users", { page, filter }],       // cache key — deps array లాంటిది
  queryFn: ({ signal }) => fetch(`/api/users?page=${page}`, { signal }).then(r =>
r.json()),
  staleTime: 5 * 60 * 1000,                    // 5 నిమిషాలు "fresh" — refetch చేయదు
  gcTime: 10 * 60 * 1000,                      // unused అయ్యాక 10 నిమిషాల్లో cache నుండి
తొలగింపు
});
```

Real-life Scenario

> **React Query = ఇంట్లో fridge.** ప్రతిసారి కూరగాయలు కావాలంటే market కి (server) పరిగెత్తరు — fridge లో ఉంటే అదే వాడతారు (**cache hit — instant**). కానీ **fridge లోది పాతదైపోతుంది** — కాబట్టి "ఇది 5 నిమిషాల్లోపు తెచ్చినదైతే fresh" (`staleTime`) అనే నియమం. Stale అయితే — మీకు fridge లోది వెంటనే ఇస్తుంది (UI ఖాళీగా ఉండదు), **అదే సమయంలో background లో మార్కెట్ కి వెళ్ళి తాజాది తెచ్చి replace చేస్తుంది** (stale-while-revalidate). ఇద్దరు ఒకేసారి అడిగితే ఒకేసారి వెళ్తుంది (**deduplication**).

`staleTime` vs `gcTime` — అత్యంత ముఖ్యమైన concept

	`staleTime` **(default: 0)**	`gcTime` **(default: 5 min)**
అర్థం	Data ఎంతసేపు "fresh"	Unused data cache లో ఎంతసేపు ఉంటుంది
ప్రభావం	ఈ సమయంలో **refetch జరగదు**	తర్వాత memory నుండి తొలగింపు
Default 0 అంటే	ప్రతి mount/focus కి refetch	

```
staleTime: 0                    // ఎప్పుడూ stale — mount/focus/reconnect లో refetch
(default)
staleTime: 60_000               // 1 నిమిషం fresh — ఆ లోపల refetch లేదు
staleTime: Infinity             // ఎప్పటికీ stale కాదు — manual invalidation మాత్రమే
```

> **Interview గోల్డ్:** " `staleTime` controls *refetching*, `gcTime` controls *memory*. A common mistake is leaving `staleTime: 0` on rarely-changing data, causing refetch storms on every focus."

Query lifecycle

```
Mount → cache లో ఉందా?
  ├─ లేదు       → queryFn run (isPending: true) → success/error
  └─ ఉంది       → cached data వెంటనే return (UI instant!)
      ├─ fresh (staleTime లోపల)  → ఏమీ చేయదు
      └─ stale                   → cached చూపిస్తూ background refetch (isFetching:
true)
```

`isPending` vs `isFetching` — `isPending` = data ఏమీ లేదు (మొదటిసారి); `isFetching` = ఏదైనా fetch నడుస్తోంది (background refetch సహా). Skeleton కి `isPending`, subtle spinner కి `isFetching`.

Mutations + invalidation

```
const queryClient = useQueryClient();

const { mutate, isPending } = useMutation({
```

```
  mutationFn: (newTodo) => api.post("/todos", newTodo),
  onSuccess: (data, variables) => {
    queryClient.invalidateQueries({ queryKey: ["todos"] });   // ⚡ refetch trigger
    // లేదా cache ని నేరుగా update (network call లేకుండా)
    queryClient.setQueryData(["todos"], (old) => [...old, data]);
  },
  onError: (error, variables, context) => toast.error(error.message),
});

mutate({ title: "Learn React Query" });
```

Optimistic updates (production pattern)

```
const { mutate } = useMutation({
  mutationFn: (updated) => api.patch(`/todos/${updated.id}`, updated),

  onMutate: async (updated) => {
    await queryClient.cancelQueries({ queryKey: ["todos"] });     // 1. in-flight
refetch ఆపు
    const previous = queryClient.getQueryData(["todos"]);         // 2. snapshot
(rollback కోసం)
    queryClient.setQueryData(["todos"], (old) =>                  // 3. UI ని వెంటనే
update
      old.map((t) => (t.id === updated.id ? { ...t, ...updated } : t))
    );
    return { previous };                                          // context గా return
  },
  onError: (err, updated, context) => {
    queryClient.setQueryData(["todos"], context.previous);        // 4. fail అయితే
rollback
    toast.error("Update failed");
  },
  onSettled: () => queryClient.invalidateQueries({ queryKey: ["todos"] }),  // 5.
server truth
});
```

Pagination & Infinite scroll

```
// Pagination — placeholderData తో "flicker-free" page changes
const { data, isPlaceholderData } = useQuery({
  queryKey: ["projects", page],
  queryFn: () => fetchProjects(page),
  placeholderData: keepPreviousData,     // కొత్త page load అయ్యేవరకు పాతది చూపిస్తుంది
});

// Infinite scroll
const { data, fetchNextPage, hasNextPage, isFetchingNextPage } = useInfiniteQuery({
  queryKey: ["feed"],
  queryFn: ({ pageParam }) => fetchFeed(pageParam),
  initialPageParam: 0,
  getNextPageParam: (lastPage) => lastPage.nextCursor ?? undefined,   // undefined =
ఇక పేజీలు లేవు
});
const items = data?.pages.flatMap((p) => p.items) ?? [];
```

Dependent & parallel queries

```
// Dependent (waterfall — తప్పనిసరి అయినప్పుడే)
const { data: user } = useQuery({ queryKey: ["user", email], queryFn: () =>
getUser(email) });
const { data: projects } = useQuery({
  queryKey: ["projects", user?.id],
  queryFn: () => getProjects(user.id),
  enabled: !!user?.id,                  // 🔑 user వచ్చేవరకు ఆగుతుంది
});

// Parallel (waterfall నివారణ — వీలైనప్పుడల్లా ఇదే)
const results = useQueries({
  queries: userIds.map((id) => ({ queryKey: ["user", id], queryFn: () => getUser(id)
})),
});
```

Global configuration

```
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60_000,
      gcTime: 5 * 60_000,
      retry: (failureCount, error) =>
        error.status >= 400 && error.status < 500 ? false : failureCount < 3,   //
4xx retry వద్దు
      refetchOnWindowFocus: true,        // tab కి తిరిగి వస్తే refresh (mobile-like UX)
      refetchOnReconnect: true,
    },
    mutations: { retry: 0 },
  },
});
<QueryClientProvider client={queryClient}><App /><ReactQueryDevtools />
</QueryClientProvider>
```

Query key design (కీలకం)

```
["todos"]                                  // అన్ని todos
["todos", { status: "done", page: 2 }]     // filtered — object serialization
deterministic
["todos", todoId]                          // ఒక్కటి

// ✅ Query key factory — typo-free, refactor-safe
export const todoKeys = {
  all: ["todos"],
  lists: () => [...todoKeys.all, "list"],
  list: (filters) => [...todoKeys.lists(), filters],
  details: () => [...todoKeys.all, "detail"],
  detail: (id) => [...todoKeys.details(), id],
};
queryClient.invalidateQueries({ queryKey: todoKeys.lists() });   // అన్ని lists
invalidate
```

SWR తో పోలిక

	TanStack Query	SWR
Size	~13KB	~4KB
Features	mutations, infinite, devtools, offline, prefetch	తక్కువ, సరళం
Devtools	✅ అద్భుతం	❌
ఎప్పుడు	complex apps	simple fetching, Next.js చిన్న apps

Gotchas (సాధారణ తప్పులు)

- `staleTime: 0` **(default) ని అలాగే వదిలేయడం** → focus మారిన ప్రతిసారి refetch storm.
- **Query key లో dependency మర్చిపోవడం** → filter మార్చినా పాత data.
- `useQuery` **ని conditionally call చేయడం** → hooks rule ఉల్లంఘన. `enabled` వాడాలి.
- **Server data ని** `useEffect` **+** `setState` **తో Redux లోకి copy చేయడం** – library యొక్క మొత్తం ప్రయోజనం నష్టం.
- `invalidateQueries` **బదులు** `refetch` **గుడ్డిగా వాడటం** – invalidation cache-aware.
- **Mutation error handling లేకపోవడం** → silent failures.
- **QueryClient ని component లోపల create చేయడం** → ప్రతి render కి కొత్త cache. `useState(() => new QueryClient())` లేదా module scope.

Key Points

- **Server state ≠ client state** – TanStack Query దీన్నే పరిష్కరిస్తుంది.
- `staleTime` = refetch నియంత్రణ; `gcTime` = memory నియంత్రణ.
- Cache-first + background revalidate = instant UI.
- Deduplication, retry, window-focus refetch – ఉచితం.
- Optimistic update: `onMutate` → snapshot → optimistic set → `onError` rollback → `onSettled` invalidate.
- Query key factory = maintainable invalidation.

Interview దృష్టి

- *"React Query ఎందుకు, useEffect+fetch సరిపోదా?"* → cache, dedup, retry, focus refetch, races, pagination, optimistic – వేల lines ఆదా.
- *"Optimistic update ఎలా?"* → పై 5 steps.
- *"Cache invalidation strategy?"* → key hierarchy + `invalidateQueries` + tag-like keys.

---

40. Forms at Scale – React Hook Form + Zod

వివరణ

Topic 10 లో basics చూశాం. ఇక్కడ **production form architecture**.

సమస్య: controlled form లో ప్రతి keystroke కి form component (మరియు దాని అన్ని children) re-render. 50 fields + validation ఉన్న form లో ఇది కనిపించే lag.

RHF పరిష్కారం: inputs ని **uncontrolled** (refs) గా ఉంచి, subscription model వాడటం – ఒక field మారితే ఆ field మాత్రమే re-render.

Code – పూర్తి production form

```
import { useForm, useFieldArray, Controller, FormProvider, useFormContext } from
"react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";

// 1️⃣ Schema – single source of truth (types + validation ఒకే చోట)
const schema = z.object({
  email: z.string().min(1, "Email required").email("Invalid email"),
  password: z.string().min(8, "Min 8 chars").regex(/[A-Z]/, "Need uppercase"),
  confirmPassword: z.string(),
  age: z.coerce.number().int().min(18, "Must be 18+"),
  role: z.enum(["admin", "user", "guest"]),
  addresses: z.array(z.object({
    line1: z.string().min(1, "Required"),
    city: z.string().min(1, "Required"),
  })).min(1, "At least one address"),
  terms: z.literal(true, { errorMap: () => ({ message: "You must accept terms" }) }),
}).refine((d) => d.password === d.confirmPassword, {
  message: "Passwords don't match",
  path: ["confirmPassword"],                   // error ఏ field కి వెందుతుందో
});

type FormValues = z.infer<typeof schema>;      // ✅ TypeScript type schema నుండే

function SignupForm() {
  const methods = useForm<FormValues>({
    resolver: zodResolver(schema),
    defaultValues: { email: "", addresses: [{ line1: "", city: "" }], role: "user" },
    mode: "onTouched",                           // ఎప్పుడు validate:
onSubmit|onBlur|onChange|onTouched
  });
  const { register, handleSubmit, control, formState, setError, reset, watch } =
methods;
  const { errors, isSubmitting, isDirty, isValid } = formState;

  // 2️⃣ Dynamic fields
  const { fields, append, remove } = useFieldArray({ control, name: "addresses" });

  // 3️⃣ Conditional field – watch (ఇది re-render trigger చేస్తుంది; ఖరీదైతే useWatch)
  const role = watch("role");
```

```
  const onSubmit = async (data: FormValues) => {
    try {
      await api.signup(data);
      reset();
    } catch (err) {
      // Server validation errors ని fields కి map చేయడం
      if (err.fieldErrors) {
        Object.entries(err.fieldErrors).forEach(([field, message]) =>
          setError(field as keyof FormValues, { type: "server", message: message as
string })
        );
      } else {
        setError("root", { message: err.message });   // form-level error
      }
    }
  };

  return (
    <FormProvider {...methods}>
      <form onSubmit={handleSubmit(onSubmit)} noValidate>
        <input {...register("email")} aria-invalid={!!errors.email} />
        {errors.email && <span role="alert">{errors.email.message}</span>}

        <input type="password" {...register("password")} />
        {errors.password && <span role="alert">{errors.password.message}</span>}

        <select {...register("role")}>
          <option value="user">User</option><option value="admin">Admin</option>
        </select>
        {role === "admin" && <input {...register("adminCode")} placeholder="Admin
code" />}

        {/* Dynamic array */}
        {fields.map((field, i) => (
          <div key={field.id}>                          {/* ⚠️ field.id వాడాలి, index
కాదు */}
            <input {...register(`addresses.${i}.line1`)} />
            <input {...register(`addresses.${i}.city`)} />
            <button type="button" onClick={() => remove(i)}>Remove</button>
          </div>
        ))}
        <button type="button" onClick={() => append({ line1: "", city: "" })}>Add
address</button>

        {/* Third-party controlled component (MUI, react-select, date picker) */}
        <Controller
          name="country"
          control={control}
          render={({ field, fieldState }) => (
            <ReactSelect {...field} options={countries} aria-invalid=
{!!fieldState.error} />
          )}
        />

        {errors.root && <p role="alert">{errors.root.message}</p>}
        <button type="submit" disabled={isSubmitting || !isDirty}>
          {isSubmitting ? "Creating…" : "Sign up"}
        </button>
      </form>
```

```
    </FormProvider>
  );
}
```

Performance – re-render తగ్గించడం

```
// ❌ watch() – form మొత్తం re-render
const email = watch("email");

// ✅ useWatch – ఆ component మాత్రమే re-render
const email = useWatch({ control, name: "email" });

// ✅ getValues – re-render లేదు (subscription లేదు; render లో వాడొద్దు)
const onClick = () => console.log(getValues("email"));

// ✅ Nested components కి FormProvider + useFormContext (prop drilling లేకుండా)
function EmailField() {
  const { register, formState: { errors } } = useFormContext();
  return <><input {...register("email")} />{errors.email && <span>
{errors.email.message}</span>}</>;
}
```

Multi-step wizard

```
function Wizard() {
  const [step, setStep] = useState(0);
  const methods = useForm({ resolver: zodResolver(fullSchema), mode: "onTouched" });

  const next = async () => {
    const fieldsPerStep = [["email", "password"], ["name", "age"], ["address"]];
    const valid = await methods.trigger(fieldsPerStep[step]);   // ఆ step fields
మాత్రమే validate
    if (valid) setStep((s) => s + 1);
  };
  // Draft ని localStorage లో save చేయడం
  useEffect(() => {
    const sub = methods.watch((values) => localStorage.setItem("draft",
JSON.stringify(values)));
    return () => sub.unsubscribe();
  }, [methods]);
}
```

Zod schema ని client + server లో share చేయడం

```
// shared/schemas/user.ts – ఒకే schema రెండు చోట్లా
export const userSchema = z.object({ email: z.string().email(), age:
z.number().min(18) });

// client: zodResolver(userSchema)
// server (Express/Next route): const parsed = userSchema.safeParse(req.body);
//   if (!parsed.success) return res.status(400).json({ errors:
parsed.error.flatten() });
```

> **ఇది monorepo/full-stack TS లో పెద్ద లాభం** – validation logic duplicate కాదు, drift ఉండదు.

Gotchas (సాధారణ తప్పులు)

- `useFieldArray` **లో** `key={index}` – `field.id` వాడాలి.
- **Third-party controlled component కి** `register` **నేరుగా ఇవ్వడం** – `Controller` వాడాలి.
- `defaultValues` **ఇవ్వకపోవడం** → uncontrolled→controlled warning, `reset()` పని చేయదు.
- **Async default values** (API నుండి) → `reset(data)` ని effect లో call చేయాలి.
- **Client validation మాత్రమే** – server లో మళ్ళీ validate తప్పనిసరి (security).
- `mode: "onChange"` పెద్ద forms లో → ప్రతి keystroke కి validation, lag.
- **Error messages ని** `aria-invalid` **/** `role="alert"` **లేకుండా చూపడం** – screen readers కి తెలియదు.

Key Points

- RHF = uncontrolled + subscriptions → **పెద్ద forms లో తక్కువ re-renders**.
- **Zod schema = validation + TypeScript types + server share**.
- `Controller` = third-party controlled inputs కి bridge.
- `useWatch` > `watch`; `getValues` = subscription-free.
- Server errors ని `setError` తో fields కి map చేయాలి.
- Field arrays: `field.id` as key.

Interview దృష్టి

- *"పెద్ద form నెమ్మది – ఏం చేస్తావు?"* → uncontrolled/RHF, field-level subscription, split steps, memoized field components, validation mode.
- *"Validation ఎక్కడ?"* → client (UX) + server (security), shared schema.

Part 6 — Routing

41. React Router (v6/v7) Deep

వివరణ

React లో routing built-in కాదు (library, framework కాదు). **React Router** = అత్యంత ప్రాచుర్యం పొందిన పరిష్కారం. v6/v7 లో API పూర్తిగా మారింది (v5 code interviews లో ఇంకా వస్తుంది).

```
import { createBrowserRouter, RouterProvider } from "react-router-dom";

const router = createBrowserRouter([
  {
    path: "/",
    element: <RootLayout />,
    errorElement: <ErrorPage />,             // ఈ route లో error వస్తే
    children: [
      { index: true, element: <Home /> },                     // "/"
      { path: "products", element: <Products /> },            // "/products"
      { path: "products/:id", element: <ProductDetail />,    // "/products/42"
        loader: productLoader },
      { path: "*", element: <NotFound /> },                   // 404 catch-all
    ],
  },
]);

<RouterProvider router={router} />
```

Real-life Scenario

> **Router = apartment complex లోని security + lift.** మీరు address (URL) చెప్తారు — security (router) ఏ floor, ఏ flat (component) కి పంపాలో నిర్ణయిస్తుంది. **Nested routes = building → floor → flat** — building lobby (layout) అలాగే ఉంటుంది, floor మారితే flat మాత్రమే మారుతుంది.

Nested routes & `Outlet` (v6 యొక్క ప్రధాన idea)

```
function RootLayout() {
  return (
    <>
      <Navbar />                    {/* ఇది ఎప్పుడూ ఉంటుంది */}
      <main><Outlet /></main>       {/* 🔑 child route ఇక్కడ render అవుతుంది */}
      <Footer />
    </>
  );
}
```

```
// Nested layouts — dashboard లోపల మరో layout
{
  path: "dashboard",
  element: <DashboardLayout />,        // sidebar + <Outlet />
  children: [
    { index: true, element: <Overview /> },
    { path: "analytics", element: <Analytics /> },
    { path: "settings", element: <Settings /> },
  ],
}
```

లాభం: URL మారినప్పుడు layout **re-mount అవ్వదు** – sidebar scroll position, state అలాగే ఉంటాయి. Nested layouts లేకపోతే ప్రతి navigation కి మొత్తం page rebuild.

Navigation APIs

```
import { Link, NavLink, useNavigate, Navigate, useLocation, useParams,
useSearchParams } from "react-router-dom";

// Declarative
<Link to="/products">Products</Link>
<Link to="/products" state={{ from: "home" }}>With state</Link>
<Link to=".." relative="path">Up one level</Link>
<NavLink to="/about" className={({ isActive }) => isActive ? "active" :
""}>About</NavLink>

// Programmatic
const navigate = useNavigate();
navigate("/checkout");
navigate("/login", { replace: true });          // history stack లో replace (back
button skip)
navigate(-1);                                     // back
navigate("/results", { state: { query } });

// Redirect (render లో)
if (!user) return <Navigate to="/login" replace state={{ from: location }} />;

// Params & query
const { id } = useParams();                              // /products/:id
const [searchParams, setSearchParams] = useSearchParams();  // ?page=2&sort=asc
const location = useLocation();                          // { pathname, search,
hash, state }
```

Protected routes (auth guard)

```
function RequireAuth({ children, roles }) {
  const { user, loading } = useAuth();
  const location = useLocation();

  if (loading) return <Spinner />;
  if (!user) return <Navigate to="/login" replace state={{ from: location }} />;   //
🔑 from
  if (roles && !roles.includes(user.role)) return <Navigate to="/403" replace />;
  return children;
}
```

```
// Layout route గా (v6 style — cleaner)
function ProtectedLayout() {
  const { user } = useAuth();
  const location = useLocation();
  return user ? <Outlet /> : <Navigate to="/login" replace state={{ from: location }}
/>;
}
{
  element: <ProtectedLayout />,
  children: [
    { path: "dashboard", element: <Dashboard /> },
    { path: "profile", element: <Profile /> },
  ],
}

// Login తర్వాత తిరిగి పంపడం
const from = location.state?.from?.pathname ?? "/";
await login(creds);
navigate(from, { replace: true });
```

Data APIs — loaders & actions (v6.4+, Remix నుండి వచ్చినవి)

```
// Loader — route render కి *ముందే* data fetch (waterfall నివారణ ⚡)
async function productLoader({ params, request }) {
  const url = new URL(request.url);
  const res = await fetch(`/api/products/${params.id}`, { signal: request.signal });
  if (!res.ok) throw new Response("Not Found", { status: 404 });   // errorElement కి
వెళ్తుంది
  return res.json();
}

function ProductDetail() {
  const product = useLoaderData();       // ✅ loading state అవసరం లేదు — data ఇప్పటికే
ఉంది
  const navigation = useNavigation();    // global loading state
  return <div style={{ opacity: navigation.state === "loading" ? 0.5 : 1 }}>
{product.name}</div>;
}

// Action — form submissions
async function productAction({ request, params }) {
  const formData = await request.formData();
  const res = await fetch(`/api/products/${params.id}`, { method: "PUT", body:
formData });
  if (!res.ok) return { error: "Failed" };
  return redirect(`/products/${params.id}`);
}

function EditProduct() {
  const actionData = useActionData();
  const navigation = useNavigation();
  return (
    <Form method="post">                    {/* ⚠️ <form> కాదు, Router యొక్క <Form> */}
      <input name="title" />
      <button disabled={navigation.state === "submitting"}>Save</button>
      {actionData?.error && <p>{actionData.error}</p>}
```

```
    </Form>
  );
}
```

Loader ఎందుకు మేలు: `useEffect` లో fetch చేస్తే – component render → effect → fetch → render (waterfall). Loader route match అయిన వెంటనే fetch మొదలుపెడుతుంది, **component code download అవుతున్నప్పుడే parallel గా**.

Lazy routes (code splitting)

```
{
  path: "dashboard",
  lazy: async () => {
    const { Dashboard, dashboardLoader } = await import("./pages/Dashboard");
    return { Component: Dashboard, loader: dashboardLoader };
  },
}
// లేదా classic React.lazy + Suspense
```

ఇతర ముఖ్య APIs

```
useRouteError()        // errorElement లో error చదవడం
useRevalidator()       // manual గా loaders ని మళ్ళీ run చేయడం
useFetcher()           // navigation లేకుండా action/loader call (like/upvote buttons)
useBlocker()           // unsaved changes ఉంటే navigation ఆపడం
ScrollRestoration      // navigation లో scroll position నిర్వహణ
```

```
// Unsaved changes guard
const blocker = useBlocker(({ currentLocation, nextLocation }) =>
  isDirty && currentLocation.pathname !== nextLocation.pathname);

{blocker.state === "blocked" && (
  <Modal>
    <p>Unsaved changes! Leave anyway?</p>
    <button onClick={() => blocker.proceed()}>Leave</button>
    <button onClick={() => blocker.reset()}>Stay</button>
  </Modal>
)}
```

v5 → v6 ముఖ్య మార్పులు (interview)

v5	v6/v7
`<Switch>`	`<Routes>`
`component={X}` / `render={}`	`element={<X />}`
`exact` prop	default exact matching
`useHistory()`	`useNavigate()`

v5	v6/v7
`<Redirect />`	`<Navigate />`
Manual nesting	`<Outlet />` + nested config
Data fetching manual	`loader` / `action`

Gotchas (సాధారణ తప్పులు)

- **`<a href>` వాడటం** → full page reload, SPA state పోతుంది. `<Link>` వాడాలి.
- **`navigate` ని render సమయంలో call చేయడం** → warning/loop. Effect లేదా handler లో, లేదా `<Navigate />`.
- **Login redirect లో `replace` వాడకపోవడం** → back button login page కి తిరిగి తీసుకెళ్తుంది.
- **404 route (`path: "*"`) మర్చిపోవడం** → తప్పు URL కి ఖాళీ page.
- **Protected route ని client-side మాత్రమే నమ్మడం – security కాదు**! Server ప్రతి API call లో authorize చేయాలి. Client guard కేవలం UX.
- **Deployment లో SPA fallback config లేకపోవడం** → `/products/42` ని refresh చేస్తే 404. Server లో అన్ని routes ని `index.html` కి rewrite చేయాలి (nginx `try_files`, Netlify `_redirects`, Vercel automatic).
- **Query params ని state లో duplicate చేయడం** → source of truth రెండు చోట్ల.

Key Points

- **Nested routes + `<Outlet />`** = layouts re-mount అవ్వకుండా.
- `useNavigate`, `<Navigate>`, `useParams`, `useSearchParams`, `useLocation` – core hooks.
- **Loaders/actions** = render కి ముందే data → waterfalls నివారణ.
- Protected routes = **UX only**, server authorization తప్పనిసరి.
- SPA deployment కి **history fallback** config తప్పనిసరి.

Interview దృష్టి

- *"Client-side routing ఎలా పని చేస్తుంది?"* → History API (`pushState`) + `popstate` listener; page reload లేకుండా URL మార్పు + matching component render.
- *"Protected routes ఎలా?"* → layout route guard + redirect with `from` state; server-side authz తప్పనిసరి అని చెప్పడం ముఖ్యం.
- *"Loader vs useEffect fetch?"* → parallel fetching, no waterfall, built-in pending/error, revalidation.

Part 7 – Performance Engineering

42. Re-render Mental Model & Profiling

వివరణ

Performance పని ఎప్పుడూ ఇలా ఉండాలి: **Measure → Identify → Fix → Measure again.** Guess చేసి optimize చేయడం సమయం వృథా (మరియు తరచుగా code ని నెమ్మది చేస్తుంది).

React DevTools Profiler – వాడకం

1. **Profiler tab → Record → interact → Stop.**
2. **Flamegraph** – ప్రతి commit లో ఏ components render అయ్యాయి, ఎంత సమయం.
3. **Ranked chart** – నెమ్మది components వరుసలో.
4. **Settings → "Record why each component rendered"** ✅ – ఇది అత్యంత ఉపయోగకరం. ఇది చూపిస్తుంది: *"Props changed: (onClick)"*, *"Hook 3 changed"*, *"Parent rendered"*, *"Context changed"*.
5. **"Highlight updates when components render"** – screen మీద re-render అవుతున్న components చుట్టూ box flash.

```
// Programmatic profiling – production లో metrics పంపడానికి
import { Profiler } from "react";

<Profiler id="Feed" onRender={(id, phase, actualDuration, baseDuration, startTime,
commitTime) => {
  // phase: "mount" | "update" | "nested-update"
  // actualDuration: ఈ commit కి పట్టిన సమయం
  // baseDuration: memoization లేకుండా ఎంత పట్టేదో అంచనా
  if (actualDuration > 16) analytics.track("slow_render", { id, actualDuration });
}}>
  <Feed />
</Profiler>
```

Performance సమస్యల 5 వర్గాలు

సమస్య	లక్షణం	పరిష్కారం
1. అనవసర re-renders	typing/click లో లాగ్; DevTools లో అనేక components flash	memo, state colocation, composition, context split
2. ఖరీదైన render	ఒకే component లో ఎక్కువ సమయం	useMemo, virtualization, web worker
3. పెద్ద bundle	మొదటి load నెమ్మది, LCP ఎక్కువ	code splitting, tree shaking, lighter libs

సమస్య	లక్షణం	పరిష్కారం
4. Network waterfalls	data ఒకదాని తర్వాత ఒకటి వస్తుంది	parallel fetch, loaders, prefetch, RSC
5. DOM పెద్దది	scroll jank, memory	virtualization, pagination, `content-visibility`

Chrome DevTools Performance tab

- **Long tasks** (>50ms) – main thread block; INP పాడవుతుంది.
- **Layout shift** regions – CLS.
- **Flame chart** – scripting vs rendering vs painting ఎక్కడ సమయం పోతోంది.

Key Points

- Profiler-first; guess చేయకూడదు.
- "Record why each component rendered" = అనవసర re-renders ని పట్టుకునే ఏకైక నమ్మకమైన మార్గం.
- 5 వర్గాలు – సమస్య ఏ వర్గమో ముందు నిర్ణయించాలి; పరిష్కారం అక్కడి నుండే.
- Production లో `<Profiler>` + RUM metrics.

---

43. Optimization Techniques

1. అనవసర re-renders తగ్గించడం

```
// (a) State colocation – state ని వాడే చోటికి కిందికి
// (b) Composition/children – Topic 17
// (c) React.memo + useCallback/useMemo – Topic 16-17
// (d) Context splitting – Topic 18
// (e) Selector-based store (Zustand/Redux) – Topic 38
```

2. List virtualization

```
// 10,000 rows → ~20 DOM nodes (Topic 9 లో పూర్తి code)
import { useVirtualizer } from "@tanstack/react-virtual";
// లేదా react-window (సరళం)
```

నియమం: 100+ rows లేదా ఒక్క row ఖరీదైతే → virtualize. దీనితో పాటు rows ని `React.memo` చేయాలి.

3. Code splitting & lazy loading

```
const Chart = lazy(() => import("./Chart"));          // component level
const dayjs = await import("dayjs");                  // library level (event
handler లో)
```

```
// Route level (అత్యధిక ప్రయోజనం) — Topic 31
```

4. Debounce & throttle

```
// Debounce — చివరి event తర్వాత X ms ఆగి ఒకసారి (search input, autosave)
// Throttle — X ms కి ఒకసారి మాత్రమే (scroll, resize, mousemove)

function throttle(fn, ms) {
  let last = 0, timer = null;
  return (...args) => {
    const now = Date.now();
    const remaining = ms - (now - last);
    if (remaining <= 0) { clearTimeout(timer); timer = null; last = now; fn(...args);
}
    else if (!timer) { timer = setTimeout(() => { last = Date.now(); timer = null;
fn(...args); }, remaining); }
  };
}

// Scroll — passive listener + rAF (అత్యుత్తమం)
useEffect(() => {
  let ticking = false;
  const onScroll = () => {
    if (ticking) return;
    ticking = true;
    requestAnimationFrame(() => { setY(window.scrollY); ticking = false; });
  };
  window.addEventListener("scroll", onScroll, { passive: true });   // 🔑 passive
  return () => window.removeEventListener("scroll", onScroll);
}, []);
```

5. Images

```
<img
  src="/hero.webp"
  srcSet="/hero-400.webp 400w, /hero-800.webp 800w, /hero-1200.webp 1200w"
  sizes="(max-width: 600px) 400px, (max-width: 1000px) 800px, 1200px"
  width={1200} height={630}          // 🔑 CLS నివారణ (aspect ratio reserve)
  loading="lazy"                      // below-the-fold కి
  decoding="async"
  fetchPriority="high"                // LCP image కి (దీనికి loading="lazy" వద్దు!)
  alt="Product hero"
/>
// Next.js లో <Image /> ఇవన్నీ automatic
```

6. Web Worker — heavy computation ని main thread నుండి తీసేయడం

```
// worker.js
self.onmessage = (e) => {
  const result = heavyComputation(e.data);       // 2 సెకన్ల CPU పని
  self.postMessage(result);
};
```

```
// component
useEffect(() => {
  const worker = new Worker(new URL("./worker.js", import.meta.url), { type: "module"
});
  worker.postMessage(bigData);
  worker.onmessage = (e) => setResult(e.data);
  return () => worker.terminate();
}, [bigData]);
// ✅ UI freeze అవ్వదు — worker వేరే thread లో
```

7. ఇతర quick wins

```
// CSS containment — browser కి "ఈ subtree బయటిదాన్ని ప్రభావితం చేయదు" అని చెప్పడం
.card { content-visibility: auto; contain-intrinsic-size: 200px; }

// Transitions — Topic 20
// Prefetch on hover — Topic 31
// Font optimization — font-display: swap, preload critical fonts, subset
```

Optimization checklist (priority క్రమంలో)

```
1. Route-level code splitting          ← అత్యధిక ప్రభావం, తక్కువ శ్రమ
2. Server/CDN caching + compression (brotli)
3. Image optimization (format, size, lazy, dimensions)
4. Bundle audit — పెద్ద libraries replace (moment→date-fns, lodash→lodash-es/native)
5. Data fetching — parallel, cache (React Query), prefetch
6. Virtualization for long lists
7. Memoization (profiler ఆధారంగా మాత్రమే)
8. Web workers (చివరి ఎంపిక)
```

Gotchas

- **Profiling లేకుండా memoization** — తరచుగా నెమ్మది చేస్తుంది.
- **`loading="lazy"` ని LCP image కి** → LCP పాడవుతుంది.
- **Image కి width/height ఇవ్వకపోవడం** → CLS.
- **Scroll listener passive లేకుండా** → scroll jank.
- **Bundle లో మొత్తం library import** — `import _ from "lodash"` ❌ vs `import debounce from "lodash/debounce"` ✅.

Key Points

- Priority: bundle/network ముందు, rendering తర్వాత.
- Virtualization = పెద్ద lists కి ఏకైక నిజమైన పరిష్కారం.
- Debounce (network) vs throttle (continuous events) vs transition (rendering).
- Web worker = CPU-heavy pure computation కి.

44. React Compiler & Core Web Vitals

React Compiler (React 19)

Build-time లో మీ components ని analyze చేసి **automatic memoization** జోడించే compiler.

```
npm install -D babel-plugin-react-compiler
```

```
// vite.config.js
export default defineConfig({
  plugins: [react({ babel: { plugins: [["babel-plugin-react-compiler", {}]] } })],
});
```

ఏం చేస్తుంది: ప్రతి component లో ఏ values ఏ inputs మీద ఆధారపడతాయో లెక్కించి, cache slots create చేస్తుంది – మీరు `useMemo` / `useCallback` / `memo` రాయనవసరం లేదు.

షరతు – Rules of React పాటించాలి:

- Components/hooks **pure** – render లో mutation, side effects వద్దు.
- Props/state mutate చేయకూడదు.
- Hooks rules పాటించాలి.

`eslint-plugin-react-compiler` ఉల్లంఘనలను చూపిస్తుంది; compiler సందేహాస్పద components ని **skip** చేస్తుంది (safe fallback).

Core Web Vitals (2026)

Metric	అర్థం	లక్ష్యం	React లో కారణాలు
LCP (Largest Contentful Paint)	అతిపెద్ద content కనిపించే సమయం	< 2.5s	పెద్ద JS bundle, CSR-only, unoptimized hero image, client-side data fetch
INP (Interaction to Next Paint)	click/type కి UI స్పందించే సమయం	< 200ms	long tasks, heavy re-renders, sync work in handlers
CLS (Cumulative Layout Shift)	layout ఎంత కదులుతుంది	< 0.1	image dimensions లేకపోవడం, ads/banners, font swap, skeleton→content size mismatch

FID ని 2024 లో INP replace చేసింది – INP అన్ని interactions ని కొలుస్తుంది (FID కేవలం మొదటిది).

React-specific పరిష్కారాలు

```
// LCP — SSR/SSG, critical CSS, image priority, bundle తగ్గింపు
<img fetchPriority="high" />                    // hero image
export const dynamic = "force-static";          // Next.js — static render

// INP — long tasks విడగొట్టడం
const onClick = () => {
```

```
  setSelected(id);                              // urgent
  startTransition(() => setHeavyView(compute(id)));  // deferred ✅
};

// CLS — స్థలం ముందే reserve
<div style={{ aspectRatio: "16/9" }}><img /></div>
<Skeleton style={{ height: 200 }} />             // నిజమైన content ఎత్తుకి సరిపోయేలా
// font-display: optional/swap + preload
```

కొలవడం

```
// web-vitals library — real user monitoring
import { onLCP, onINP, onCLS } from "web-vitals";
onLCP((m) => analytics.send("LCP", m.value));
onINP((m) => analytics.send("INP", m.value));
onCLS((m) => analytics.send("CLS", m.value));
```

Lab vs Field: Lighthouse = lab (మీ machine); **CrUX/RUM = field (నిజమైన users)**. Field data నే Google ranking కి వాడుతుంది. Lab score 100 అయినా field లో పేలవంగా ఉండొచ్చు (slow devices, 3G).

Key Points

- **React Compiler** = auto-memoization; purity తప్పనిసరి.
- **LCP/INP/CLS** – ప్రతి దానికి React-specific కారణాలు & పరిష్కారాలు తెలియాలి.
- **INP** = FID స్థానంలో; transitions + task splitting కీలకం.
- Lab ≠ Field; RUM తో నిజమైన users ని కొలవాలి.

Interview దృష్టి

- *"App నెమ్మదిగా ఉంది – ఎలా debug చేస్తావు?"* → measure (Profiler + Lighthouse + RUM) → వర్గీకరించు (bundle/render/network/DOM) → priority ప్రకారం fix → మళ్ళీ measure. ఈ structured answer చాలా ముఖ్యం.
- *"INP ఎలా మెరుగుపరుస్తావు?"* → long tasks విడగొట్టడం, transitions, debounce, virtualization, worker.

---

45. Bundle Size & Build Optimization

వివరణ

Frontend performance లో **JavaScript bundle size** అతిపెద్ద lever. 1 MB JS = mid-range Android phone లో ~3-5 సెకన్ల parse+execute – network వేగం ఎంతైనా.

Bundle ని analyze చేయడం

```
# Vite
npm i -D rollup-plugin-visualizer
# vite.config.js: plugins: [react(), visualizer({ open: true, gzipSize: true })]
npm run build

# webpack
npm i -D webpack-bundle-analyzer

# Next.js
npm i -D @next/bundle-analyzer && ANALYZE=true npm run build

# ఏ package ఎంత ఖరీదో ముందే చూడటానికి: bundlephobia.com
```

సాధారణ bundle దోషులు & ప్రత్యామ్నాయాలు

Library	Size (gzip)	ప్రత్యామ్నాయం
`moment`	~70KB (+ locales)	`date-fns` (tree-shakeable), `dayjs` (~3KB), native `Intl.DateTimeFormat`
`lodash` (full)	~25KB	`lodash-es` + named imports, లేదా native methods
`axios`	~14KB	native `fetch` + చిన్న wrapper
Icon library (full)	100KB+	individual icon imports / SVG sprite
`chart.js` + wrapper	~70KB	lazy load, లేదా lightweight alternative

```
// ❌ మొత్తం library
import _ from "lodash";
import * as Icons from "react-icons/fa";

// ✅ Named/deep imports (tree-shaking పని చేస్తుంది)
import debounce from "lodash/debounce";
import { FaUser } from "react-icons/fa";
```

Tree shaking పని చేయాలంటే

1. **ESM** (`import/export`) వాడాలి – CommonJS (`require`) tree-shake కాదు.
2. Package `package.json` లో `"sideEffects": false` (లేదా CSS files list).
3. Barrel files (`index.ts` re-exports) జాగ్రత్తగా – పెద్దవి tree-shaking ని ఆపొచ్చు.
4. Production build లోనే tree shaking జరుగుతుంది (dev లో కాదు).

Manual chunking (vendor splitting)

```
// vite.config.js
export default defineConfig({
  build: {
```

```
    rollupOptions: {
      output: {
        manualChunks: {
          "react-vendor": ["react", "react-dom", "react-router-dom"],
          "ui-vendor": ["@mui/material", "@emotion/react"],
          "chart-vendor": ["recharts"],
        },
      },
    },
    chunkSizeWarningLimit: 500,
  },
});
```

ఎందుకు: vendor code అరుదుగా మారుతుంది → browser cache లో ఉంటుంది. మీ app code మార్చినప్పుడు user vendor chunk మళ్ళీ download చేయనవసరం లేదు.

Caching strategy

```
index.html              → no-cache (ఎప్పుడూ తాజాది)
assets/*.[hash].js      → Cache-Control: public, max-age=31536000, immutable
                          (hash మారితే filename మారుతుంది → cache bust automatic)
```

ఇతర build wins

- **Compression:** brotli > gzip (~15-20% చిన్నది). Server/CDN లో enable చేయాలి.
- **Modern build target:** `esnext` – పాత browsers కి polyfills bundle పెంచుతాయి. `browserslist` తో నియంత్రణ.
- **Source maps:** production లో `hidden` (Sentry కి upload చేసి, public గా serve చేయకూడదు).
- **Preconnect/dns-prefetch** critical origins కి: `<link rel="preconnect" href="https://api.example.com" />`.
- **Font subsetting** – Telugu/Devanagari fonts పెద్దవి; అవసరమైన glyphs మాత్రమే.

Performance budget (team discipline)

```
// CI లో enforce – bundle పెరిగితే PR fail
// package.json
"bundlesize": [
  { "path": "dist/assets/index-*.js", "maxSize": "150 kB" },
  { "path": "dist/assets/vendor-*.js", "maxSize": "120 kB" }
]
```

Gotchas

- **`devDependencies` లో ఉండాల్సినవి `dependencies` లో** – bundle పెరగదు కానీ install నెమ్మది; నిజమైన సమస్య dev-only code ని import చేయడం.
- **Dynamic import ని variable తో** – `import(path)` bundler statically analyze చేయలేదు.
- **Polyfills ని అన్ని browsers కి** – modern/legacy dual build (`@vitejs/plugin-legacy`).
- **CSS ని JS bundle లో ఉంచడం** – extract చేయాలి (Vite default చేస్తుంది).

Key Points

- Analyze మొదట (visualizer/bundlephobia).
- Heavy libs ని replace/lazy చేయాలి; named imports + ESM తో tree-shaking.
- Vendor chunking + content-hash caching.
- Brotli + modern target + performance budget in CI.

Part 8 — Styling

46. Styling in React — అన్ని విధానాలు

వివరణ – 6 ఎంపికలు

విధానం	Runtime cost	Scoping	RSC compatible	ఎప్పుడు
Plain CSS / SCSS	0	❌ global	✅	చిన్న projects
CSS Modules	0	✅ automatic	✅	సురక్షితమైన default ✅
Tailwind CSS	0	✅ utility	✅	వేగవంతమైన development ✅
CSS-in-JS runtime (styled-components, emotion)	⚠️ ఉంది	✅	❌ (client only)	dynamic theming heavy apps
Zero-runtime CSS-in-JS (vanilla-extract, Panda, Linaria)	0	✅	✅	type-safe + zero runtime
Inline styles	తక్కువ	✅	✅	dynamic values మాత్రమే

CSS Modules

```
/* Button.module.css */
.button { padding: 8px 16px; border-radius: 6px; }
.primary { composes: button; background: var(--brand); color: white; }
.danger  { composes: button; background: var(--red); }
```

```
import styles from "./Button.module.css";
import clsx from "clsx";

function Button({ variant = "primary", isLoading, className, ...rest }) {
  return (
    <button
      className={clsx(styles[variant], isLoading && styles.loading, className)}
      {...rest}
    />
  );
}
// build లో class names hash అవుతాయి: "Button_primary__x7f2a" → collisions లేవు ✅
```

Tailwind CSS

```
function Card({ featured }) {
  return (
    <div className={clsx(
      "rounded-lg border p-4 shadow-sm transition hover:shadow-md",
      "dark:border-gray-700 dark:bg-gray-800",
      featured && "border-blue-500 ring-2 ring-blue-200"
    )}>
      <h3 className="text-lg font-semibold text-gray-900 dark:text-white">Title</h3>
    </div>
  );
}
```

Tailwind ఎందుకు గెలిచింది: naming ఆలోచించనవసరం లేదు, dead CSS ఉండదు (unused purge), design constraints (spacing/color scale) built-in, co-located styles, zero runtime. **విమర్శ:** HTML "కంగాళీ"గా కనిపిస్తుంది → `clsx` + component extraction + `cva` (class-variance-authority) తో పరిష్కారం.

```
// cva — variant API (shadcn/ui ఇలానే చేస్తుంది)
import { cva } from "class-variance-authority";
const button = cva("rounded font-medium transition", {
  variants: {
    intent: { primary: "bg-blue-600 text-white hover:bg-blue-700",
              danger: "bg-red-600 text-white hover:bg-red-700" },
    size: { sm: "px-2 py-1 text-sm", lg: "px-6 py-3 text-lg" },
  },
  defaultVariants: { intent: "primary", size: "sm" },
});
<button className={button({ intent: "danger", size: "lg" })} />
```

CSS-in-JS (styled-components / emotion)

```
import styled, { css, ThemeProvider } from "styled-components";

const Button = styled.button`
  padding: ${(p) => (p.$size === "lg" ? "12px 24px" : "8px 16px")};
  background: ${(p) => p.theme.colors.primary};
  ${(p) => p.$outline && css`background: transparent; border: 2px solid
${p.theme.colors.primary};`}
  &:hover { opacity: 0.9; }
  @media (max-width: 768px) { width: 100%; }
`;
<ThemeProvider theme={{ colors: { primary: "#0070f3" } }}><Button $size="lg" />
</ThemeProvider>
```

⚠️ **2026 వాస్తవం:** runtime CSS-in-JS **Server Components తో పని చేయదు** (runtime + context అవసరం). React team కూడా దీన్ని discourage చేస్తోంది (runtime cost, SSR complexity). కొత్త projects లో **CSS Modules, Tailwind, లేదా zero-runtime (vanilla-extract/Panda)** ఎంచుకోవడం మేలు.

> **Transient props (`$size`)** – `$` prefix ఉన్న props DOM కి pass కావు (React warning నివారణ).

Zero-runtime CSS-in-JS — vanilla-extract

```
// button.css.ts — TypeScript లో CSS, build time లో static CSS గా మారుతుంది
import { style, styleVariants } from "@vanilla-extract/css";
const base = style({ padding: 8, borderRadius: 6 });
export const button = styleVariants({
  primary: [base, { background: "blue", color: "white" }],
  danger:  [base, { background: "red", color: "white" }],
});
```

Theming & dark mode (CSS variables — ఉత్తమ మార్గం)

```
:root { --bg: #fff; --text: #111; --brand: #0070f3; }
[data-theme="dark"] { --bg: #111; --text: #eee; }
@media (prefers-color-scheme: dark) { :root:not([data-theme]) { --bg: #111; --text:
#eee; } }
body { background: var(--bg); color: var(--text); }
```

```
function useTheme() {
  const [theme, setTheme] = useState(() => localStorage.getItem("theme") ??
"system");
  useEffect(() => {
    const resolved = theme === "system"
      ? (matchMedia("(prefers-color-scheme: dark)").matches ? "dark" : "light")
      : theme;
    document.documentElement.dataset.theme = resolved;
    localStorage.setItem("theme", theme);
  }, [theme]);
  return [theme, setTheme];
}
```

Flash of wrong theme (FOUC) నివారణ – React hydrate అవ్వకముందే `<head>` లో blocking script:

```
<script>
  (function () {
    const t = localStorage.getItem("theme");
    const dark = t === "dark" || (!t && matchMedia("(prefers-color-scheme:
dark)").matches);
    document.documentElement.dataset.theme = dark ? "dark" : "light";
  })();
</script>
```

Gotchas (సాధారణ తప్పులు)

- **Runtime CSS-in-JS ని RSC app లో** → `'use client'` తప్పనిసరి → RSC ప్రయోజనం నష్టం.

- **Inline styles ని అన్నిటికీ** – pseudo-classes (`:hover`), media queries, animations పని చేయవు; ప్రతి render కి కొత్త object.
- `styled` **component ని render లోపల define చేయడం** → ప్రతి render కి కొత్త class + component → remount.
- **Global CSS collisions** – CSS Modules/Tailwind/scoping వాడాలి.
- **Tailwind class names ని dynamically construct చేయడం** – `` `text-${color}-500` `` ❌ purge చేసేస్తుంది. పూర్తి class names map వాడాలి.
- **Dark mode flash** – blocking script లేకపోతే.
- `!important` **వరద** – specificity war; design system + layers (`@layer`) వాడాలి.

Key Points

- **CSS Modules / Tailwind = 2026 default** (zero runtime, RSC-safe).
- Runtime CSS-in-JS = RSC-incompatible + runtime cost.
- **CSS variables** = theming కి ఉత్తమం (JS re-render లేకుండా theme మారుతుంది).
- `clsx` + `cva` = variant-based component styling.
- Dark mode FOUC కి blocking script.

Interview దృష్టి

- *"CSS-in-JS vs Modules vs Tailwind?"* → runtime cost, scoping, RSC compatibility, DX, team scale.
- *"Design system ఎలా build చేస్తావు?"* → tokens (CSS vars) → primitives (Button/Input) → variants (cva) → composition + a11y + docs (Storybook).

Part 9 — Data Fetching, SSR & React Server Components

47. Client-side Data Fetching Patterns

Waterfall — అతిపెద్ద performance హంతకుడు

```
// ❌ Sequential waterfall — మొత్తం 900ms
function Profile({ userId }) {
  const { data: user } = useQuery(["user", userId], fetchUser);          // 300ms
  const { data: posts } = useQuery(["posts", user?.id], fetchPosts, { enabled: !!user
});   // +300ms
  const { data: comments } = useQuery(["comments", posts?.[0]?.id], fetchComments, {
enabled: !!posts });  // +300ms
}

// ✅ Parallel — 300ms (అన్నీ ఒకేసారి)
const [user, posts, stats] = useQueries({
  queries: [
    { queryKey: ["user", userId], queryFn: () => fetchUser(userId) },
    { queryKey: ["posts", userId], queryFn: () => fetchPosts(userId) },
    { queryKey: ["stats", userId], queryFn: () => fetchStats(userId) },
  ],
});

// ✅ ఇంకా మెరుగు — backend లో ఒకే aggregated endpoint (BFF pattern)
const { data } = useQuery(["profile", userId], () =>
fetch(`/api/profile/${userId}`));
```

Component-level waterfall (దాగిన రకం):

```
<Layout>                    ← fetch (200ms)
  <Sidebar>                 ← fetch (200ms) — Layout render అయ్యాకే మొదలవుతుంది
    <UserWidget>            ← fetch (200ms) — Sidebar render అయ్యాకే
```

మొత్తం 600ms. **పరిష్కారం:** route loader / RSC / prefetch — data ని component tree కంటే ముందుకు తీసుకెళ్ళడం.

Prefetching

```
// Hover లో prefetch — click చేసేసరికి data సిద్ధం
const queryClient = useQueryClient();
<Link
  to={`/products/${id}`}
  onMouseEnter={() => queryClient.prefetchQuery({
    queryKey: ["product", id], queryFn: () => fetchProduct(id), staleTime: 60_000,
```

```
  })}
/>
```

Polling & real-time

```
// Polling
useQuery({ queryKey: ["status"], queryFn: getStatus, refetchInterval: 5000 });
// Smart polling — pending అయినంతసేపు మాత్రమే
refetchInterval: (query) => (query.state.data?.status === "processing" ? 2000 :
false),

// WebSocket + query cache
useEffect(() => {
  const ws = new WebSocket(url);
  ws.onmessage = (e) => {
    const msg = JSON.parse(e.data);
    queryClient.setQueryData(["messages"], (old) => [...(old ?? []), msg]);
  };
  return () => ws.close();
}, [url, queryClient]);

// SSE (server-sent events) — one-way streams కి తేలికైనది
useEffect(() => {
  const es = new EventSource("/api/events");
  es.onmessage = (e) => setEvents((prev) => [...prev, JSON.parse(e.data)]);
  return () => es.close();
}, []);
```

API layer design

```
// lib/api.js — centralized client (interceptors, auth, error normalization)
const BASE = import.meta.env.VITE_API_URL;

async function request(path, { method = "GET", body, signal, ...rest } = {}) {
  const res = await fetch(`${BASE}${path}`, {
    method,
    headers: { "Content-Type": "application/json", ...getAuthHeader() },
    body: body ? JSON.stringify(body) : undefined,
    credentials: "include",
    signal,
    ...rest,
  });

  if (res.status === 401) { await refreshTokenOrLogout(); throw new
ApiError("Unauthorized", 401); }
  if (!res.ok) {
    const data = await res.json().catch(() => ({}));
    throw new ApiError(data.message ?? res.statusText, res.status, data.fieldErrors);
  }
  return res.status === 204 ? null : res.json();
}

export const api = {
  get: (p, o) => request(p, o),
  post: (p, body, o) => request(p, { ...o, method: "POST", body }),
```

```
  put: (p, body, o) => request(p, { ...o, method: "PUT", body }),
  del: (p, o) => request(p, { ...o, method: "DELETE" }),
};

export class ApiError extends Error {
  constructor(message, status, fieldErrors) {
    super(message);
    this.name = "ApiError"; this.status = status; this.fieldErrors = fieldErrors;
  }
}
```

Key Points

- **Waterfalls** = #1 data performance సమస్య → parallel queries, loaders, RSC, BFF.
- Prefetch on hover/intent.
- Polling → conditional `refetchInterval`; real-time → WS/SSE + cache write.
- Centralized API layer: auth, error normalization, abort support.

---

48. CSR vs SSR vs SSG vs ISR & Hydration

వివరణ – rendering strategies

	CSR	SSR	SSG	ISR	Streaming SSR
HTML ఎక్కడ	browser	server (per request)	build time	build + background regen	server (chunks)
TTFB	వేగం (static shell)	నెమ్మది (server work)	అత్యంత వేగం (CDN)	అత్యంత వేగం	వేగం
FCP/LCP	నెమ్మది	వేగం	అత్యంత వేగం	అత్యంత వేగం	వేగం
Data freshness	తాజాది	తాజాది	build నాటిది	TTL ప్రకారం	తాజాది
SEO	బలహీనం	బలం	బలం	బలం	బలం
Server cost	తక్కువ	ఎక్కువ	సున్నా	తక్కువ	ఎక్కువ
ఎప్పుడు	dashboards, internal tools (login వెనుక)	personalized pages, auth-heavy	blogs, docs, marketing	e-commerce catalog	data-heavy pages

Real-life Scenario

> **CSR = ఖాళీ ప్లేటు + వంట సామాగ్రి ఇచ్చి "నువ్వే వండుకో" అనడం.** వేగంగా ఇస్తారు కానీ తినడానికి చాలా సమయం. **SSR = order చేసినప్పుడు వండి ఇవ్వడం.** కొంచెం వేచి ఉండాలి, కానీ వచ్చినది వెంటనే తినొచ్చు. **SSG = ముందే వండి pack చేసి పెట్టడం (frozen meal).** అత్యంత వేగం, కానీ తాజాది కాదు. **ISR = ముందే వండినది ఇస్తూ, ప్రతి గంటకి కొత్తది వండి replace చేయడం. Streaming = starter, main course, dessert ఒక్కొక్కటిగా వచ్చినప్పుడు పంపడం** – అన్నీ సిద్ధమయ్యేదాకా ఆగకుండా.

Hydration – ఏమిటి, ఎందుకు ఖరీదు

Hydration = server పంపిన static HTML కి React event listeners attach చేసి, "సజీవం" చేయడం.

```
1. Server → HTML పంపుతుంది (user కి content కనిపిస్తుంది, కానీ click పని చేయదు)
2. Browser → JS bundle download
3. React → HTML tree ని walk చేసి, virtual tree తో match చేసి, listeners attach
(hydration)
4. ఇప్పుడు page interactive ✅
```

సమస్య: 2 & 3 మధ్య **"uncanny valley"** – content కనిపిస్తుంది కానీ clicks పని చేయవు. బలహీన device లో ఇది సెకన్లు ఉండొచ్చు.

```
// React 18 — hydrateRoot
import { hydrateRoot } from "react-dom/client";
hydrateRoot(document.getElementById("root"), <App />);
```

Hydration mismatch – అత్యంత సాధారణ SSR bug

```
// ❌ Server లో ఒక value, client లో వేరే → mismatch error
function Bad() {
  return <p>{new Date().toLocaleString()}</p>;        // server time ≠ client time
  return <p>{Math.random()}</p>;                       // ❌
  return <p>{window.innerWidth}</p>;                   // ❌ server లో window లేదు
  return <p>{localStorage.getItem("name")}</p>;        // ❌
}

// ✅ పరిష్కారం 1 — mount తర్వాత మాత్రమే render
function Good() {
  const [mounted, setMounted] = useState(false);
  useEffect(() => setMounted(true), []);
  return <p>{mounted ? new Date().toLocaleString() : null}</p>;
}

// ✅ పరిష్కారం 2 — suppressHydrationWarning (ఒకే text node కి మాత్రమే)
<time suppressHydrationWarning>{new Date().toISOString()}</time>

// ✅ పరిష్కారం 3 — client-only component
const Chart = dynamic(() => import("./Chart"), { ssr: false });   // Next.js
```

ఇతర mismatch కారణాలు: invalid HTML nesting (`<div>` inside `<p>`), browser extensions DOM ని మార్చడం, `typeof window` checks, locale-dependent formatting.

Streaming SSR + Suspense (React 18)

```
// server.js
import { renderToPipeableStream } from "react-dom/server";

app.get("*", (req, res) => {
  const { pipe, abort } = renderToPipeableStream(<App url={req.url} />, {
    bootstrapScripts: ["/client.js"],
    onShellReady() {                    // shell (Suspense బయటిది) సిద్ధం
      res.setHeader("Content-Type", "text/html");
      pipe(res);                        // ⚡ మిగిలినది ఇంకా వండుతూనే HTML పంపడం
మొదలు
    },
    onShellError(err) { res.statusCode = 500; res.send("<h1>Error</h1>"); },
    onError(err) { console.error(err); },
  });
  setTimeout(abort, 10_000);            // timeout
});
```

```
function App() {
  return (
    <Layout>
      <Header />                                    {/* shell — వెంటనే */}
      <Suspense fallback={<Skeleton />}>
        <SlowFeed />                                {/* ready అయినప్పుడు stream */}
      </Suspense>
    </Layout>
  );
}
```

Selective hydration: streaming తో React ఏ భాగం ముందు hydrate చేయాలో ఎంచుకోగలదు – user ఒక భాగం మీద click చేస్తే, React **ఆ భాగాన్ని ముందు** hydrate చేస్తుంది.

`renderToString` vs `renderToPipeableStream`

	`renderToString`	`renderToPipeableStream`
Blocking	✅ మొత్తం tree ready అయ్యేవరకు	❌ chunks గా
Suspense	❌ support లేదు	✅
TTFB	నెమ్మది	వేగం
React 18+	legacy	✅ recommended

Key Points

- CSR/SSR/SSG/ISR/Streaming – ప్రతి page కి సరైనది ఎంచుకోవాలి (ఒకే app లో కలపొచ్చు).

- **Hydration** = HTML + JS → interactive; ఖరీదైనది.
- **Mismatch** కారణాలు: time, random, browser APIs, invalid HTML.
- **Streaming SSR + Suspense + selective hydration** = React 18 యొక్క పెద్ద SSR ముందడుగు.

Interview దృష్టి

- *"SSR vs CSR ఎప్పుడు?"* → SEO, TTFB/LCP, personalization, server cost, team complexity.
- *"Hydration అంటే ఏమిటి, ఖరీదు ఎందుకు?"* → tree walk + listener attach + double data (HTML + JSON payload).
- *"Hydration mismatch ఎలా fix చేస్తావు?"* → deterministic render, mount-gate, `suppressHydrationWarning`, `ssr: false`.

---

49. React Server Components (RSC) Deep

వివరణ

RSC = server లో మాత్రమే run అయ్యే components. వాటి JavaScript **client కి ఎప్పుడూ పంపబడదు**. React 19 లో stable; Next.js App Router లో default.

```
// app/page.tsx — Server Component (default, 'use client' లేదు)
import db from "@/lib/db";                    // ✅ database నేరుగా! bundle లోకి వెళ్ళదు

export default async function ProductsPage({ searchParams }) {
  const products = await db.product.findMany({ where: { active: true } });   // ✅
async component
  return (
    <ul>
      {products.map((p) => (
        <li key={p.id}>
          {p.name}
          <AddToCartButton productId={p.id} />     {/* client component */}
        </li>
      ))}
    </ul>
  );
}
```

```
// components/AddToCartButton.tsx — Client Component
"use client";                                  // 🔑 ఈ directive ఫైల్ మొదటి line లో
import { useState } from "react";

export function AddToCartButton({ productId }) {
  const [adding, setAdding] = useState(false);
  return <button onClick={() => addToCart(productId)}>Add</button>;
}
```

Real-life Scenario

> **RSC = restaurant kitchen vs dining table.** పాత విధానంలో వాళ్ళు మీ table కి **వంటగది మొత్తం** (recipes, గిన్నెలు, పచ్చి కూరగాయలు = JS bundle + data fetching code) పంపి "నువ్వే వండుకో" అనేవాళ్ళు. RSC లో – వంట వంటగదిలోనే (server) జరుగుతుంది, **తయారైన dish (rendered output) మాత్రమే** table కి వస్తుంది. మీకు కావాల్సినది కేవలం fork & spoon (interactive client components).

Server vs Client Components

	Server Component	**Client Component**
ఎక్కడ run	**server మాత్రమే**	server (SSR) + browser
Bundle కి వెళ్తుందా	❌ **సున్నా JS**	✅
`async/await`	✅ నేరుగా	❌
DB/filesystem/secrets	✅	❌
`useState`, `useEffect`	❌	✅
Event handlers (`onClick`)	❌	✅
Browser APIs	❌	✅
Context	❌ (provide చేయలేదు)	✅

Composition నియమాలు (అత్యంత ముఖ్యం)

```
// ✅ Server component → client component ని import చేయొచ్చు
// ❌ Client component → server component ని import చేయలేదు
"use client";
import ServerComp from "./ServerComp";      // ❌ ఇది client component గా మారిపోతుంది!

// ✅ కానీ children గా pass చేయొచ్చు! (కీలకమైన pattern)
// app/page.tsx (server)
<ClientWrapper>
  <ServerComponent />        {/* ✅ ఇది server లోనే render అవుతుంది */}
</ClientWrapper>

// ClientWrapper.tsx
"use client";
export function ClientWrapper({ children }) {
  const [open, setOpen] = useState(false);
  return <div>{open && children}</div>;      // children ఇప్పటికే rendered output
}
```

`'use client'` **= boundary, కాదు label.** ఒక file లో `'use client'` రాస్తే, అది import చేసే **అన్ని modules** client bundle లోకి వెళ్తాయి. అందుకే దాన్ని **tree లో వీలైనంత కిందికి** (leaf components కి) పెట్టాలి.

Server Actions (`'use server'`)

Client నుండి server function ని నేరుగా call చేయడం – API route రాయనవసరం లేదు.

```
// app/actions.ts
"use server";
import { revalidatePath } from "next/cache";
import { z } from "zod";

const schema = z.object({ title: z.string().min(1) });

export async function createPost(prevState, formData) {
  const parsed = schema.safeParse({ title: formData.get("title") });
  if (!parsed.success) return { error: "Title required" };

  const session = await auth();                          // 🔑 ప్రతి action లో authorize!
  if (!session) return { error: "Unauthorized" };

  await db.post.create({ data: { ...parsed.data, userId: session.userId } });
  revalidatePath("/posts");                              // cache invalidate
  return { success: true };
}
```

```
// app/new-post/page.tsx
"use client";
import { useActionState } from "react";
import { createPost } from "../actions";

export default function NewPost() {
  const [state, action, pending] = useActionState(createPost, {});
  return (
    <form action={action}>
      <input name="title" />
      <button disabled={pending}>{pending ? "Saving…" : "Create"}</button>
      {state.error && <p>{state.error}</p>}
    </form>
  );
}
```

> 🔒 **Security:** Server Actions **public HTTP endpoints** – client ఏదైనా payload పంపొచ్చు. **ప్రతి action లో authentication + authorization + input validation తప్పనిసరి.** "ఇది server code కాబట్టి safe" అనుకోవడం తీవ్రమైన తప్పు.

RSC payload – ఎలా పని చేస్తుంది

Server components HTML కాదు – ఒక ప్రత్యేక **serialized format** (RSC payload) గా stream అవుతాయి:

```
0:["$","div",null,{"children":[["$","h1",null,{"children":"Products"}],
["$","$L1",null,{"productId":42}]]}]
1:I["./AddToCartButton.js",["chunk1"],"AddToCartButton"]      ← client component
reference
```

Client React ఈ payload ని చదివి, existing tree లోకి merge చేస్తుంది – **full page reload లేకుండా server component ని re-render చేయొచ్చు** (state ని కోల్పోకుండా!). ఇదే RSC యొక్క పెద్ద మ్యాజిక్.

RSC లాభాలు & పరిమితులు

✅ లాభాలు	❌ పరిమితులు
Zero JS for non-interactive UI	Framework అవసరం (Next.js/Waku)
Data fetching component లోనే (waterfall తగ్గింపు)	Mental model కొత్తది (`'use client'` boundaries)
Secrets/DB నేరుగా (API layer అవసరం లేదు)	Server infrastructure అవసరం
Large libraries server లోనే (markdown, syntax highlight)	Runtime CSS-in-JS incompatible
Automatic code splitting at client boundaries	Props **serializable** ఉండాలి (functions, Dates class instances వద్దు)

Gotchas (సాధారణ తప్పులు)

- `'use client'` **ని root layout లో పెట్టడం** → మొత్తం app client bundle లోకి. **Leaf కి దగ్గరగా** ఉంచాలి.
- **Server component కి function/class instance ని prop గా పంపడం** → serialization error. (Server Actions మాత్రమే exception.)
- **Server Action లో auth check మర్చిపోవడం** → security hole.
- **Server component లో** `useState/useEffect` → error.
- **Client component లో** `async` → support లేదు (`use()` + Suspense వాడాలి).
- **Server component లో environment secret ని client కి prop గా పంపడం** → payload లో leak అవుతుంది!
- **Server components ని SSR అనుకోవడం** – వేరు: SSR = HTML generate, RSC = component-level server rendering + client bundle exclusion.

Key Points

- RSC = **server-only components, zero client JS**.
- `'use client'` = **boundary**; కిందికి తరలించాలి.
- Server → client import ✅ ; client → server import ❌ (కానీ `children` గా pass ✅).
- **Server Actions** = public endpoints → validate + authorize తప్పనిసరి.
- Props serializable ఉండాలి.
- RSC payload = streaming, mergeable, state-preserving.

Interview దృష్టి

- *"RSC vs SSR?"* → SSR = HTML generation (component ఇంకా client లో hydrate అవుతుంది); RSC = component ఎప్పటికీ client కి రాదు.
- *"'use client' ఏం చేస్తుంది?"* → module boundary; ఆ subtree client bundle లోకి.

- *"RSC లో data fetching?"* → component లోనే `await`; parallel fetching కి `Promise.all`; Suspense తో streaming.

---

50. Next.js App Router

వివరణ

Next.js = React యొక్క production framework – routing, SSR/SSG/ISR, RSC, bundling, image/font optimization, API routes, caching అన్నీ ఇస్తుంది. 2026 లో React docs కూడా కొత్త apps కి framework వాడమని సిఫార్సు చేస్తాయి.

File-based routing (App Router)

```
app/
  layout.tsx              → root layout (అన్ని pages కి; <html>, <body> ఇక్కడ)
  page.tsx                → "/"
  loading.tsx             → automatic Suspense boundary
  error.tsx               → automatic Error Boundary ('use client' తప్పనిసరి)
  not-found.tsx           → 404
  global-error.tsx        → root layout errors
  products/
    page.tsx              → "/products"
    layout.tsx            → products routes కి nested layout
    [id]/
      page.tsx            → "/products/42"   (params.id)
    [...slug]/            → catch-all "/products/a/b/c"
    [[...slug]]/          → optional catch-all
  (marketing)/            → route group – URL లో కనిపించదు (organization కోసం)
    about/page.tsx        → "/about"
  @modal/                 → parallel route (slot)
  api/
    users/route.ts        → API endpoint "/api/users" (GET/POST exports)
```

Data fetching & caching (Next 15 semantics)

```
// Server component లో నేరుగా fetch
async function Page() {
  // Next.js 15: fetch default uncached (no-store)
  const dynamic = await fetch("https://api.example.com/live");              //
ప్రతిసారి fresh
  const cached  = await fetch("https://api.example.com/config",
                              { cache: "force-cache" });                    //
build/first request లో cache
  const isr     = await fetch("https://api.example.com/products",
                              { next: { revalidate: 60 } });                // 60
సెకన్ల ISR
  const tagged  = await fetch("https://api.example.com/posts",
                              { next: { tags: ["posts"] } });               //
tag ఆధారిత invalidation
```

```
  // ✅ Parallel — waterfall నివారణ
  const [user, posts] = await Promise.all([getUser(id), getPosts(id)]);
}

// Route segment config
export const revalidate = 3600;          // ఈ route కి ISR
export const dynamic = "force-dynamic";  // ఎప్పుడూ SSR
export const dynamic = "force-static";   // ఎప్పుడూ static
```

```
// On-demand revalidation (webhook/CMS update తర్వాత)
import { revalidatePath, revalidateTag } from "next/cache";
revalidateTag("posts");
revalidatePath("/products/[id]", "page");
```

Caching layers (Next.js లో గందరగోళానికి ప్రధాన కారణం)

Layer	ఏమి cache చేస్తుంది	ఎక్కడ	ఎలా clear
Request Memoization	ఒకే render లో duplicate `fetch` calls	server, per-request	automatic
Data Cache	`fetch` results	server, persistent	`revalidateTag/Path`, `cache: "no-store"`
Full Route Cache	rendered HTML/RSC payload	server, build/runtime	revalidate, dynamic
Router Cache	client-side RSC payload	browser, session	`router.refresh()`, navigation

Streaming & loading UI

```
// app/dashboard/loading.tsx — automatic <Suspense fallback>
export default function Loading() { return <DashboardSkeleton />; }

// Granular streaming
export default function Page() {
  return (
    <>
      <Header />                                      {/* వెంటనే */}
      <Suspense fallback={<ChartSkeleton />}><SlowChart /></Suspense>
      <Suspense fallback={<TableSkeleton />}><SlowTable /></Suspense>
    </>
  );
}
```

Metadata & SEO

```
// Static
export const metadata = {
  title: "Products | MyShop",
  description: "Browse our catalog",
```

```
  openGraph: { title: "Products", images: ["/og.png"] },
  alternates: { canonical: "https://shop.com/products" },
};

// Dynamic
export async function generateMetadata({ params }) {
  const product = await getProduct(params.id);
  return { title: product.name, description: product.summary };
}

// Static params (SSG)
export async function generateStaticParams() {
  const products = await getAllProducts();
  return products.map((p) => ({ id: String(p.id) }));
}
```

Route handlers & middleware

```
// app/api/users/route.ts
import { NextResponse } from "next/server";

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const users = await db.user.findMany({ take: Number(searchParams.get("limit") ??
10) });
  return NextResponse.json(users, { headers: { "Cache-Control": "s-maxage=60" } });
}
export async function POST(request: Request) {
  const body = await request.json();
  return NextResponse.json(await db.user.create({ data: body }), { status: 201 });
}
```

```
// middleware.ts — ప్రతి request కి (edge runtime లో run అవుతుంది)
import { NextResponse } from "next/server";
export function middleware(request) {
  const token = request.cookies.get("session")?.value;
  if (!token && request.nextUrl.pathname.startsWith("/dashboard")) {
    return NextResponse.redirect(new URL("/login", request.url));
  }
  return NextResponse.next();
}
export const config = { matcher: ["/dashboard/:path*", "/admin/:path*"] };
```

Built-in optimizations

```
import Image from "next/image";
import { Inter } from "next/font/google";
import dynamic from "next/dynamic";
import Link from "next/link";

<Image src="/hero.jpg" alt="" width={1200} height={630} priority />   {/* WebP/AVIF,
lazy, CLS-safe */}
const inter = Inter({ subsets: ["latin"], display: "swap" });          {/* self-
hosted, no layout shift */}
```

```
const Chart = dynamic(() => import("./Chart"), { ssr: false, loading: () => <Skeleton
/> });
<Link href="/about" prefetch>About</Link>                    {/* viewport
లోకి వస్తే prefetch */}
```

App Router vs Pages Router

	Pages Router (legacy)	App Router (2026 default)
Data	`getServerSideProps`, `getStaticProps`	async server components, `fetch`
Layouts	`_app.js` hacks	nested `layout.tsx` ✅
RSC	❌	✅
Streaming	పరిమితం	✅
Loading/Error UI	manual	file conventions ✅

Gotchas (సాధారణ తప్పులు)

- **`error.tsx` లో `'use client'` మర్చిపోవడం** → error boundaries client components కావాలి.
- **Server component లో `useSearchParams` / `usePathname`** → client hooks; server లో `searchParams` prop వాడాలి.
- **`searchParams` వాడితే route dynamic అవుతుందని తెలియకపోవడం** → static optimization పోతుంది.
- **Client component లో secrets** – `NEXT_PUBLIC_` prefix ఉన్నవి **అన్నీ public**.
- **Caching గందరగోళం** – Next 15 లో `fetch` default uncached; 14 లో cached. Version-specific behaviour తెలియాలి.
- **`middleware` లో heavy logic** – edge runtime, ప్రతి request; DB calls వద్దు.
- **`router.push` తర్వాత data stale** → `router.refresh()` లేదా `revalidatePath`.

Key Points

- App Router = **RSC + nested layouts + streaming + file conventions**.
- 4 caching layers – ఏది ఎప్పుడు clear అవుతుందో తెలియడం కీలకం.
- `loading.tsx` / `error.tsx` = automatic Suspense/ErrorBoundary.
- Middleware = edge-level auth/redirects (heavy work వద్దు).
- Image/Font/Link/dynamic = built-in performance.

Interview దృష్టి

- *"Next.js ఎందుకు React కంటే?"* → routing, SSR/SSG/ISR, RSC, image/font optimization, API routes – decisions ready.
- *"App Router caching explain చెయ్యి"* → 4 layers table.
- *"Server Actions vs API routes?"* → colocation, type safety, progressive enhancement vs public REST API/third-party consumers.

Part 10 — TypeScript + React

51. TypeScript with React

వివరణ

2026 లో production React = TypeScript. Props, state, events, hooks, generics — అన్నీ type చేయడం SDE2/SSE కి తప్పనిసరి నైపుణ్యం.

Props typing

```
// 1. Basic
type ButtonProps = {
  label: string;
  count?: number;                         // optional
  variant: "primary" | "secondary";       // union — typo compile error
  onClick: (e: React.MouseEvent<HTMLButtonElement>) => void;
  children: React.ReactNode;              // ఏ renderable అయినా
};

function Button({ label, count = 0, variant, onClick, children }: ButtonProps) { }

// 2. HTML attributes ని extend చేయడం (wrapper components కి)
type Props = React.ComponentProps<"button"> & { variant: "primary" | "danger" };
function Button({ variant, ...rest }: Props) {
  return <button className={variant} {...rest} />;    // onClick, disabled, type… అన్నీ
typed ✅
}

// 3. ఇంకో component యొక్క props ని reuse
type IconProps = React.ComponentProps<typeof Icon>;

// 4. Discriminated union — impossible props ని అసాధ్యం చేయడం
type AlertProps =
  | { severity: "error"; error: Error; onRetry: () => void }
  | { severity: "info"; message: string };
// severity: "info" ఇచ్చినప్పుడు `error` prop ఇవ్వలేం ✅
```

Children రకాలు

```
children: React.ReactNode          // ✅ default choice — elements, strings, numbers,
null, arrays
children: React.ReactElement       // ఒకే element మాత్రమే
children: (data: T) => React.ReactNode    // render prop
children: string                   // text మాత్రమే
```

Hooks typing

```
// useState — inference సాధారణంగా సరిపోతుంది
const [count, setCount] = useState(0);                   // number
const [user, setUser] = useState<User | null>(null);     // ✅ explicit అవసరం
const [items, setItems] = useState<Item[]>([]);          // [] నుండి never[] వస్తుంది →
explicit

// useRef — రెండు రకాలు
const inputRef = useRef<HTMLInputElement>(null);         // DOM ref (readonly
current)
const timerRef = useRef<number | null>(null);            // mutable box
const countRef = useRef<number>(0);                      // initial ఉంటే mutable

// useReducer — discriminated union actions
type State = { count: number; status: "idle" | "loading" };
type Action =
  | { type: "increment"; by: number }
  | { type: "reset" }
  | { type: "setStatus"; status: State["status"] };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "increment": return { ...state, count: state.count + action.by };  //
action.by typed ✅
    case "reset": return { count: 0, status: "idle" };
    case "setStatus": return { ...state, status: action.status };
    default: {
      const _exhaustive: never = action;      // ✅ కొత్త action type add చేస్తే compile
error
      return state;
    }
  }
}

// Context — null-safe pattern
const AuthContext = createContext<AuthValue | null>(null);
export function useAuth(): AuthValue {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be inside AuthProvider");
  return ctx;                                 // ✅ ఇక్కడ నుండి non-null typed
}

// Custom hook — tuple return కి `as const`
function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => setOn((v) => !v), []);
  return [on, toggle] as const;               // ✅ [boolean, () => void] — లేకపోతే
(boolean | fn)[]
}
```

Event types

```
onChange: (e: React.ChangeEvent<HTMLInputElement>) => void
onSubmit: (e: React.FormEvent<HTMLFormElement>) => void
onClick:  (e: React.MouseEvent<HTMLButtonElement>) => void
onKeyDown: (e: React.KeyboardEvent<HTMLInputElement>) => void
```

```
onFocus:  (e: React.FocusEvent<HTMLInputElement>) => void
onDrop:   (e: React.DragEvent<HTMLDivElement>) => void
```

Generic components

```
// Reusable typed list
type ListProps<T> = {
  items: T[];
  renderItem: (item: T, index: number) => React.ReactNode;
  keyExtractor: (item: T) => string;
};

function List<T>({ items, renderItem, keyExtractor }: ListProps<T>) {
  return <ul>{items.map((it, i) => <li key={keyExtractor(it)}>{renderItem(it, i)}
</li>)}</ul>;
}

<List
  items={users}                                  // User[]
  keyExtractor={(u) => u.id}                     // u: User ✅ inferred
  renderItem={(u) => <span>{u.name}</span>}
/>

// Polymorphic component (`as` prop) — advanced
type PolymorphicProps<E extends React.ElementType> = {
  as?: E;
  children: React.ReactNode;
} & Omit<React.ComponentPropsWithoutRef<E>, "as" | "children">;

function Text<E extends React.ElementType = "span">({ as, ...rest }:
PolymorphicProps<E>) {
  const Component = as || "span";
  return <Component {...rest} />;
}
<Text as="a" href="/x">link</Text>              // ✅ href typed (anchor కి మాత్రమే
అనుమతి)
```

Utility types (తరచూ వాడేవి)

```
Partial<T>              // అన్ని props optional
Required<T>             // అన్నీ mandatory
Pick<T, "a" | "b">      // కొన్ని మాత్రమే
Omit<T, "a">            // కొన్ని తీసేయడం
Record<string, User>    // dictionary
NonNullable<T>          // null/undefined తీసేయడం
Awaited<ReturnType<typeof fetchUser>>    // async function యొక్క data type
React.PropsWithChildren<P>               // P + children
```

API types — runtime validation తో

```
// ❌ కేవలం type assertion — runtime లో ఏ హామీ లేదు
const user = await res.json() as User;      // API మారితే silent crash

// ✅ Zod — runtime validation + type inference
```

```
const UserSchema = z.object({ id: z.string(), name: z.string(), email:
z.string().email() });
type User = z.infer<typeof UserSchema>;

async function getUser(id: string): Promise<User> {
  const res = await fetch(`/api/users/${id}`);
  return UserSchema.parse(await res.json());   // ✅ తప్పుంటే ఇక్కడే పట్టుబడుతుంది
}
```

Gotchas (సాధారణ తప్పులు)

- `any` **వాడటం** – type safety పూర్తిగా పోతుంది. తెలియకపోతే `unknown` + narrowing.
- `React.FC` **వాడటం** – పాత versions లో implicit `children` ఇచ్చేది, generics కష్టం; **plain function + typed props** మేలు.
- `useState([])` → `never[]`; `useState<Item[]>([])`.
- `as` **assertions అతిగా** – compiler ని మోసం చేయడం; validation వాడాలి.
- **Event type తప్పు** → `e.target.value` కి error. `ChangeEvent<HTMLInputElement>` వాడాలి.
- `e.target` **vs** `e.currentTarget` **typing** – `currentTarget` ఎప్పుడూ సరైన type.
- `strict: false` – tsconfig లో `strict: true` తప్పనిసరి (`strictNullChecks` లేకపోతే TS విలువ సగం పోతుంది).

Key Points

- `React.ComponentProps<"button">` = HTML props extend చేయడానికి ఉత్తమ మార్గం.
- **Discriminated unions** = impossible props/states ని అసాధ్యం చేయడం.
- `as const` = tuple returns కి.
- Context: `createContext<T | null>(null)` + throwing hook.
- Runtime validation (**Zod**) + static types కలిపి – assertions కాదు.
- `React.FC` వద్దు; `strict: true` తప్పనిసరి.

Interview దృష్టి

- *"Generic component రాయి"* → typed `<List<T> />`.
- *"API response ని ఎలా type చేస్తావు?"* → Zod schema → `z.infer` → runtime parse.
- *"Discriminated union ఎందుకు?"* → invalid combinations ని compile time లో ఆపడం.

Part 11 – Testing

52. Testing React – RTL, Vitest, MSW, E2E

వివరణ – Testing philosophy

> **"The more your tests resemble the way your software is used, the more confidence they can give you."** – Kent C. Dodds (React Testing Library రచయిత)

అంటే: **implementation details ని test చేయకూడదు** (state variable పేరు, internal methods). **User ఏమి చూస్తాడు, ఏమి చేస్తాడు** – అదే test చేయాలి.

Testing pyramid (frontend కి trophy)

```
        /\        E2E (Playwright/Cypress) — తక్కువ, నెమ్మది, ఖరీదు, అత్యధిక నమ్మకం
       /  \
      /----\      Integration (RTL — అనేక components కలిసి) ← 🏆 ఎక్కువ ROI ఇక్కడే
     /      \
    /--------\    Unit (pure functions, hooks, reducers) — వేగం, చవక
   /  Static  \   TypeScript + ESLint
  /____________\
```

Setup (Vitest + RTL)

```
npm i -D vitest @testing-library/react @testing-library/user-event @testing-
library/jest-dom jsdom
```

```
// vitest.config.ts
export default defineConfig({
  test: { globals: true, environment: "jsdom", setupFiles: "./src/test/setup.ts",
css: true },
});
// src/test/setup.ts
import "@testing-library/jest-dom/vitest";
```

Code – component test

```
import { render, screen, waitFor, within } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { describe, it, expect, vi } from "vitest";

describe("LoginForm", () => {
  it("shows validation error for invalid email", async () => {
    const user = userEvent.setup();
```

```
    render(<LoginForm onSubmit={vi.fn()} />);

    await user.type(screen.getByLabelText(/email/i), "not-an-email");
    await user.click(screen.getByRole("button", { name: /sign in/i }));

    expect(await screen.findByText(/invalid email/i)).toBeInTheDocument();
  });

  it("submits valid credentials", async () => {
    const user = userEvent.setup();
    const onSubmit = vi.fn();
    render(<LoginForm onSubmit={onSubmit} />);

    await user.type(screen.getByLabelText(/email/i), "a@b.com");
    await user.type(screen.getByLabelText(/password/i), "password123");
    await user.click(screen.getByRole("button", { name: /sign in/i }));

    await waitFor(() =>
      expect(onSubmit).toHaveBeenCalledWith({ email: "a@b.com", password:
"password123" })
    );
  });
});
```

Query priority (RTL యొక్క ముఖ్య నియమం)

```
1. getByRole              ✅ ఉత్తమం — a11y tree; user & screen reader చూసేది
   getByLabelText         ✅ form fields కి
   getByPlaceholderText
   getByText              ✅ non-interactive content
   getByDisplayValue
2. getByAltText, getByTitle
3. getByTestId            ⚠️ చివరి ఎంపిక — ఇతర query సాధ్యం కానప్పుడు మాత్రమే
```

`getBy` vs `queryBy` vs `findBy`:

	దొరక్కపోతే	Async?	ఎప్పుడు
`getBy`	throw	❌	element ఉండాలి
`queryBy`	`null`	❌	**లేదని నిరూపించడానికి** (`expect(...).not.toBeInTheDocument()`)
`findBy`	throw (retry తర్వాత)	✅	async గా వచ్చేదానికి

API mocking — MSW (Mock Service Worker)

```
// mocks/handlers.js — network level mocking (fetch/axios ఏదైనా పని చేస్తుంది)
import { http, HttpResponse } from "msw";

export const handlers = [
  http.get("/api/users", () =>
    HttpResponse.json([{ id: "1", name: "Surya" }])),
```

```
  http.post("/api/users", async ({ request }) => {
    const body = await request.json();
    return HttpResponse.json({ id: "2", ...body }, { status: 201 });
  }),
];

// mocks/server.js
import { setupServer } from "msw/node";
export const server = setupServer(...handlers);

// setup.ts
beforeAll(() => server.listen({ onUnhandledRequest: "error" }));
afterEach(() => server.resetHandlers());
afterAll(() => server.close());
```

```
// Error case ని ఆ test లో మాత్రమే override చేయడం
it("shows error when API fails", async () => {
  server.use(http.get("/api/users", () => new HttpResponse(null, { status: 500 })));
  render(<UserList />);
  expect(await screen.findByRole("alert")).toHaveTextContent(/failed/i);
});
```

> **MSW ఎందుకు `vi.mock("axios")` కంటే మేలు:** implementation (axios/fetch) మారినా tests మారవు; అదే handlers ని dev mode లో, Storybook లో, E2E లో reuse చేయొచ్చు.

Custom hooks testing

```
import { renderHook, act, waitFor } from "@testing-library/react";

it("useCounter increments", () => {
  const { result } = renderHook(() => useCounter(5));
  expect(result.current.count).toBe(5);
  act(() => result.current.increment());
  expect(result.current.count).toBe(6);
});

// Provider అవసరమైతే wrapper
const wrapper = ({ children }) => (
  <QueryClientProvider client={new QueryClient({ defaultOptions: { queries: { retry:
false } } })}>
    {children}
  </QueryClientProvider>
);
const { result } = renderHook(() => useUsers(), { wrapper });
await waitFor(() => expect(result.current.isSuccess).toBe(true));
```

Custom render (providers తో)

```
// test/utils.jsx — ప్రతి test లో providers repeat చేయకుండా
function AllProviders({ children }) {
  const qc = new QueryClient({ defaultOptions: { queries: { retry: false } } });
  return (
```

```
    <MemoryRouter>
      <QueryClientProvider client={qc}>
        <ThemeProvider>{children}</ThemeProvider>
      </QueryClientProvider>
    </MemoryRouter>
  );
}
export function renderWithProviders(ui, options) {
  return render(ui, { wrapper: AllProviders, ...options });
}
export * from "@testing-library/react";
export { userEvent };
```

E2E – Playwright

```
import { test, expect } from "@playwright/test";

test("user can complete checkout", async ({ page }) => {
  await page.goto("/products");
  await page.getByRole("button", { name: "Add to cart" }).first().click();
  await page.getByRole("link", { name: "Cart" }).click();
  await expect(page.getByText("1 item")).toBeVisible();

  await page.getByRole("button", { name: "Checkout" }).click();
  await page.getByLabel("Card number").fill("4242424242424242");
  await page.getByRole("button", { name: "Pay" }).click();

  await expect(page.getByRole("heading", { name: "Order confirmed" })).toBeVisible();
});
```

ఏం test చేయాలి / చేయకూడదు

✅ Test చేయాలి	❌ చేయకూడదు
User flows (login, checkout)	Implementation details (state variable names)
Conditional rendering	CSS classes (behaviour కాదు)
Error/loading/empty states	Third-party libraries
Form validation & submission	Trivial props pass-through
Accessibility (roles, labels)	Snapshot everything (brittle)
Business logic (reducers, utils)	

Gotchas (సాధారణ తప్పులు)

- `getByTestId` **ని default గా వాడటం** → a11y ని test చేయదు; role/label queries మేలు.
- `fireEvent` **vs** `userEvent` – `userEvent` నిజమైన user behaviour ని simulate చేస్తుంది (focus, keydown, keyup, input, change). `userEvent` వాడాలి.
- `await user.click(...)` **లో await మర్చిపోవడం** → flaky tests (v14+ లో అన్నీ async).

- `act()` **warning** – state update ని `act` బయట చేయడం; `waitFor` / `findBy` వాడితే సాధారణంగా పోతుంది.
- **Test లో `setTimeout` / `sleep`** → flaky. `findBy` / `waitFor` వాడాలి.
- **Snapshot tests అతిగా** – ప్రతి UI మార్పుకి fail, ఎవరూ చదవరు; targeted assertions మేలు.
- **React Query tests లో `retry: true`** → failure tests 3 రెట్లు నెమ్మది.

Key Points

- **User లా test చేయాలి** – implementation కాదు.
- Query priority: **role** → **label** → **text** → **testId**.
- `getBy` / `queryBy` (absence) / `findBy` (async).
- **MSW** = network-level mocking (dev/test/Storybook లో reuse).
- Integration tests = అత్యధిక ROI (testing trophy).
- E2E = critical flows కి మాత్రమే (నెమ్మది, ఖరీదు).

Interview దృష్టి

- *"React component ని ఎలా test చేస్తావు?"* → RTL + role queries + user-event + MSW; user behaviour focus.
- *"Implementation details అంటే?"* → state names, internal functions, CSS classes; refactor చేస్తే fail అయ్యే ఏదైనా.
- *"Coverage 100% కావాలా?"* → కాదు; critical paths + edge cases; coverage ఒక signal, లక్ష్యం కాదు.

Part 12 – Quality, Security & Accessibility

53. Accessibility (a11y) in React

వివరణ

a11y = అందరూ (screen readers, keyboard-only users, low vision, motor disability) మీ app ని వాడగలగడం. ఇది nice-to-have కాదు – భారత్‌లో RPwD Act, US లో ADA/Section 508, EU లో EAA – చట్టపరమైన అవసరం. Interview లో a11y గురించి మాట్లాడితే **సీనియారిటీ signal**.

1. Semantic HTML – 80% పని ఇక్కడే

```
// ❌ div soup — screen reader కి ఏమీ అర్థం కాదు, keyboard పని చేయదు
<div onClick={submit}>Submit</div>
<div className="heading">Title</div>

// ✅ Semantic elements — free a11y (focus, Enter/Space, role, announcement)
<button onClick={submit}>Submit</button>
<h1>Title</h1>
<nav><ul><li><a href="/">Home</a></li></ul></nav>
<main>...</main>
<form onSubmit={...}>...</form>
```

> **నియమం:** *"ఏ ARIA లేకపోవడం, తప్పు ARIA కంటే మేలు."* ముందు సరైన HTML element వాడండి; ARIA ని అది సాధ్యం కానప్పుడు మాత్రమే.

2. Forms & labels

```
// ✅ ప్రతి input కి label
const id = useId();
<label htmlFor={id}>Email</label>
<input id={id} type="email" aria-describedby={`${id}-error`} aria-invalid={!!error}
required />
{error && <span id={`${id}-error`} role="alert">{error}</span>}

// Label కనిపించకూడదంటే — placeholder కాదు!
<label htmlFor={id} className="sr-only">Search</label>
// లేదా <input aria-label="Search" />
```

```
/* screen-reader-only utility */
.sr-only {
  position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px;
```

```
  overflow: hidden; clip: rect(0,0,0,0); white-space: nowrap; border: 0;
}
```

3. Keyboard navigation

```
// ప్రతి interactive element keyboard తో చేరుకోగలగాలి
<button>OK</button>                          {/* ✅ automatic */}
<div role="button" tabIndex={0}              {/* custom అయితే మనమే అన్నీ చేయాలి */}
     onClick={h}
     onKeyDown={(e) => { if (e.key === "Enter" || e.key === " ") {
e.preventDefault(); h(); } }}>
  OK
</div>

// tabIndex నియమాలు
tabIndex={0}    // natural tab order లోకి
tabIndex={-1}   // programmatic focus మాత్రమే (tab తో రాదు) — modals, error summaries
tabIndex={5}    // ❌ ఎప్పుడూ positive వాడొద్దు — tab order పాడవుతుంది
```

Focus visible ని ఎప్పుడూ తీసేయకూడదు:

```
/* ❌ */ *:focus { outline: none; }
/* ✅ */ :focus-visible { outline: 2px solid var(--brand); outline-offset: 2px; }
```

4. ARIA – అవసరమైనప్పుడు

```
// Modal
<div role="dialog" aria-modal="true" aria-labelledby="t" aria-describedby="d">
  <h2 id="t">Confirm</h2><p id="d">Are you sure?</p>
</div>

// Live regions — dynamic content ని announce చేయడం
<div role="status" aria-live="polite">{count} items found</div>       {/* నెమ్మదిగా */}
<div role="alert" aria-live="assertive">{error}</div>                  {/* వెంటనే */}

// State
<button aria-expanded={open} aria-controls="menu">Menu</button>
<button aria-pressed={active}>Bold</button>
<div aria-busy={loading} aria-hidden={decorative}>...</div>
```

5. SPA-specific problems

```
// (a) Route మారినప్పుడు screen reader కి తెలియదు (page reload లేదు)
function RouteAnnouncer() {
  const { pathname } = useLocation();
  const [msg, setMsg] = useState("");
  useEffect(() => { setMsg(`Navigated to ${document.title}`); }, [pathname]);
  return <div role="status" aria-live="polite" className="sr-only">{msg}</div>;
}

// (b) Route మారితే focus reset
useEffect(() => { mainRef.current?.focus(); }, [pathname]);   // <main tabIndex={-1}
```

```
ref={mainRef}>

// (c) Skip link — keyboard users nav ని దాటవేయడానికి
<a href="#main" className="skip-link">Skip to main content</a>

// (d) Modal focus trap + restore — Topic 34
```

6. Images, icons, motion

```
<img src="/chart.png" alt="Sales increased 20% in Q4" />   {/* అర్థవంతమైన alt */}
<img src="/decoration.png" alt="" />                        {/* decorative → ఖాళీ alt
*/}
<button aria-label="Close"><XIcon aria-hidden="true" /></button>

// Reduced motion గౌరవించడం
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation-duration: 0.01ms !important; transition-
duration: 0.01ms !important; }
}
```

Testing a11y

```
npm i -D eslint-plugin-jsx-a11y jest-axe @axe-core/react
```

```
// Automated (30-40% సమస్యలు మాత్రమే పట్టుకుంటుంది)
import { axe } from "jest-axe";
it("has no a11y violations", async () => {
  const { container } = render(<Form />);
  expect(await axe(container)).toHaveNoViolations();
});

// Manual testing తప్పనిసరి:
// 1. Mouse లేకుండా Tab తో మొత్తం app వాడండి
// 2. Screen reader (macOS VoiceOver: Cmd+F5, Windows NVDA)
// 3. Browser zoom 200%
// 4. Color contrast checker (4.5:1 normal text, 3:1 large)
```

Gotchas (సాధారణ తప్పులు)

- `outline: none` — keyboard users కి focus కనిపించదు.
- `<div onClick>` — keyboard/screen reader కి అందదు.
- **Placeholder ని label గా వాడటం** — typing మొదలైతే మాయం, contrast తక్కువ.
- **Color మాత్రమే information carrier** — color-blind users కి పోతుంది (icon/text కూడా ఇవ్వాలి).
- `aria-label` **ని native text ఉన్న element కి** → screen reader రెండూ చదవడం/overwrite.
- **Modal లో focus trap లేకపోవడం** — tab background లోకి వెళ్తుంది.
- **Auto-focus అతిగా** — screen reader users ని గందరగోళపరుస్తుంది.
- **Dynamic content ని live region లేకుండా update చేయడం** — screen reader కి ఏమీ తెలియదు.

Key Points

- **Semantic HTML మొదట**, ARIA తర్వాత.
- ప్రతి input కి **label**; ప్రతి interactive element **keyboard-reachable**.
- `:focus-visible` ఎప్పుడూ ఉంచాలి.
- SPA: route announcement + focus management + skip link.
- Automated (axe) + **manual (keyboard, screen reader)** రెండూ.

Interview దృష్టి

- *"React app ని accessible ఎలా చేస్తావు?"* → semantic HTML, labels, keyboard, focus management, live regions, testing.
- *"Modal ని accessible గా రాయి"* → role/aria-modal/labelledby + focus trap + ESC + restore focus.

---

54. Security in React

వివరణ

Frontend security = **"client ని ఎప్పుడూ నమ్మకూడదు"** అనే సూత్రం మీద. React చాలా XSS ని default గా ఆపుతుంది, కానీ మిగిలినవి మన బాధ్యత.

1. XSS (Cross-Site Scripting)

```
// ✅ React default గా escape చేస్తుంది
const userInput = '<script>alert(document.cookie)</script>';
<div>{userInput}</div>          // text గా చూపిస్తుంది, execute కాదు

// ❌ ప్రమాదకరం
<div dangerouslySetInnerHTML={{ __html: userInput }} />

// ✅ తప్పనిసరి అయితే sanitize
import DOMPurify from "dompurify";
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(html, {
  ALLOWED_TAGS: ["b", "i", "em", "strong", "a", "p", "ul", "li"],
  ALLOWED_ATTR: ["href", "target", "rel"],
}) }} />

// ❌ URL-based XSS — React ఇది ఆపదు!
<a href={userProvidedUrl}>Click</a>          // javascript:alert(1) → XSS!
// ✅ Protocol validate
const safeUrl = /^https?:\/\//i.test(url) ? url : "#";
<a href={safeUrl} target="_blank" rel="noopener noreferrer">Click</a>
```

`rel="noopener noreferrer"` ఎందుకు: `target="_blank"` లేకుండా, తెరిచిన page `window.opener` ద్వారా మీ page ని redirect చేయగలదు (tabnabbing). ఆధునిక browsers automatic చేస్తాయి, కానీ explicit ఉండటం మేలు.

2. Token storage – auth యొక్క అతిపెద్ద నిర్ణయం

ఎక్కడ	XSS ప్రమాదం	CSRF ప్రమాదం	సిఫార్సు
localStorage	❌ అధికం (JS చదవగలదు)	✅ లేదు	❌ tokens కి వద్దు
sessionStorage	❌ అధికం	✅ లేదు	❌
httpOnly cookie	✅ సురక్షితం (JS చదవలేదు)	⚠️ ఉంది	✅ **ఉత్తమం** (+ SameSite)
Memory (JS variable)	⚠️ మధ్యస్థం	✅ లేదు	✅ access token కి

```
// ✅ సిఫార్సు చేసిన pattern
// - Refresh token: httpOnly + Secure + SameSite=Strict cookie (server sets)
// - Access token: memory లో (React state/module variable), short TTL (5-15 min)
// - Page reload లో /refresh endpoint తో కొత్త access token
Set-Cookie: refresh=xyz; HttpOnly; Secure; SameSite=Strict; Path=/auth; Max-
Age=604800
```

3. CSRF

Cookie-based auth వాడితే CSRF ప్రమాదం. రక్షణ: `SameSite=Strict/Lax` **cookies** (ఆధునిక primary defense) + **CSRF token** (double-submit) + state-changing requests కి POST/PUT/DELETE మాత్రమే.

4. Environment variables – అత్యంత సాధారణ leak

```
// ❌ ఇవన్నీ bundle లోకి plain text గా వెళ్తాయి — browser DevTools లో ఎవరైనా చూడొచ్చు!
VITE_API_SECRET=sk_live_xxxxx
NEXT_PUBLIC_DB_PASSWORD=xxx
REACT_APP_STRIPE_SECRET=sk_xxx

// ✅ Client లో publishable/public keys మాత్రమే
VITE_STRIPE_PUBLISHABLE_KEY=pk_live_xxx
NEXT_PUBLIC_ANALYTICS_ID=G-XXX

// Secrets ఎప్పుడూ server లోనే (API route, server component, backend)
```

5. ఇతర ముఖ్య రక్షణలు

```
// Prototype pollution — user JSON ని గుడ్డిగా merge చేయకూడదు
const clean = JSON.parse(input, (k, v) =>
  (k === "__proto__" || k === "constructor") ? undefined : v);

// Dependency security
// npm audit --production ; Dependabot/Renovate ; lockfile commit ; typosquatting
జాగ్రత్త

// Content Security Policy (server header)
Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-{random}';
  style-src 'self' 'unsafe-inline'; img-src 'self' data: https:; object-src 'none';
  frame-ancestors 'none'; base-uri 'self'
```

```
// ఇతర headers
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), camera=(), microphone=()
```

6. Client-side validation ≠ security

```
// ❌ ఇది కేవలం UX — attacker DevTools లో మార్చొచ్చు, లేదా నేరుగా API కి request పంపొచ్చు
{user.role === "admin" && <DeleteButton />}
<input maxLength={10} />

// ✅ Server ప్రతి request లో authorize + validate చేయాలి
// Client checks = UX; server checks = security
```

7. Sensitive data leaks

```
// ❌ Console logs లో tokens/PII — production లో strip చేయాలి
console.log("user", user);                    // build లో drop: esbuild drop:
['console']

// ❌ Server component నుండి client కి secret prop
<ClientComp apiKey={process.env.SECRET_KEY} />  // RSC payload లో కనిపిస్తుంది!

// ❌ Error messages లో internal details
catch (e) { setError(e.stack); }              // stack trace user కి చూపొద్దు

// ❌ Source maps ని public గా serve చేయడం
```

Gotchas (సాధారణ తప్పులు)

- **`dangerouslySetInnerHTML` sanitize లేకుండా.**
- **`href={userUrl}`** — `javascript:` protocol.
- **JWT ని localStorage లో.**
- **`NEXT_PUBLIC_` / `VITE_` లో secrets.**
- **Client-side route guard ని security అనుకోవడం.**
- **`npm audit` ని ignore చేయడం.**
- **CORS ని `*` తో credentials సహా** (browser ఇది block చేస్తుంది, కానీ misconfiguration సాధారణం).

Key Points

- React JSX escaping = XSS కి బలమైన default; `dangerouslySetInnerHTML` + URLs exceptions.
- **httpOnly + SameSite cookies** = token storage కి ఉత్తమం; localStorage వద్దు.
- Client env vars = **public**.
- **Client validation = UX; server validation = security.**
- CSP + security headers + dependency audit.

Interview దృష్టి

- *"React లో XSS ఎలా ఆపుతావు?"* → default escaping, sanitize, URL protocol check, CSP.
- *"JWT ఎక్కడ store చేస్తావు?"* → httpOnly cookie (refresh) + memory (access) + trade-offs స్పష్టంగా.
- *"Frontend security ఏం చేయలేదు?"* → authorization – అది ఎప్పుడూ server బాధ్యత.

---

55. Error Handling, Logging & Monitoring

Error handling strategy – layers

```
1. TypeScript + validation (Zod)     → errors ని ముందే ఆపడం
2. try/catch in handlers/async       → expected errors
3. Query library (retry/onError)     → network errors
4. Error Boundaries                  → render errors
5. window.onerror / onunhandledrejection → global catch-all
6. Monitoring (Sentry)               → production visibility
```

```
// Global handlers – boundaries పట్టుకోని errors కి
useEffect(() => {
  const onError = (e) => logService.capture(e.error ?? e.message, { type:
"window.error" });
  const onRejection = (e) => logService.capture(e.reason, { type:
"unhandledrejection" });
  window.addEventListener("error", onError);
  window.addEventListener("unhandledrejection", onRejection);
  return () => {
    window.removeEventListener("error", onError);
    window.removeEventListener("unhandledrejection", onRejection);
  };
}, []);
```

Sentry setup

```
import * as Sentry from "@sentry/react";

Sentry.init({
  dsn: import.meta.env.VITE_SENTRY_DSN,
  environment: import.meta.env.MODE,
  release: __APP_VERSION__,                        // source maps తో match కావడానికి
  integrations: [Sentry.browserTracingIntegration(), Sentry.replayIntegration()],
  tracesSampleRate: 0.1,                            // 10% performance traces
  replaysOnErrorSampleRate: 1.0,                    // error వచ్చినప్పుడు session replay
  beforeSend(event) {
    // PII ని strip చేయడం
    if (event.request?.cookies) delete event.request.cookies;
    return event;
```

```
  },
});
```

User-facing error messages

```
// ❌ technical
"Error: Cannot read properties of undefined (reading 'map')"

// ✅ user-friendly + actionable + recoverable
function ErrorFallback({ error, resetErrorBoundary }) {
  const message = error.status === 404 ? "ఈ page దొరకలేదు"
    : error.status === 403 ? "మీకు ఈ page చూసే అనుమతి లేదు"
    : error.name === "NetworkError" ? "Internet connection తనిఖీ చేయండి"
    : "ఏదో తప్పు జరిగింది. మేము దీన్ని పరిశీలిస్తున్నాము.";
  return (
    <div role="alert">
      <p>{message}</p>
      <button onClick={resetErrorBoundary}>మళ్ళీ ప్రయత్నించండి</button>
      <a href="/support">సహాయం</a>
    </div>
  );
}
```

Feature flags (safe rollouts)

```
const flags = useFlags();
{flags.newCheckout ? <NewCheckout /> : <OldCheckout />}
// LaunchDarkly/Unleash/PostHog; kill switch — deploy లేకుండా feature ఆపొచ్చు
```

Key Points

- Layered handling: types → try/catch → query retry → boundaries → global → monitoring.
- Sentry: release + source maps + PII scrubbing + replay.
- User messages: friendly, actionable, recoverable.
- Feature flags = risk-free rollout + instant rollback.

Part 13 – Architecture & Frontend System Design

56. Scalable Folder Structure & Architecture

వివరణ

Codebase 50 files నుండి 5000 files కి పెరిగినప్పుడు మనుగడ సాగించే architecture – SSE role యొక్క ప్రధాన బాధ్యత.

Feature-Sliced architecture (సిఫార్సు)

```
src/
  app/                     # Application layer – setup మాత్రమే
    providers/             # QueryClient, Theme, Auth, Router providers
    router/
    styles/
    App.tsx
  pages/                   # Route-level compositions (business logic తక్కువ)
    ProductListPage.tsx
    CheckoutPage.tsx
  features/                # Business features – ఇక్కడే 90% code
    cart/
      api/cartApi.ts
      components/CartDrawer.tsx CartItem.tsx
      hooks/useCart.ts
      model/cartStore.ts    # state
      lib/calculateTotal.ts # pure helpers
      types.ts
      index.ts              # 🔑 PUBLIC API – బయటివారు ఇది మాత్రమే import చేయాలి
    auth/
    checkout/
  entities/                # Business objects (User, Product) – features మధ్య share
    product/ui/ProductCard.tsx
  shared/                  # Feature-agnostic
    ui/                    # design system (Button, Modal, Input)
    api/                   # http client, interceptors
    hooks/                 # useDebounce, useMediaQuery
    lib/                   # formatters, validators
    config/                # constants, env
```

Dependency rule (అత్యంత ముఖ్యం)

```
app → pages → features → entities → shared
     (ఒకే దిశలో మాత్రమే! ⬇️)

❌ shared → features    (shared ఏ feature గురించి తెలియకూడదు)
```

```
❌ features/cart → features/auth/components/LoginForm  (deep import)
✅ features/cart → features/auth (index.ts public API ద్వారా)
```

```
// ESLint తో enforce చేయడం (manual discipline మీద ఆధారపడొద్దు)
// eslint-plugin-boundaries / import/no-restricted-paths
"import/no-restricted-paths": ["error", {
  zones: [
    { target: "./src/shared", from: "./src/features", message: "shared cannot import
features" },
    { target: "./src/features/*/", from: "./src/features/*/!(index.ts)" },
  ],
}]
```

Component organization

```
// ఒకే component ఒకే file; పెద్దదైతే folder
features/cart/components/CartDrawer/
  CartDrawer.tsx
  CartDrawer.test.tsx
  CartDrawer.module.css
  index.ts

// Component size heuristic — ఈ signals వస్తే విడగొట్టాలి:
// - 200+ lines
// - 5+ useState
// - "and" తో వర్ణించాల్సి రావడం ("this shows the list AND handles filters AND...")
// - JSX లో 3+ levels conditional nesting
```

Layered separation (concerns)

```
UI layer           → components (presentational, props మాత్రమే)
Container layer    → data + state ని కలిపే components
Domain layer       → hooks, stores, business rules (React-agnostic ఉంటే testing సులభం)
Data layer         → api clients, query hooks, schemas
```

```
// ✅ Business logic ని React నుండి వేరు చేయడం — testing + reuse
// features/cart/lib/pricing.ts (pure — React లేదు)
export function calculateTotal(items, coupon, taxRate) { /* ... */ }

// hooks లో React తో కలపడం
export function useCartTotal() {
  const { items, coupon } = useCart();
  return useMemo(() => calculateTotal(items, coupon, TAX), [items, coupon]);
}
```

Monorepo (పెద్ద orgs)

```
apps/
  web/           # Next.js customer app
  admin/         # Vite admin panel
```

```
  mobile/         # React Native
packages/
  ui/             # shared design system
  config/         # eslint, tsconfig, tailwind presets
  api-client/     # generated API types + client
  utils/
```

సాధనాలు: pnpm workspaces + Turborepo (caching, task orchestration) / Nx. **లాభాలు:** atomic cross-app changes, shared code, ఒకే CI. **ఖర్చులు:** build complexity, CI time, tooling నేర్చుకోవడం.

Micro-frontends (అరుదుగా అవసరం)

```
Module Federation (webpack 5 / Vite plugin):
  shell app → remote apps ని runtime లో load చేస్తుంది
  ప్రతి team స్వతంత్రంగా deploy చేయగలదు
```

ఎప్పుడు: అనేక స్వతంత్ర teams, వేర్వేరు release cycles, legacy migration. **ఖర్చులు:** duplicate dependencies (bundle పెరుగుదల), version skew, cross-app state, debugging కష్టం, shared design system అవసరం. **నిజం:** చాలా companies కి **monorepo + మంచి module boundaries** సరిపోతాయి. Micro-frontends ఒక **org-structure పరిష్కారం**, technical పరిష్కారం కాదు.

Key Points

- **Feature-sliced** + **unidirectional dependency rule** + ESLint enforcement.
- ప్రతి feature కి **public API (`index.ts`)**; deep imports నిషేధం.
- Business logic ని React నుండి వేరు చేయడం → test + reuse.
- Monorepo = shared code + atomic changes; micro-frontends = team autonomy (అధిక ఖర్చు).

---

57. Component API Design Principles

1. Props vs Composition

```
// ❌ Props explosion — ప్రతి కొత్త అవసరానికి కొత్త prop
<Card title="x" subtitle="y" image="z" imagePosition="top" showBadge badgeText="New"
      footerButtons={[...]} headerAlign="center" />

// ✅ Composition — infinite flexibility, zero new props
<Card>
  <Card.Image src="z" />
  <Card.Header><Badge>New</Badge><h3>x</h3></Card.Header>
  <Card.Body>y</Card.Body>
  <Card.Footer><Button>OK</Button></Card.Footer>
</Card>
```

నియమం: *"ఈ prop UI structure గురించా?"* – అవును అయితే **composition**. *"ఇది behaviour/data గురించా?"* – అవును అయితే **prop**.

2. Controlled + Uncontrolled రెండూ (Topic 28)

3. Sensible defaults + escape hatches

```
function DatePicker({
  format = "DD/MM/YYYY",          // ✅ default – 90% cases పని చేస్తుంది
  locale = "en-IN",
  className,                       // ✅ escape hatch – styling override
  renderDay,                       // ✅ escape hatch – custom rendering
  ...rest                          // ✅ escape hatch – native props
}) {}
```

4. ప్రవర్తనని ప్రకటించే props (boolean flags కాదు)

```
// ❌ boolean explosion – 2^4 combinations, చాలావి అర్థరహితం
<Button isPrimary isLarge isDisabled isLoading />

// ✅ enums/variants
<Button variant="primary" size="lg" state="loading" />
```

5. Naming conventions

```
onClick, onChange, onSubmit          // event handlers → on* prefix
isOpen, isLoading, hasError          // booleans → is/has/should
renderItem, renderHeader             // render props → render*
defaultValue vs value                // uncontrolled vs controlled
```

6. Accessibility by default

```
// Component library ఎప్పుడూ a11y ని built-in గా ఇవ్వాలి –
// consumers దాన్ని గుర్తుపెట్టుకోవాల్సిన అవసరం ఉండకూడదు
function Modal({ title, children }) {
  const titleId = useId();
  return (
    <div role="dialog" aria-modal="true" aria-labelledby={titleId}>
      <h2 id={titleId}>{title}</h2>{children}
    </div>
  );
}
```

7. Documentation (Storybook)

```
export default { title: "UI/Button", component: Button,
  argTypes: { variant: { control: "select", options: ["primary", "danger"] } } };
export const Primary = { args: { variant: "primary", children: "Click" } };
export const Loading = { args: { state: "loading" } };
```

Key Points

- **Structure → composition; behaviour → props.**
- Controlled + uncontrolled support.
- Defaults + escape hatches (`className`, `...rest`, render props).
- Variants > boolean flags; consistent naming.
- a11y built-in, Storybook documented.

---

58. Frontend System Design Round (SSE)

వివరణ

SSE interviews లో ఒక పెద్ద frontend system ని design చేయమని అడుగుతారు (45-60 నిమిషాలు). **Backend system design లా కాదు** – ఇక్కడ rendering strategy, state, caching, performance, accessibility, offline, real-time మీద దృష్టి.

ప్రామాణిక framework (ఈ క్రమంలో మాట్లాడాలి)

```
1. Requirements clarification (5 min)
   - Functional: ఏ features? ఎవరు users?
   - Non-functional: scale, devices, browsers, latency, SEO, a11y, i18n, offline
   - Constraints: team size, timeline, existing stack

2. High-level architecture (10 min)
   - Rendering: CSR/SSR/SSG/ISR/RSC — ఎందుకు
   - Component tree / page structure
   - Data flow diagram

3. Data model & API (10 min)
   - API shape (REST/GraphQL/tRPC), pagination (offset vs cursor)
   - Caching layers, invalidation
   - Real-time (polling/SSE/WebSocket)

4. Deep dive (15 min) — interviewer ఎంచుకున్న ఒక భాగం
   - Component design, state management, edge cases

5. Performance & scalability (10 min)
   - Bundle, virtualization, images, Core Web Vitals, CDN

6. Cross-cutting (5 min)
   - a11y, i18n, error handling, monitoring, testing, security
```

Example 1 – Design a News Feed (Twitter/LinkedIn)

Requirements: infinite scroll, likes/comments, real-time updates, images/videos, mobile+desktop.

Architecture:

```
Rendering:  SSR/RSC మొదటి page (SEO + LCP) → client-side infinite scroll
Data:       cursor-based pagination (offset కాదు — కొత్త posts వస్తే duplicates/misses
వస్తాయి)
            GET /feed?cursor=abc&limit=20 → { items, nextCursor }
State:      TanStack useInfiniteQuery (server state) + Zustand (UI state)
Real-time:  WebSocket → "5 new posts" pill (auto-inject చేయొద్దు — scroll position
jump)
Rendering perf: virtualization (variable heights → dynamic measurement)
Media:      lazy loading, blurhash placeholders, IntersectionObserver తో video
autoplay
Optimistic: like button → useOptimistic / React Query onMutate
Offline:    service worker + cached feed
```

కీలక సవాళ్లు & పరిష్కారాలు:

సవాలు	పరిష్కారం
Variable-height virtualization	dynamic measurement + `ResizeObserver` (TanStack Virtual)
Scroll position restore (back navigation)	scroll position + cache ని sessionStorage లో
కొత్త posts scroll ని jump చేయడం	"new posts" pill; user click చేస్తేనే prepend
Duplicate posts	cursor pagination + client-side dedupe by id
Image CLS	aspect ratio placeholder
Memory (10k posts)	window-based virtualization + పాత pages ని unload

Example 2 – Design Autocomplete/Typeahead

```
UX:        debounce 300ms, min 2 chars, keyboard nav (↑↓ Enter Esc), highlight match
Network:   AbortController తో previous request cancel, race condition handling
Caching:   query → results Map (LRU, max 50 entries), prefix cache reuse
Perf:      virtualize if >100 results; transition తో render deprioritize
a11y:      role="combobox" aria-expanded aria-activedescendant, live region
announcement
Edge:      empty state, error state, slow network (skeleton), offline
Scale:     server-side search (Elasticsearch), CDN cache for popular queries
```

Example 3 – Design a Chat App (WhatsApp Web)

```
Real-time:   WebSocket (bidirectional) + reconnect with exponential backoff +
heartbeat
Ordering:    server timestamps + client-side sort; optimistic messages "sending"
state
Offline:     IndexedDB queue → reconnect లో flush
State:       messages normalized by conversation; virtualized reverse list
Perf:        pagination (older messages upward scroll), image lazy load, typing
indicator throttle
Multi-tab:   BroadcastChannel / SharedWorker తో ఒకే WS connection
Security:    E2E encryption (client-side), XSS sanitization in message rendering
```

తరచూ అడిగే ఇతర designs

Netflix/YouTube (video, adaptive streaming, prefetch) · E-commerce PLP+PDP (SSG/ISR, filters in URL, cart persistence) · Google Docs (CRDT/OT, presence) · Dashboard/analytics (chart perf, data aggregation, date ranges) · File upload (chunked, resumable, progress) · Photo gallery (masonry, lazy, lightbox)

మంచి vs చెడు సమాధానం

❌ చెడు	✅ మంచి
వెంటనే code రాయడం	Requirements ముందు clarify
ఒకే పరిష్కారం	Trade-offs తో 2-3 ఎంపికలు
Happy path మాత్రమే	Loading/error/empty/offline/slow network
a11y/i18n మర్చిపోవడం	Cross-cutting concerns ప్రస్తావించడం
"React వాడతాను"	ఎందుకు అనేది justify చేయడం
Numbers లేకుండా	"20 items/page, 300ms debounce, 5min staleTime"

Key Points

- Structured framework: requirements → architecture → data → deep dive → perf → cross-cutting.
- **Trade-offs మాట్లాడటమే ఇక్కడ అసలు skill.**
- Edge cases (offline, slow, error, empty) ప్రస్తావించడం seniority signal.
- Numbers ఇవ్వండి (page size, debounce, cache TTL, bundle budget).

---

59. Design Systems, Monorepo & Module Federation

Design System – layers

```
1. Tokens          → colors, spacing, typography, radii, shadows (CSS variables /
JSON)
2. Primitives      → Box, Text, Stack, Grid (layout building blocks)
3. Components      → Button, Input, Modal, Select (a11y + variants built-in)
4. Patterns        → LoginForm, DataTable, PageHeader (composed, opinionated)
5. Documentation   → Storybook + usage guidelines + do/don't
```

```
// tokens.js – single source of truth (design ↔ code)
export const tokens = {
  color: { brand: { 50: "#eff6ff", 500: "#3b82f6", 900: "#1e3a8a" },
           semantic: { danger: "#ef4444", success: "#22c55e" } },
  space: { 1: "4px", 2: "8px", 3: "12px", 4: "16px", 6: "24px", 8: "32px" },
```

```
  radius: { sm: "4px", md: "8px", full: "9999px" },
  fontSize: { sm: "14px", base: "16px", lg: "18px", xl: "24px" },
};
// → CSS variables generate చేయడం → Tailwind config / vanilla-extract theme
```

Design system యొక్క నిజమైన సవాళ్లు (technical కంటే social): adoption (teams పాత components వాడుతూ ఉంటాయి), versioning & breaking changes, contribution model, design-dev drift, documentation maintenance.

Versioning: semver + changesets; breaking changes కి codemods ఇవ్వడం; deprecation warnings 2 releases ముందు.

Module Federation (micro-frontends)

```
// vite.config.js (host/shell)
import federation from "@originjs/vite-plugin-federation";
export default defineConfig({
  plugins: [federation({
    name: "shell",
    remotes: { cart: "https://cart.example.com/assets/remoteEntry.js" },
    shared: ["react", "react-dom"],       // 🔑 duplicate React ని ఆపడం
  })],
});

// remote (cart app)
federation({
  name: "cart",
  filename: "remoteEntry.js",
  exposes: { "./CartWidget": "./src/CartWidget" },
  shared: ["react", "react-dom"],
});

// Host లో వాడకం
const CartWidget = lazy(() => import("cart/CartWidget"));
<ErrorBoundary fallback={<CartUnavailable />}>      {/* remote fail అయితే app crash
కాకూడదు */}
  <Suspense fallback={<Skeleton />}><CartWidget /></Suspense>
</ErrorBoundary>
```

ప్రమాదాలు: React version skew (shared singleton తప్పనిసరి), remote downtime, cross-app state/auth, CSS conflicts, debugging కష్టం, bundle duplication.

Key Points

- Design system = tokens → primitives → components → patterns → docs.
- నిజమైన సవాలు = adoption + governance, code కాదు.
- Module Federation = runtime remote loading; `shared` singletons + error boundaries తప్పనిసరి.

Part 14 – Interview Preparation

60. Machine Coding Round – 10 Must-Practice Components

> ఈ 10 components భారతీయ product companies (Flipkart, Swiggy, Razorpay, Atlassian, Uber, PhonePe…) machine coding rounds లో పదే పదే వస్తాయి. **45-90 నిమిషాల్లో working code + edge cases + a11y.**

1. Debounced Search with Autocomplete

```
function Autocomplete({ fetchSuggestions, onSelect, placeholder = "Search…" }) {
  const [query, setQuery] = useState("");
  const [items, setItems] = useState([]);
  const [open, setOpen] = useState(false);
  const [highlighted, setHighlighted] = useState(-1);
  const [loading, setLoading] = useState(false);
  const cache = useRef(new Map());
  const listId = useId();
  const wrapRef = useRef(null);

  const debounced = useDebounce(query, 300);

  useEffect(() => {
    if (debounced.trim().length < 2) { setItems([]); return; }

    if (cache.current.has(debounced)) { setItems(cache.current.get(debounced));
return; }

    const ctrl = new AbortController();
    setLoading(true);
    fetchSuggestions(debounced, ctrl.signal)
      .then((res) => {
        cache.current.set(debounced, res);
        if (cache.current.size > 50)
cache.current.delete(cache.current.keys().next().value); // LRU-ish
        setItems(res);
      })
      .catch((e) => { if (e.name !== "AbortError") setItems([]); })
      .finally(() => setLoading(false));

    return () => ctrl.abort();          // 🔑 race condition
  }, [debounced, fetchSuggestions]);

  useOnClickOutside(wrapRef, () => setOpen(false));

  const onKeyDown = (e) => {
    if (!open || !items.length) return;
    if (e.key === "ArrowDown") { e.preventDefault(); setHighlighted((i) => (i + 1) %
items.length); }
    if (e.key === "ArrowUp")   { e.preventDefault(); setHighlighted((i) => (i - 1 +
```

```
items.length) % items.length); }
    if (e.key === "Enter" && highlighted >= 0) { e.preventDefault();
pick(items[highlighted]); }
    if (e.key === "Escape")    { setOpen(false); setHighlighted(-1); }
  };

  const pick = (item) => { setQuery(item.label); setOpen(false); setHighlighted(-1);
onSelect(item); };

  return (
    <div ref={wrapRef} className="autocomplete">
      <input
        role="combobox" aria-expanded={open} aria-controls={listId} aria-
autocomplete="list"
        aria-activedescendant={highlighted >= 0 ? `${listId}-${highlighted}` :
undefined}
        value={query} placeholder={placeholder}
        onChange={(e) => { setQuery(e.target.value); setOpen(true);
setHighlighted(-1); }}
        onFocus={() => setOpen(true)} onKeyDown={onKeyDown}
      />
      {loading && <Spinner />}
      {open && (
        <ul id={listId} role="listbox">
          {items.length === 0 && !loading && debounced.length >= 2 && <li>No
results</li>}
          {items.map((it, i) => (
            <li key={it.id} id={`${listId}-${i}`} role="option" aria-selected={i ===
highlighted}
                className={i === highlighted ? "highlighted" : ""}
                onMouseEnter={() => setHighlighted(i)}
                onMouseDown={(e) => e.preventDefault()}     // blur ని ఆపడం
                onClick={() => pick(it)}>
              {highlightMatch(it.label, debounced)}
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}

function highlightMatch(text, q) {
  const i = text.toLowerCase().indexOf(q.toLowerCase());
  if (i === -1) return text;
  return <>{text.slice(0, i)}<mark>{text.slice(i, i + q.length)}</mark>{text.slice(i
+ q.length)}</>;
}
```

Interviewer చూసేది: debounce, abort/race, cache, keyboard nav, a11y roles, empty/loading states, outside click.

2. Infinite Scroll List

```
function InfiniteList({ fetchPage }) {
  const [items, setItems] = useState([]);
```

```
  const [cursor, setCursor] = useState(null);
  const [hasMore, setHasMore] = useState(true);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const sentinelRef = useRef(null);
  const loadingRef = useRef(false);        // 🔑 double-fire ఆపడానికి

  const load = useCallback(async () => {
    if (loadingRef.current || !hasMore) return;
    loadingRef.current = true; setLoading(true); setError(null);
    try {
      const { items: newItems, nextCursor } = await fetchPage(cursor);
      setItems((prev) => {
        const seen = new Set(prev.map((i) => i.id));
        return [...prev, ...newItems.filter((i) => !seen.has(i.id))];   // dedupe
      });
      setCursor(nextCursor);
      setHasMore(Boolean(nextCursor));
    } catch (e) { setError(e); }
    finally { loadingRef.current = false; setLoading(false); }
  }, [cursor, hasMore, fetchPage]);

  useEffect(() => {
    const el = sentinelRef.current;
    if (!el) return;
    const obs = new IntersectionObserver(
      ([entry]) => { if (entry.isIntersecting) load(); },
      { rootMargin: "200px" }              // కనిపించే ముందే load
    );
    obs.observe(el);
    return () => obs.disconnect();
  }, [load]);

  return (
    <>
      <ul>{items.map((it) => <Row key={it.id} item={it} />)}</ul>
      {error && <button onClick={load}>Retry</button>}
      {loading && <Skeleton count={3} />}
      {!hasMore && <p>No more items</p>}
      <div ref={sentinelRef} style={{ height: 1 }} />
    </>
  );
}
```

3. Accessible Modal (Topic 32 లో పూర్తి code – portal + focus trap + ESC + scroll lock)

4. Star Rating

```
function StarRating({ value = 0, max = 5, onChange, readOnly = false, allowHalf =
false }) {
  const [hover, setHover] = useState(null);
  const display = hover ?? value;

  const onKeyDown = (e) => {
    if (readOnly) return;
    if (e.key === "ArrowRight") onChange(Math.min(max, value + (allowHalf ? 0.5 :
```

```
1)));
    if (e.key === "ArrowLeft")  onChange(Math.max(0, value - (allowHalf ? 0.5 : 1)));
  };

  return (
    <div role="slider" aria-valuenow={value} aria-valuemin={0} aria-valuemax={max}
         aria-label="Rating" tabIndex={readOnly ? -1 : 0} onKeyDown={onKeyDown}
         onMouseLeave={() => setHover(null)}>
      {Array.from({ length: max }, (_, i) => {
        const starValue = i + 1;
        const filled = display >= starValue;
        const half = allowHalf && display >= starValue - 0.5 && display < starValue;
        return (
          <span key={i}
                onMouseMove={(e) => {
                  if (readOnly) return;
                  const { left, width } = e.currentTarget.getBoundingClientRect();
                  const isLeftHalf = e.clientX - left < width / 2;
                  setHover(allowHalf && isLeftHalf ? starValue - 0.5 : starValue);
                }}
                onClick={() => !readOnly && onChange(hover ?? starValue)}
                style={{ cursor: readOnly ? "default" : "pointer" }}>
            {filled ? "★" : half ? "□" : "☆"}
          </span>
        );
      })}
    </div>
  );
}
```

5. Nested Comments (recursive rendering)

```
function Comment({ comment, depth = 0, onReply, maxDepth = 5 }) {
  const [collapsed, setCollapsed] = useState(false);
  const [replying, setReplying] = useState(false);

  return (
    <div style={{ marginLeft: Math.min(depth, maxDepth) * 20 }}>
      <div className="comment">
        <button onClick={() => setCollapsed((c) => !c)} aria-expanded={!collapsed}>
          {collapsed ? "▶" : "▼"}
        </button>
        <strong>{comment.author}</strong>
        <p>{comment.text}</p>
        <button onClick={() => setReplying((r) => !r)}>Reply</button>
      </div>

      {replying && <ReplyBox onSubmit={(t) => { onReply(comment.id, t);
setReplying(false); }} />}

      {!collapsed && comment.replies?.map((r) => (
        <Comment key={r.id} comment={r} depth={depth + 1} onReply={onReply} maxDepth=
{maxDepth} />
      ))}
    </div>
  );
}
```

```
// ⚠️ చాలా deep trees లో recursion + rendering ఖరీదు → depth limit + "show more
replies"
```

6. Pagination component

```
function Pagination({ currentPage, totalPages, onPageChange, siblingCount = 1 }) {
  const range = useMemo(() => {
    const total = siblingCount * 2 + 5;            // first, last, current, 2
siblings, 2 dots
    if (totalPages <= total) return Array.from({ length: totalPages }, (_, i) => i +
1);

    const left = Math.max(currentPage - siblingCount, 1);
    const right = Math.min(currentPage + siblingCount, totalPages);
    const showLeftDots = left > 2, showRightDots = right < totalPages - 1;

    if (!showLeftDots && showRightDots)
      return [...Array.from({ length: 3 + siblingCount * 2 }, (_, i) => i + 1), "…",
totalPages];
    if (showLeftDots && !showRightDots)
      return [1, "…", ...Array.from({ length: 3 + siblingCount * 2 },
              (_, i) => totalPages - (3 + siblingCount * 2) + 1 + i)];
    return [1, "…", ...Array.from({ length: right - left + 1 }, (_, i) => left + i),
"…", totalPages];
  }, [currentPage, totalPages, siblingCount]);

  return (
    <nav aria-label="Pagination">
      <button onClick={() => onPageChange(currentPage - 1)} disabled={currentPage ===
1}>Prev</button>
      {range.map((p, i) =>
        p === "…" ? <span key={`d${i}`}>…</span>
          : <button key={p} onClick={() => onPageChange(p)}
                   aria-current={p === currentPage ? "page" : undefined}
                   className={p === currentPage ? "active" : ""}>{p}</button>
      )}
      <button onClick={() => onPageChange(currentPage + 1)} disabled={currentPage ===
totalPages}>Next</button>
    </nav>
  );
}
```

7. Toast/Notification system (context + queue + auto-dismiss)

```
const ToastContext = createContext(null);

export function ToastProvider({ children, max = 3 }) {
  const [toasts, setToasts] = useState([]);
  const timers = useRef(new Map());

  const remove = useCallback((id) => {
    setToasts((t) => t.filter((x) => x.id !== id));
    clearTimeout(timers.current.get(id));
    timers.current.delete(id);
  }, []);
```

```
  const show = useCallback((message, { type = "info", duration = 4000 } = {}) => {
    const id = crypto.randomUUID();
    setToasts((t) => [...t, { id, message, type }].slice(-max));
    if (duration) timers.current.set(id, setTimeout(() => remove(id), duration));
    return id;
  }, [max, remove]);

  useEffect(() => () => timers.current.forEach(clearTimeout), []);   // unmount
cleanup

  const value = useMemo(() => ({ show, remove }), [show, remove]);

  return (
    <ToastContext.Provider value={value}>
      {children}
      {createPortal(
        <div className="toast-container" role="region" aria-label="Notifications">
          {toasts.map((t) => (
            <div key={t.id} role="alert" aria-live="polite" className={`toast
${t.type}`}>
              {t.message}
              <button onClick={() => remove(t.id)} aria-label="Dismiss">×</button>
            </div>
          ))}
        </div>,
        document.body
      )}
    </ToastContext.Provider>
  );
}
export const useToast = () => {
  const ctx = useContext(ToastContext);
  if (!ctx) throw new Error("useToast must be inside ToastProvider");
  return ctx;
};
```

8. Todo List with filters (classic, కానీ quality చూస్తారు)

```
function TodoApp() {
  const [todos, setTodos] = useState(() =>
    JSON.parse(localStorage.getItem("todos") ?? "[]"));
  const [filter, setFilter] = useState("all");      // all | active | done
  const [editingId, setEditingId] = useState(null);

  useEffect(() => { localStorage.setItem("todos", JSON.stringify(todos)); },
[todos]);

  const add = (text) => {
    if (!text.trim()) return;
    setTodos((t) => [...t, { id: crypto.randomUUID(), text: text.trim(), done: false
}]);
  };
  const toggle = (id) => setTodos((t) => t.map((x) => x.id === id ? { ...x, done:
!x.done } : x));
  const remove = (id) => setTodos((t) => t.filter((x) => x.id !== id));
  const edit = (id, text) => setTodos((t) => t.map((x) => x.id === id ? { ...x, text
```

```
} : x));

  // ✅ derived — state లో duplicate కాదు
  const visible = useMemo(() => todos.filter((t) =>
    filter === "active" ? !t.done : filter === "done" ? t.done : true), [todos,
filter]);
  const remaining = todos.filter((t) => !t.done).length;

  return (/* UI: input + filter buttons + list + "N remaining" + clear completed */);
}
```

9. Carousel / Image Slider

```
function Carousel({ images, autoPlayMs = 0 }) {
  const [index, setIndex] = useState(0);
  const [paused, setPaused] = useState(false);
  const touchStart = useRef(null);

  const next = useCallback(() => setIndex((i) => (i + 1) % images.length),
[images.length]);
  const prev = useCallback(() => setIndex((i) => (i - 1 + images.length) %
images.length), [images.length]);

  useEffect(() => {
    if (!autoPlayMs || paused) return;
    const id = setInterval(next, autoPlayMs);
    return () => clearInterval(id);
  }, [autoPlayMs, paused, next]);

  const onKeyDown = (e) => { if (e.key === "ArrowRight") next(); if (e.key ===
"ArrowLeft") prev(); };

  return (
    <div className="carousel" tabIndex={0} onKeyDown={onKeyDown}
         onMouseEnter={() => setPaused(true)} onMouseLeave={() => setPaused(false)}
         onTouchStart={(e) => { touchStart.current = e.touches[0].clientX; }}
         onTouchEnd={(e) => {
           const delta = e.changedTouches[0].clientX - touchStart.current;
           if (Math.abs(delta) > 50) delta < 0 ? next() : prev();
         }}
         role="region" aria-roledescription="carousel" aria-label="Image gallery">
      <div className="track" style={{ transform: `translateX(-${index * 100}%)`,
transition: "transform .3s" }}>
        {images.map((src, i) => (
          <img key={src} src={src} alt="" aria-hidden={i !== index}
               loading={i === 0 ? "eager" : "lazy"} />
        ))}
      </div>
      <button onClick={prev} aria-label="Previous">‹</button>
      <button onClick={next} aria-label="Next">›</button>
      <div role="tablist">
        {images.map((_, i) => (
          <button key={i} role="tab" aria-selected={i === index}
                  aria-label={`Slide ${i + 1}`} onClick={() => setIndex(i)} />
        ))}
      </div>
      <div className="sr-only" aria-live="polite">Slide {index + 1} of
```

```
{images.length}</div>
    </div>
  );
}
```

10. Drag & Drop Kanban (native HTML5 DnD)

```
function Board({ initialColumns }) {
  const [columns, setColumns] = useState(initialColumns);   // { todo: [...], doing:
[...], done: [...] }
  const dragItem = useRef(null);

  const onDragStart = (e, colId, index) => {
    dragItem.current = { colId, index };
    e.dataTransfer.effectAllowed = "move";
  };

  const onDrop = (e, targetCol, targetIndex) => {
    e.preventDefault();
    const src = dragItem.current;
    if (!src) return;
    setColumns((cols) => {
      const next = { ...cols, [src.colId]: [...cols[src.colId]] };
      const [moved] = next[src.colId].splice(src.index, 1);
      next[targetCol] = src.colId === targetCol ? next[src.colId] :
[...cols[targetCol]];
      next[targetCol].splice(targetIndex, 0, moved);
      return next;
    });
    dragItem.current = null;
  };

  return (
    <div className="board">
      {Object.entries(columns).map(([colId, items]) => (
        <div key={colId} className="column"
             onDragOver={(e) => e.preventDefault()}          // 🔑 లేకపోతే drop fire
కాదు
             onDrop={(e) => onDrop(e, colId, items.length)}>
          <h3>{colId}</h3>
          {items.map((item, i) => (
            <div key={item.id} draggable
                 onDragStart={(e) => onDragStart(e, colId, i)}
                 onDrop={(e) => { e.stopPropagation(); onDrop(e, colId, i); }}>
              {item.title}
            </div>
          ))}
        </div>
      ))}
    </div>
  );
}
// ⚠️ Production లో dnd-kit వాడాలి — keyboard a11y, touch, auto-scroll, collision
detection
```

Machine coding round – evaluation checklist

```
✅ Working core functionality (అన్నిటికంటే ముఖ్యం — ముందు పని చేసేలా చేయండి)
✅ Component decomposition (ఒక్క 500-line file కాదు)
✅ State design (derived vs stored, immutability)
✅ Edge cases: empty, loading, error, boundary values
✅ Cleanup (timers, listeners, aborts)
✅ Keyboard + a11y (roles, labels, focus)
✅ Performance awareness (debounce, memo, virtualize — అవసరమైతే)
✅ Naming & readability, small functions
✅ మాట్లాడుతూ code రాయడం (thought process చెప్పడం)
⏱ సమయ నిర్వహణ: 60% core, 20% edge cases, 20% polish
```

---

61. Rapid-Fire Interview Q&A (SDE2 + SSE)

A. Fundamentals

1. React అంటే ఏమిటి? UI build చేయడానికి declarative, component-based JavaScript library. `UI = f(state)`.

2. JSX ఏమవుతుంది compile అయ్యాక? `React.createElement()` / `jsx()` calls → plain JS objects (React elements) tree.

3. Element vs Component? Element = ఒక plain object description (చవక, immutable). Component = elements return చేసే function/class (reusable).

4. Virtual DOM React ని fast చేస్తుందా? VDOM diffing overhead ఉంది. అది ఇచ్చేది – declarative code రాసినా DOM writes minimal గా ఉంటాయనే హామీ. నిజమైన లాభం maintainability.

5. Reconciliation అంటే? కొత్త element tree ని current fiber tree తో పోల్చి, తేడాలను గుర్తించి, కనీస DOM operations లెక్కించే ప్రక్రియ. Heuristics: type మారితే subtree పారేస్తుంది; keys ద్వారా children match.

6. Keys ఎందుకు, index key ఎందుకు చెడ్డది? Identity కోసం. Index వాడితే reorder/delete లో React తప్పు nodes ని reuse చేసి state/input values తప్పు rows లోకి జారతాయి.

7. Props vs State? Props parent నుండి, immutable; state component-owned, setter ద్వారా మారుతుంది, మారితే re-render.

8. Controlled vs uncontrolled? Value React state లోనా (controlled) DOM లోనా (uncontrolled). File input ఎప్పుడూ uncontrolled.

9. Component ఎప్పుడు re-render అవుతుంది? own state change, parent re-render, context value change, external store change, key change (remount).

10. Fragment ఎందుకు? Extra DOM wrapper లేకుండా multiple children return చేయడానికి (table/flex/grid layouts కి కీలకం).

B. Hooks

11. Hooks ఎందుకు వచ్చాయి? Class లో logic reuse కష్టం (HOC/render props wrapper hell), related logic lifecycle లో చెల్లాచెదురు, `this` గందరగోళం.

12. Hooks ని condition లో ఎందుకు call చేయకూడదు? React వాటిని fiber లో linked list గా, **call order ద్వారా** track చేస్తుంది. Order మారితే state తప్పు hook కి వెళ్తుంది.

13. `useEffect` vs `useLayoutEffect`? Passive effect paint తర్వాత async; layout effect DOM mutation తర్వాత paint కి ముందు sync (blocking).

14. `useEffect` cleanup ఎప్పుడు run అవుతుంది? Unmount లో **మరియు** ప్రతి తర్వాతి effect run కి ముందు.

15. StrictMode లో effects ఎందుకు రెండుసార్లు? Cleanup correctness test (dev-only). Production లో ఒక్కసారే.

16. `useMemo` vs `useCallback`? Value vs function identity. `useCallback(f, d) === useMemo(() => f, d)`.

17. `React.memo` ఎప్పుడు విఫలం? Props identity ప్రతి render కి మారితే (inline objects/arrays/functions/children).

18. `useRef` vs `useState`? Ref mutate చేస్తే re-render కాదు; state అవుతుంది. Ref = timers, DOM handles, previous values, latest-callback.

19. `useReducer` ఎప్పుడు? Complex/multi-field transitions, testable logic, stable `dispatch` కావాలంటే.

20. `useTransition` vs debounce? Transition = priority-based, interruptible (rendering కి); debounce = time-based delay (network calls కి).

21. `useDeferredValue` ఎప్పుడు? Value మన నియంత్రణలో లేనప్పుడు (props), లేదా search-as-you-type లో; `useMemo` తో కలిపి వాడాలి.

22. `useSyncExternalStore` ఎందుకు? External store subscription ని tearing-safe గా చేయడానికి + SSR snapshot.

23. `useId` ఎందుకు? SSR/client లో ఒకే id → hydration mismatch నివారణ (a11y attributes కి; keys కి కాదు).

24. Custom hook state share చేస్తుందా? లేదు – logic share చేస్తుంది; ప్రతి caller కి వేరే instance.

25. React 19 `use()` ప్రత్యేకత? Promise/context చదువుతుంది, **conditional గా call చేయొచ్చు** (ఏకైక hook).

C. Performance

26. App నెమ్మది – ఎలా debug? Profiler + "why did this render" → వర్గీకరణ (bundle/render/network/DOM) → priority fix → re-measure.

27. Re-render = DOM update? కాదు; diff ఖాళీ అయితే DOM touch అవ్వదు.

28. Child state reset ఎలా? `key` prop మార్చడం.

29. 100k rows ఎలా? Virtualization + pagination + memoized rows + server-side filtering.

30. Code splitting ఎక్కడ? Route-level మొదట, heavy libraries, modals/rare UI; hover prefetch.

31. Context ఎందుకు re-render storms కలిగిస్తుంది? Selector-based subscription లేదు – value మారితే అన్ని consumers re-render (memo ఆపదు). పరిష్కారం: split contexts, dispatch separation, external store.

32. React Compiler ఏం చేస్తుంది? Build time లో auto-memoization; Rules of React పాటిస్తే `useMemo/useCallback/memo` దాదాపు అవసరం లేదు.

D. Internals

33. Fiber ఏమిటి? Reconciliation engine rewrite – unit of work + linked-list tree → interruptible rendering.

34. Render vs commit phase? Render = pure, interruptible, discardable. Commit = synchronous, atomic DOM mutations + layout effects.

35. Double buffering? current ↔ workInProgress trees; commit లో pointer swap → torn UI ఎప్పుడూ కనిపించదు.

36. Lanes? Bitmask priority model (31 lanes) – sync input, continuous, default, transition, retry, idle.

37. Tearing? Concurrent render మధ్యలో external store మారితే ఒకే frame లో వేర్వేరు values → `useSyncExternalStore`.

38. Batching React 17 vs 18? 17: event handlers లో మాత్రమే. 18 (`createRoot`): **automatic batching everywhere**.

39. Suspense లోపల ఎలా? Component render లో promise throw → React దగ్గరి boundary fallback చూపిస్తుంది → resolve అయ్యాక retry lane లో re-render.

40. Portal లో events ఎక్కడికి bubble అవుతాయి? React tree parent కి (DOM parent కి కాదు).

E. State & Data

41. Redux ఎప్పుడు అవసరం లేదు? Server state మాత్రమే అయితే; Context/Zustand సరిపోతే; app చిన్నదైతే.

42. Server state vs client state? Server state remote-owned, stale అవుతుంది, caching/invalidation/retry అవసరం → TanStack Query/RTK Query/RSC.

43. `staleTime` vs `gcTime` ? staleTime = refetch నియంత్రణ; gcTime = unused cache memory retention.

44. Optimistic update ఎలా? `onMutate` (cancel + snapshot + optimistic set) → `onError` rollback → `onSettled` invalidate.

45. Filters ఎక్కడ ఉంచుతావు? URL (`useSearchParams`) – shareable, bookmarkable, back button, SSR.

46. Prop drilling ఎలా ఆపుతావు? Composition మొదట → context → state library (ఈ క్రమంలో).

F. SSR / RSC / Next.js

47. Hydration అంటే? Server HTML కి React listeners attach చేసి interactive చేయడం.

48. Hydration mismatch కారణాలు? `Date` / `Math.random` , browser APIs, locale formatting, invalid HTML nesting, extensions.

49. RSC vs SSR? SSR = HTML generate, component ఇంకా client లో hydrate అవుతుంది. RSC = component ఎప్పటికీ client bundle లోకి రాదు.

50. `'use client'` ఏం చేస్తుంది? Client boundary – ఆ file మరియు దాని imports client bundle లోకి. Leaf కి దగ్గరగా ఉంచాలి.

51. Server Actions security? అవి public endpoints – ప్రతి action లో auth + authorization + validation తప్పనిసరి.

52. Streaming SSR లాభం? Shell వెంటనే పంపడం + Suspense chunks + selective hydration → TTFB/FCP మెరుగు.

G. Quality

53. Error boundary ఏం పట్టుకోదు? Event handlers, async code, SSR, boundary దానిలోని errors.

54. Testing లో implementation details అంటే? State variable names, internal functions, CSS classes – refactor చేస్తే fail అయ్యేవి.

55. RTL query priority? role → label → text → ... → testId (చివరి ఎంపిక).

56. XSS ని React ఎలా ఆపుతుంది, ఎక్కడ ఆపదు? JSX escaping ద్వారా ఆపుతుంది; `dangerouslySetInnerHTML` మరియు `href={userUrl}` (javascript:) లో ఆపదు.

57. JWT ఎక్కడ store? Refresh token = httpOnly+Secure+SameSite cookie; access token = memory. localStorage వద్దు (XSS).

58. a11y లో మొదటి పని? Semantic HTML – ARIA తర్వాత ("no ARIA is better than bad ARIA").

H. Behavioural/Design (SSE)

59. React app ని ఎలా structure చేస్తావు? Feature-sliced + public API per feature + unidirectional dependency rule + ESLint enforcement.

60. Component API ఎలా design చేస్తావు? Structure → composition; behaviour → props; controlled+uncontrolled; defaults + escape hatches; a11y built-in.

61. Legacy class codebase ని hooks కి ఎలా migrate చేస్తావు? Incremental – కొత్త code hooks లో; leaf components మొదట; error boundaries classes గానే; strangler pattern; ప్రతి step లో tests.

62. పెద్ద team లో consistency ఎలా? ESLint/Prettier + TypeScript strict + design system + code review checklist + ADRs (architecture decision records) + CI gates (bundle budget, coverage, a11y).

---

62. Anti-Patterns & Common Mistakes Cheat-Sheet

State

❌ Anti-pattern	✅ సరైనది
`state.items.push(x); setItems(items)`	`setItems([...items, x])`
`setCount(count + 1)` (multiple times)	`setCount(c => c + 1)`
Derived data ని state లో (`fullName`)	Render లో compute (అవసరమైతే `useMemo`)
`useState(props.value)` (props sync)	`key` prop, లేదా నేరుగా props వాడటం
`isLoading + isError + isSuccess`	`status: 'idle' / 'loading' / 'success' / 'error'` (ఒకే state machine)
అన్ని state ని App లో	State colocation (కిందికి తరలించడం)
Server data ని Redux/Context లో	TanStack Query / RTK Query
`useState` కి బదులు ref UI data కి	UI data ఎప్పుడూ state లో

Effects

❌	✅
Cleanup లేకపోవడం	timers/listeners/subscriptions/aborts ని cleanup
`useEffect(async () => {})`	లోపల IIFE / named async function
Derived state కోసం effect	Render లో compute
Event logic ని effect లో	Event handler లో
`[]` + stale closure	functional updater / ref / సరైన deps
`eslint-disable exhaustive-deps`	నిజమైన కారణం fix చేయడం
Chained effects (dominoes)	ఒకే handler/reducer

❌	✅
Effect లో object dep	primitives / `useMemo` / లోపలికి తరలింపు

Rendering

❌	✅
`key={index}` (dynamic lists)	`key={item.id}`
`key={Math.random()}`	stable id
Component లోపల component define	బయట define
Render లో `Math.random()` / `Date.now()`	effect/handler లో, లేదా props గా
`{items.length && <X/>}`	`{items.length > 0 && <X/>}`
Nested ternaries (3+)	lookup map / early returns / switch
Render లో side effects	effects/handlers
Render లో `ref.current` mutate	effects లో

Performance

❌	✅
అన్నిటినీ `useMemo` / `useCallback`	Profiler ఆధారంగా మాత్రమే (లేదా Compiler)
`useCallback` + memo లేని child	జంటగా వాడాలి
10k rows ని నేరుగా render	Virtualization
ప్రతి keystroke కి API	debounce + cancel
ఒకే giant context	Split contexts / external store
`import _ from "lodash"`	`import debounce from "lodash/debounce"`
Image కి width/height లేకపోవడం	CLS నివారణకు తప్పనిసరి

Architecture & Security

❌	✅
500-line components	Decomposition (UI/container/logic)
Deep cross-feature imports	Public API (`index.ts`) + ESLint boundaries
Client-side auth guard ని security అనుకోవడం	Server authorization తప్పనిసరి
Secrets ని `VITE_` / `NEXT_PUBLIC_` లో	Server-only

❌	✅
`dangerouslySetInnerHTML` sanitize లేకుండా	DOMPurify + allowlist
JWT ని localStorage లో	httpOnly cookie + memory
`<div onClick>`	`<button>`
`outline: none`	`:focus-visible`

---

63. Memory Tips – ఒకసారి చదివి గుర్తుపెట్టుకునే Framework

1. ఒక్క వాక్యంలో React

> `UI = f(state)` – state ఇవ్వు, React UI ని sync లో ఉంచుతుంది.

మిగిలినదంతా ఈ ఒక్క వాక్యం చుట్టూ ఉన్న వివరాలే: state ఎలా మారుతుంది (`useState`), బయటి ప్రపంచంతో ఎలా sync అవుతుంది (`useEffect`), ఆ sync ని ఎలా వేగం చేయాలి (memo, transitions), ఎక్కడ run అవుతుంది (client/server).

2. ప్రతి update యొక్క 3 దశలు (ఇది గుర్తుంటే internals అన్నీ గుర్తుంటాయి)

```
Trigger   →  Render   →  Commit  →  (paint)  →  useEffect
(setState)  (pure,      (DOM,
             ఆపగలం)      ఆపలేం)
```

3. Hook ఎంపిక – 6 ప్రశ్నలు

```
UI లో కనిపించాలా?              → useState
మారితే re-render వద్దా?        → useRef
Complex transitions?           → useReducer
బయటి system తో sync?           → useEffect (cleanup తో!)
Paint కి ముందు measure?        → useLayoutEffect
ఖరీదైన compute / memo child? → useMemo / useCallback
```

4. "నేను ఏమి తప్పు చేస్తున్నాను?" – 5 సాధారణ నేరాలు

```
1. State mutate చేశానా?                    → కొత్త object/array
2. Effect లో cleanup ఉందా?                  → లేకపోతే leak
3. Derived data ని state లో పెట్టానా?  → render లో compute
4. key = index వాడానా?                      → stable id
5. ఇది నిజంగా effect పనేనా?               → handler లో ఉండాలేమో
```

5. Analogy map (కథలుగా గుర్తుంచుకోవడం)

Concept	Analogy
Declarative UI	Restaurant order (ఏం కావాలో చెప్పడం)
VDOM	Blueprint మీద edits
Fiber	Bookmark పెట్టుకుంటూ చదవడం
Lanes/priorities	Highway lanes + ambulance
Hooks order	Tiffin box అరలు (వరుస సంఖ్య)
useEffect	కేబుల్ connection (ఇల్లు మారితే cut+కొత్తది)
Context	ఇంటి WiFi
React Query	Fridge (stale-while-revalidate)
Error Boundary	MCB/circuit breaker
Portal	Terrace మీద party
RSC	Kitchen vs dining table
Keys	Roll number vs bench position
Suspense	Starter ఇవ్వడం
useReducer	బ్యాంకు voucher

6. Version timeline (ఒక్క line)

```
16 = Fiber · 16.8 = Hooks · 17 = root delegation · 18 = Concurrent + batching · 19 =
Actions + Compiler
```

7. Interview కి ముందు 30 నిమిషాల revision క్రమం

```
1. UI = f(state) + 3 phases                    (2 min)
2. Reconciliation + keys + state-position rule    (3 min)
3. Hooks rules + linked list + closure trap       (5 min)
4. useEffect: cleanup, deps, races, StrictMode    (5 min)
5. Re-render కారణాలు + memo mental model         (3 min)
6. State types decision tree                      (3 min)
7. Fiber/lanes/concurrent ఒక్క paragraph        (3 min)
8. RSC vs SSR + hydration                         (3 min)
9. Anti-patterns table (Topic 62)                 (3 min)
```

8. చివరి మాట

React లో **API లు గుర్తుపెట్టుకోవడం కాదు – mental model** ముఖ్యం. `useMemo` యొక్క signature Google చేయొచ్చు; కానీ *"ఈ component ఎందుకు re-render అవుతోంది?"*, *"ఈ state ఇక్కడ ఉండాలా?"*,

"ఇది *effect* పనేనా?" – ఈ ప్రశ్నలు అడగడం అలవాటైతే మీరు React ని నేర్చుకున్నట్టే.

SDE2 అంటే – feature ని సరిగ్గా, పరీక్షించదగినట్టు build చేయగలగడం. **SSE అంటే** – ఎందుకు ఈ approach అని justify చేయగలగడం, trade-offs తెలియడం, ఇతరులు దాని మీద build చేయగలిగే foundation వేయడం.

> **గుర్తుంచుకోండి:** ప్రతి React సమస్యకి సమాధానం మూడింటిలో ఒకటి – **(1) state తప్పు చోట ఉంది, (2) identity మారుతోంది (keys/props/deps), (3) ఇది effect పని కాదు.**

అనుబంధం – వేగవంతమైన Reference

అన్ని hooks ఒక్క చూపులో

```
const [s, setS] = useState(init);                     // local state
const [s, dispatch] = useReducer(reducer, init, fn); // complex state
useEffect(fn, deps);                                  // paint తర్వాత sync
useLayoutEffect(fn, deps);                            // paint ముందు (measure)
useInsertionEffect(fn, deps);                         // CSS-in-JS libs
const ref = useRef(init);                             // mutable box / DOM
useImperativeHandle(ref, () => ({ api }), deps);      // parent కి API
const v = useContext(Ctx);                            // context read
const m = useMemo(() => compute(), deps);             // value cache
const c = useCallback(fn, deps);                      // fn identity cache
const id = useId();                                   // SSR-safe id
const [pending, start] = useTransition();             // non-urgent update
const d = useDeferredValue(v);                        // lagging copy
const v = useSyncExternalStore(sub, get, getServer); // external store
useDebugValue(v);                                     // devtools label
const v = use(promiseOrContext);                      // React 19
const [st, action, pending] = useActionState(fn, i); // React 19 forms
const { pending } = useFormStatus();                  // React 19 (react-dom)
const [opt, addOpt] = useOptimistic(state, reducer); // React 19
```

Production checklist (release కి ముందు)

```
□ Error boundaries (app + route + widget levels)
□ Loading / error / empty states ప్రతి async UI కి
□ Code splitting (routes + heavy libs) + chunk-fail retry
□ Images: dimensions, lazy, modern format, LCP priority
□ a11y: keyboard walk-through, axe clean, focus visible, labels
□ SEO/metadata (title, description, OG, canonical)
□ Security: CSP, no secrets in bundle, sanitized HTML, httpOnly cookies
□ Monitoring: Sentry + source maps + web-vitals RUM
□ Bundle budget in CI; Lighthouse score check
□ Tests: critical flows (E2E) + integration (RTL) + unit (logic)
□ SPA deployment: history fallback, cache headers, versioned assets
□ Env config per environment; feature flags for risky features
```

> **ఈ guide పూర్తి చేసినందుకు అభినందనలు!** React ని నిజంగా నేర్చుకోవడం అంటే – ఈ concepts ని ఒక real project లో వాడటం. ఒక app build చేయండి: authentication + list/detail + forms + optimistic updates + tests. అప్పుడు ఈ document ని మళ్ళీ చదవండి – ప్రతి section కి "అవును, ఇది నేను ఎదుర్కొన్నాను" అనిపిస్తుంది. అదే నిజమైన నేర్చుకోవడం.
>
> **Companion docs:** `JavaScript_Telugu.md` · `OOPS_Telugu.md` · `HLD_Telugu.md` · `LLD_Telugu.md` · `SystemDesign_Go_Telugu.md` · `Security_Telugu.md` · `SoftwareEngineering_Telugu.md` · `DSA_00..10_Telugu.md`